مرجع شخصي عملي · بالعامية المصري

Terminal

إصدار macOS

من أول أمر لحد ما تكتب Script يشتغل لوحده

مكتوب للماك من الأول للآخر: zsh و Homebrew و launchd، والفروق بين BSD و GNU، ومعاه المقابل على سيرفرات Linux. كل أمر متشرّح حتة حتة.

`grep -rn "error" .` `pbcopy < key.pub` `open .` `brew install git`

`chmod 755 run.sh` `launchctl list` `sed -i '' 's/a/b/' f`

`ssh -i ~/.ssh/key user@host` `set -euo pipefail`

مرجعك الشخصي

من المبتدئ للمحترف

إصدار macOS · 16 فصل + تحديات + مشروع نهائي

+ Cheat Sheet وجدول Flags وجدول رموز

المحتويات

الكتاب ده مرجع ليك إنت، ومكتوب للماك: كل مثال وكل ناتج على macOS و zsh و Homebrew، والمقابل على سيرفرات Linux في صناديق لوحدها. ماشي على 6 مراحل من "يعني إيه Terminal أصلاً" لحد ما تكتب Scripts احترافية وتشتغل بأمان. كل أمر متشرّح حتة حتة، وكل Flag أو رمز بيتكرر بين الأوامر عليه علامة **حاجة مشتركة** وإحالة لمكانه الأصلي في الكتاب.

إزاي تستخدم الكتاب ده؟ دليل الألوان

- **معلومة مهمة**: حاجة لازم تفضل فاكرها
- **خلي بالك**: غلطة شائعة أو فخ
- **حاجة مشتركة**: Flag أو فكرة بتتكرر في أوامر كتير
- **لو حابب تتعمق**: تفاصيل من جوه، تقدر تعدّيها أول مرة
- **تشريح الأمر**: كل حتة في الأمر عليها رقم وشرح
- **تحديات الفصل**: سهل / متوسط / صعب، والحلول في الملحق
- **Best Practice**: الطريقة الصح في الشغل الحقيقي
- **مثال عملي**: جرّبه بإيدك
- **على Linux**: نفس الفكرة على السيرفر (بإطار متقطّع)
- **إحالة**: رقم الجزء والصفحة اللي فيها الشرح الأصلي
- **مشكلة شائعة**: العَرَض والسبب والحل
- **ملخص + صح ولا غلط + أسئلة**: آخر كل فصل

01 جوه الـ Terminal: إيه اللي بيحصل بجد؟ 8
1.1 حكاية الشاشة السودا: جت منين أصلاً؟ 8
1.2 إيه اللي بيحصل لما تدوس Enter؟ 10
1.3 الـ Builtin مقابل البرنامج الخارجي 11
1.4 الـ PATH: إزاي الـ Shell بيلاقي البرنامج؟ 12
1.5 شوف بعينك: مين "أبو" الأمر اللي شغّلته؟ 13
1.6 تشريح الـ Prompt 14
1.7 مشاكل شائعة 15

02 تشريح أي أمر: إزاي تقرا وتفهم أي حاجة تتكتب 17
2.1 الشكل العام لأي أمر 17
2.2 أنواع الـ Options 18
2.3 الشرطتين -- والشرطة لوحدها - 19
2.4 إزاي تجيب المساعدة لأي أمر 20
2.5 الـ Flags المشتركة: القاموس الكبير 21
2.6 ليه أوامر من النت مبتشتغلش على الماك؟ GNU ضد BSD 23
2.7 مشاكل شائعة 24

03 الـ Shells: sh و bash و zsh و fish و PowerShell 26
3.1 المشكلة: ليه مش Shell واحد وخلاص؟ 26
3.2 الجد الكبير: sh و POSIX 27
3.3 كل Shell في كارت 27
3.4 ليه الماك ساب bash وراح لـ zsh؟ 28
3.5 مقارنة الكتابة (Syntax) 29
3.6 PowerShell: عالم تاني (Objects مش نصوص) 31
3.7 مقارنة المميزات 31
3.8 إنت شغال على أنهي Shell؟ وإزاي تغيره؟ 32
3.9 مشاكل شائعة 35

04 الـ Filesystem على الماك: كل حاجة ليها مكان 37
4.1 المشكلة: الـ Finder مش بيقولك كل حاجة 37
4.2 خريطة المجلدات على الماك 38
4.3 المسارات: المطلق والنسبي 41
4.4 جوه الملف: الـ inode 42
4.5 الـ Hard link والـ Soft link 43
4.6 الديسكات والمساحة 44
4.7 مشاكل شائعة 46

05 التنقل والتعامل مع الملفات 48
5.1 إنت فين؟ وفيه إيه هنا؟ pwd و ls 48
5.2 التنقل: cd 50
5.3 إنشاء: mkdir و touch 50
5.4 النسخ والنقل والمسح: cp و mv و rm 51
5.5 قراية الملفات: cat و less و head و tail 53
5.6 تفاصيل الملف: file و stat و tree 55
5.7 مشاكل شائعة 56

06 الصلاحيات والمستخدمين 58
6.1 الفكرة: 3 أسئلة × 3 أنواع ناس 58
6.2 chmod: تغيير الصلاحيات 59
6.3 chown و chgrp: مين المالك؟ 61
6.4 umask: الصلاحيات الافتراضية 62
6.5 المستخدمين والجروبات 63
6.6 sudo و su 66
6.7 حماية الماك الزيادة: الـ Extended attributes والـ Quarantine 67
6.8 Full Disk Access و SIP: حتى root ليه حدود 69
6.9 مشاكل شائعة 70

07 قوة الـ Shell: الـ Pipes والـ Redirection والـ Quoting 72
7.1 الـ Streams التلاتة 72
7.2 الـ Redirection: وجّه الـ Streams 73
7.3 الـ | Pipe: ركّب الأوامر 74
7.4 الـ Globbing: النجمة وأخواتها 75
7.5 الـ Quoting: إمتى الـ Shell يفهم الرموز وإمتى لأ 76
7.6 الـ Command Substitution: $() 77
7.7 الـ Exit Codes و && و || و ; 78
7.8 المتغيرات الخاصة 80
7.9 مشاكل شائعة 80

08 معالجة النصوص والبحث 82
8.1 grep: دوّر على سطور 82
8.2 sed: عدّل النص وهو ماشي 85
8.3 awk: لغة الأعمدة 86
8.4 cut: قص الأعمدة 87
8.5 tr: بدّل الحروف 88
8.6 sort و uniq: رتّب وشيل المكرر 88
8.7 wc: عِد السطور والكلمات 90
8.8 tee: شوف واحفظ في نفس الوقت 90
8.9 xargs: من سطور لـ Arguments 90
8.10 find و mdfind: دوّر على ملفات 91
8.11 Pipelines حقيقية لتحليل اللوجات 94
8.12 مشاكل شائعة 95

09 العمليات والخدمات 97
9.1 ps: صورة للعمليات دلوقتي 97
9.2 top و htop: مراقبة لايف 99
9.3 الـ Signals و kill 100
9.4 الخلفية: & و jobs و fg و bg و nohup 101
9.5 الخدمات على الماك: launchd و launchctl و brew services 103
9.6 اللوجات على الماك: log show و log stream 107
9.7 مشاكل شائعة 110

10 الشبكات والاتصال بالسيرفرات 113
10.1 التشخيص: ping و ip 113
10.2 curl و wget: كلّم السيرفرات 115
10.3 الـ DNS: dig و nslookup 117
10.4 ssh: ادخل على سيرفر بعيد 119
10.5 ملف ~/.ssh/config: اختصر كل ده 121
10.6 scp: انسخ من وإلى السيرفر 122
10.7 rsync: المزامنة الذكية 122
10.8 tar: أرشف واضغط 123

10.9 البورتات و lsof 125
10.10 مشاكل شائعة 127

11 Homebrew ومديري الحزم: تسطيب البرامج صح 129
11.1 الفكرة: المخزن والحزمة والاعتماديات 129
11.2 تسطيب Homebrew نفسه 130
11.3 brew كل يوم: install و upgrade و info 131
11.4 Brewfile: نقل كل برامجك لماك جديد 134
11.5 على السيرفر: apt في Debian و Ubuntu و Mint 135
11.6 على السيرفر: dnf في Fedora و RHEL و Rocky 137
11.7 على Linux: pacman في Arch و Manjaro 137
11.8 ولزمايلك على Windows: winget و choco 138
11.9 جدول المقارنة الكبير 139
11.10 مشاكل شائعة 140

12 الـ Bash Scripting: خلّي الأوامر تشتغل لوحدها 143
12.1 أول Script: الـ Shebang 143
12.2 المتغيرات والحساب 144
12.3 الشروط: test و [ ] و [[ ]] 146
12.4 اللوبات: for و while 148
12.5 الدوال 149
12.6 الـ Arguments: $1 و $@ و $# 150
12.7 الوضع الصارم: set -euo pipefail و trap 151
12.8 Scripts عملية كاملة 152
12.9 مشاكل شائعة 155

13 تخصيص الـ Shell: ملفات الإعدادات والـ Aliases والـ Prompt 157
13.1 مين بيقرا إيه وإمتى؟ 157
13.2 الـ Aliases: اختصارات 159
13.3 دوال في ملف الإعدادات 161
13.4 متغيرات البيئة و export 161
13.5 الـ PATH: ضيف مجلد صح 163
13.6 الـ Prompt على مزاجك 164
13.7 مشاكل شائعة 167

14 السرعة والإنتاجية: الاختصارات والـ History و tmux 169
14.1 اختصارات السطر 169
14.2 Terminal.app و iTerm2: اختصارات Cmd 171
14.3 أدوات الماك اللي مش في أي حتة تانية 171
14.4 الـ History: متكتبش حاجة مرتين 174
14.5 الـ Tab Completion 176
14.6 tmux: شغلك ميضيعش 177
14.7 fzf: البحث التفاعلي 178
14.8 oh-my-zsh باختصار 179

15 الأوامر الخطيرة وإزاي تحمي نفسك 182
15.1 rm -rf: المسح من غير رجعة 182
15.2 dd: الـ Disk Destroyer 183
15.3 chmod -R 777 و chown -R على النظام 185
15.4 الـ Fork Bomb 185
15.5 > اللي بيمسح من غير ما يسأل 186
15.6 curl | sh: تشغيل Script من النت 187
15.7 قائمة الحماية 188

16 المشروع النهائي: أداة opskit 190
16.1 الخطة 190
16.2 الكود كامل 191
16.3 تشريح أصعب السطور 193
16.4 التثبيت والتشغيل اليومي 195
16.5 طوّرها بنفسك 197

أ Cheat Sheet وجدول الـ Flags وجدول الرموز 199
أ.1 جدول الـ Flags المشتركة 204

أ.2 جدول الرموز 206
ب قاموس المصطلحات 209
ج الإجابات والحلول 213

خريطة الكتاب

تخيل إنك داخل مدينة جديدة: أول حاجة بتعملها إنك تبص على الخريطة. بص على الطريق كله مرة واحدة قبل ما تبدأ. كل مرحلة بتبني على اللي قبلها: لو حسيت إن حاجة مش واضحة، ارجع للمرحلة اللي قبلها. الأرقام اللي جوه الدوايز هي أرقام المراحل، وفوقها أرقام الفصول.

الطريق من أول مرة تفتح فيها الـ Terminal لحد المشروع النهائي.

1 · الأساسيات

إيه اللي بيحصل جوه Terminal.app لما تدوس Enter، إزاي تقرا وتفك أي أمر (وليه أوامر النت ساعات مبتشتغلش على الماك: BSD ضد GNU)، وليه الماك بقى zsh وسايب bash 3.2.

2 · الملفات والصلاحيات

الماك من جوه: /Users و ~/Library و /opt/homebrew و APFS، التنقل والتعامل مع الملفات، والصلاحيات والمستخدمين و Quarantine و SIP.

3 · قوة الـ Shell

الـ Pipes والـ Redirection والـ Globbing والـ Quoting، ومعالجة النصوص بـ grep و sed و awk و find و mdfind.

4 · النظام والشبكة

الـ Processes والـ Signals و launchd و log، أوامر الشبكة و SSH، و Homebrew (ومعاه apt و dnf للسيرفرات).

5 · الأتمتة والتخصيص

الـ Bash Scripting اللي يشتغل على الماك و Linux، ملفات zsh والـ PATH، واختصارات Terminal.app و iTerm2 وأدوات الماك الخاصة.

6 · الاحتراف بأمان

الأوامر الخطيرة وإزاي تحمي نفسك منها (و Time Machine)، ومشروع نهائي بيجمع كل اللي اتعلمته.

Best Practice

استخدم الكتاب كمرجع: اقرا الفصل مرة، جرّب كل أمر بنفسك، حل **التحديات**، وبعدين خلي الـ **Cheat Sheet** وجدول الـ **Flags** وجدول **الرموز** في الملاحق جنبك وانت بتشتغل.

01

المرحلة 1 · الأساسيات

الفصل الأول

جوه الـ Terminal: إيه اللي بيحصل بجد؟

أول مرة تفتح فيها Terminal.app بتلاقي شباك فاضي وحرف % بيرمش ومستنيك. لا زرار ولا قوايم، ومش عارف تكتب إيه. الفصل ده هيحكيلك الحكاية من الأول: الشاشة دي جت منين، ومين اللي قاعد جواها بيفهم كلامك، وإيه اللي بيحصل من ساعة ما تدوس Enter لحد ما الرد يظهر. ولما تفهم ده، رسايل زي command not found هتبطّل تخوّفك.

بعد الفصل ده هتكون عارف

- الفرق بين Terminal و Shell و Console و TTY
- الخطوات اللي بيعملها الـ Shell لما تدوس Enter
- يعني إيه Builtin ويعني إيه برنامج خارجي
- إزاي الـ Shell بيلاقي البرنامج عن طريق الـ PATH
- يعني إيه fork و exec
- تقرا الـ Prompt وتفهم كل حتة فيه

1.1 حكاية الشاشة السودا: جت منين أصلاً؟

خلينا نرجع بالزمن شوية. في الستينات والسبعينات الكمبيوتر كان أوضة كاملة، ومكانش فيه شاشة ولا ماوس. طب الناس كانت بتكلّمه إزاي؟ كانوا بيوصّلوا بيه آلة كاتبة كهربا اسمها **Teletype**: تكتب الأمر على الورق، والكمبيوتر يرد عليك ويطبع الرد على نفس الورقة. الآلة دي اختصارها **TTY**، وافتكر الاسم ده لأنه لسه عايش معانا لحد النهارده.

بعد كده الورق اتشال وبقى مكانه شاشة وكيبورد، والجهاز ده اتسمى **Terminal**، يعني "الطرف" اللي إنت قاعد عليه والكمبيوتر الحقيقي في حتة تانية. وبعدين جت الواجهات الرسومية والماوس والأيقونات، والمفروض إن الحكاية تخلص هنا، صح؟ بس اللي حصل إن المبرمجين لقوا إن الكتابة أسرع بكتير في حاجات كتير. تخيل كده إنك عايز تغيّر اسم 200 صورة في `Downloads` : بالماوس هتقعد ساعة، وبسطر واحد في الـ Terminal هتخلص في ثانية.

فعملوا برنامج بيقلّد الـ Terminal القديم جوه شباك عادي، وده اللي إنت بتفتحه على الماك وإسمه **Terminal.app**. طب مين بقى اللي بيفهم الكلام اللي بتكتبه؟ مش الشباك نفسه. جوه الشباك فيه برنامج تاني قاعد مستنيك اسمه **Shell**، وده "المخ". يعني عندنا 4 كلمات هتسمعهم كتير، وكل واحدة فيهم ليها شغلانة:

Terminal (Terminal Emulator)

البرنامج اللي بيرسم الشباك: بياخد الزرايز اللي بتدوسها وبيعتها، ويعرض الحروف اللي راجعة بالألوان. على الماك عندك **Terminal.app** اللي جاي مع النظام (في `/Applications/Utilities`)، وبدايله المشهورة: iTerm2، Ghostty، Warp، kitty، WezTerm، Alacritty. هو "الشاشة والكيبورد" بس، ميعرفش يعني إيه `ls` .

Shell

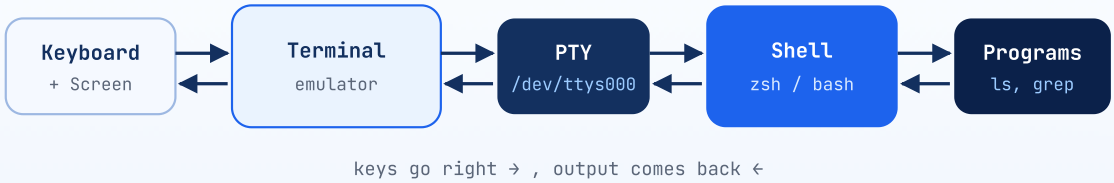
البرنامج اللي شغال جوه الـ Terminal و**بيفهم** الكلام اللي بتكتبه: بيفك الأمر، يدور على البرنامج، يشغله، ويرجعلك النتيجة. على الماك الـ Shell الافتراضي هو **zsh** من أيام macOS Catalina، وفيه كمان bash قديم وfish و PowerShell لو سطبتهم. ده "المخ".

Console

الشاشة النصية "الحقيقية" اللي النظام نفسه بيوفرها من غير واجهة رسومية، على الماك مفيش Console نصي بتوصله بزرار في الاستخدام العادي؛ أقرب حاجة ليه هي الـ Terminal اللي جوه **Recovery Mode**. أما على Linux فبتظهر لو دست `Ctrl+Alt+F3`، أو شاشة السيرفر لو ملهوش Desktop.

TTY و PTY

الـ TTY اختصار **TeleTYpewriter**: أجهزة الطباعة القديمة اللي كانت بتتوصل بالكمبيوتر. النهارده هي "الجهاز" اللي الـ Kernel بيتعامل معاه كأنه طرف كتابة وقراية. أما الـ **PTY** (Pseudo-TTY) فهو TTY "وهمي" بيعمله الـ Kernel لكل شباك Terminal بتفتحه.



شكل 1.1 الطبقات من الكيبورد لحد البرنامج: الـ Terminal بيرسم، والـ Shell بيفهم، والبرامج بتشتغل.

الكلمة	هو إيه؟	تشبيه
Terminal	برنامج الشباك (إدخال وعرض)	التليفون نفسه
Shell	المفسّر اللي بيفهم الأوامر ويشغلها	الموظف اللي بيرد عليك
Console	شاشة النظام النصية الأساسية	التليفون الأرضي اللي في الشركة نفسها
PTY / TTY	الجهاز (الحقيقي أو الوهمي) اللي بيوصّل الاتنين بالـ Kernel	خط التليفون

جدول 1.1 الأربع كلمات في سطر واحد

لو حابب تتعمق

تقدر تعدّيه في أول قراية

الـ Teletype القديم لسه موجود جوه النظام كـ "جهاز وهمي". كل ما تفتح تابة جديدة، الـ Kernel بيعمل لها **PTY** (Pseudo-TTY) خاص بيها، والـ Terminal والـ Shell بيتكلموا من خلاله. تقدر تشوف اسمه بنفسك:

تشريح الأمر

```
tty
```

1. `tty` أمر بيطبعلك اسم ملف الـ TTY اللي إنت متوصل بيه دلوقتي. على الماك هيطلع حاجة زي `/dev/ttys000` (يعني PTY رقم 000، والتابة اللي بعدها `/dev/ttys001`). على Linux نفس الفكرة بس اسمها `/dev/pts/0`. مالوش أي Flags محتاجها دلوقتي.

> **معلومة مهمة**
>
> عشان كده تقدر تغير الـ Terminal (مثلاً من Terminal.app لـ iTerm2) من غير ما تغير الـ Shell، وتغير الـ Shell (من zsh لـ bash) من غير ما تغير الـ Terminal. هما حاجتين منفصلتين تماماً.

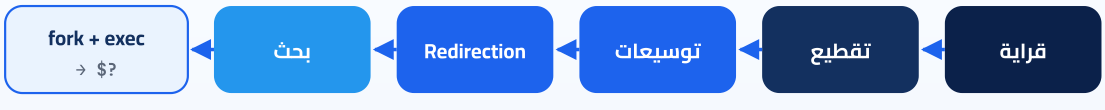
1.2 إيه اللي بيحصل لما تدوس Enter؟

تخيل إنك داخل مطعم وقلت للجرسون: "عايز 2 شاورما، واحدة من غير ثومية، وابعتهم على الترابيزة اللي جنب الشباك". الجرسون مش بيجري على المطبخ على طول. الأول بيسمع الجملة كلها، وبعدين يقسمها في دماغه (2 شاورما، من غير ثومية، الترابيزة دي)، ويعرف هيبعت الأوردر لمين، وفي الآخر يرجعلك يقولك "تمام". الـ Shell بيعمل نفس الحكاية بالظبط مع كل سطر بتكتبه.

خلينا نمشي ورا أمر واحد ونشوف الـ Shell بيعمل فيه إيه خطوة خطوة. الأمر ده بيعمل لستة بكل ملفات الـ PDF اللي في `Downloads` ويحطها في ملف اسمه `list.txt`:

```
Terminal
ls -l ~/Downloads/*.pdf > list.txt
```

1. **القراية (Read):** الـ Shell بيستنى لحد ما تدوس Enter، وبياخد السطر كله كنص.
2. **التقطيع (Tokenizing):** بيقسم السطر لكلمات (Tokens) على المسافات، مع احترام علامات التنصيص: `ls` و `-l` و `~/Downloads/*.pdf` و `>` و `list.txt`. وبيتعرف على الرموز الخاصة زي `>` و `|` و `&&`.
3. **التوسيعات (Expansions) بالترتيب:** الـ `~` تتحول لـ `/Users/user`، والمتغيرات زي `$HOME` تتحط قيمتها، و `$( )` يتنفذ ويتحط ناتجه، وفي الآخر الـ `*.pdf` تتحول لأسماء الملفات الحقيقية (Globbing).
4. **الـ Redirection:** قبل التشغيل، الـ Shell بيفتح `list.txt` للكتابة ويوجّه الناتج ليه بدل الشاشة.
5. **البحث عن الأمر:** هل `ls` ده Alias؟ ولا Function؟ ولا Builtin جوه الـ Shell؟ ولا برنامج على الديسك؟ (هنفصّلها تحت).
6. **التشغيل (exec + fork):** لو برنامج خارجي، الـ Shell بيعمل نسخة من نفسه (**fork**)، والنسخة دي بتستبدل نفسها بالبرنامج `/bin/ls` (**exec**)، والـ Shell الأصلي بيستنى.
7. **النهاية:** البرنامج يخلص ويرجع رقم اسمه **Exit Code** (صفر = نجاح)، والـ Shell يحفظه في `$?` ويطبعلك الـ Prompt تاني.



نفس الترتيب ده بيتكرر مع كل أمر بتكتبه

شكل 1.2 الرحلة من ساعة ما تدوس Enter لحد ما الـ Prompt يرجع.

حاجة مشتركة

الترتيب ده هو سر حاجات كتير هتشوفها بعدين: ليه `"*.txt"` بين علامات تنصيص مبتتوسعش، وليه `echo $HOME` بيطبع المسار مش الكلمة، وليه `>` بيمسح الملف حتى لو الأمر فشل (لأن الـ Redirection بيحصل **قبل** التشغيل). كل ده متفصّل في ↩ 7.5 ص 76 و ↩ 7.2 ص 73.

1.3 الـ Builtin مقابل البرنامج الخارجي

هنا بقى فيه سؤال شكله بسيط بس إجابته هتفتحلك حاجات كتير: لما تكتب `ls`، هو فين الـ `ls` ده؟ الإجابة إنه ملف عادي على الديسك في `/bin/ls`، زيه زي أي برنامج. طب و `cd`؟ هنا المفاجأة: مفيش ملف اسمه `cd` بيشتغل. مش كل أمر بتكتبه برنامج على الديسك. فيه أوامر "مبنية جوه" الـ Shell نفسه اسمها **Builtins**. ليه؟ خد مثال `cd`: لو `cd` كان برنامج خارجي، الـ Shell هيعمل له fork، والبرنامج هيغير المجلد بتاع **نفسه**، ويخلص ويموت، والـ Shell بتاعك هيفضل في مكانه! عشان كده `cd` لازم يبقى جوه الـ Shell عشان يغيّر حالة الـ Shell نفسه.

	Builtin	برنامج خارجي
مكانه	جوه الـ Shell نفسه	ملف على الديسك، زي `/bin/ls`
بيحتاج fork؟	لأ، أسرع	أيوه
يقدر يغير حالة الـ Shell؟	أيوه (المجلد، المتغيرات)	لأ
أمثلة	`cd` `export` `alias` `source` `exit` `read` `type` `history` `jobs`	`ls` `grep` `find` `cp` `ssh` `curl`
المساعدة	`man zshbuiltins` في zsh، و `help cd` في bash	`man ls` (الـ `ls` بتاع الماك مبيفهمش `--help`)

جدول 1.2 Builtin ولا خارجي؟

تشريح الأمر

```
type -a echo
 1   2  3
```

1. `type` Builtin بيقولك الأمر ده "نوعه إيه": alias ولا function ولا builtin ولا ملف.
2. `-a` اختصار **all**: اعرض **كل** الأماكن اللي ممكن الاسم ده يتلاقى فيها، مش أول واحد بس.
3. `echo` الاسم اللي بنسأل عليه.

```
Terminal
user@MacBook ~ % type -a echo
echo is a shell builtin
echo is /bin/echo
user@MacBook ~ % type ll
ll is an alias for ls -lAhG
user@MacBook ~ % type cd
cd is a shell builtin
```

الـ `ll` هنا alias إنت اللي عامله (الماك مفيهوش `ll` جاهز زي Ubuntu)؛ هنعمله في ↩ 13.2 ص 159.

لاحظ إن `echo` موجود مرتين: Builtin وبرنامج. الـ Shell بيختار الـ Builtin الأول لأنه أسرع. وترتيب البحث في zsh (و bash نفس الفكرة) هو:

1. **Alias** (اختصارات إنت عاملها، ↩ 13.2 ص 159)
2. **Function** (دوال، ↩ 12.5 ص 149)
3. **Builtin**
4. **برنامج في الـ PATH** (الجزء الجاي)

> **أوامر شبيهة: `which` و `command -v`**
>
> في **zsh** الـ `which` نفسه Builtin وبيعرف الـ aliases والـ builtins (زي `whence -c`). لكن في **bash** هو برنامج خارجي بيدور في الـ PATH بس، فنتيجته ممكن تختلف من Shell للتاني. أما `command -v ls` فده الطريقة القياسية (POSIX) اللي بتشتغل في أي Shell، وأحسن حاجة تستخدمها جوه الـ Scripts عشان تتأكد إن برنامج موجود.

1.4 الـ PATH: إزاي الـ Shell بيلاقي البرنامج؟

لو حصلك قبل كده إنك سطبت حاجة، وكتبت اسمها، والـ Terminal قالك "مش لاقيها"، فإنت قابلت الـ PATH من غير ما تعرف. تخيل الماك فيه آلاف الملفات، والـ Shell مش هيقعد يدور فيهم كلهم كل ما تكتب أمر، كده هيفضل يدور للصبح. فبدل كده إنت بتديله لستة صغيرة بالأماكن اللي الـ برامج بتبقى فيها، ويقوله "دوّر هنا بس".

فلما تكتب `grep`، الـ Shell مش بيدور في الديسك كله. هو بيبص في متغير اسمه **PATH**: لستة مجلدات مفصولة بـ `:`، ويدور فيها بالترتيب، وأول ما يلاقي ملف اسمه `grep` وقابل للتشغيل، يشغله ويقف.

تشريح الأمر

```
echo $PATH
```

1. `echo` اطبع اللي بعدي على الشاشة.
2. `$` علامة الدولار قبل اسم متغير معناها "حط قيمة المتغير ده هنا". من غيرها هيطبع كلمة PATH نفسها.
3. `PATH` اسم المتغير. الأسماء حساسة لحالة الحروف: `$path` حاجة تانية.

```
Terminal
user@MacBook ~ % echo $PATH
/Users/user/.local/bin:/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

- الترتيب مهم: لو فيه `python3` متسطب من Homebrew في `/opt/homebrew/bin`، وواحد تاني جاي مع أدوات Apple في `/usr/bin`، اللي في `/opt/homebrew/bin` هو اللي هيشتغل لأنه جاي الأول.
- الـ Shell بيحفظ مكان الأوامر اللي لقاها في ذاكرة (hash table) عشان ميدورش كل مرة. لو نقلت برنامج أو سطبت واحد جديد والـ Shell لسه "مش لاقيه"، اعمل `rehash` في zsh (أو `hash -r` في bash) عشان تمسح الذاكرة دي.
- إزاي تضيف مجلد للـ PATH بشكل دايم؟ ده في ↩ 13.5 ص 163.

ليه لازم تكتب `./script.sh`؟

المجلد الحالي (`.`) **مش** موجود في الـ PATH عن قصد، عشان الأمان: لو حد حط ملف اسمه `ls` في مجلد، وإنت دخلت المجلد ده وكتبت `ls`، مش عايز ملفه هو اللي يشتغل! فعشان تشغل حاجة في المجلد الحالي لازم تقول صراحةً `./script.sh`، يعني "الملف ده اللي هنا".

1.5 شوف بعينك: مين "أبو" الأمر اللي شغّلته؟

الجزء ده للي حابب يشوف اللي جوه المطبخ. لو حاسس إنه تقيل عليك دلوقتي عدّيه عادي، وارجعله بعد فصل الـ Processes.

لو حابب تتعمق — تقدر تعدّيه في أول قراية

فاكر خطوة الـ fork و exec؟ يعني الـ Shell بيعمل نسخة من نفسه، والنسخة تتحول للبرنامج. تعالى نتأكد إن ده بيحصل فعلاً. كل Process ليها رقم اسمه **PID**، ورقم أبوها اسمه **PPID**. خلينا نشوف إن الـ Shell هو "أبو" أي أمر بتشغله:

تشريح الأمر

```
ps -o pid,ppid,command
```

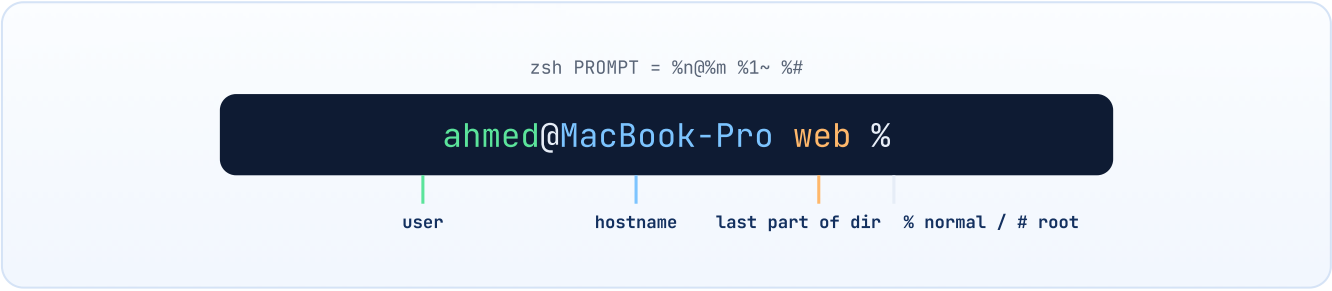
1. `ps` — اعرض الـ Processes (الشرح الكامل في ↩ 9.1 ص 97).
2. `-o` — اختصار **output format**: إنت اللي تحدد الأعمدة اللي عايزها.
3. `pid,ppid,command` — الأعمدة مفصولة بفاصلة من غير مسافات: رقم الـ Process، ورقم أبوها، والأمر نفسه. على الماك اسم العمود `command`؛ أما `cmd` فده اسمه في نسخة Linux.

```
Terminal
user@MacBook ~ % echo $$
4210
user@MacBook ~ % ps -o pid,ppid,command
  PID  PPID COMMAND
 4210  4198 -zsh
 4377  4210 ps -o pid,ppid,command
```

الـ `$$` متغير خاص فيه PID الـ Shell نفسه (4210). ولاحظ إن الـ `ps` رقم أبوه (PPID) هو 4210: يعني الـ Shell عمل fork، والنسخة عملت exec لـ `ps`. والشرطة اللي قبل `-zsh` معناها إنه **Login Shell**. وعلى الماك، Terminal.app بيفتح **كل** تابة جديدة كـ Login Shell، عكس Linux (هنعرف ليه ده مهم في ↩ 13.1 ص 157).

1.6 تشريح الـ Prompt

أول ما فتحت الـ Terminal شفت حاجة زي `ahmed@MacBook-Pro ~ %`، ويمكن قلت "إيه الكلام ده؟". السطر ده اسمه **Prompt**، وهو الـ Shell بيقولك "أنا جاهز، اتفضل". ومش مجرد ديكور، ده بطاقة تعريف صغيرة: بيقولك إنت مين، وعلى أنهي جهاز، وواقف فين دلوقتي. وده هيفرق معاك جداً لما تبقى فاتح 3 تابات، واحدة منهم متوصلة بسيرفر:



شكل 1.3 كل حتة في الـ Prompt ومعناها.

zsh	bash (PS1)	بيتحول لإيه
`%n`	`\u`	اسم المستخدم (user)
`%m`	`\h`	اسم الجهاز لحد أول نقطة (host)
`%~`	`\w`	المجلد الحالي كامل، والـ home بيتكتب `~`
`%1~`	`\W`	آخر جزء بس من المجلد الحالي (ده اللي الماك بيستخدمه افتراضياً)
`%#`	`\$`	`%` في zsh (أو `$` في bash) لو مستخدم عادي، و `#` لو root
`%*`	`\t`	الساعة HH:MM:SS

جدول 1.3 رموز الـ Prompt في zsh (متغير PROMPT)، وقصادها bash

على Linux (السيرفرات)

الـ Prompt الافتراضي على Ubuntu (bash) شكله `ahmed@laptop:~/projects/web$` : فيه `:` بين الجهاز والمسار، والمسار كامل، وفي الآخر `$` بدل `%` . هتشوفه كتير لما تدخل على سيرفر بـ ssh.

الـ $ و # في الكتب والمواقع

لما تشوف في أي شرح سطر بيبدأ بـ `$` (أو `%` زي الماك) فده معناه "اكتب الأمر ده كمستخدم عادي" والرمز نفسه **متتكتبش**. في الكتاب ده هنكتب الـ Prompt بشكل الماك `%` . ولو يبدأ بـ `#` يبقى "لازم root". وجوه ملف أو Script، الـ `#` معناها تعليق (Comment). نفس الرمز، معاني مختلفة حسب المكان.

ولو كتبت أمر ناقص (زي علامة تنصيص مفتوحة) هيظهرلك Prompt تاني؛ في zsh شكله `dquote>` (وفي bash `>` بس). ده اسمه **PS2**، ومعناه "كمّل، أنا مستني باقي الأمر". اخرج منه بـ `Ctrl+C` . وتخصيص الـ Prompt بالألوان هنعمله في ↩ 13.6 ص 164 .

1.7 مشاكل شائعة

مشكلة 1 command not found

كتبت اسم أمر والـ Shell قالك `zsh: command not found: htop` . ده معناه إن الـ Shell دوّر في الـ alias والـ functions والـ builtins وكل مجلدات الـ PATH وملقاش حاجة بالاسم ده.

الحل: (1) اتأكد من الإملاء وحالة الحروف. (2) يمكن البرنامج مش متسطب: سطّبه بـ `brew install htop` (↩ 11.1 ص 129). (3) يمكن متسطب في مجلد مش في الـ PATH: لو لسه مسطب Homebrew ومش لاقي `brew` نفسه، غالباً ناقصك سطر `brew shellenv` في `~/.zprofile` . دور بـ `ls /opt/homebrew/bin` وضيفه للـ PATH، وبعدها `rehash` .

مشكلة 2 Permission denied وإنت بتشغل Script

كتبت `./backup.sh` وطلع `Permission denied` . الملف موجود، بس مش عليه صلاحية التشغيل (x).

الحل: `chmod +x backup.sh` وبعدين شغله تاني. الصلاحيات متفصّلة في ↩ 6.1 ص 58 .

ملخص الفصل

- الـ Terminal بيرسم ويستقبل الزراير، والـ Shell بيفهم الأوامر، والـ TTY/PTY هو الخط بينهم وبين الـ Kernel.
- مع كل Enter: قراية ← تقطيع ← توسيعات ← Redirection ← بحث ← fork/exec ← Exit Code في `$?` .
- ترتيب البحث: Alias ← Function ← Builtin ← PATH. واستخدم `type -a` عشان تعرف.
- `cd` لازم يبقى Builtin لأن الـ Process الابن ميقدرش يغيّر مجلد أبوه.
- الـ PATH لستة مجلدات بـ `:` ، وأول ملف يتلاقي هو اللي يشتغل، والمجلد الحالي مش فيه.

صح ولا غلط؟

1. لو غيرت الـ Terminal من Terminal.app لـ iTerm2 لازم تغير الـ Shell كمان. صح / غلط
2. الـ Globbing (تحويل `*.pdf` لأسماء ملفات) بيعمله الـ Shell قبل ما البرنامج يشتغل. صح / غلط
3. `which` هو أدق طريقة تعرف بيها لو الأمر Builtin في أي Shell. صح / غلط

؟ أسئلة مراجعة

1. اشرح بكلامك الفرق بين الـ Terminal والـ Shell.
2. ليه `cd` مينفعش يبقى برنامج خارجي؟
3. عندك نسختين من `node` في مجلدين مختلفين في الـ PATH. مين هيشتغل؟ وإزاي تتأكد؟
4. ليه بنكتب `./script.sh` مش `script.sh` بس؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل شغّل `tty` في تابتين Terminal مختلفين (`Cmd+T` بتفتح تابة جديدة)، وقارن الناتج.
2. سهل استخدم `type` على: `cd` و `ls` و `pwd` و `type` نفسه. مين فيهم Builtin؟
3. متوسط اطبع الـ PATH كل مجلد في سطر لوحده. (تلميح: `echo $PATH | tr ':' '\n'` ، وفكّر كل حتة بتعمل إيه بعد ما تقرا ↩ 8.5 ص 88).
4. صعب افتح bash جوه zsh (اكتب `bash`)، واطبع `$$` في كل واحد، واستخدم `ps -o pid,ppid,command` عشان ترسم "شجرة العيلة". اخرج بـ `exit`.

02

المرحلة 1 · الأساسيات

الفصل التاني

تشريح أي أمر: إزاي تقرا وتفهم أي حاجة تتكتب

أكيد حصلك قبل كده: بتدوّر على حل لمشكلة، تلاقي واحد كاتب على Stack Overflow أمر زي tar -xzvf backup.tar.gz -C /opt وقايلك "اكتب ده وخلاص". فتنسخه وتلزقه وانت مش عارف كل حرف فيه بيعمل إيه، ومستني ربنا يستر. الفصل ده هو القاموس اللي هيخليك تقرا أي أمر زي ما بتقرا جملة عربي: شكل الأمر، وأنواع الـ Options، والـ -- ، وإزاي تجيب المساعدة، والـ Flags اللي بتتكرر في أوامر كتير والفخاخ بتاعتها. وكمان هتعرف ليه أوامر كتير من النت مبتشتغلش على الماك.

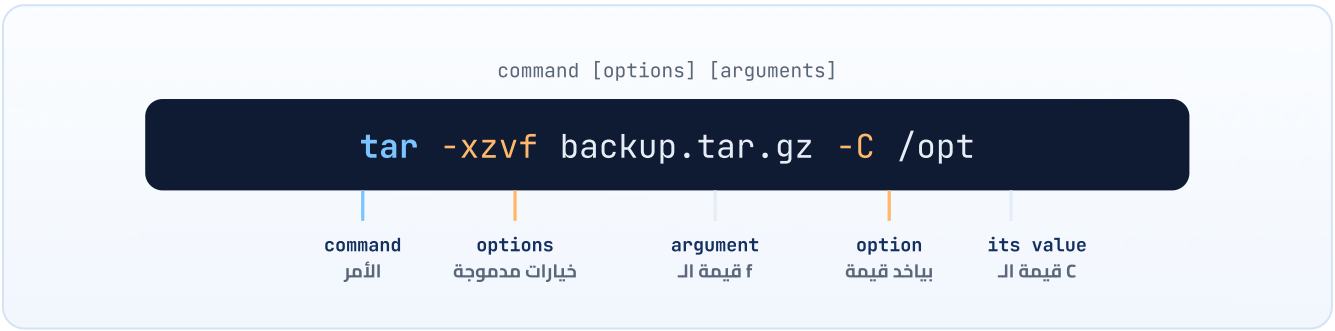
بعد الفصل ده هتكون عارف

- الشكل العام لأي أمر: أمر + Options + Arguments
- معنى -- و - لوحدها
- الفرق بين الـ- و الـ-- ودمج الحروف زي -la
- إزاي تقرا man page وسط الـ SYNOPSIS
- الـ Options اللي بتاخد قيمة: -n 5 و --lines=5
- الـ Flags المشتركة بين الأوامر والفخاخ بتاعتها

2.1 الشكل العام لأي أمر

خلينا نبدأ بحاجة مريحة: الأوامر شكلها مرعب من بره بس، لكن جواها ماشية على قواعد ثابتة زي قواعد اللغة. فكّر فيها كإنك بتكلم حد: فيه **فعل** (اعمل إيه؟)، وفيه **طريقة** (اعمله إزاي؟)، وفيه **مفعول به** (اعمله في إيه؟). "افتح الشباك بالراحة" = فعل + مفعول به + طريقة. الأوامر بالظبط كده، والفرق إن "بالراحة" بتتكتب حروف بشرطة قدامها.

وأحلى حاجة إن تقريباً كل أوامر الـ Terminal ماشية على نفس الشكل، فلو فهمته مرة هتقرا أي حاجة. خد الأمر اللي خوّفنا في الأول ده وتعالى نشرّحه:



شكل 2.1 أمر، وبعده Options بتعدّل سلوكه، وبعدها Arguments هي الحاجات اللي هيشتغل عليها.

Command

اسم البرنامج أو ال Builtin. دايماً أول كلمة.

(Flags) Options

بتبدأ بـ `-` أو `--` وبتغير **طريقة** شغل الأمر. اختيارية في الغالب.

Arguments

الحاجات اللي الأمر هيشتغل **عليها**: ملفات، مجلدات، نصوص، عناوين.

تشريح الأمر

```
1   2        3          4  5
tar -xzvf backup.tar.gz -C /opt
```

1. `tar` برنامج بيجمّع ملفات كتير في ملف واحد (Archive) أو يفكها. الاسم جاي من Tape ARchive.
2. `-xzvf` أربع Options مدموجين: `-x` (eXtract: فك) و `-z` (الملف مضغوط بـ gzip) و `-v` (Verbose: اطبع كل ملف وانت بتفكه) و `-f` (File: الكلمة الجاية هي اسم الأرشيف).
3. `backup.tar.gz` Argument: قيمة الـ `-f`. عشان كده الـ `f` لازم تبقى آخر حرف في المجموعة.
4. `-C` Change directory: فك الملفات جوه مجلد تاني بدل المجلد الحالي. حرف كبير، ومختلف تماماً عن `-c` (Create).
5. `/opt` قيمة الـ `-C`: المجلد اللي الملفات هتتفك فيه.

2.2 أنواع الـ Options

طب ليه مرة تلاقي `-l` ومرة `--long`، ومرة `-n 5` ومرة `-n5`؟ هل كل واحد كاتبها على مزاجه؟ الحقيقة إن فيه كام شكل متفق عليه، وكلهم اتعملوا لسبب. زمان الشاشات كانت صغيرة والكتابة بطيئة، فالناس كانت عايزة أقصر حاجة ممكنة: حرف واحد. ولما الأوامر كترت والحروف خلصت، ظهرت الأسماء الطويلة اللي مفهومة من غير ما تحفظ. والنتيجة إن النهارده عندك الاتنين، وده جدول بكل الأشكال اللي هتقابلها:

الشكل	مثال	المعنى
قصير (حرف واحد)	`ls -l`	شرطة واحدة + حرف
طويل (كلمة)	`grep --count`	شرطتين + كلمة، أوضح في الـ Scripts. أدوات الماك القديمة (BSD) زي `ls` و `cp` أغلبها ملهاش أشكال طويلة
مدموج	`ls -la`	بالظبط زي `ls -l -a`
قصير بقيمة	`head -n 5` أو `head -n5`	القيمة بعده بمسافة أو لازقة فيه
طويل بقيمة	`grep --max-count=5` أو `grep --max-count 5`	بـ `=` أو بمسافة
Subcommand	`git commit`، `brew install`، `launchctl list`	كلمة تانية بتحدد "العملية"، وليها Options خاصة بيها
قديم من غير شرطة	`tar xzf a.tgz`، `ps aux`	أسلوب BSD قديم لسه شغال في أوامر معدودة (والماك أصلاً مبني على BSD)

جدول 2.1 أشكال كتابة الـ Options

الحروف حساسة جداً

`ls -r` معناها **reverse** (اعكس الترتيب)، و `ls -R` معناها **Recursive** (ادخل جوه المجلدات). `cp -r` و `cp -R` نفس الحاجة، بس `chmod -r` معناها "شيل صلاحية القراية"! دايماً اتأكد من الـ man قبل ما تفترض.

ترتيب الـ Options على الماك

أدوات الماك (نسخ **BSD**) بتبطّل تقرا Options بعد أول Argument، فـ `ls /tmp -l` هيقولك `ls: -l: No such file or directory` لأنه افتكر `-l` اسم ملف. القاعدة: **الـ Options الأول، وبعدها الـ Arguments**. دي هتنفعك على أي نظام.

يعني إيه BSD و GNU؟

الأوامر الأساسية (`ls` و`sed` و`grep` و`date`...) ليها "عيلتين": نسخ **GNU** اللي على Linux، ونسخ **BSD** اللي جاية مع الماك. الاتنين بيعملوا نفس الشغلانة، بس بعض الـ Flags مختلفة. عشان كده أي شرح على النت مكتوب لـ Linux ممكن أمر فيه ميشتغلش معاك. هتلاقي الفروقات دي متعلّمة في الكتاب كله، وجدول كامل بيها في ↩ 2.6 ص 23.

2.3 الشرطتين `--` والشرطة لوحدها `-`

دي حكاية حقيقية بتحصل كتير: واحد كتب أمر غلط بالسهو فطلعله ملف اسمه `-rf` (أيوه، الاسم يبدأ بشرطة). وعايز تمسحه. لو كتبت `rm -rf` الأمر هيفتكرها Options. الحل: `--` معناها "خلصت الـ Options، أي حاجة بعدي Argument حتى لو بتبدأ بشرطة".

تشريح الأمر

```
rm -- -rf
1  2  3
```

1. `rm` امسح ملف (↩ 5.4 ص 51).
2. `--` نهاية الـ Options. ده اتفاق عام بتفهمه أغلب الأوامر، مش خاص بـ rm.
3. `-rf` هنا بقى اسم ملف عادي، مش Options. وفيه حل تاني: `rm ./-rf` لأن الاسم بقى يبدأ بنقطة.

حاجة مشتركة

نفس الـ `--` هتلاقيها في أماكن تانية بنفس المعنى: `grep -- -v file` عشان تدور على النص "-v"، و`kubectl exec pod -- ls` و`docker run img -- args` عشان تفصل Options الأمر عن الأمر اللي هيتنفذ جوه. وجوه الـ Scripts: `set -- a b c` بيغير الـ Arguments (↩ 12.6 ص 150).

أما الشرطة **لوحدها** `-` ففي أوامر كتير معناها "**اقرا من الـ stdin** بدل ملف" أو "اكتب على الـ stdout": `cat -`، `tar -czf - dir | ssh host "tar -xzf -"`. وفي `cd -` معناها حاجة تالتة خالص: "ارجع للمجلد اللي كنت فيه". الـ stdin والـ stdout متفصلين في ↩ 7.1 ص 72.

2.4 إزاي تجيب المساعدة لأي أمر

مفيش حد في الدنيا حافظ كل الـ Flags، ولا حتى الناس اللي بقالها 20 سنة شغالة على الـ Terminal. السر إنهم عارفين **يسألوا مين**. وكل أمر تقريباً جاي معاه "كتالوج" جوه الجهاز نفسه، من غير نت ومن غير ما تفتح المتصفح. خلينا نشوف الطرق دي من الأسرع للأكمل:

`man ls`	الدليل الكامل (Manual)، وده مرجعك الأساسي على الماك. اتحرك بالأسهم، دوّر بـ `/word`، التالي بـ `n`، اخرج بـ `q`.
`brew --help`	ملخص سريع للأدوات الحديثة (git و brew و node...). أدوات BSD زي `ls` مبتفهمش `--help`، بس لو كتبت Option غلط بتطبعلك سطر `usage:`.
`man zshbuiltins`	مساعدة الـ Builtins في zsh (زي `cd` و `export`). في bash بتكتب `help cd`.
`man -k password`	دوّر على أوامر ليها علاقة بكلمة (نفس `apropos`).
`tldr tar`	أمثلة عملية مختصرة. مش متسطب افتراضياً: `brew install tlrc` (أو `tldr`).
`type -a cmd`	هو إيه أصلاً؟ (↩ 1.3 ص 11)

> ⚠ **فخ الـ `-h`**
>
> في أوامر كتير `-h` معناها help، لكن في `ls -lh` و `df -h` و `du -h` و `sort -h` معناها **human-readable** (أحجام بـ K و M و G). عشان كده `man` هو الأضمن على الماك، و `--help` للأدوات الحديثة.

أقسام الـ man

الـ man متقسم لأقسام بأرقام، وعشان كده بتشوف حاجات زي `passwd(1)` و `passwd(5)`:

القسم	فيه إيه	مثال
1	أوامر المستخدم العادية	`man 1 passwd` (أمر تغيير الباسورد)
5	شكل ملفات الإعدادات	`man 5 passwd` (شكل ملف /etc/passwd)
7	مفاهيم عامة	`man 7 signal`
8	أوامر إدارة النظام (root)	`man 8 launchd`

جدول 2.2 أهم أقسام الـ man

إزاي تقرا سطر الـ SYNOPSIS

man cp · SYNOPSIS (macOS)

```
cp [-R [-H | -L | -P]] [-fi | -n] [-aclpSsvXx] source_file target_file
cp [-R [-H | -L | -P]] [-fi | -n] [-aclpSsvXx] source_file ... target_directory
```

الرمز	المعنى
`[ ]`	اختياري، ينفع تسيبه
`...`	ينفع يتكرر أكتر من مرة (`source_file ...` = ملف أو أكتر)
`source_file` أو `CAPS` أو `<name>`	مكان تحط فيه قيمتك إنت (متكتبش الكلمة نفسها)
`[-aclpv]`	مجموعة حروف، اختار منها اللي عايزه وادمجهم
`a | b`	واحد من الاتنين
`{a,b}`	لازم تختار واحد

جدول 2.3 رموز الـ SYNOPSIS

فالسطر التاني معناه: "cp، وبعدها Options اختيارية (`-R` ومعاها واحد من `-H` أو `-L` أو `-P`، و `-f` أو `-i` أو `-n` أو...)، وبعدها ملف أو أكتر، وفي الآخر مجلد". يعني `cp a.txt b.txt c.txt backup/` صح.

على Linux (السيرفرات)

نفس الصفحة على Linux شكلها أبسط: `cp [OPTION]... SOURCE... DIRECTORY`، لأن GNU بيحط كل الـ Options في كلمة `[OPTION]...` ويشرحها تحت.

2.5 الـ Flags المشتركة: القاموس الكبير

لو اتعلمت كذا لغة قبل كده، هتلاحظ إن كلمات كتير بتتشابه بينهم. نفس الكلام في الـ Terminal: أحلى حاجة فيه إن فيه "عُرف": حروف معينة بتتكرر بنفس المعنى في أوامر كتير. لو حفظتها هتخمّن نص الـ Flags صح من غير ما تبص. بس خلي بالك من الاستثناءات اللي في العمود التالت:

الـ Flag	المعنى الأشهر	معاني تانية (الفخاخ)	فين في الكتاب
`-r` / `-R`	Recursive: ادخل جوه المجلدات (`cp -r`، `rm -r`، `grep -r`، `scp -r`)	`chmod` / `chown` بالكبيرة بس `-R`. `ls -r` = reverse و `sort -r` = reverse. على Linux `grep -R` بيتبع الـ symlinks و `-r` لأ	↩ 5.4 ص 51
`-f`	Force: من غير ما تسأل ولا تشتكي (`rm -f`، `cp -f`، `ln -sf`)	`tar -f` = اسم الملف. `tail -f` = follow (تابع لايف). `grep -f` = خد الـ patterns من ملف	↩ 5.5 ص 53

الـ Flag	المعنى الأشهر	معاني تانية (الفخاخ)	فين في الكتاب
`-i`	Interactive: اسألني قبل أي حاجة (`rm -i`، `cp -i`، `mv -i`)	`grep -i` و `sort -f`: تجاهل حالة الحروف. `sed -i ''`: عدّل الملف نفسه (In-place)، وعلى الماك لازم `''` بعدها	↩ 8.2 ص 85
`-v`	Verbose: احكيلي بتعمل إيه (`cp -v`، `tar -v`، `ssh -v`)	`grep -v` = اعكس (السطور اللي مفيهاش). في بعض البرامج `-v` = version	↩ 8.1 ص 82
`-n`	Number: رقّم السطور (`grep -n`، `cat -n`)	`head/tail -n 5` = عدد. `sort -n` = ترتيب رقمي. `echo -n` = من غير سطر جديد. `rsync -n` = dry-run	↩ 8.6 ص 88
`-a`	All: كله (`ls -a` يظهر المخفي)	`cp -a` و `rsync -a` = Archive (احفظ كل الخصائص). `tee -a` = Append	↩ 10.7 ص 122
`-l`	Long: تفاصيل (`ls -l`)	`wc -l` = عد السطور. `grep -l` = أسماء الملفات بس	↩ 8.7 ص 90
`-h`	Human-readable: K4.0 و G1.2	في أوامر تانية = help	↩ 2.4 ص 20
`-p`	Parents: `mkdir -p a/b/c`	`cp -p` = Preserve. `ssh -p 22` = Port، بس `scp -P` كبيرة!	↩ 10.6 ص 122
`-q`	Quiet: اسكت، متطبعش غير الضروري		
`-y`	Yes: وافق على كل الأسئلة (`apt install -y` على Linux)	`brew` أصلاً مبيسألش، فمش محتاجها	↩ 11.1 ص 129
`-o`	Output: اسم ملف الناتج (`curl -o`، `sort -o`)	`grep -o` = اطبع الجزء المطابق بس. `ps -o` = شكل الأعمدة	↩ 10.2 ص 115
`-e`	Expression: `grep -e`، `sed -e`	`echo -e` في bash = افهم `\n` و `\t` (الـ echo بتاع zsh بيفهمهم من غيرها)	
`-c`	Count: `grep -c`، `uniq -c`	`tar -c` = Create. `bash -c "cmd"` = نفّذ النص ده	
`--version` / `--help`	المساعدة / رقم الإصدار، في الأدوات الحديثة (`git` و `brew` و `node`)	أدوات BSD على الماك مبتفهمهمش؛ استخدم `man`	

جدول 2.4 الـ Flags المشتركة ومعانيها والاستثناءات

Best Practice

في ال Scripts استخدم ال Options **الطويلة** (`--recursive` بدل `-r`) لما تكون متاحة: أي حد هيقرا ال Script بعدك (حتى إنت بعد 6 شهور) هيفهم من غير ما يفتح ال man.

2.6 ليه أوامر من النت مبتشتغلش على الماك؟ BSD ضد GNU

الحكاية دي هتوفر عليك ساعات من الحيرة، فخليني أحكيهالك من الأول. في السبعينات، شركة T&AT عملت نظام اسمه **Unix**، وفيه الأوامر اللي إنت بتستخدمها لحد النهارده: `ls` و `cp` و `grep` . الجامعة اللي في Berkeley في كاليفورنيا خدت Unix وطوّرت نسخة منه اسمها **BSD**. وبعدها بشوية، واحد اسمه Richard Stallman بدأ مشروع اسمه **GNU** علشان يكتب نفس الأوامر دي من الصفر وتبقى مجانية ومفتوحة لأي حد.

وهنا بقى بتحصل التفريعة: **Linux** خد الأدوات من GNU، والماك اتبنى على أساس BSD (ال Kernel بتاعه اسمه Darwin، وجذوره من BSD). يعني الاتنين عندهم `sed` ، بس كل واحد عنده "لهجة" مختلفة شوية، زي المصري والشامي: نفس اللغة، وكلمات معينة بتفرق. وبما إن أغلب الشروحات على النت مكتوبة لسيرفرات Linux، هتقابل أوامر كتير تشتغل هناك وتعلّق عندك. الجدول ده فيه أهم الفروقات اللي هتقابلها في الكتاب:

الأمر	على الماك (BSD)	على Linux (GNU)	فين في الكتاب
`sed -i`	`sed -i '' 's/a/b/' f` : لازم قيمة لامتداد ال Backup، و `''` = من غير Backup	`sed -i 's/a/b/' f`	↩ 8.2 ص 85
`date`	`date -v-1d` لإمبارح، و `date -r 1700000000` لتحويل Timestamp	`date -d yesterday` و `date -d @1700000000`	↩ 12.2 ص 144
`stat`	`stat -f '%z %N' f` (الحجم والاسم)	`stat -c '%s %n' f`	↩ 5.5 ص 53
`ls` **بالألوان**	`ls -G` (أو متغير `CLICOLOR=1`)	`ls --color=auto`	↩ 5.1 ص 48
`grep -P`	مش موجود: استخدم `grep -E` أو سطّب `ggrep`	Regex على طريقة Perl	↩ 8.1 ص 82
`xargs`	مفيش داعي لـ `-r` (وممكن يرفضها): الماك أصلاً مبيشغّلش الأمر لو مفيش مدخلات	`xargs -r` عشان ميشتغلش على فاضي	↩ 8.9 ص 90
`readlink -f`	شغال في الإصدارات الحديثة (من macOS 12.3)، ومعاه `realpath` ؛ في الأقدم مكانش موجود	شغال	
`base64 -d`	`base64 -d` (و `-D` القديمة)، ومفيش `-w` : الناتج سطر واحد افتراضياً، و `-b 76` لو عايز تقسّمه	`base64 -d` و `-w 0`	
`find`	لازم تكتب المجلد: `find . -name x`	ينفع `find -name x` من غير مجلد	↩ 8.10 ص 91

الأمر	على الماك (BSD)	على Linux (GNU)	فين في الكتاب
`sort` / `du`	`du -d 1` للعمق	`du --max-depth=1` (و `-d 1` شغالة برضه)	↩ 4.6 ص 44
الخيارات الطويلة	كتير من أدوات BSD ملهاش `--long` ولا `--help`	تقريباً كل أوامر GNU ليها	↩ 2.2 ص 18

جدول 2.5 أهم الفروقات بين أدوات الماك (BSD) و Linux (GNU)

عايز أدوات GNU على الماك؟

تقدر تسطّبها جنب أدوات الماك من غير ما تبوّظ حاجة: `brew install coreutils gnu-sed grep findutils` . هتلاقيها بحرف **g** في الأول: `gls` و `gdate` و `gstat` و `gsed` و `ggrep` و `gfind` و `gxargs` . نصيحتي: اتعلّم نسخ الماك الأصلية الأول، واستخدم نسخ `g` لما تحتاج Script يشتغل بنفس الشكل على الماك والسيرفر.

لو حابب تتعمق

تقدر تعدّيه في أول قراية

ممكن تخلّي `sed` نفسها تبقى نسخة GNU، عن طريق إنك تحط مجلد `gnubin` في أول الـ PATH: `export PATH="$(brew --prefix)/opt/gnu-sed/libexec/gnubin:$PATH"` . الأمر `brew --prefix` بيطبع مكان Homebrew (`/opt/homebrew`)، و `$( )` بتحط ناتجه في النص، و `:$PATH` بتحط الـ PATH القديم بعده. بس خلي بالك: أي Script مكتوب للماك وبيعتمد على `sed -i ''` هيبوظ ساعتها.

2.7 مشاكل شائعة

مشكلة 1 **unrecognized option أو invalid option**

نسخت أمر من شرح على النت زي `ls --group-directories-first` وطلعلك `ls: unrecognized option `--group-directories-first'` وتحتها سطر `usage:` . الشرح ده مكتوب لـ Linux (أدوات GNU)، والـ Option دي مش موجودة في نسخة BSD اللي على الماك.

الحل: شوف `man cmd` على **جهازك إنت**، ومتعتمدش على شرح لنظام تاني. ولو محتاج نسخة GNU بالذات، سطّبها بـ `brew install coreutils` ، وهتلاقيها بحرف `g` في الأول: `gls` و `gcp` و `gdate` (التفاصيل في ↩ 2.6 ص 23).

ملخص الفصل

- أي أمر = Command + Options + Arguments، والـ Options الأول أضمن.
- `-la` = `-l -a`، والـ Option اللي بياخد قيمة يبقى آخر حرف في المجموعة.
- `--` = نهاية الـ Options، و `-` لوحدها غالباً = stdin/stdout.
- المساعدة: `man` هو الأساس على الماك، و `--help` للأدوات الحديثة، و `man zshbuiltins` للـ Builtins.
- أدوات الماك نسخ BSD، وفيها Flags مختلفة عن GNU اللي على Linux.
- الـ Flags ليها معاني مشهورة، بس فيه فخاخ: `-h` و `-v` و `-n` و `-f` و `-p`.

صح ولا غلط؟

1. `tar -xvfz a.tgz` زي `tar -xzvf a.tgz` بالظبط. صح / غلط
2. `ls -R` و `ls -r` نفس المعنى. صح / غلط
3. في `[OPTION]...` الـ Options اختيارية وينفع تتكرر. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين BSD و GNU، وليه أمر `sed -i 's/a/b/' f` من شرح Linux مش هيشتغل على الماك زي ما هو؟
2. فكّك الأمر `grep --max-count=20 -i error a.log b.log` لكل حتة فيه.
3. إزاي تمسح ملف اسمه `--help`؟ اكتب طريقتين.
4. اذكر 3 أوامر `-n` فيها معناها مختلف.
5. ليه `man 5 passwd` مختلفة عن `man passwd`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح `man ls`، ودور جواه على كلمة `human` بـ `/human`، واعرف الـ Flag. اخرج بـ `q`.
2. متوسط اعمل ملف اسمه `-n` بـ `touch -- -n`، وبعدين امسحه بطريقتين مختلفتين.
3. صعب من غير ما تشغله، فكّك الأمر: `tar -czf - logs/ | ssh srv "tar -xzf - -C /backup"`. (هترجعله تاني بعد فصل الشبكات).

03

المرحلة 1 · الأساسيات

الفصل التالت

الـ Shells: sh و bash و zsh و fish و PowerShell

سنة 2019 الناس اللي فتحت الـ Terminal على الماك بعد تحديث Catalina لقت رسالة غريبة بتقول: The default interactive shell is now zsh. ناس كتير قالت "يعني إيه؟ هو أنا كنت شغال على إيه أصلاً؟". الفصل ده بيحكي حكاية الـ Shells: ليه فيه أكتر من واحد، وليه Apple سابت bash وراحت لـ zsh، وليه الـ bash اللي على جهازك قديم جداً، وإزاي تختار وتغيّر. وهتعرف كمان ليه أمر نسخته من صاحبك ممكن ميشتغلش عندك.

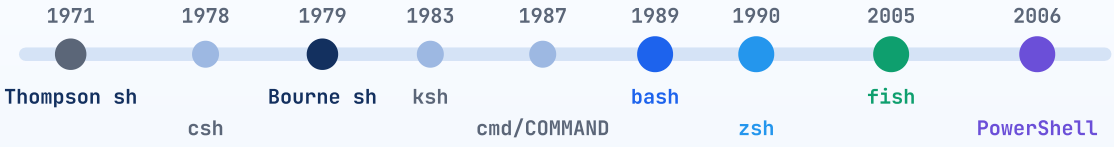
بعد الفصل ده هتكون عارف

- تاريخ الـ Shells وليه فيه أكتر من واحد
- ليه الماك بيستخدم zsh و bash 3.2 القديم، وإزاي تسطب bash حديث
- يعني إيه POSIX وليه sh مهم
- الفروق العملية بين bash و zsh و fish
- إزاي PowerShell مختلف من الأساس (Objects مش نصوص)
- جدول مقارنة للكتابة والمميزات وملفات الإعدادات
- تعرف الـ Shell الحالي وتغيره بـ chsh

3.1 المشكلة: ليه مش Shell واحد وخلاص؟

سؤال منطقي جداً. يعني لو الـ Shell ده "المخ" اللي بيفهم الأوامر، ليه مش فيه واحد بس وكلنا نستخدمه؟ الإجابة شبه حكاية الموبايلات: كل جيل بيطلع عشان يحل حاجة كانت مضايقة الناس في اللي قبله.

الأول خالص كان فيه Shell بسيط جداً بيشغّل أوامر وخلاص. الناس استخدمته وبدأت تقول "طب عايزين نكتب برامج صغيرة بيه، فيها شروط و Loops"، فاتعمل Shell أقوى في الـ Scripting. وبعدها الناس قالت "طب إحنا قاعدين عليه طول اليوم، عايزين ألوان واقتراحات وهو يكمّل الكلام معانا"، فاتعمل Shell مريح أكتر في الاستخدام اليومي. ولما ويندوز احتاج حاجة بتفهم نظامه هو، اتعملت حاجة مختلفة خالص. والنتيجة: عيلة كبيرة، وكل فرد فيها اتولد علشان سبب.



2018: PowerShell Core open-source cross-platform · 2019: macOS default = zsh · 2025: fish 4.0 rewritten in Rust

شكل 3.1 عيلة الـ Shells على خط زمني.

3.2 الجد الكبير: sh و POSIX

تخيل لو كل شركة عربيات عملت الدركسيون في مكان مختلف، مرة على اليمين ومرة في النص. هتحتاج تتعلم سواقة من الأول مع كل عربية. نفس المشكلة كانت هتحصل مع الـ Shells، فالناس اتفقت على "قواعد مشتركة". خلينا نشوف الحكاية دي:

الـ **Bourne Shell** (`sh`) اتعمل سنة 1979 وبقى الأساس. بعدين اتعمل معيار اسمه **POSIX** بيحدد "الحد الأدنى" اللي أي Shell متوافق لازم يفهمه. يعني لو كتبت Script بـ POSIX sh بس، هيشتغل على أي نظام Unix تقريباً.

> ⚠ **sh على جهازك مش هو نفسه في كل حتة**
>
> على الماك، `/bin/sh` بيشغّل في الحقيقة **bash 3.2** في وضع اسمه "POSIX mode"، فحاجات زي `[[ ]]` هتعدّي معاك. وده فخ، لأن نفس الـ Script هيبوظ على السيرفر. اكتب `sh --version` وهتشوف بنفسك إنه بيقولك GNU bash.

> **على Linux (السيرفرات)**
>
> على Debian و Ubuntu، `/bin/sh` بيشاور على **dash**: Shell صغير وسريع جداً بيفهم POSIX بس. فلو كتبت Script فيه حاجات خاصة بـ bash (زي `[[ ]]` أو Arrays) وشغلته بـ `sh script.sh` هيبوظ. اعرف بـ `ls -l /bin/sh` . والماك نفسه فيه `/bin/dash` لو عايز تجرّب الـ Script عليه قبل ما ترفعه: `dash script.sh` .

3.3 كل Shell في كارت

خلينا نتعرّف على أفراد العيلة واحد واحد. مش مطلوب منك تحفظهم، المهم تعرف كل واحد بيتقال عليه إيه علشان لما تقابل اسمه في شرح متتلخبطش:

Bourne Again SHell · bash

اتعمل سنة 1989 كبديل حر لـ sh (مشروع GNU). الـ Default في أغلب توزيعات Linux، وهو اللي معظم الـ Scripts والشروحات مكتوبة بيه. آخر إصدار رئيسي **5.3** (يوليو 2025). خلي بالك: macOS لسه جاي معاه bash **3.2** القديم جداً لأسباب رخصة (الحكاية كاملة تحت).

Z shell · zsh

اتعمل سنة 1990. متوافق مع bash في 90% من الكتابة، بس أقوى في الاستخدام التفاعلي: Tab completion ذكي، Globbing متقدم، تصحيح أخطاء الكتابة، وPlugins (أشهرها Oh My Zsh). الـ Default على **macOS من Catalina (2019)**، ولحد النهارده في macOS 27 Golden Gate. الماك جاي معاه zsh **5.9**، وده الـ Shell اللي الكتاب ده ماشي بيه.

Friendly Interactive SHell · fish

اتعمل سنة 2005 وفلسفته "يشتغل حلو من غير أي إعدادات": اقتراحات أوتوماتيك من الـ History، ألوان للأوامر الصح والغلط، وإعدادات من المتصفح. **مش متوافق مع POSIX** عن قصد، فـ Scripts الـ bash مش هتشتغل عليه. من الإصدار 4.0 (فبراير 2025) اتكتب من جديد بلغة Rust.

pwsh · PowerShell

من Microsoft (2006)، ومن 2018 مفتوح المصدر وشغال على Linux و macOS كمان. الفرق الجوهري: الـ Pipe بيعدّي **Objects** مش نصوص. الإصدار الحالي **LTS 7.6** (مارس 2026). وعلى Windows فيه كمان "Windows PowerShell 5.1" القديم المدمج، ودول برنامجين منفصلين.

cmd.exe

الـ Command Prompt القديم بتاع Windows، وارث DOS. محدود جداً في الـ Scripting (ملفات `.bat`). موجود عشان التوافق مع الحاجات القديمة، ومايكروسوفت نفسها بتنصح بـ PowerShell.

والباقي

dash (sh سريع في Debian)، **ksh** (KornShell، لسه موجود في أنظمة Unix قديمة)، **csh/tcsh** (كتابة شبه لغة C، قديم)، و **nushell** (حديث، بيتعامل مع البيانات كجداول زي فكرة PowerShell).

3.4 ليه الماك ساب bash وراح لـ zsh؟

دي حكاية فيها شوية قانون، بس هتفهم منها حاجات كتير. لحد سنة 2019 كان الـ Shell الافتراضي على الماك هو bash. وفي 2007 bash طلع بإصدار 3.2، وبعدها بشوية مشروع GNU غيّر الرخصة بتاعته لنسخة جديدة اسمها **GPLv3**. الرخصة دي فيها شروط Apple مكانتش مرتاحة لها. فـ Apple عملت إيه؟ وقفت عند bash 3.2 ومحدّثتهوش تاني خالص.

السنين عدّت، و bash وصل لـ 4 و 5، وفيه حاجات جديدة كتير، والماك لسه قاعد على نسخة من 2007. ففي Apple، Catalina قررت تخلّي **zsh** هو الافتراضي، لأن رخصته (MIT-like) مريحة ليها، وهو أصلاً أقوى في الاستخدام اليومي. بس سابت bash 3.2 موجود في `/bin/bash` علشان الـ Scripts القديمة متبوظش.

Terminal

```
user@MacBook ~ % zsh --version
zsh 5.9 (arm64-apple-darwin)
user@MacBook ~ % /bin/bash --version | head -n 1
GNU bash, version 3.2.57(1)-release (arm64-apple-darwin)
```

آخر الكلام اللي بين القوسين بيختلف حسب جهازك وإصدار النظام.

طب ده يفرق معاك في إيه؟ الحاجات دي **مش موجودة** في bash 3.2، وهتلاقيها في Scripts كتير على النت:

الحاجة	من bash رقم	بتعمل إيه
`declare -A`	4.0	Associative Arrays (مفتاح وقيمة، زي Object في JavaScript)
`mapfile` / `readarray`	4.0	تقرا ملف سطر سطر جوه Array
`${var,,}` و `${var^^}`	4.0	تحويل لحروف صغيرة أو كبيرة
`shopt -s globstar` (`**`)	4.0	الـ `**` اللي بتدخل جوه كل المجلدات (zsh عنده دي من غير أي إعداد)
`|&`	4.0	اختصار `2>&1 |`
`${arr[-1]}`	4.3	آخر عنصر في Array

جدول 3.1 حاجات ناقصة في bash 3.2 اللي على الماك

تسطّب bash حديث بـ Homebrew

الحل سهل: سطّب نسخة حديثة جنب القديمة، ومتلمسش القديمة خالص (النظام ممكن يكون معتمد عليها). Homebrew متشرح بالتفصيل في ↩ 11.3 ص 131:

تشريح الأمر

```
brew install bash
 1    2      3
```

1. `brew` — مدير الحزم Homebrew.
2. `install` — Subcommand: سطّب.
3. `bash` — اسم الحزمة. هتتسطب في `/opt/homebrew/bin/bash` على أجهزة Apple Silicon (أو `/usr/local/bin/bash` على أجهزة Intel القديمة).

Terminal

```
user@MacBook ~ % which -a bash
/opt/homebrew/bin/bash
/bin/bash
user@MacBook ~ % bash --version | head -n 1
GNU bash, version 5.3.x(1)-release (aarch64-apple-darwin)
```

لاحظ إن `which -a` لقى الاتنين، والنسخة الجديدة جت الأول لأن `/opt/homebrew/bin` قبل `/bin` في الـ PATH (فاكر ↩ 1.4 ص 12؟). وعشان كده أي Script أوله `#!/usr/bin/env bash` هيشتغل بالنسخة الجديدة، أما اللي أوله `#!/bin/bash` هيشتغل بالقديمة. الفرق ده متشرح في ↩ 12.1 ص 143.

3.5 مقارنة الكتابة (Syntax)

دلوقتي بعد ما عرفت العيلة، تعالى نشوف "لهجة" كل واحد. نفس الفكرة بتتكتب إزاي في كل Shell؟ هتلاحظ إن bash و zsh و sh تقريباً نفس الكلام، وإن fish وعيلة Windows مختلفين:

الفكرة	sh / zsh / bash	fish
متغير	`name=Ali` (من غير مسافات)	`set name Ali`
استخدامه	`$name`	`$name`
Environment Variable	`export NAME=v`	`set -gx NAME v`
ناتج أمر	`$(date)`	`$(date)` أو `(date)`
شرط	`if [ -f a ]; then ...; fi`	`if test -f a; ...; end`
Loop	`for f in *; do ...; done`	`for f in *; ...; end`
و / أو	`&&` `||`	`and` `or` أو `&&` `||`
ملف الإعدادات	`~/.bashrc` / `~/.zshrc`	`~/.config/fish/config.fish`
امتداد الـ Script	`.sh`	`.fish`

جدول 3.2 نفس الفكرة في كل Shell: عيلة Unix

الفكرة	PowerShell	cmd
متغير	`$name = "Ali"`	`set name=Ali`
استخدامه	`$name`	`%name%`
Environment Variable	`$env:NAME = "v"`	`set NAME=v`
ناتج أمر	`$(Get-Date)`	معقد (`for /f`)
شرط	`if (Test-Path a) { ... }`	`if exist a ( ... )`
Loop	`foreach ($f in ls) { ... }`	`for %f in (*) do ...`
و / أو	`&&` `||` (من 7.0)	`&&` `||`
ملف الإعدادات	المسار في `$PROFILE`	مفيش (Registry)
امتداد الـ Script	`.ps1`	`.bat` / `.cmd`

جدول 3.3 نفس الفكرة في كل Shell: عيلة Windows

فروق صغيرة بين bash و zsh بتفرق

- **الـ Arrays:** في bash أول عنصر رقمه 0 (`${arr[0]}`)، وفي zsh رقمه 1 افتراضياً.
- **الـ Word splitting:** في bash، `$var` من غير علامات تنصيص بيتقسم على المسافات. في zsh لأ. عشان كده Script شغال في zsh ممكن يبوظ في bash لو فيه أسماء ملفات بمسافات (↩ 7.5 ص 76).
- **الـ Glob اللي مالوش نتيجة:** `ls *.xyz` في bash بيعدّي النص `*.xyz` زي ما هو، وفي zsh بيدي Error اسمه `no matches found`.

3.6 PowerShell: عالم تاني (Objects مش نصوص)

إنت كمبرمج JavaScript هتفهم الجزء ده بسرعة. تخيل الفرق بين إنك تبعت لحد رسالة WhatsApp فيها "الاسم أحمد، السن 25"، وبين إنك تبعتله Object `{name, age}` جاهز. في الأولى هو لازم يقرا ويقص الكلام علشان يطلع منه المعلومة، وفي التانية المعلومة جاهزة. ده بالظبط الفرق بين bash و PowerShell.

في bash لما تعمل `ps aux | grep chrome` إنت بتعدّي **نص**، والأمر التاني لازم "يقص" النص عشان يفهمه. في PowerShell كل أمر (اسمه **Cmdlet**) بيطلع **Objects** ليها خصائص، فتقدر تفلتر وترتب بالاسم:

تشريح الأمر

```
Get-Process | Where-Object CPU -gt 100 | Sort-Object CPU -Descending | Select-Object -First 5
```

1. `Get-Process` هات كل الـ Processes كـ Objects. الأسماء دايماً **Verb-Noun** (فعل-اسم).
2. `|` الـ Pipe، بس بيعدّي Objects كاملة مش سطور نص.
3. `Where-Object CPU -gt 100` فلتر: خلّي بس اللي خاصية CPU بتاعته أكبر من 100. والـ `-gt` = greater than (بدل `>` اللي معناها Redirection).
4. `Sort-Object CPU -Descending` رتّب حسب الـ CPU من الكبير للصغير.
5. `Select-Object -First 5` خد أول 5 بس (زي `head -n 5`).

الـ Aliases المضللة

في PowerShell على Windows، `ls` و `cat` و `rm` شغالين، بس هما Aliases لـ `Get-ChildItem` و `Get-Content` و `Remove-Item`، والـ Flags بتاعتهم مختلفة: `rm -rf` مش هتشتغل، المقابل `Remove-Item -Recurse -Force`.

3.7 مقارنة المميزات

طب في الآخر، مين الأحسن؟ الإجابة الصريحة: مفيش "أحسن" على طول الخط. كل واحد شاطر في حاجة. الجدولين دول بيلخصوا الكلام اللي فات:

الميزة	sh/dash	bash	zsh	fish	PowerShell
متوافق مع POSIX	✓	✓ (تقريباً)	✓ (تقريباً)	✗	✗
اقتراحات أوتوماتيك وانت بتكتب	✗	✗ (بـ أدوات خارجية)	بـ Plugin	✓ مدمجة	✓ (PSReadLine)
ألوان للأوامر (Syntax highlighting)	✗	✗	بـ Plugin	✓ مدمجة	✓
Tab completion ذكي	بسيط	كويس	ممتاز	ممتاز	ممتاز
Scripts تشتغل في كل حتة	الأفضل	ممتاز (Linux)	كويس	ضعيف	Windows أساساً
Objects في الـ Pipe	✗	✗	✗	✗	✓

جدول 3.4 المميزات وجهاً لوجه

الموقف	الاختيار
Script هيشتغل على سيرفرات و Docker images صغيرة	sh (POSIX)
Script عادي هيشتغل على الماك والسيرفر	bash: هو المعيار الفعلي على السيرفرات، وده اللي الكتاب بيكتب بيه الـ Scripts
استخدامك اليومي على الماك	zsh (موجود جاهز، وده اللي الكتاب ماشي بيه) أو fish لو عايز حاجة جاهزة من غير إعدادات
إدارة Windows و Active Directory و Azure	PowerShell
Windows وعايز أدوات Linux	WSL وجواه bash

جدول 3.5 إمتى تستخدم أنهي واحد؟

Best Practice

القاعدة الذهبية: **استخدم اللي يريحك للشغل التفاعلي (zsh أو fish)، واكتب الـ Scripts بـ bash أو sh** وحط الـ Shebang في أول سطر (↩ 12.1 ص 143). كده الـ Script بيشتغل بالـ Shell الصح مهما كان الـ Shell اللي إنت قاعد فيه.

3.8 إنت شغال على أنهي Shell؟ وإزاي تغيره؟

ممكن حد يكون غيّر الـ Shell على جهازك قبل كده، أو إنت نفسك كتبت `bash` وانت مش واخد بالك ونسيت. خلينا نتأكد إحنا فين بالظبط:

تشريح الأمر

```
1  2  3     4
ps -p $$ -o comm=
```

1. `ps` اعرض معلومات عن Process.
2. `-p` Process ID: عايز Process معينة برقمها.
3. `$$` متغير خاص فيه رقم الـ Shell الحالي (↩ 7.8 ص 80).
4. `-o comm=` اعرض عمود اسم الأمر بس، و `=` الفاضية بعده معناها "من غير عنوان للعمود".

Terminal

```
user@MacBook ~ % ps -p $$ -o comm=
-zsh
user@MacBook ~ % echo $SHELL
/bin/zsh
user@MacBook ~ % cat /etc/shells
.List of acceptable shells for chpass(1) #
/bin/bash
/bin/csh
/bin/dash
/bin/ksh
/bin/sh
/bin/tcsh
/bin/zsh
```

$SHELL مش الـ Shell الحالي

المتغير `$SHELL` فيه الـ **Login Shell** الافتراضي بتاعك (اللي متسجل في حسابك؛ على الماك بتشوفه بـ `dscl . -read ~ UserShell`، وعلى Linux في `/etc/passwd`)، مش اللي شغال دلوقتي. لو كتبت `bash` جوه zsh، الـ `$SHELL` هيفضل zsh. الأدق هو `ps -p $$` و `echo $0` بتديك فكرة سريعة برضه.

جرّب Shell مؤقتاً

اكتب اسمه بس: `zsh` أو `fish`، وهيفتح جوه الـ Shell الحالي. اخرج منه بـ `exit` أو `Ctrl+D`. لو مش متسطب: `brew install fish` (↩ 11.1 ص 129). والـ bash اللي جاي مع الماك تقدر تفتحه بـ `bash`، وهتلاحظ إن الـ Prompt بقى شكله `bash-3.2$`.

غيّر الـ Default بشكل دايم

تشريح الأمر

```
chsh -s $(command -v bash)
```

1. `chsh` — CHange SHell: غيّر الـ Login Shell بتاعك.
2. `-s` — Shell: الكلمة الجاية هي مسار الـ Shell الجديد.
3. `$( )` — Command Substitution: نفّذ اللي جوه وحط ناتجه مكانه (↩ 7.6 ص 77).
4. `command -v bash` — بيطبع المسار الكامل لأول bash في الـ PATH، زي `/opt/homebrew/bin/bash`. والمسار لازم يكون مكتوب في `/etc/shells`، وإلا chsh هيرفض بـ `non-standard shell`.

بس الـ bash بتاع Homebrew مش مكتوب في `/etc/shells`، فلازم تضيفه الأول:

تشريح الأمر

```
echo /opt/homebrew/bin/bash | sudo tee -a /etc/shells
```

1. `echo /opt/homebrew/bin/bash` — اطبع المسار ده.
2. `|` — الـ Pipe: ابعت الناتج للأمر اللي بعده (↩ 7.3 ص 74).
3. `sudo` — نفّذ اللي بعدي بصلاحيات الأدمن، لأن `/etc/shells` ملف نظام (↩ 6.6 ص 66).
4. `tee` — اكتب اللي جايلك في ملف (↩ 8.8 ص 90). استخدمناه بدل `>>` لأن `sudo echo ... >> file` مش هيشتغل: الـ Redirection بيعمله الـ Shell بتاعك العادي قبل الـ sudo.
5. `-a` — Append: ضيف في آخر الملف، متمسحش اللي فيه.

هيطلب الباسورد، والتغيير بيظهر في أي Terminal **جديد**. ولو حابب ترجع لـ zsh: `chsh -s /bin/zsh`. وتقدر كمان تغيّره من **System Settings ← Groups & Users**، كليك يمين (أو Control+Click) على اسمك ← **Advanced Options**. نصيحتي: سيب zsh هو الافتراضي، واستخدم bash الجديد للـ Scripts بس. وعلى Windows: نوع الـ Shell بتختاره من إعدادات Windows Terminal، ورقم إصدار PowerShell بيظهر بـ `$PSVersionTable`.

3.9 مشاكل شائعة

مشكلة 1 invalid option :-A :declare

نزّلت Script من GitHub وشغلته، وطلعلك `declare: -A: invalid option` أو `mapfile: command not found`. السبب: الـ Script محتاج bash 4 أو أحدث، وإنت شغلته بـ bash 3.2 اللي جاي مع الماك.

الحل: `brew install bash`، وخلي أول سطر في الـ Script `#!/usr/bin/env bash` بدل `#!/bin/bash` علشان ياخد النسخة الجديدة من الـ PATH. اتأكد بـ `bash --version`.

مشكلة 2 [[: not found على السيرفر بس

الـ Script شغال على الماك بـ `sh script.sh`، ولما رفعته على سيرفر Ubuntu طلع `[[: not found` أو `Syntax error: "(" unexpected`. السبب: `sh` على الماك هو bash متنكّر، وعلى Ubuntu هو dash اللي مبيفهمش حاجات bash.

الحل: حط `#!/usr/bin/env bash` في أول سطر، وشغله بـ `./script.sh` أو `bash script.sh`، أو اكتب الـ Script بـ POSIX بس (`[ ]` بدل `[[ ]]`) وجرّبه على الماك بـ `dash script.sh`.

مشكلة 3 نسخت أمر bash في fish ومشتغلش

مثلاً `export PATH=$PATH:~/bin` أو `VAR=1 cmd` أو `for` بـ `do/done`.

الحل: استخدم كتابة fish (`fish_add_path ~/bin`، `for x in ...; end`)، أو شغّل الأمر جوه bash مؤقتاً: `bash -c '...'`.

ملخص الفصل

- sh هو الأساس، و POSIX هو المعيار المشترك. على الماك الـ sh هو bash في وضع POSIX، وعلى Debian/Ubuntu هو dash.
- الماك جاي بـ zsh 5.9 كـ Default، و bash 3.2 قديم من 2007 بسبب الرخصة؛ سطّب bash حديث بـ `brew install bash`.
- bash هو المعيار الفعلي على السيرفرات، zsh أريح تفاعلياً (Default الماك)، fish أسهل بس مش POSIX.
- PowerShell بيعدّي Objects مش نصوص، وأسماء أوامره Verb-Noun.
- اعرف الـ Shell الحالي بـ `ps -p $$`، وغيّر الـ Default بـ `chsh -s`.
- اشتغل باللي يريحك، واكتب الـ Scripts بـ bash/sh مع Shebang.

صح ولا غلط؟

1. Script مكتوب بـ bash هيشتغل على fish من غير تعديل. صح / غلط
2. `echo $SHELL` دايماً بيقولك الـ Shell اللي شغال دلوقتي. صح / غلط
3. في PowerShell الـ Pipe بيعدّي Objects. صح / غلط

أسئلة مراجعة

1. يعني إيه POSIX، وليه ممكن تكتب Script بـ sh بدل bash؟
2. اذكر فرقين بين bash و zsh ممكن يبوظوا Script.
3. اكتب تعريف متغير `NAME=Ali` كـ Environment Variable في bash و fish و PowerShell.
4. إيه الخطوات عشان تخلي bash بتاع Homebrew هو الـ Default بتاعك؟ وليه لازم تعدّل `/etc/shells` ؟
5. ليه Apple وقفت عند bash 3.2؟ واذكر حاجتين مش هتشتغل فيه.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعرف الـ Shell الحالي بـ 3 طرق مختلفة وقارن النتايج.
2. متوسط سطّب fish بـ `brew install fish` وجرّبه 10 دقايق: اكتب نص أمر قديم ولاحظ الاقتراح الرمادي، ودوس السهم اليمين لقبوله.
3. صعب اكتب Loop بتطبع الأرقام من 1 لـ 3 في bash وفي fish وفي PowerShell (`brew install --cask powershell` لو حابب).
4. صعب اكتب سطر فيه `declare -A` وجرّبه في `/bin/bash` وفي bash بتاع Homebrew، وقارن.

04

المرحلة 2 · الملفات والصلاحيات

الفصل الرابع

الـ Filesystem على الماك: كل حاجة ليها مكان

إنت متعود تشوف ملفاتك من الـ Finder: Desktop و Downloads و Applications، وكله شكله مرتب ولطيف. بس أول ما تكتب ls / في الـ Terminal هتلاقي أسامي غريبة زي private و usr و Volumes و opt، وتقول "هو ده نفس الجهاز؟". أيوه هو، بس إنت كنت شايف نص الحقيقة. الفصل ده بيديك الخريطة الكاملة للماك: كل مجلد فيه إيه، وفين Homebrew بيسطّب حاجاته، وإزاي تكتب أي مسار صح، ويعني إيه inode، والفرق بين الـ Hard link والـ Soft link.

بعد الفصل ده هتكون عارف

- فكرة الشجرة الواحدة و "كل حاجة ملف"
- خريطة الماك: Users/ و Applications/ و Library/ و Library/~ و System/ و Volumes/ و opt/homebrew/ و private/
- ليه etc/ و tmp/ على الماك Links لـ private/
- المسار المطلق والنسبي، و ~ و . و ..
- الملفات المخفية، وإن الماك مش حساس لحالة الحروف افتراضياً
- يعني إيه inode، والفرق بين Hard link و Soft link و Alias بتاع الـ Finder
- الديسكات على الماك: df و du و diskutil، ومقارنة بـ FHS بتاع Linux

4.1 المشكلة: الـ Finder مش بيقولك كل حاجة

افتح الـ Finder ودوس على **Macintosh HD**. هتلاقي 4 مجلدات بس تقريباً: `Applications` و `Library` و `System` و `Users`. شكلها نضيف وبسيط، صح؟ طب تعالى نسأل الـ Terminal نفس السؤال:

```
Terminal

user@MacBook ~ % ls /
Applications    System  Volumes cores   etc     opt     sbin    usr
Library Users   bin     dev     home    private tmp     var
```

إيه ده كله؟ `bin` و `etc` و `private` و `usr`؟ دول كانوا فين؟ الحقيقة إن الماك تحت الواجهة الحلوة دي نظام **Unix** كامل (فاكر حكاية BSD في ↩ 2.6 ص 23؟). و Apple عملت حاجة ذكية: سابت مجلدات Unix في مكانها علشان كل الأدوات تشتغل، بس خبّتها من الـ Finder علشان المستخدم العادي ميتلخبطش ولا يمسح حاجة بالغلط.

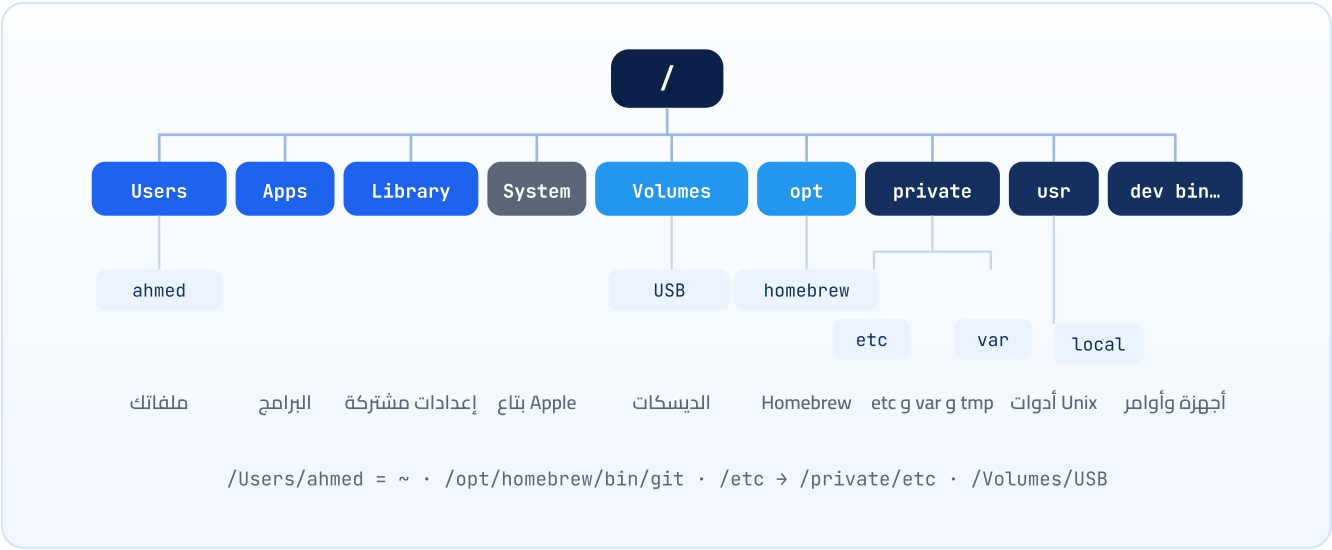
شوف الحاجات المخفية في الـ Finder

في أي شباك Finder دوس `Cmd+Shift+.` (نقطة)، وهتلاقي المجلدات المخفية ظهرت باهتة. دوس نفس الزرار تاني علشان تخفيها. ولو عايز تروح لأي مسار بالكتابة: `Cmd+Shift+G` واكتب `/usr/local` مثلاً.

وخلينا نتفق على فكرتين أساسيتين هتمشي معاك طول الكتاب. الأولى: فيه **شجرة واحدة بس** بتبدأ من الجذر `/` (اسمه Root). مفيش `C:` و `D:` زي Windows. أي ديسك أو فلاشة بتوصلها بتتركّب (Mount) كمجلد جوه الشجرة دي، وعلى الماك المكان ده هو `/Volumes`. والتانية قاعدة مشهورة في عالم Unix: **"كل حاجة ملف"**. الديسك نفسه ملف في `/dev` (زي `/dev/disk3`)، والـ Terminal اللي بتكتب فيه ملف (فاكر `/dev/ttys000`؟)، وحتى "الزبالة" اللي بترمي فيها الناتج اللي مش عايزه ملف اسمه `/dev/null`.

4.2 خريطة المجلدات على الماك

تخيل الماك عمارة. فيه دور لـ Apple بس محدش يلمسه، وفيه شقة لكل ساكن، وفيه مخزن مشترك، وفيه جراج للعربيات اللي بتيجي وتمشي. خلينا نلف على العمارة دي دور دور:



شكل 4.1 شجرة الماك: واحدة بتبدأ من /، وكل مجلد ليه دور.

المجلد	فيه إيه؟	مثال
`/`	الجذر، أبو كل حاجة	
`/Users`	مجلد لكل مستخدم ("شقته")، فيه Desktop و Documents و Downloads وإعداداته	`/Users/ahmed` (و `~` اختصاره)
`/Applications`	البرامج اللي بتشوفها في Launchpad. كل برنامج `.app` هو في الحقيقة **مجلد**	`/Applications/Safari.app`
`~/Library`	إعدادات وبيانات البرامج **بتاعتك إنت**: Application Support و Caches و Logs و Preferences و LaunchAgents. مخفي في الـ Finder	`~/Library/Application Support/Code`
`/Library`	نفس الفكرة بس لكل المستخدمين على الجهاز (محتاج أدمن)	`/Library/LaunchDaemons`

المجلد	فيه إيه؟	مثال
`/System`	النظام نفسه بتاع Apple. **للقراية بس** ومحمي، حتى root ميقدرش يعدّل فيه	`/System/Applications`
`/Volumes`	"الجراج": كل ديسك أو فلاشة أو ملف `.dmg` بتفتحه بيتركّب هنا	`/Volumes/USB`
`/opt/homebrew`	كل حاجة بتسطبها بـ Homebrew على أجهزة **Apple Silicon** (M1 وما بعدها)	`/opt/homebrew/bin/node`
`/usr/local`	برامج إنت سطّبتها بنفسك، ومكان Homebrew على أجهزة **Intel** القديمة	`/usr/local/bin`
`/usr/bin` و `/bin` و `/sbin`	أدوات Unix اللي جاية مع الماك (sbin = للأدمن). محمية زي `/System`	`/bin/zsh` ، `/usr/bin/grep`
`/private/etc`	ملفات **الإعدادات** بتاعة Unix (نصوص عادية). و `/etc` مجرد Link ليه	`/etc/hosts` ، `/etc/shells` ، `/etc/ssh/`
`/private/var`	بيانات **بتتغير** (Variable): Logs و Caches. و `/var` Link ليه	`/var/log` ، `/var/folders`
`/private/tmp`	ملفات مؤقتة لأي حد، بتتمسح مع الـ Restart. و `/tmp` Link ليه	`/tmp/test.txt`
`/dev`	الأجهزة كملفات	`/dev/disk3` ، `/dev/null`

جدول 4.1 أهم المجلدات على الماك وفيها إيه

طب ليه `/etc` و `/var` و `/tmp` جوه حاجة اسمها `private`؟ ده موروث من نظام قديم اسمه NeXTSTEP (اللي الماك اتولد منه)، وكانت الفكرة إن الحاجات الخاصة بكل جهاز تبقى في مكان واحد. ولما تبص بعينك هتلاقيها Links:

Terminal

```
user@MacBook ~ % ls -ld /etc /tmp /var
lrwxr-xr-x@ 1 root  wheel  11 Sep 14 09:00 /etc -> private/etc
lrwxr-xr-x@ 1 root  wheel  11 Sep 14 09:00 /tmp -> private/tmp
lrwxr-xr-x@ 1 root  wheel  11 Sep 14 09:00 /var -> private/var
```

تشريح الأمر

```
ls -ld /etc /tmp /var
```

1. `ls` اعرض (↩ 5.1 ص 48).
2. `-ld` اتنين مدموجين: `-l` (Long: تفاصيل) و `-d` (Directory: اعرض المجلد نفسه مش اللي جواه).
3. `/etc /tmp /var` تلات Arguments: اعرض التلاتة. والـ `l` في أول السطر والسهم `->` معناهم إنهم Soft links (هنفهمها في آخر الفصل)، والـ `@` بعد الصلاحيات معناها إن عليهم Extended attributes (↩ 6.7 ص 67).

البرنامج app. ده مجلد!

جرّب `ls /Applications/Safari.app/Contents`. هتلاقي مجلدات زي `MacOS` و `Resources` وملف `Info.plist`. الـ Finder بيعرض المجلد ده كأنه ملف واحد علشان يبقى سهل، والاسم الرسمي ليه **Bundle**. والبرنامج الحقيقي اللي بيشتغل جوه `Contents/MacOS`. وتقدر تفتح أي برنامج من الـ Terminal بـ `open -a Safari` (↩ 14.3 ص 171).

لو حابب تتعمق: مفيش /proc على الماك

تقدر تعدّيه في أول قراية

على Linux فيه مجلد سحري اسمه `/proc` فيه ملفات **وهمية** بيولّدها الـ Kernel لحظياً: الذاكرة والـ CPU وكل Process برقمها. الماك **معندوش** `/proc`. بدل كده بتسأل الـ Kernel مباشرة بأمر اسمه `sysctl`:

تشريح الأمر

```
sysctl -n hw.memsize machdep.cpu.brand_string
```

1. `sysctl` SYStem ConTroL: اقرا (أو غيّر) إعدادات ومعلومات الـ Kernel.
2. `-n` اطبع القيمة بس من غير اسم المفتاح قبلها.
3. `hw.memsize` مفتاح: حجم الرامات بالـ Bytes (يعني `17179869184` = 16 GB).
4. `machdep.cpu.brand_string` مفتاح تاني: اسم الـ CPU، زي `Apple M3`. والمفاتيح مكتوبة بنقط زي العناوين: `hw` = Hardware.

وفيه كمان `vm_stat` لتفاصيل الذاكرة، و `sysctl -a | grep cpu` لو عايز تتفرج على كل المفاتيح اللي فيها cpu.

على Linux: خريطة FHS

السيرفرات ماشية على معيار اسمه **FHS** (Filesystem Hierarchy Standard). أغلب الأسماء هتعرفها: `/etc` و `/var` و `/tmp` و `/usr` و `/dev` و `private` (بس من غير `/`). والمختلف: `/home/ahmed` بدل `/Users/ahmed`، و `/root` هو home بتاع root، و `/proc` و `/sys` للمعلومات اللحظية (`head -n 3 /proc/meminfo`)، و `/boot` لـ Kernel وملفات الإقلاع، و `/mnt` و `/media` بدل `/Volumes`، و `/opt` لبرامج الطرف التالت، و `/var/log/syslog` لـ Logs، و `/run` و `/srv`. والماك نفسه فيه مجلد `/home` بس فاضي ومش مستخدم.

4.3 المسارات: المطلق والنسبي

تخيل إنك بتوصف لصاحبك مكان بيتك. ممكن تقوله العنوان الكامل: "القاهرة، مدينة نصر، شارع عباس العقاد، عمارة 5". العنوان ده هيوصله من أي حتة في الدنيا. وممكن لو هو واقف قدام المسجد اللي جنبك تقوله "تاني شارع يمين". ده أقصر، بس مش هيفهم منه حاجة لو كان واقف في إسكندرية. المسارات (Paths) في الـ Terminal نفس الحكاية بالظبط، وفيه نوعين:

مسار مطلق (Absolute)

بيبدأ بـ `/` من الجذر، وبيوصل لنفس المكان مهما إنت فين: `/Users/ahmed/projects/web` . زي العنوان الكامل بالمحافظة والشارع.

مسار نسبي (Relative)

مبيبدأش بـ `/` ، وبيتحسب من المجلد اللي إنت واقف فيه دلوقتي: `projects/web` أو `../docs` . زي "تاني شارع يمين من هنا".

الرمز	المعنى	مثال
`/`	في الأول: الجذر. في النص: فاصل بين المجلدات	`/etc/ssh`
`.`	المجلد الحالي	`./run.sh` ، `cp a.txt .` ، `open .`
`..`	المجلد اللي فوق (الأب)	`cd ..` ، `../../etc`
`~`	الـ home بتاعك (الـ Shell بيحولها لـ `/Users/ahmed`)	`~/Downloads`
`~sara`	الـ home بتاع مستخدم تاني	`ls ~sara`
`-` **(مع cd بس)**	المجلد اللي كنت فيه قبل كده	`cd -`

جدول 4.2 الرموز الخاصة في المسارات

تشريح الأمر

```
cd ../../etc
```

1. `cd` Change Directory (Builtin، ↩ 1.3 ص 11).
2. `..` اطلع مجلد لفوق.
3. `/` فاصل بين خطوة والتانية.
4. `../` اطلع مجلد كمان لفوق. (لو إنت في `/Users/ahmed` كده وصلت `/`).
5. `etc` وبعدين ادخل `etc` . النتيجة: `/etc` ، اللي هو في الحقيقة `/private/etc` . ولو كتبت `pwd -P` هيقولك المسار الحقيقي بعد ما يفك الـ Links.

متكتبش المسار، اسحبه!

أحلى حركة على الماك: اسحب أي ملف أو مجلد من الـ Finder وارميه جوه شباك الـ Terminal، وهيتكتب مساره الكامل لوحده، وكمان بيحط `\` قبل المسافات. والعكس: `open .` بتفتح المجلد اللي إنت واقف فيه في الـ Finder.

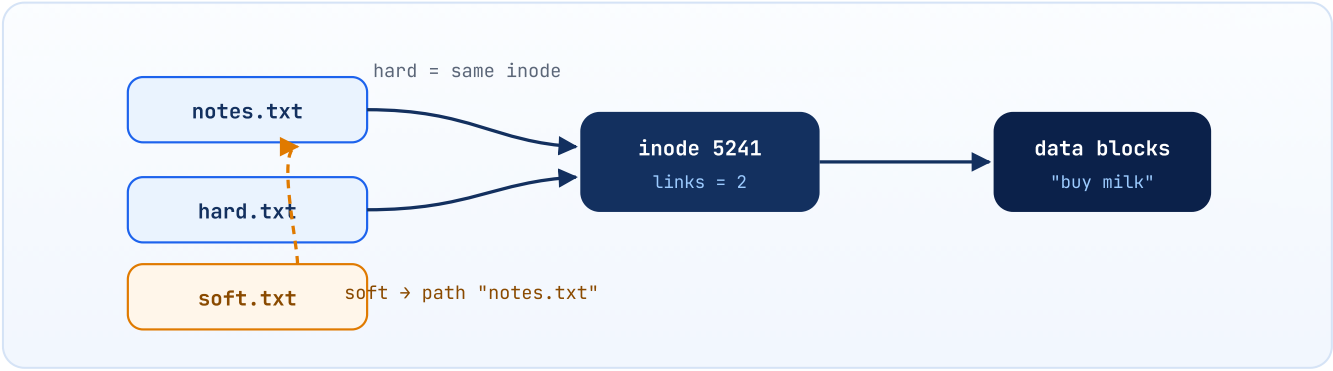
⚠ 3 حاجات بتوقع الناس

- **الحروف مش حساسة على الماك، بس حساسة على السيرفر:** الـ Filesystem بتاع الماك (اسمه **APFS**) افتراضياً *Case-insensitive*: يعني `Report.txt` و `report.txt` نفس الملف، رغم إنه بيحفظ الاسم بالشكل اللي كتبته. على Linux دول ملفين مختلفين. والفرق ده بيعمل مشاكل حقيقية: تعمل `import './Header.js'` والملف اسمه `header.js`، يشتغل عندك ويبوظ على السيرفر. اعرف نوع الديسك بتاعك بـ `diskutil info / | grep -i personality`.
- **المسافات:** ملف اسمه `my file.txt` لازم يتكتب `"my file.txt"` أو `my\ file.txt`، وإلا الـ Shell هيعتبره ملفين (↩ 7.5 ص 76). وعلى الماك هتقابل ده كتير، زي `~/Library/Application Support`.
- **الامتداد مالوش لازمة للنظام:** `.txt` و `.sh` مجرد جزء من الاسم. عشان تعرف الملف نوعه إيه بجد استخدم `file` (↩ 5.6 ص 55).

الملفات المخفية: أي ملف أو مجلد اسمه يبدأ بنقطة (`.zshrc`، `.ssh/`) بيبقى مخفي، ومبيظهرش غير بـ `ls -a`. ده مجرد عُرف، مش حماية. وعشان كده ملفات الإعدادات اسمها "Dotfiles". والماك عنده كمان طريقة تانية للإخفاء بعلامة اسمها `hidden` (ده اللي مخبّي `~/Library`)، وتشيلها بـ `chflags nohidden ~/Library`. وهتلاقي في كل مجلد فتحته بالـ Finder ملف مخفي اسمه `.DS_Store`: ده الـ Finder بيحفظ فيه شكل الأيقونات وترتيبها، ودايماً حطه في `.gitignore`.

4.4 جوه الملف: الـ inode

هنا هنعمل حاجة شبه ما تفك الموبايل وتشوف جواه. لما بتشوف ملف اسمه `notes.txt` في الـ Finder، إنت متخيل إن الاسم ده هو الملف. بس الحقيقة إن الاسم مجرد "يافطة". عشان تفهم الـ Links لازم تعرف إن اسم الملف **مش** هو الملف. الملف الحقيقي عبارة عن **inode**: رقم جوه الـ Filesystem مربوط بيه البيانات والصلاحيات والمالك والتواريخ والحجم. والاسم ده مجرد "يافطة" في المجلد بتشاور على الـ inode، زي اسم في الـ Contacts بيشاور على رقم تليفون. وينفع يبقى عندك اسمين بيشاوروا على نفس الرقم.



شكل 4.2 الـ Hard link اسم تاني لنفس الـ inode. الـ Soft link ملف صغير فيه "مسار" لاسم تاني.

تشريح الأمر

```
ls -li
```

1. `ls` اعرض محتويات المجلد (↩ 5.1 ص 48).
2. `-li` اتنين مدموجين: `-l` (Long: تفاصيل) و `-i` (Inode: اعرض رقم الـ inode في أول عمود). لاحظ: `-i` هنا مش interactive، ده فخ تاني من فخاخ ↩ 2.5 ص 21.

4.5 الـ Hard link والـ Soft link

طب الكلام ده هيفيدني في إيه؟ هتلاقيه في كل حتة من غير ما تاخد بالك. مثلاً لما بتسطّب `git` بـ Homebrew، البرنامج الحقيقي بيتحط في مجلد عميق جداً اسمه `Cellar`، و Homebrew بيعمل "يافطة" صغيرة في `/opt/homebrew/bin` بتشاور عليه، علشان الـ PATH يلاقيه. اتفرج:

Terminal

```
user@MacBook ~ % ls -l /opt/homebrew/bin/git
lrwxr-xr-x  1 ahmed  admin  29 Oct  2 11:20 /opt/homebrew/bin/git ->
../Cellar/git/2.51.0/bin/git
```

الحاجة دي اسمها **Soft link**. وتعالى نعمل الاتنين بإيدنا ونشوف الفرق:

Terminal

```
user@MacBook ~ % echo "buy milk" > notes.txt
user@MacBook ~ % ln notes.txt hard.txt
user@MacBook ~ % ln -s notes.txt soft.txt
user@MacBook ~ % ls -li
12893301 -rw-r--r--  2 ahmed  staff  9 Oct 22 10:01 hard.txt
12893301 -rw-r--r--  2 ahmed  staff  9 Oct 22 10:01 notes.txt
12893355 lrwxr-xr-x  1 ahmed  staff  9 Oct 22 10:01 soft.txt -> notes.txt
```

- `notes.txt` و `hard.txt` ليهم **نفس رقم الـ inode** (12893301)، والعمود التالت (2) هو عدد الأسماء اللي بتشاور عليه.
- `soft.txt` ليه inode مختلف، وأول حرف `l` (Link)، وبيشاور بـ `->` على الاسم.
- و `staff` هو الجروب الافتراضي لأي مستخدم على الماك (على Linux غالباً الجروب باسمك إنت). هنفهم الجروبات في ↩ 6.5 ص 63.

تشريح الأمر

```
ln -s ~/tools/node-v22.11 ~/tools/node-current
```

1. `ln` — LiNk: اعمل Link. من غير Options بيعمل Hard link.
2. `-s` — Symbolic: اعمل Soft link بدل Hard. ودايماً الترتيب **زي cp**: الأصل الأول، وبعدين الاسم الجديد.
3. `~/tools/node-v22.11` — الهدف (Target): الحاجة الحقيقية.
4. `~/tools/node-current` — اسم الـ Link الجديد. ولو عايز تحدّثه بعدين لنسخة جديدة: `ln -sfn` (`-f` = اكتب فوق القديم، `-n` = متدخلش جوه الـ Link لو بيشاور على مجلد). والـ `ln` دي موجودة في بتاع الماك ومعناها نفس `-h`.

	Hard link	link (Symbolic) Soft	Finder Alias
بيتعمل إزاي	`ln target name`	`ln -s target name`	كليك يمين ← Make Alias
هو إيه	اسم تاني لنفس الـ inode	ملف صغير جواه مسار	ملف خاص بالماك بيفتكر الهدف حتى لو اتنقل
لو مسحت الأصل	البيانات فاضلة (لسه فيه اسم)	الـ Link يبوظ (Dangling)	يبوظ
على مجلد؟	لأ	أيوه	أيوه
بين ديسكين مختلفين؟	لأ (نفس الـ Filesystem بس)	أيوه	أيوه
الـ Terminal بيفهمه؟	أيوه	أيوه	**لأ:** `cd` مش هيدخل جواه
استخدام شائع	Time Machine و Backups، نادر يدوياً	Homebrew، "current" لنسخة برنامج، Dotfiles	اختصارات في الـ Finder بس

جدول 4.3 Hard مقابل Soft مقابل Alias بتاع الـ Finder

حاجة مشتركة

الـ Soft links بتظهر في أوامر كتير بخيارات مشتركة: `cp -L` و `find -L` و `ls -L` بيـ"تبعوا" الـ Links ويتعاملوا مع الهدف. ولو عايز المسار الحقيقي اللي الـ Link بيوصله: `readlink -f soft.txt` أو `realpath soft.txt`. ومن غير `-f`، `readlink` بيطبع اللي مكتوب جوه الـ Link بس.

4.6 الديسكات والمساحة

أكيد جالك قبل كده إشعار "Your disk is almost full"، وفتحت **System Settings ← General ← Storage** ولقيت لون رمادي كبير مكتوب عليه "System Data" ومحدش عارف فيه إيه. الـ Terminal هنا بيديك إجابات أوضح بكتير. خلينا نبدأ بالسؤال الأول: فاضل كام؟

تشريح الأمر

```
df -h ~
```

1. `df` — Disk Free: المساحة المستخدمة والفاضية لكل Filesystem متركّب.
2. `-h` — Human-readable: بـ Gi و Mi بدل أرقام الـ Blocks.
3. `~` — اختياري: اعرض بس الديسك اللي عليه المسار ده (الـ home). من غيره بيعرض كله.

Terminal

```
user@MacBook ~ % df -h ~
Filesystem      Size    Used   Avail Capacity iused ifree %iused  Mounted on
/dev/disk3s5   460Gi   201Gi   244Gi    46%  2.1M  2.5G    0%   /System/Volumes/Data
```

العمود الأخير **Mounted on** هو المجلد اللي الديسك ده متركّب عليه في الشجرة. ولاحظ حاجة غريبة: ملفاتك مش على `/`، دي على `/System/Volumes/Data`! والعواميد `iused` و `ifree` هي عدد الـ inodes المستخدمة والفاضية، اللي لسه فاهمينها فوق.

لو حابب تتعمق: الديسك متقسم لنصين

تقدر تعدّيه في أول قراية

من macOS Catalina، Apple قسمت الديسك لـ **Volumes** كذا جوه نفس الـ Container: Volume للنظام اسمه **Signed System Volume**، ده للقراية بس ومختوم بتوقيع Apple، فلو اتغير فيه حرف الجهاز مش هيقوم. و Volume تاني للبيانات (`Data`) فيه ملفاتك وبرامجك. والاتنين بيتعرضوا قدامك كأنهم شجرة واحدة بفضل حاجة اسمها **firmlinks**، علشان كده `/Users` شكله جوه `/` وهو في الحقيقة على الـ Data volume. شوف بنفسك بـ `diskutil list`، وبيان أي Volume بـ `diskutil info /`.

وعشان تعرف مين واكل المساحة، استخدم `du` (Disk Usage):

تشريح الأمر

```
du -h -d 1 ~ | sort -h
```

1. `du` — Disk Usage: حجم الملفات والمجلدات.
2. `-h` — Human-readable زي اللي في df.
3. `-d 1` — Depth: ادخل مستوى واحد بس، يعني اديني رقم لكل مجلد جوه الـ home. (على Linux الشكل الطويل `--max-depth=1`، و `-d` شغالة برضه).
4. `~` — ابدأ من الـ home.
5. `| sort -h` — رتّب الناتج من الصغير للكبير، و `-h` هنا معناها "افهم الأحجام اللي فيها K و M و G" (↩ 8.6 ص 88). فأكبر مجلد هيبقى آخر سطر.

ولو عايز رقم واحد بس لمجلد: `du -sh ~/Library/Caches` (`-s` = Summary). هتتفاجئ بحجم الـ Caches و `node_modules` !

على Linux (السيرفرات)

على Linux الـ `df -h` بيطلع عمود `Use%` بدل `Capacity` ومن غير عواميد الـ inodes (بتشوفها بـ `df -i`)، والديسك اسمه حاجة زي `/dev/nvme0n1p2`، وبتشوف الديسكات وتقسيماتها بـ `lsblk` بدل `diskutil list`.

4.7 مشاكل شائعة

مشكلة 1 **No such file or directory مع إن الملف موجود**

غالباً بتكتب مسار نسبي وإنت واقف في مجلد غير اللي فاكره، أو فيه مسافة في الاسم زي `Application Support`.

الحل: `pwd` عشان تعرف إنت فين، و `ls` عشان تشوف الأسماء بالظبط، واستخدم Tab عشان الـ Shell يكمّل الاسم صح (↩ 14.5 ص 176)، أو اسحب الملف من الـ Finder للـ Terminal.

مشكلة 2 **Operation not permitted وإنت بتعمل ls على Desktop أو Downloads**

كتبت `ls ~/Desktop` أو `ls ~/Library/Mail` وطلع `Operation not permitted`، مع إنها ملفاتك إنت! ده مش نظام الصلاحيات العادي، ده نظام حماية الخصوصية بتاع الماك: كل برنامج (ومنهم Terminal.app) لازم ياخد إذن علشان يقرا المجلدات دي.

الحل: هتلاقي نافذة بتسألك "Terminal would like to access files in your Desktop folder"، دوس **Allow**. ولو رفضت قبل كده: **System Settings ← Security & Privacy ← Files and Folders** (أو **Full Disk Access**) وفعّل Terminal. التفاصيل في ↩ 6.8 ص 69.

مشكلة 3 **Soft link بيقول No such file**

الهدف اتمسح أو اتنقل، أو الـ Link معمول بمسار نسبي واتنقل لمكان تاني. (وده بيحصل كتير مع Homebrew بعد ما برنامج يتحدّث).

الحل: `readlink soft.txt` تشوف بيشاور على إيه، وأعد عمله بـ `ln -sfn`. ولو هتنقل الـ Link، اعمله بمسار مطلق. ولو المشكلة في Homebrew: `brew link --overwrite اسم_الحزمة` أو `brew doctor`.

ملخص الفصل

- شجرة واحدة من `/`، والديسكات بتتركّب في `/Volumes`، وكل حاجة ملف. والـ Finder بيخبّي مجلدات Unix (Cmd+Shift+.) تظهرها).
- `/Users` ملفاتك، `/Applications` البرامج (كل `.app` مجلد)، `~/Library` إعداداتك، `/Library` مشترك، `/System` بتاع Apple ومقفول، `/opt/homebrew` لـ Homebrew على Apple Silicon (و `/usr/local` على Intel).
- `/etc` و `/var` و `/tmp` Links لـ `/private`. ومفيش `/proc`: استخدم `sysctl`.
- المطلق بيبدأ بـ `/`، والنسبي من مكانك. `.` هنا، `..` فوق، `~` الـ home. والماك مش حساس لحالة الحروف افتراضياً، والسيرفر حساس.
- الاسم بيشاور على inode. الـ Hard link اسم تاني لنفس الـ inode، والـ Soft link ملف فيه مسار، والـ Finder Alias الـ Terminal مبيفهموش.
- `df -h` للمساحة الفاضية، `du -h -d 1` لمين واكلها، `diskutil list` للديسكات.

صح ولا غلط؟

1. لو مسحت الملف الأصلي، الـ Hard link بيفضل شغال. صح / غلط
2. الماك فيه مجلد `/proc` زي Linux. صح / غلط
3. على الماك، `Report.txt` و `report.txt` افتراضياً نفس الملف. صح / غلط
4. `../a.txt` مسار مطلق. صح / غلط

أسئلة مراجعة

1. فين تدوّر على ملف إعدادات SSH؟ وفين إعدادات VS Code بتاعتك؟ وفين برنامج سطبته بـ Homebrew على جهاز M2؟
2. إنت في `/Users/ahmed/projects/web`. اكتب مسار نسبي يوصلك لـ `/Users/ahmed/Downloads`.
3. ليه مينفعش تعمل Hard link لمجلد أو لملف على فلاشة تانية؟
4. ليه `ls -l /etc` بيوريك سهم؟ والمسار الحقيقي فين؟
5. إزاي تعرف أكبر 3 مجلدات جوه الـ home؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل ادخل كل مجلد من الجدول 4.1 بـ `cd` واعمل `ls`، وارجع بـ `cd -`. وجرّب `open .` في كل واحد.
2. سهل اعرف رقم الرامات واسم الـ CPU بـ `sysctl`، واعرف نوع الـ Filesystem بـ `diskutil info /`.
3. متوسط اعمل ملف، وHard link، وSoft link، وبعدين امسح الأصل، وجرّب `cat` على الاتنين. فسّر اللي حصل.
4. متوسط جرّب: `touch Test.txt` وبعدين `ls test.txt`. إيه اللي حصل؟ وليه ده ممكن يبوظ مشروع JavaScript على السيرفر؟
5. صعب ادخل جوه `/Applications/Calculator.app` (أو `/System/Applications/Calculator.app`) ولاقي الملف التنفيذي الحقيقي، وشغله من الـ Terminal بالمسار بتاعه.

05

المرحلة 2 · الملفات والصلاحيات

الفصل الخامس

التنقل والتعامل مع الملفات

بص على مجلد Downloads بتاعك دلوقتي. غالباً فيه مئات الملفات: PDFs من سنة فاتت، صور، ملفات dmg لبرامج سطبتها ونسيت تمسحها. لو عايز تعرف أكبر 5 ملفات فيهم، أو تنقل كل الـ PDFs لمجلد لوحدها، بالماوس هتقعد نص ساعة. الفصل ده بيعلمك الأوامر اللي هتعملها كل يوم: تعرف إنت فين، تشوف فيه إيه، تنتقل، تعمل وتنسخ وتنقل وتمسح، وتقرا اللي جوه الملفات. الأوامر بسيطة، بس الـ Flags بتاعتها هي اللي بتفرق بين واحد بيستخدم الـ Terminal وواحد بيحترفه.

بعد الفصل ده هتكون عارف

- pwd و ls وقراية كل عمود في ls -l
- cd و mkdir و touch
- cp و mv و rm و rmdir بأمان
- cat و less و head و tail وإمتى تستخدم كل واحد
- file و stat و tree
- مقارنة cp و mv و rsync

5.1 إنت فين؟ وفيه إيه هنا؟ pwd و ls

في الـ Finder إنت دايماً شايف إنت فين: اسم المجلد مكتوب فوق، والملفات قدامك. في الـ Terminal مفيش الرفاهية دي، فأول سؤالين هتسألهم طول الوقت هما: "أنا واقف فين؟" و "فيه إيه حواليا؟". زي ما تصحى في أوضة ضلمة وتولّع النور الأول:

تشريح الأمر

```
pwd
```

1. `pwd` — Print Working Directory: اطبع المسار المطلق للمجلد اللي إنت فيه دلوقتي. أول حاجة تعملها لما تتوه.

تشريح الأمر

```
ls -lahG ~/Downloads
```

1. `ls` — LiSt: اعرض محتويات مجلد. من غير أي Argument بيعرض المجلد الحالي.
2. `-lahG` — أربعة مدموجين: `-l` (Long: سطر تفاصيل لكل ملف) و `-a` (All: اعرض المخفي كمان، اللي يبدأ بنقطة) و `-h` (Human: الأحجام بـ B و K و M) و `-G` (لوّن الأنواع: مجلدات أزرق، Links بنفسجي، قابل للتشغيل أحمر، وده بس لو الناتج رايح للشاشة). الـ `-G` دي خاصة بـ `ls` بتاع الماك؛ على Linux الألوان بتبقى بـ `--color=auto`. ولو عايز ألوان على طول من غير `-G`، حط `export CLICOLOR=1` في `~/.zshrc`.
3. `~/Downloads` — المجلد اللي عايز تشوفه. ممكن تحط أكتر من مسار.

```
Terminal
user@MacBook ~ % ls -lah ~/Downloads
total 23816
drwx------@  5 ahmed  staff   160B Oct 22 09:14 .
drwxr-x---+ 42 ahmed  staff   1.3K Oct 22 08:02 ..
-rw-r--r--@  1 ahmed  staff    11M Oct 21 18:40 notes.pdf
-rwxr-xr-x   1 ahmed  staff   812B Oct 20 11:05 setup.sh
lrwxr-xr-x   1 ahmed  staff     8B Oct 19 10:00 latest -> setup.sh
```

شكل 5.1 كل عمود في ls -l. أول حرف هو النوع: - ملف، d مجلد، l Link.

الـ `.` و `..` اللي ظهروا فوق هما المجلد الحالي والأب (↩ 4.3 ص 41)، والـ `total` هو مجموع المساحة المحجوزة بوحدات اسمها Blocks (مبيتأثرش بـ `-h` على الماك). والصلاحيات (العشر حروف الأولانية) ليها فصل كامل: ↩ 6.1 ص 58.

وفيه حاجتين هتشوفهم على الماك بس. الأولى: علامة `@` بعد الصلاحيات، ومعناها إن الملف عليه **Extended attributes**: معلومات زيادة الماك بيلزقها في الملف، زي "الملف ده اتنزل من النت" (ده سبب إن الـ PDF عليه `@`). والتانية: علامة `+` ومعناها إن عليه **ACL**: قواعد صلاحيات زيادة (زي اللي على الـ home بتاعك علشان محدش يمسحه). الاتنين متشرحين في ↩ 6.7 ص 67.
وكمان المجلد حجمه `160B` مش `4.0K` زي Linux، لأن APFS بيحسب حجم المجلد بعدد اللي جواه.

الـ Flag	المعنى
`-l`	Long: تفاصيل
`-a` / `-A`	All: المخفي كمان. `-A` نفسها من غير `.` و `..`
`-h`	أحجام مقروءة (مع `-l`)
`-t`	رتّب بالوقت: الأحدث الأول
`-S`	رتّب بالحجم: الأكبر الأول
`-r`	Reverse: اعكس أي ترتيب (`-ltr` = الأحدث في الآخر، مفيد جداً مع الـ Logs)
`-R`	Recursive: ادخل جوه كل المجلدات
`-d`	اعرض المجلد نفسه مش اللي جواه (`ls -ld /etc`)
`-1`	اسم واحد في كل سطر (مفيد مع الـ Pipes)
`-i`	رقم الـ inode (↩ 4.4 ص 42)

ال Flag	المعنى
`-G`	ألوان (على الماك). على Linux `-G` معناها حاجة تانية خالص (متعرضش الجروب)!
`-@`	اعرض أسماء ال Extended attributes تحت كل ملف (ماك بس)
`-e`	اعرض ال ACL (ماك بس)
`-O`	اعرض ال File flags زي `hidden` و `uchg` (ماك بس)

جدول 5.1 أهم Flags ال ls

5.2 التنقل: cd

ال `cd` هو رجليك في ال Terminal. كل اللي بتعمله في ال Finder بالدبل كليك على مجلد، هنا بتعمله بـ `cd` واسم المجلد. وهتستخدمه أكتر من أي أمر تاني في حياتك، فخلينا نعرف كل أشكاله:

الأمر	بيوديك فين؟
`cd /etc`	مسار مطلق
`cd projects`	مجلد جوه المجلد الحالي (نسبي)
`cd ..`	مجلد لفوق
`cd` أو `cd ~`	ال home بتاعك
`cd -`	المجلد اللي كنت فيه قبل كده (بيبدّل بين الاتنين)
`cd "My Files"`	اسم فيه مسافة لازم بين علامات تنصيص

جدول 5.2 أشكال cd

popd و pushd

لو بتتنقل بين أكتر من مجلدين: `pushd /etc` بيروح هناك ويحفظ مكانك في "كومة"، و `popd` بيرجّعك. و `dirs -v` بيعرض الكومة بأرقام، وفي zsh تقدر تكتب `cd -2` علشان تروح لرقم 2 فيها على طول.

حركات zsh في التنقل

لو كتبت `setopt AUTO_CD` في `~/.zshrc`، هتقدر تكتب اسم المجلد بس من غير `cd` وتدخله. وفي zsh كمان `cd` بيكمّل الأسماء بـ Tab حتى لو كتبت جزء من النص، وبيعرضلك قايمة تختار منها بالأسهم (↩ 14.5 ص 176).

5.3 إنشاء: mkdir و touch

إنت كمبرمج هتبدأ مشاريع كتير، وكل مشروع ليه نفس الهيكل تقريباً: مجلد لـ CSS، ومجلد لـ JS، ومجلد للصور. بال Finder: كليك يمين، New Folder، اكتب الاسم، كرر 4 مرات. بال Terminal: سطر واحد. اتفرج:

تشريح الأمر

```
mkdir -pv projects/web/{css,js,img}
```

1. `mkdir` — MaKe DIRectory: اعمل مجلد.
2. `-pv` — `-p` (Parents: اعمل المجلدات اللي في النص لو مش موجودة، ومتشتكيش لو المجلد موجود أصلاً) و `-v` (Verbose: قولي عملت إيه).
3. `projects/web/` — المسار. من غير `-p` هيفشل لو `projects` مش موجود.
4. `{css,js,img}` — Brace Expansion: الـ Shell (مش mkdir) بيحولها لـ 3 مسارات: `projects/web/css` و `projects/web/js` و `projects/web/img`. من غير مسافات جوه الأقواس! (↩ 7.4 ص 75).

تشريح الأمر

```
touch notes.txt
```

1. `touch` — لو الملف مش موجود: اعمله فاضي. ولو موجود: حدّث وقت آخر تعديل بتاعه لـ "دلوقتي" من غير ما تغير المحتوى. (أصل الاسم: "المسه" عشان الوقت يتحدث).
2. `notes.txt` — اسم الملف. ينفع أكتر من واحد: `touch a.txt b.txt`.

5.4 النسخ والنقل والمسح: cp و mv و rm

هنا بقى لازم نتكلم بجد شوية. في الـ Finder لو مسحت حاجة بالغلط بتروح الـ Trash وترجعها عادي. ولو نسخت ملف فوق ملف بنفس الاسم، الماك بيسألك "Replace؟". الـ Terminal **مبيعملش أي حاجة من دي**. هو بيفترض إنك عارف إنت بتعمل إيه، ويعمله على طول. عشان كده الأوامر التلاتة دي هي أكتر حاجة لازم تفهم الـ Flags بتاعتها كويس:

تشريح الأمر

```
cp -rv src/ backup/
```

1. `cp` — CoPy: انسخ. الترتيب دايماً: **المصدر الأول، وبعدين الوجهة**.
2. `-rv` — `-r` (Recursive: انسخ المجلد باللي جواه، من غيرها cp بيرفض ينسخ مجلد) و `-v` (اطبع كل ملف بيتنسخ).
3. `src/` — المصدر.
4. `backup/` — الوجهة. لو `backup` مش موجود، هيتعمل وفيه محتويات src.

> ⚠ **الـ / في آخر المصدر بتفرق على الماك**
>
> على الماك (BSD)، لو المصدر آخره `/` زي `src/` ، ال `cp` بينسخ **اللي جوه** المجلد في الوجهة. ولو كتبته من غير `/` (`src`) بينسخ المجلد نفسه، فتبقى النسخة `backup/src` . على Linux ال `/` دي ملهاش أي تأثير مع cp. نفس فكرة `rsync` بالظبط (↩ 10.7 ص 122).

الـ Flag	المعنى
`-i`	اسألني قبل ما تكتب فوق ملف موجود
`-n`	No-clobber: متكتبش فوق أي ملف موجود خالص
`-p`	Preserve: احفظ الصلاحيات والتواريخ (وعلى الماك كمان ال Extended attributes وال ACL)
`-a`	Archive: زي `-pPR` : كله، وال Links زي ما هي. أحسن حاجة لل Backups
`-c`	Clone (ماك بس): نسخة لحظية على APFS من غير ما تاخد مساحة زيادة، لحد ما تعدّل في واحدة منهم
`-u`	Update: انسخ بس لو المصدر أحدث. **على Linux بس**؛ على الماك استخدم `rsync -u`

جدول 5.3 مهمة J Flags cp

تشريح الأمر

```
mv -i old_name.txt reports/2026-report.txt
```

1. `mv` MoVe: انقل. وهو نفسه أمر إعادة التسمية (Rename)، لأن إعادة التسمية = نقل لنفس المكان باسم تاني. مش محتاج `-r` للمجلدات.
2. `-i` Interactive: اسأل قبل ما تكتب فوق ملف موجود. (`mv` افتراضياً بيكتب فوقه من غير ما يسألك!)
3. `old_name.txt` المصدر.
4. `reports/2026-report.txt` الوجهة: نقل وغيّر الاسم في نفس الخطوة.

تشريح الأمر

```
rm -ri old_project/
```

1. `rm` ReMove: امسح. **مفيش Trash**، المسح نهائي، ومش هتلاقي الملف في سلة الماك. لو عايز تمسح من ال Terminal وتروح ال Trash، استخدم ال Finder، أو سطّب أداة زي `brew install trash` .
2. `-ri` `-r` (Recursive: امسح المجلد وكل اللي جواه) و `-i` (اسألني على كل ملف). وفيه `-I` (كبيرة): اسأل مرة واحدة بس لو هتمسح أكتر من 3 ملفات.
3. `old_project/` اللي هيتمسح. وللمجلد الفاضي فيه `rmdir` اللي مبيمسحش غير مجلد فاضي، وده أأمن.

rm -rf

`-f` معناها Force: متسألش، ومتشتكيش لو الملف مش موجود. `rm -rf` مع مسار غلط (أو متغير فاضي زي `rm -rf $DIR/`) ممكن تمسح حاجات مكنتش تقصدها. ليها جزء كامل في ↩ 15.1 ص 182.

حاجة مشتركة

لاحظ إن `-i` و `-v` و `-f` و `-r` بنفس المعنى في `cp` و `mv` و `rm` (ومعظم الأوامر اللي بتغير ملفات). ده العُرف اللي في ↩ 2.5 ص 21.

	`cp`	`mv`	`rsync`
الأصل بيفضل؟	أيوه	لأ (اتنقل)	أيوه
مجلدات	محتاج `-r`	عادي	محتاج `-r` أو `-a`
لو اتقطع في النص	تبدأ من الأول	نفس الديسك: لحظي	يكمل من مكان ما وقف
بينسخ اللي اتغير بس؟	لأ (إلا `-u`)	.	أيوه، وده أهم ميزة
على جهاز تاني	لأ	لأ	أيوه عبر SSH
تستخدمه في	نسخة سريعة	نقل وإعادة تسمية	Backups ومزامنة (↩ 10.7 ص 122)

جدول 5.4 cp مقابل mv مقابل rsync

5.5 قراية الملفات: cat و less و head و tail

عندك ملف وعايز تعرف فيه إيه. على الماك ممكن تدوس Space في الـ Finder وتشوف Quick Look. بس لما الملف يبقى Log فيه نص مليون سطر، أو ملف إعدادات على سيرفر مفيهوش Finder أصلاً، هتحتاج الأوامر دي. وكل واحد فيهم ليه شخصية: واحد بيدلق الملف كله مرة واحدة، وواحد بيقلّب صفحة صفحة زي الكتاب، واتنين بيبصوا على الأول أو الآخر بس:

تشريح الأمر

```
cat -n app.conf
1  2   3
```

1. `cat` — conCATenate: اطبع محتوى ملف أو أكتر ورا بعض. اسمه جاي من "وصّل الملفات ببعض": `cat a b > c`.
2. `-n` — Number: رقّم كل السطور. (و `-et` على الماك بيظهر الحروف المخفية: `-e` بيحط `$` في آخر كل سطر، و `-t` بيبين الـ Tabs كـ `^I`. على Linux نفس الحكاية اسمها `-A`).
3. `app.conf` — الملف.

تشريح الأمر

```
1    2  3      4
less -N +G /var/log/install.log
```

1. `less` اعرض الملف صفحة صفحة من غير ما يتحمّل كله في الذاكرة، فمناسب لملفات ضخمة. (اسمه نكتة على برنامج قديم اسمه more: "less is more").
2. `-N` رقّم السطور (هنا كبيرة، و `-n` الصغيرة معناها حاجة تانية!).
3. `+G` نفّذ أمر `G` أول ما تفتح: روح لآخر الملف (مفيد للـ Logs).
4. `/var/log/install.log` الملف: Log بكل حاجة اتسطبت على الماك. (على Linux الـ Log الرئيسي غالباً `/var/log/syslog`، والماك بقى بيحفظ أغلب الـ Logs في نظام تاني بتقراه بـ `log show`، ↩ 9.5 ص 103).

الزرار	بيعمل إيه
Space / b	صفحة لتحت / لفوق
g / G	أول الملف / آخره
/word ثم **n / N**	دوّر لقدام، التالي / السابق (و `?word` لورا)
F	تابع الملف لايف زي `tail -f` (اخرج بـ Ctrl+C)
q	خروج

جدول 5.5 زرار less (نفسها في man)

تشريح الأمر

```
1    2  3  4
tail -n 50 -f /opt/homebrew/var/log/nginx/access.log
```

1. `tail` اعرض آخر الملف (و `head` أوله، بنفس الـ Flags).
2. `-n 50` آخر 50 سطر (الافتراضي 10). و `-n +2` بالـ `+` معناها "من السطر رقم 2 لحد الآخر"، مفيد عشان تشيل سطر العناوين.
3. `-f` Follow: متقفلش، وكل سطر جديد يتكتب في الملف اطبعه لايف. اخرج بـ `Ctrl+C`. ولو الملف بيتعمله Rotate (يتغير باسم جديد) استخدم `-F`.
4. `/opt/homebrew/var/log/nginx/access.log` الملف: Log بتاع nginx لو مسطبه بـ Homebrew (Homebrew بيحط الـ Logs والبيانات في `/opt/homebrew/var`). على سيرفر Linux نفس الملف بيبقى في `/var/log/nginx/access.log`.

الأمر	إمتى؟	ميزة	عيب
`cat`	ملف صغير، أو تبعته لـ Pipe	سريع وبسيط	ملف كبير هيغرق الشاشة

الأمر	إمتى؟	ميزة	عيب
`less`	تقرا وتدور في ملف كبير	بحث وتنقل وميحملش الملف كله	تفاعلي، مش للـ Scripts
`head`	تشوف شكل الملف (أول سطور)	سريع لأي حجم	جزء بس
`tail` / `tail -f`	آخر الـ Log، أو تتابعه لايف	المتابعة اللحظية	جزء بس

جدول 5.6 cat مقابل less مقابل head/tail

5.6 تفاصيل الملف: file و stat و tree

مرة صاحبك بعتلك ملف اسمه `photo.jpg` والـ Preview رفض يفتحه. هل الملف بايظ؟ ولا هو أصلاً مش JPG؟ الامتداد ممكن يكدب، بس محتوى الملف مبيكدبش. الأوامر دي بتخليك "تستجوب" الملف وتعرف عنه كل حاجة:

تشريح الأمر

```
file photo.jpg setup.sh
```

1. `file` بيعرف نوع الملف من **محتواه** (أول بايتات)، مش من الامتداد.
2. `photo.jpg setup.sh` ملف أو أكتر.

Terminal

```
user@MacBook ~ % file photo.jpg setup.sh
photo.jpg: PNG image data, 800 x 600, 8-bit/color RGBA, non-interlaced
setup.sh:  Bourne-Again shell script text executable, ASCII text
```

لاحظ إن `photo.jpg` طلع PNG فعلياً: الامتداد كان كذاب!

تشريح الأمر

```
stat -f '%N %z bytes, modified %Sm' notes.txt
```

1. `stat` كل تفاصيل الملف من الـ inode: الحجم، الصلاحيات بالأرقام، المالك، والتواريخ (Access و Modify و Change و Birth: وقت إنشاء الملف، والماك بيحفظه).
2. `-f` Format: إنت تحدد شكل الناتج. ده شكل الماك (BSD).
3. `'%N %z bytes, modified %Sm'` `%N` = الاسم، `%z` = الحجم بالبايت، `%Sm` = وقت آخر تعديل (`m` = modify، و `S` قبلها معناها "اكتبه كنص مقروء" مش كرقم). والعلامات المفردة عشان الـ Shell ميلمسش الـ `%` والمسافات.
4. `notes.txt` الملف.

```
Terminal
user@MacBook ~ % stat -f '%N %z bytes, modified %Sm' notes.txt
notes.txt 9 bytes, modified Oct 22 10:01:44 2026
user@MacBook ~ % stat -f '%Lp' notes.txt
644
```

و `%Lp` بتطبع الصلاحيات بالأرقام (هنفهمها في ↩ 6.2 ص 59). ولو عايز كل التفاصيل في شكل مرتب شبه Linux: `stat -x notes.txt`.

على Linux (السيرفرات)

على Linux (GNU) نفس الأمر بيتكتب `stat -c '%n %s bytes, modified %y' notes.txt` : `-c` بدل `-f` ، و `%n` و `%s` و `%y` بدل `%N` و `%z` و `%Sm` . والصلاحيات بالأرقام `%a` . ده من أكتر الأوامر اللي بتختلف بين النظامين، فلو هتكتب Script للاتنين استخدم `gstat` من `brew install coreutils` .

تشريح الأمر

```
tree -L 2 -a -I node_modules
```

1. `tree` اعرض المجلدات كشجرة. مش جاي مع الماك، سطّبه بـ `brew install tree` .
2. `-L 2` Level: انزل مستويين بس.
3. `-a` All: المخفي كمان (نفس معنى `ls -a`).
4. `-I node_modules` Ignore: استبعد اللي اسمه كده (ينفع Pattern زي `'*.log'`).

5.7 مشاكل شائعة

مشكلة 1 **cp: src is a directory (not copied)**

بتحاول تنسخ مجلد من غير `-R` . (على Linux نفس الغلطة رسالتها `-r not specified; omitting directory`).

الحل: `cp -R src dst` (و `-r` الصغيرة شغالة برضه)، أو `cp -a` لو عايز تحافظ على الصلاحيات والتواريخ. وافتكر فرق الـ `/` في آخر المصدر.

مشكلة 2 mv مسح ملف موجود من غير ما يسأل

نقلت `a.txt` لمجلد فيه ملف بنفس الاسم، فالقديم اتكتب فوقه.

الحل للمستقبل: استخدم `mv -i` أو `mv -n`، أو اعمل Alias دايم (↩ 13.2 ص 159). ولو اتمسح خلاص، مفيش Undo: Backups بس.

ملخص الفصل

- `pwd` إنت فين، `ls -lah` فيه إيه بالتفاصيل، و `ls -ltr` الأحدث في الآخر.
- `mkdir -p` بيعمل المسار كله، و `touch` بيعمل ملف فاضي أو يحدّث وقته.
- `cp` و `mv` و `rm`: المصدر قبل الوجهة، و `-i` للأمان، و `-r` للمجلدات (ما عدا mv).
- `cat` للصغير، `less` للكبير، `head` / `tail` للأطراف، `tail -f` للمتابعة.
- `file` بيعرف النوع الحقيقي، `stat -f` كل التفاصيل (على Linux `stat -c`)، `tree` الشجرة (`brew install tree`).
- على الماك: `ls -G` للألوان، `@` و `+` في `ls -l` = Extended attributes و ACL، و `cp` بيفرق معاه الـ `/` في آخر المصدر.

صح ولا غلط؟

1. `mv` محتاج `-r` عشان ينقل مجلد. صح / غلط
2. `tail -n +2 file` بيعرض آخر سطرين. صح / غلط
3. `file` بيعرف نوع الملف من الامتداد. صح / غلط

أسئلة مراجعة

1. اكتب أمر يعرض ملفات `~/Downloads` مرتبة بالوقت والأحدث في الآخر وبأحجام مقروءة وبالألوان.
2. إيه الفرق بين `cp -r` و `cp -a`؟ وإيه الفرق على الماك بين `cp -R src dst` و `cp -R src/ dst`؟
3. إزاي تغير اسم ملف من `draft.md` لـ `final.md`؟
4. عايز تتابع Log بيتكتب دلوقتي وتشوف آخر 100 سطر. اكتب الأمر.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل بأمر واحد المجلدات `lab/{in,out,tmp}`، وجوه `in` اعمل 3 ملفات بأمر واحد.
2. متوسط انسخ `lab` لـ `lab-backup` مع الحفاظ على التواريخ، وتأكد بـ `stat` إن وقت التعديل زي ما هو.
3. صعب افتح `/var/log/install.log` في less، روح للآخر، ودوّر على كلمة `error` لورا.

06

المرحلة 2 · الملفات والصلاحيات

الفصل السادس

الصلاحيات والمستخدمين

أشهر رسالة هتشوفها في حياتك: Permission denied. وأول مرة تشوفها غالباً هتبقى وانت بتسطب حاجة بـ npm install -g، فتدوّر على النت وتلاقي حد بيقولك "حط sudo قبلها وخلاص". وفعلاً بتشتغل، وبعدها بأسبوع كل حاجة في npm تبوظ ومش فاهم ليه. الفصل ده بيشرح نظام الصلاحيات من الأساس: مين يقدر يعمل إيه، إزاي تقرا وتكتب الصلاحيات بالحروف والأرقام، والمستخدمين والجروبات و sudo صح. وكمان طبقات الحماية الزيادة اللي الماك بيضيفها: الـ Extended attributes والـ Quarantine والـ ACL و SIP و Full Disk Access.

بعد الفصل ده هتكون عارف

- نظام r و w و x للمالك والجروب والباقي
- الفرق بين الصلاحيات على الملف وعلى المجلد
- chmod بالحروف وبالأرقام (755 و 644 و 600)
- chown و chgrp و umask
- المستخدمين والجروبات على الماك (dscl و admin) ومقارنتها بـ /etc/passwd
- الفرق بين sudo و su وليه `sudo echo > file` مبتشتغلش
- حماية الماك الزيادة: xattr و com.apple.quarantine و ACL و SIP و Full Disk Access

6.1 الفكرة: 3 أسئلة × 3 أنواع ناس

تخيل إنك ساكن في شقة مع عيلتك، وعندك أوضة خاصة بيك. إنت تقدر تدخل أوضتك وتقرا الكتب اللي فيها وتغيّر ترتيبها. إخواتك (الجروب) ممكن يدخلوا يقروا بس، من غير ما يحركوا حاجة. والضيوف (الباقي) ملهمش دعوة بأوضتك خالص. ده بالظبط نظام الصلاحيات في Unix، والماك ماشي عليه زي Linux بالظبط.

طب ليه النظام ده موجود أصلاً على جهاز إنت بس اللي بتستخدمه؟ لأن الماك من جوه فيه "مستخدمين" كتير غيرك: كل خدمة في النظام شغالة باسم مستخدم خاص بيها. والصلاحيات هي اللي بتمنع برنامج اتخرّب إنه يلمس ملفاتك، وبتمنعك إنت كمان إنك تبوّظ النظام بالغلط.

كل ملف ليه **مالك** (Owner) و**جروب** (Group). ولكل ملف، النظام بيسأل 3 أسئلة لـ 3 فئات:

4 = Read · r	2 = Write · w	1 = eXecute · x
تقدر تقرا المحتوى؟	تقدر تعدّل؟	تقدر تشغّله كبرنامج؟

شكل 6.1 -rwxr-x--- = ملف عادي بصلاحيات 750.

يعني `-rwxr-x---` معناها: ملف عادي (`-`)، المالك يقرا ويكتب ويشغّل، الجروب يقرا ويشغّل بس، والباقي ملهمش أي حاجة.

الحرف	على ملف	على مجلد
`r`	تقرا المحتوى	تعرض الأسماء اللي جواه (`ls`)
`w`	تعدّل المحتوى	تعمل وتمسح وتغير أسماء ملفات **جواه**
`x`	تشغّله كبرنامج أو Script	تدخله (`cd`) وتوصل للملفات اللي جواه

جدول 6.1 نفس الحرف معناه إيه على الملف وعلى المجلد

> **معلومة بتفاجئ الناس**
>
> مسح ملف بيعتمد على صلاحية **w على المجلد** اللي فيه، مش على الملف نفسه. يعني ممكن تمسح ملف مش من حقك تعدّله لو المجلد بتاعك! (وعشان كده `/tmp` عليه Sticky bit، تحت).

6.2 chmod: تغيير الصلاحيات

أول مرة هتحتاج الأمر ده غالباً هتبقى كتبت Script صغير، وحاولت تشغله بـ `./backup.sh`، والـ Terminal قالك `zsh: permission denied`. الملف موجود والكلام اللي جواه صح، بس محدش قال للماك إن الملف ده "برنامج" ينفع يتشغل. وهنا بييجي دور `chmod`. وليه طريقتين للكتابة: بالحروف (أوضح) وبالأرقام (أسرع):

بالحروف (Symbolic)

تشريح الأمر

```
chmod u+x,g-w,o= deploy.sh
```

1. `chmod` — CHange MODe: غيّر الصلاحيات.
2. `u+x` — User = `u` (المالك)، `+` = ضيف، `x` = تشغيل. يعني "ضيف للمالك صلاحية التشغيل".
3. `,` — فاصلة من غير مسافة: اعمل كذا تغيير في نفس الأمر.
4. `g-w` — Group = `g`، `-` = شيل، `w` = كتابة.
5. `o=` — Others = `o`، `=` = خليها بالظبط كده، ومفيش حاجة بعدها = ملهمش أي صلاحية. (وفيه `a` = All للتلاتة).
6. `deploy.sh` — الملف.

بالأرقام (Octal)

كل فئة رقم واحد = مجموع (r=4، w=2، x=1). فـ rwx = 7، -rw = 6، r-x = 5، r-- = 4، --- = 0. والأرقام التلاتة بالترتيب: المالك، الجروب، الباقي.

الرقم	بالحروف	الاستخدام
755	rwxr-xr-x	Scripts وبرامج ومجلدات عامة
644	rw-r--r--	ملفات عادية (الافتراضي)
700	rwx------	مجلد خاص بيك بس (`~/.ssh`)
600	rw-------	ملفات سرية: مفاتيح SSH، ملفات `.env`
640	rw-r-----	إعدادات الجروب يقراها والباقي لأ
775	rwxrwxr-x	مجلد شغل مشترك لفريق
777	rwxrwxrwx	**تقريباً أبداً**: أي حد على الجهاز يعدّل ويشغّل

جدول 6.2 أشهر الصلاحيات بالأرقام

تشريح الأمر

```
1     2  3      4
chmod -R 750 ~/projects/app
```

1. `chmod` غيّر الصلاحيات.
2. `-R` Recursive (حرف كبير بس في chmod/chown!): طبّق على المجلد وكل اللي جواه.
3. `750` المالك rwx، الجروب r-x، الباقي ولا حاجة.
4. `~/projects/app` المجلد.

فخ chmod -R على ملفات ومجلدات مع بعض

`chmod -R 755` هيخلي **كل الملفات** قابلة للتشغيل كمان، وده مش مطلوب. الأصح إنك تفرق بين المجلدات والملفات باستخدام `find` (↩ 8.10 ص 91):

Terminal

```
find ~/projects/app -type d -exec chmod 755 {} +    # المجلدات
find ~/projects/app -type f -exec chmod 644 {} +    # الملفات
```

وفيه حل أسهل بالحروف: `chmod -R a+rX dir`، الـ `X` الكبيرة معناها "x للمجلدات بس (واللي عليه x أصلاً)".

الصلاحيات الخاصة (رقم رابع في الأول)

الاسم	الرقم	بيظهر	بيعمل إيه	مثال
setuid	`4` xxx	`s` مكان x المالك	البرنامج يشتغل بصلاحيات **صاحبه** مش اللي شغّله	`/usr/bin/sudo` = `-r-s--x--x`
setgid	`2` xxx	`s` مكان x الجروب	على مجلد: أي ملف جديد ياخد جروب المجلد	مجلدات الفرق
sticky	`1` xxx	`t` في الآخر	على مجلد: محدش يمسح غير ملفاته	`/private/tmp` = `drwxrwxrwt`

جدول 6.3 Setuid و Setgid و Sticky

6.3 chown و chgrp: مين المالك؟

فاكر حكاية `sudo npm install -g` اللي في أول الفصل؟ تعالى نفهم إيه اللي حصل بالظبط. لما كتبت `sudo`، الـ npm اشتغل كـ **root**، فكل الملفات اللي عملها في الـ Cache بتاعه (`~/.npm`) بقى مالكها root مش إنت. وبعد كده لما تشغّل npm عادي من غير sudo، يحاول يكتب في ملفات مش بتاعتك، فيطلعلك `EACCES: permission denied`. الحل مش إنك تفضل تحط sudo، الحل إنك ترجّع الملفات لصاحبها:

تشريح الأمر

```
1    2     3    4              5      6
sudo chown -R $(whoami):staff ~/.npm
```

1. `sudo` نفّذ الأمر كـ root، لأن تغيير المالك لملف مش بتاعك محتاج صلاحيات Admin (تحت).
2. `chown` CHange OWNer: غيّر المالك.
3. `-R` Recursive: المجلد وكل اللي جواه (نفس chmod).
4. `$(whoami)` Command Substitution: نفّذ `whoami` وحط ناتجه (اسمك، زي `ahmed`) مكانه (↩ 7.6 ص 77).
5. `:staff` `user:group` . و `staff` هو الجروب الافتراضي لكل مستخدم على الماك. لو كتبت `ahmed` بس يتغير المالك بس. ولو `:staff` (من غير user) يتغير الجروب بس، زي `chgrp staff` .
6. `~/.npm` المسار.

Best Practice

متستخدمش `sudo` مع `npm install -g` خالص. على الماك، الأحسن إنك تسطّب Node عن طريق أداة زي `nvm` أو `fnm` أو `brew install node` ، وكلهم بيسطبوا في أماكن بتاعتك إنت، فمش هتحتاج sudo أصلاً.

على Linux (السيرفرات)

على السيرفر هتشوف كتير `sudo chown -R ahmed:www-data /var/www/site` : الجروب `www-data` هو اللي nginx شغال بيه على Ubuntu، فلازم يقدر يقرا ملفات الموقع.

6.4 umask: الصلاحيات الافتراضية

ليه أي ملف جديد بتعمله بيطلع `644` وأي مجلد `755` ؟ بسبب **umask**: "قناع" بيتشال من الصلاحيات الكاملة. الملفات بتبدأ من 666 والمجلدات من 777، وبيتشال منهم الـ umask.

Terminal

```
user@MacBook ~ % umask
0022
files: 666 - 022 = 644   dirs: 777 - 022 = 755 #
```

تشريح الأمر

```
umask 027
```
(1) (2)

1. `umask` — Builtin: من غير Argument بيعرض القيمة، ومعاه بيغيرها للـ Shell الحالي بس (حطها في `~/.zshrc` عشان تفضل، ↩ 13.1 ص 157).
2. `027` — شيل w من الجروب (2)، وشيل كله من الباقي (7). النتيجة: ملفات `640` ومجلدات `750`. (الحساب الدقيق مش طرح، ده "شيل البتات دي"، بس الطرح بيوصل لنفس النتيجة في الحالات المشهورة).

6.5 المستخدمين والجروبات

لما فتحت الماك أول مرة عملت حساب باسمك، وده كل اللي شفته. بس جوه النظام فيه عشرات المستخدمين التانيين اللي مبيظهروش في شاشة الدخول: مستخدم للـ Spotlight، ومستخدم للطباعة، ومستخدم لكل خدمة تقريباً. خلينا نتعرف على نفسنا الأول:

Terminal

```
user@MacBook ~ % whoami
ahmed
user@MacBook ~ % id
uid=501(ahmed) gid=20(staff)
groups=20(staff),12(everyone),61(localaccounts),80(admin),701(com.apple.sharepoint.group.1),33
(_appstore),98(_lpadmin),100(_lpoperator),204(_developer)
```

الجروبات عندك ممكن تكون أكتر أو أقل حسب إعدادات جهازك.

- كل مستخدم ليه رقم **UID**، و root دايماً UID = 0. على الماك أول مستخدم عادي رقمه **501** (على Linux بيبدأوا من 1000).
- الجروب الأساسي **staff** (رقمه 20)، وده مشترك بين كل المستخدمين العاديين.
- أهم جروب في اللستة: **admin** (80). اللي فيه هو اللي يقدر يستخدم `sudo` ويسطب برامج. لو حسابك "Administrator" في System Settings، يبقى إنت فيه.
- المستخدمين بتوع النظام أساميهم بتبدأ بـ `_` زي `_www` و `_spotlight`.

فين متخزنة بيانات المستخدمين؟

على Linux كل ده متخزن في ملف نصي اسمه `/etc/passwd`. الماك عنده الملف ده برضه، بس للزينة تقريباً: الماك بيستخدم قاعدة بيانات اسمها **Directory Services**، وبتسألها بأمر اسمه `dscl`:

تشريح الأمر

```
dscl . -read /Users/ahmed UniqueID PrimaryGroupID NFSHomeDirectory UserShell
```

1. `dscl` — Directory Service Command Line: اسأل (أو عدّل) قاعدة بيانات المستخدمين.
2. `.` — النقطة هنا معناها "الجهاز ده" (الـ Local directory)، مش المجلد الحالي.
3. `-read` — اقرا سجل.
4. `/Users/ahmed` — السجل: المستخدم ahmed. (ده مسار جوه قاعدة البيانات، مش مجلد الـ home، رغم إن شكلهم واحد).
5. `UniqueID PrimaryGroupID NFSHomeDirectory UserShell` — المفاتيح اللي عايزها: الـ UID، والجروب الأساسي، والـ home، والـ Login Shell. من غيرهم هيطبعلك كل حاجة.

Terminal

```
user@MacBook ~ % dscl . -read /Users/ahmed UniqueID PrimaryGroupID NFSHomeDirectory UserShell
NFSHomeDirectory: /Users/ahmed
PrimaryGroupID: 20
UniqueID: 501
UserShell: /bin/zsh
```

وعلى سيرفر Linux نفس المعلومات دي في سطر واحد في `/etc/passwd`، مفصولة بـ `:`. الجدول ده بيقطّع السطر ده عمود عمود، وقصاد كل عمود اسمه في dscl:

Terminal

```
user@server:~$ grep ahmed /etc/passwd
ahmed:x:1000:1000:Ahmed Ali,,,:/home/ahmed:/bin/bash
```

#	العمود على Linux	الماك (dscl)	المعنى
1	`ahmed`	`RecordName`	اسم المستخدم
2	`x`	متخزن متشفر في مكان محمي	الباسورد مش هنا؛ على Linux في `/etc/shadow` (اللي root بس يقراه)
3	`1000`	`UniqueID` = 501	الـ UID
4	`1000`	`PrimaryGroupID` = 20	الـ GID: الجروب الأساسي
5	`Ahmed Ali,,,`	`RealName`	الاسم الكامل (GECOS)
6	`/home/ahmed`	`NFSHomeDirectory`	الـ home
7	`/bin/bash`	`UserShell`	الـ Login Shell (اللي بيتغير بـ `chsh`، ↩ 3.8 ص 32)

جدول 6.4 أعمدة /etc/passwd على Linux وقصادها في dscl على الماك

تضيف مستخدم لجروب

على الماك غالباً هتعمل ده من **System Settings ← Groups & Users**. بس لو عايز تعمله بالأمر (مثلاً تخلي مستخدم Admin):

تشريح الأمر

```
sudo dseditgroup -o edit -a sara -t user admin
     ①          ②     ③     ④     ⑤
```

① `dseditgroup` — عدّل جروب في Directory Services.

② `-o edit` — Operation: العملية هي "تعديل" (فيه كمان `read` و `create` و `delete`).

③ `-a sara` — Add: ضيف sara. (و `-d` = امسحها من الجروب).

④ `-t user` — Type: اللي بنضيفه نوعه مستخدم (ممكن يبقى جروب جوه جروب).

⑤ `admin` — اسم الجروب. ولو عايز تتأكد: `dseditgroup -o checkmember -m sara admin` .

على Linux: usermod

تشريح الأمر

```
sudo usermod -aG docker ahmed
     ①       ②  ③      ④
```

①	`usermod`	USER MODify: عدّل مستخدم موجود. (و `useradd` تعمل جديد، و `passwd` تغير الباسورد).
②	`-aG`	`-G` = الجروبات الإضافية، و `-a` = Append: **ضيف** عليهم. من غير `-a` المستخدم هيتشال من كل الجروبات التانية (حتى sudo!).
③	`docker`	اسم الجروب. (على الماك Docker Desktop مش محتاج الجروب ده أصلاً).
④	`ahmed`	اسم المستخدم. والتغيير بيبان بعد ما تعمل Log out و Log in (أو `newgrp docker`).

وعلى Linux جروب الأدمن اسمه `sudo` (Ubuntu) أو `wheel` (Fedora)، مش `admin`. وبيانات المستخدمين في `/etc/passwd` اللي بتقراه بـ `grep ahmed /etc/passwd`.

6.6 sudo و su

فكّر في `sudo` كإنه "الكارنيه الأحمر" اللي بتطلّعه للأمن علشان يدخلك أماكن ممنوعة. مش بتمشي بيه طول اليوم، بتطلعه للحظة وتشيله. كل مرة تكتب `sudo` قبل أمر، الماك بيسألك على باسوردك (أو بصمتك لو ظبطتها تحت) علشان يتأكد إن إنت فعلاً اللي عايز تعمل الحاجة الخطيرة دي.

	`sudo cmd`	`su -`
بيعمل إيه	ينفّذ أمر **واحد** كـ root	يفتح Shell كامل كمستخدم تاني (root افتراضياً)
باسورد مين	باسوردك إنت	باسورد المستخدم التاني (root)
مين يقدر	اللي في جروب `admin` على الماك (مكتوب في `/etc/sudoers`)، أو `sudo` / `wheel` على Linux	أي حد عارف الباسورد. وعلى الماك حساب root **مقفول** افتراضياً، فـ `su -` لوحدها مش هتشتغل
تسجيل	كل أمر بيتسجل في الـ Logs	أقل تتبع
النصيحة	استخدمه دايماً	نادراً

جدول 6.5 sudo مقابل su

`sudo dscacheutil -flushcache`	أمر واحد كـ root (هنا: امسح كاش الـ DNS، ↩ 10.3 ص 117).
`sudo -i`	Shell كامل كـ root بإعداداته (زي `su -` بس بباسوردك). اخرج بـ `exit`.
`sudo -u nobody ls`	`-u` = User: نفّذ كمستخدم تاني مش root (هنا `nobody`: مستخدم ملوش صلاحيات خالص، ممتاز للتجارب).

`su - sara`	ادخل كـ sara (محتاج باسوردها). الـ `-` معناها Login shell: حمّل الـ home والمتغيرات بتاعتها.
`sudo !!`	أعد آخر أمر بـ sudo (↩ 14.4 ص 174).
`sudo visudo`	عدّل ملف sudoers بأمان (بيتأكد من الكتابة قبل الحفظ). متعدلوش بـ nano مباشرة.

خلّي بصمتك هي اللي تعمل sudo

لو الماك بتاعك فيه Touch ID، تقدر تخلي `sudo` يطلب البصمة بدل الباسورد. من macOS Sonoma فيه ملف جاهز للتعديل ده ومبيتمسحش مع التحديثات:

```
Terminal
sudo cp /etc/pam.d/sudo_local.template /etc/pam.d/sudo_local
sudo nano /etc/pam.d/sudo_local    # شيل علامة # من أول سطر auth sufficient pam_tid.so
```

الـ `pam.d` هو المكان اللي فيه قواعد "إزاي النظام يتأكد إنك إنت"، و `pam_tid.so` هو الجزء اللي بيكلم Touch ID. افتح تابة جديدة وجرّب `sudo -v` (الـ `-v` = Validate: اتأكد مني بس من غير ما تشغل أمر).

على Linux: sudo على Ubuntu الحديث

من Ubuntu 25.10، الأمر `sudo` بقى بيتنفذ افتراضياً بـ **sudo-rs** (نسخة مكتوبة بـ Rust)، والـ sudo الأصلي لسه موجود باسم `sudo.ws`. الاستخدام اليومي زي ما هو، بس لو حاجة متقدمة في sudoers مشتغلتش، ده ممكن يكون السبب.

Least Privilege

اشتغل كمستخدم عادي، واستخدم `sudo` للأمر اللي محتاجه بس. ولو حاجة "مش شغالة غير بـ sudo" وهي المفروض متحتاجش (زي ملف في الـ home بتاعك)، المشكلة غالباً في المالك: صلّحها بـ `chown` بدل ما تفضل تستخدم sudo.

6.7 حماية الماك الزيادة: الـ Extended attributes والـ Quarantine

أكيد حصلك الموقف ده: نزّلت برنامج من النت، دبل كليك عليه، وطلعتلك رسالة "*can't be opened because Apple cannot check it for malicious software*". أو نزّلت أداة CLI من GitHub، وأول ما شغلتها الماك قالك إنها "from an unidentified developer". طب الماك عرف منين إن الملف ده جاي من النت؟ ده مش في الصلاحيات `rwx` خالص.

هنا بقى بييجي دور الـ **Extended attributes** (أو xattr): معلومات زيادة الماك بيلزقها في الملف من غير ما تغيّر محتواه، زي "استيكر" على الكرتونة. ولما المتصفح ينزّل ملف، بيلزق عليه استيكر اسمه `com.apple.quarantine` فيه "الملف ده جه من النت، من موقع كذا، يوم كذا". وبعد كده نظام اسمه **Gatekeeper** بيشوف الاستيكر ده ويقرر يفتح الملف ولا يحذّرك. وده نفس سبب علامة `@` اللي شفناها في `ls -l`.

تشريح الأمر

```
xattr -l ~/Downloads/tool.zip
```

1. `xattr` اعرض أو عدّل الـ Extended attributes.
2. `-l` Long: اعرض اسم كل Attribute وقيمته. (من غيرها بيعرض الأسماء بس).
3. `~/Downloads/tool.zip` الملف.

Terminal

```
user@MacBook ~ % xattr -l ~/Downloads/tool.zip
com.apple.macl:
com.apple.quarantine: 0083;6717a2c1;Safari;9E1A4C3B-...
user@MacBook ~ % ls -l@ ~/Downloads/tool.zip
-rw-r--r--@ 1 ahmed  staff  48211 Oct 22 10:30 tool.zip
        com.apple.macl       72
        com.apple.quarantine         57
```

الـ `-@` مع `ls -l` بيعرض أسماء الـ Attributes وحجمها تحت الملف. ولو إنت **متأكد** إن الملف ده من مصدر موثوق، تقدر تشيل الاستيكر:

تشريح الأمر

```
xattr -dr com.apple.quarantine ~/Applications/Tool.app
```

1. `xattr` نفس الأمر.
2. `-dr` اتنين مدموجين: `-d` (Delete: امسح Attribute معين) و `-r` (Recursive: على المجلد وكل اللي جواه، ومهمة مع الـ `.app` لأنه مجلد).
3. `com.apple.quarantine` اسم الـ Attribute اللي هيتمسح.
4. `~/Applications/Tool.app` البرنامج. (وفيه `xattr -c` بتمسح **كل** الـ Attributes، وده أوسع من اللي محتاجه).

> **خلي بالك**
>
> الاستيكر ده موجود علشان يحميك. متشيلوش غير من حاجة إنت عارف مصدرها كويس. والطريقة الأأمن من غير Terminal: كليك يمين على البرنامج ← **Open**، أو **System Settings** ← **Security & Privacy** وانزل لتحت هتلاقي زرار **Open Anyway**. ولاحظ إن الملفات اللي بتنزلها بـ `curl` من الـ Terminal مبيتلزقش عليها الاستيكر ده أصلاً، فإنت مسؤول عنها بالكامل (↩ 15.6 ص 187).

الـ ACL: صلاحيات أدق من rwx

الـ `rwx` بيقسم الناس لـ 3 فئات بس. طب لو عايز تقول "سارة بالذات تقدر تقرا الملف ده"؟ هنا بييجي دور الـ **ACL** (Access Control List): لستة قواعد زيادة على الملف. والماك بيستخدمها فعلاً على مجلداتك، وده سبب علامة `+` على الـ home:

```
Terminal
user@MacBook ~ % ls -led ~/Desktop
drwx------+ 5 ahmed  staff  160 Oct 22 09:00 /Users/ahmed/Desktop
 0: group:everyone deny delete
```

الـ `-e` بتعرض الـ ACL تحت الملف (ماك بس)، و `-d` المجلد نفسه. والقاعدة دي معناها "أي حد، حتى إنت، ممنوع يمسح مجلد Desktop نفسه". وتقدر تضيف قاعدة بـ `chmod +a` :

تشريح الأمر

```
chmod +a "user:sara allow read" report.pdf
```

1. `chmod` نفس الأمر بتاع الصلاحيات.
2. `+a` Add ACL entry: ضيف قاعدة. (و `-a` تمسح قاعدة، و `-N` تمسح كل الـ ACL).
3. `"user:sara allow read"` القاعدة بين علامات تنصيص لأن فيها مسافات: مين (`user:sara`)، مسموح ولا ممنوع (`deny` / `allow`)، وإيه (`read` ، `write` ، `delete` ...).
4. `report.pdf` الملف.

6.8 SIP و Full Disk Access: حتى root ليه حدود

على Linux الـ root هو الملك، يقدر يعمل أي حاجة في أي حتة. على الماك لأ. Apple قالت: "طب لو برنامج خبيث خد صلاحيات root، كده هيبوظ النظام كله؟" فعملت طبقتين حماية فوق الصلاحيات العادية:

System Integrity Protection · SIP

بيمنع **أي حد**، حتى root بـ `sudo` ، إنه يعدّل ملفات النظام في `/System` و `/bin` و `/sbin` و `/usr` (ما عدا `/usr/local`). علشان كده عمرك ما هتسطب حاجة في `/usr/bin` ، وده سبب إن Homebrew بيستخدم `/opt/homebrew` . اعرف حالته بـ `csrutil status` .

Full Disk Access & Privacy · TCC

بيحمي **خصوصيتك**: كل برنامج لازم ياخد إذن صريح منك علشان يوصل لـ Desktop و Documents و Downloads والكاميرا و Mail و Photos. والـ Terminal برنامج زي أي برنامج، فلو عايزه يوصل لكل حاجة: **System Settings** ← **Security & Privacy** ← **Full Disk Access** وفعّل Terminal.

```
Terminal
user@MacBook ~ % csrutil status
System Integrity Protection status: enabled.
user@MacBook ~ % sudo touch /usr/bin/test
touch: /usr/bin/test: Operation not permitted
```

> ⚠ **خلي بالك**
>
> هتلاقي شروحات قديمة بتقولك "اقفل SIP" علشان تعمل حاجة. متعملش كده: ده محتاج تدخل Recovery Mode، وبيفتح الجهاز لأي برنامج خبيث. في 2026 مفيش تقريباً أي حاجة عادية محتاجة كده. ولاحظ إن `Operation not permitted` رسالة مختلفة عن `Permission denied` : الأولى غالباً معناها SIP أو TCC، والتانية معناها `rwx` .

6.9 مشاكل شائعة

مشكلة 1 — `sudo echo "..." > /etc/file` بيقول Permission denied

عايز تخلي `myapp.test` يفتح على جهازك وانت بتجرب مشروع، فكتبت `sudo echo "127.0.0.1 myapp.test" > /etc/hosts` وفشل! ليه؟ لأن الـ `>` بيعملها الـ **Shell بتاعك** (مستخدم عادي) قبل ما الأمر يشتغل (↩ 1.2 ص 10). الـ sudo بيطبّق على `echo` بس.

الحل: خلي برنامج شغال بـ sudo هو اللي يكتب: `echo "127.0.0.1 myapp.test" | sudo tee -a /etc/hosts` .
(الـ `tee -a` بيضيف في آخر الملف، ↩ 8.8 ص 90). وبعدها على الماك امسح كاش الـ DNS علشان التغيير يبان:
`sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder` .

مشكلة 2 — npm: EACCES permission denied

بعد ما استخدمت `sudo npm ...` مرة، بقى npm بيطلع `EACCES` كل شوية.

الحل: رجّع الملفات لصاحبها: `sudo chown -R $(whoami):staff ~/.npm` ، ومتستخدمش sudo مع npm تاني (↩ 6.3 ص 61).

مشكلة 3 — Bad permissions لمفتاح SSH

SSH بيرفض المفتاح ويقول `UNPROTECTED PRIVATE KEY FILE!` لأن صلاحياته مفتوحة لغيرك.

الحل: `chmod 700 ~/.ssh` و `chmod 600 ~/.ssh/id_ed25519` (↩ 10.4 ص 119).

ملخص الفصل

- r=4، w=2، x=1، لـ 3 فئات: المالك والجروب والباقي.
- على المجلد: r = تعرض، w = تعمل وتمسح جواه، x = تدخله.
- `chmod u+x` بالحروف أو `chmod 755` بالأرقام، و `-R` كبيرة للـ Recursive.
- `chown user:group`، و `umask` بيحدد الافتراضي.
- على الماك: أول UID = 501، الجروب الأساسي `staff`، والأدمن جروب `admin`، والبيانات في `dscl` مش `/etc/passwd`. وعلى Linux `usermod -aG` بالـ `-a` دايماً.
- sudo لأمر واحد أحسن من su، و root مقفول على الماك افتراضياً.
- الماك بيضيف: `xattr` و `com.apple.quarantine` للملفات اللي من النت، والـ ACL (`ls -le`، `chmod +a`)، و SIP اللي بيحمي النظام حتى من root، و Full Disk Access.

صح ولا غلط؟

1. صلاحية 640 معناها الجروب يقدر يكتب في الملف. صح / غلط
2. عشان تدخل مجلد بـ cd لازم يكون عليه x. صح / غلط
3. `sudo echo hi > /etc/test.txt` بتشتغل لأن sudo بيشمل الـ Redirection. صح / غلط
4. بـ `sudo` تقدر تعدّل أي ملف في `/usr/bin` على الماك. صح / غلط

أسئلة مراجعة

1. حوّل `rwxr-x---` و `rw-rw-r--` لأرقام.
2. إيه الصلاحية المناسبة لمفتاح SSH الخاص؟ ولـ Script اللي هيتشغل؟
3. إيه اللي هيحصل على سيرفر Linux لو كتبت `usermod -G docker ahmed` من غير `-a`؟
4. نزّلت أداة من GitHub بالمتصفح والماك رافض يشغلها. إيه السبب؟ واذكر طريقتين للحل.
5. اشرح ليه `/tmp` صلاحياتها `drwxrwxrwt`.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل — اعمل Script فيه `echo hi`، وجرّب تشغله قبل وبعد `chmod +x`.
2. متوسط — اعمل مجلد `secret` بصلاحية 700 وجواه ملف 600، وجرّب تقراه بمستخدم تاني (`sudo -u nobody cat ...`).
3. متوسط — نزّل أي ملف من المتصفح، واعرض الـ Attributes بتاعته بـ `xattr -l`، وقارنها بملف نزلته بـ `curl -O`.
4. صعب — غيّر الـ umask لـ 077 في Shell مؤقت، اعمل ملف ومجلد، وتوقع صلاحياتهم قبل ما تعمل `ls -l`.

07

المرحلة 3 · قوة الـ Shell

الفصل السابع

قوة الـ Shell: الـ Pipes والـ Redirection والـ Quoting

لحد دلوقتي كنا بنستخدم الـ Terminal كأنه Finder بالكتابة: افتح، انسخ، امسح. الفصل ده هو اللحظة اللي هتقول فيها "آاااه، هو ده ليه الناس بتحبه!". هنا الأوامر الصغيرة بتتركب في بعض زي الليجو: أمر يطلّع داتا، والتاني يفلترها، والتالت يعدّها، والرابع يحفظها في ملف أو ينسخها على الـ Clipboard. والرموز اللي بتخوف في أوامر الناس (`>` و `2>&1` و `|` و `$()` و `*` و `&&`) كلها هتتفك هنا رمز رمز.

بعد الفصل ده هتكون عارف

- الـ Streams التلاتة: stdin و stdout و stderr
- الـ Redirection و Here-doc: `> >> < 2> 2>&1`
- الـ Pipe وإزاي الأوامر بتتركب
- الـ Globbing: `* ? [] {}`
- الـ Quoting: `'' "" \ $()` والـ Backticks
- الـ Exit codes و `&&` و `;` والـ OR

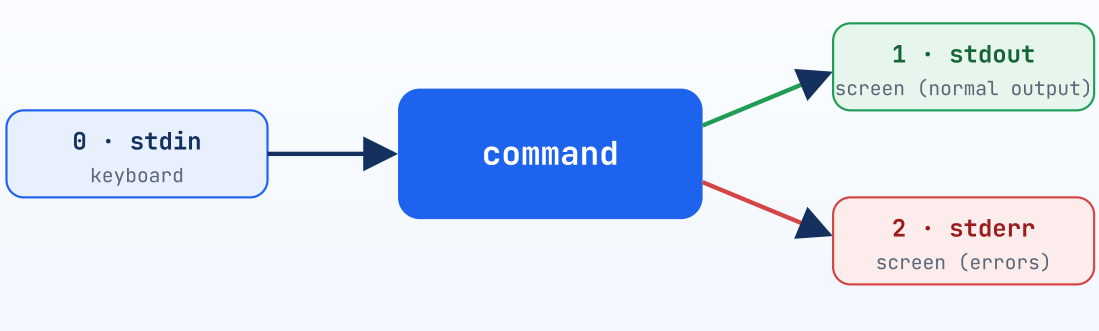
الفصل ده كله حاجة مشتركة

كل الرموز هنا مش تبع أمر معين. ده شغل الـ **Shell نفسه** قبل ما الأمر يشتغل (↩ 1.2 ص 10). يعني `>` بتشتغل مع `ls` و `grep` و `python` وأي حاجة. اتعلمها مرة واحدة، واستخدمها مع كل الأوامر.

7.1 الـ Streams التلاتة

خلينا نبدأ بسؤال: لما تكتب `ls` والأسماء تظهر قدامك، هي "راحت" فين؟ وليه لما الأمر يفشل الرسالة الحمرا بتظهر في نفس المكان؟ تخيل كل برنامج كأنه مطبخ مطعم صغير: ليه شباك بيستلم منه الطلبات، وشباك بيطلّع منه الأكل، وشباك تالت بيطلّع منه الشكاوي لو حصلت مشكلة. الشبابيك دي موجودة في كل برنامج من غير استثناء، وده اللي بيخلي البرامج تقدر تتكلم مع بعض.

أي برنامج بيتولد ومعاه 3 "قنوات" مفتوحة، ولكل واحدة رقم اسمه **File Descriptor** (fd):



شكل 7.1 كل أمر: مدخل واحد (0) ومخرجين (1 للنتيجة و 2 للأخطاء). الاتنين بيروحوا للشاشة افتراضياً.

ليه مخرجين؟ عشان لو حولت النتيجة لملف، الأخطاء تفضل قدامك على الشاشة ومتتخلطش بالبيانات. وفيه ملف خاص اسمه `/dev/null` : "ثقب أسود" أي حاجة بتتكتب فيه بتختفي.

7.2 الـ Redirection: وجّه الـ Streams

طب ما دام الأكل بيطلع من شباك، إيه رأيك نحط تحته علبة بدل ما يقع على الشاشة؟ ده بالظبط الـ Redirection. مثلاً عايز تبعت لزميلك لستة بكل الملفات اللي في مشروعك: بدل ما تعمل Screenshot أو تنسخ من الشاشة، بتكتب `ls > files.txt` والناتج يروح على طول في ملف. الجدول ده فيه كل الرموز اللي هتقابلها، وبعده هنشرّح أهمهم:

الرمز	المعنى	مثال
`>`	stdout لملف، **يمسح** القديم (= `1>`)	`ls > list.txt`
`>>`	stdout لملف، **يضيف** في الآخر	`date >> log.txt`
`<`	stdin من ملف بدل الكيبورد	`wc -l < list.txt`
`2>`	stderr لملف	`find / -name x 2> err.txt`
`2>/dev/null`	اخفي الأخطاء	`find / -name x 2>/dev/null`
`2>&1`	ودّي stderr لنفس مكان stdout	`cmd > all.log 2>&1`
`&>`	الاتنين لملف (bash/zsh اختصار)	`cmd &> all.log`
`<<EOF`	Here-doc: نص متعدد السطور كـ stdin	تحت
`<<<`	Here-string: سطر واحد كـ stdin	`wc -w <<< "a b c"`
`>|`	اكتب فوق الملف حتى لو noclobber شغال (و zsh بيقبل كمان `!>`)	`ls >| list.txt`

جدول 7.1 كل رموز الـ Redirection

تشريح الأمر

```
1          2  3          4
./backup.sh > backup.log 2>&1
```

1. `./backup.sh` الأمر اللي هيتنفذ (`./` = من المجلد الحالي، ↩ 1.4 ص 12).
2. `>` وجّه stdout (fd 1) لـ...
3. `backup.log` الملف ده. لو موجود بيتمسح ويتكتب من الأول.
4. `2>&1` = stderr (`2`) و = وجّه (`>`)، و = "لنفس مكان fd رقم 1". الـ `&` هنا معناها "ده رقم fd مش اسم ملف"؛ من غيرها `1` هيعمل ملف اسمه !

الترتيب مهم

`cmd > f 2>&1` صح: الأول stdout راح للملف، وبعدين stderr راح "لنفس مكان stdout" = الملف. لكن `cmd 2>&1 > f` غلط: stderr راح لمكان stdout **القديم** (الشاشة)، وبعدين stdout بس راح للملف. الـ Shell بيقرا من الشمال لليمين.

تشريح الأمر

```
cat <<'EOF' > config.ini[server]
port = $PORTEOF
```

1. `cat` اطبع اللي داخلك على stdin.
2. `<<` Here-doc: اقرا stdin من السطور الجاية لحد ما تلاقي الكلمة الفاصلة.
3. `'EOF'` الكلمة الفاصلة (أي كلمة، بس EOF متعارف عليها). الـ **Quotes** حواليها معناها "متفكش `$PORT`"؛ من غيرها الـ Shell هيحط قيمة المتغير (↩ 7.5 ص 76).
4. `> config.ini` والناتج يتكتب في الملف.
5. `EOF` سطر النهاية: لازم يكون لوحده في السطر ومن غير مسافات قبله (إلا لو استخدمت `<<-` مع Tabs).

احمي نفسك من الكتابة فوق الملفات بالغلط

`setopt noclobber` في zsh (أو `set -o noclobber` في bash) بيخلي `>` ترفض تكتب فوق ملف موجود وتقولك `file exists`. حطها في `~/.zshrc`، وأكتر عن المخاطر في ↩ 15.5 ص 186.

7.3 الـ | Pipe: ركّب الأوامر

هنا بقى السحر الحقيقي. في السبعينات، واحد اسمه Doug McIlroy كان شغال في Bell Labs، وكان عنده فكرة بسيطة جداً: "بدل ما نعمل برنامج عملاق بيعمل كل حاجة، نعمل برامج صغيرة كل واحد بيعمل حاجة واحدة بس، ونوصلهم ببعض زي مواسير المية". والفكرة دي اتحولت لرمز واحد: `|`، واسمه **Pipe** يعني ماسورة. ولحد النهارده، بعد 50 سنة، ده أقوى رمز في الـ Terminal.

الـ `|` بتوصل **stdout** أمر بـ **stdin** الأمر اللي بعده. كل أمر بيعمل حاجة واحدة صغيرة كويس، والـ Pipe بيخليهم فريق. دي فلسفة Unix كلها.

تشريح الأمر

```
cat access.log | grep " 404 " | wc -l
```

1. `cat access.log` اطبع اللوج كله على stdout.
2. `|` Pipe: خد stdout اللي فات وخليه stdin للي جاي. الأوامر التلاتة بيشتغلوا **في نفس الوقت**، والبيانات بتعدي بينهم.
3. `grep " 404 "` سيب السطور اللي فيها 404 بس (↩ 8.1 ص 82).
4. `|` Pipe تاني.
5. `wc -l` عِد السطور (↩ 8.7 ص 90).

الـ cat الزيادة

`grep " 404 " access.log | wc -l` نفس النتيجة من غير `cat`، لأن معظم الأوامر بتاخد اسم ملف. بل `grep -c " 404 " access.log` لوحده بيعد. بس `cat file |` مش جريمة وبتسهّل القراية.

`cmd 2>&1 | less`	الـ Pipe بيوصل stdout بس؛ عشان الأخطاء كمان تعدي، حولها لـ stdout الأول.
`cmd |& less`	اختصار zsh (و bash 4+) لـ `2>&1 |`. مش شغال في bash 3.2 بتاع الماك.
`ls -1 | pbcopy`	ماك بس: ابعت الناتج على الـ Clipboard علشان تلزقه بـ Cmd+V في أي حتة. والعكس `pbpaste | wc -l` (↩ 14.3 ص 171).
`cmd | tee out.txt`	شوف الناتج **وكمان** احفظه (↩ 8.8 ص 90).

7.4 الـ Globbing: النجمة وأخواتها

عندك 200 صورة في `Downloads` جاية من الموبايل، أساميها `IMG_0001.HEIC` لحد `IMG_0200.HEIC`، ومعاهم PDFs و Zips. وعايز تنقل الصور بس لمجلد لوحدها. هتختارهم واحدة واحدة؟ لأ طبعاً. هنا بييجي دور النجمة: `mv ~/Downloads/*.HEIC ~/Pictures/phone/`. سطر واحد وخلصت. الـ `*` دي اسمها Glob، وليها إخوات كتير.

قبل ما الأمر يشتغل، الـ Shell بيحول الـ Patterns لأسماء ملفات حقيقية. يعني `rm *.log` : الـ `rm` مبيشوفش نجمة خالص، بيشوف `rm a.log b.log c.log`.

الرمز	بيطابق	مثال	النتيجة
`*`	أي عدد حروف (حتى صفر)	`*.jpg`	كل ملفات jpg
`?`	حرف واحد بالظبط	`img?.png`	img1.png، imgA.png

الرمز	بيطابق	مثال	النتيجة
`[abc]`	حرف واحد من دول	`[Rr]eadme*`	readme.md ،Readme
`[0-9]`	Range	`log[0-9].txt`	log9 ل log0
`[!a]`	أي حرف ماعدا	`*[!.]`	اللي مبيبدأش بنقطة
`**`	أي عمق من المجلدات	`**/*.py`	zsh افتراضي، +bash 4 بعد `shopt -s globstar` (مش موجود في bash 3.2)
`{a,b}`	Brace: يولد كلمات (مش بيدور)	`file.{txt,md}`	file.txt file.md
`{1..5}`	Brace: تسلسل	`day{1..5}`	day1 ... day5

جدول 7.2 رموز الـ Globbing والـ Brace expansion

> **فرق مهم بين * و {}**
>
> الـ Glob (`*`) بيطابق ملفات **موجودة**. الـ Brace (`{}`) بيولّد كلمات سواء موجودة ولا لأ، وعشان كده `mkdir {css,js}` شغالة (↩ 5.3 ص 50). والـ `*` مبيطابقش الملفات المخفية افتراضياً. ولو مفيش ولا ملف طابق، bash بيسيب النجمة زي ما هي (والأمر يشتكي)، و zsh بيطلع خطأ `no matches found` .

> **لو حابب تتعمق: Glob qualifiers في zsh**
>
> تقدر تعدّيه في أول قراية
>
> zsh عنده ميزة مش في bash خالص: تحط "شرط" بين قوسين في آخر الـ Glob. مثلاً `ls *(.)` = الملفات العادية بس (من غير مجلدات)، و `ls *(/)` = المجلدات بس، و `ls *(om[1,5])` = أحدث 5 ملفات (`o` = order، `m` = modification time، و `[1,5]` = من الأول للخامس)، و `ls **/*(.Lm+100)` = كل ملف أكبر من 100 ميجا في أي مجلد (`L` = Length، `m` = Megabytes، `+` = أكبر من). دي حاجات بتعملها في bash بـ `find` (↩ 8.10 ص 91).

> **جرّب قبل ما تمسح**
>
> قبل `rm *.log` اكتب `echo rm *.log` : هيطبع الأمر الحقيقي بعد التوسيع من غير ما ينفذ. نفس الحيلة شغالة مع أي أمر خطير.

7.5 الـ Quoting: إمتى الـ Shell يفهم الرموز وإمتى لأ

على الماك المشكلة دي هتقابلك بدري جداً، لأن الماك بيحب المسافات في الأسماء: `Application Support` و `Screen Recording 2026-10-22.mov` و `My Project` . ولو كتبت `cd Application Support` ، الـ Shell هيفهم إنك عايز تدخل مجلد اسمه `Application` وبعدّله كلمة زيادة اسمها `Support` . محتاج طريقة تقول بيها للـ Shell "ده كله اسم واحد، متقطعوش". وهنا بييجي دور الـ Quotes.

المسافة بتفصل الـ Arguments (↩ 2.1 ص 17)، والرموز زي `$` و `*` ليها معنى. الـ Quotes بتقول للـ Shell "ده نص، متلعبش فيه".

الشكل	المتغيرات $X	$(...)	Glob *	الاستخدام
Single '...'	لأ	لأ	لأ	نص حرفي 100%
Double "..."	أيوه	أيوه	لأ	نص فيه متغيرات (الافتراضي الآمن)
Backslash \x	بيلغي معنى الحرف اللي بعده بس			my\ file.txt
من غير Quotes	أيوه + بيتقطع على المسافات	أيوه	أيوه	خطر مع المتغيرات

جدول 7.3 أنواع الـ Quoting

Terminal

```
user@MacBook ~ % name="Ahmed Ali"
user@MacBook ~ % echo 'Hi $name'
Hi $name
user@MacBook ~ % echo "Hi $name"
Hi Ahmed Ali
user@MacBook ~ % echo "Price: \$5"
Price: $5
```

تشريح الأمر

```
rm "$file"
```

1 `rm` امسح.

2 `"$file"` الـ `$` = هات قيمة المتغير، والـ Double quotes بتخلي القيمة **Argument واحد** حتى لو فيها مسافات. لو `file="my notes.txt"` وكتبت `rm $file` من غير Quotes في **bash**، هيمسح ملفين اسمهم `my` و `notes.txt` ! (zsh مبيقطّعش المتغيرات افتراضياً فهتعدي معاك في الـ Terminal، وده بالظبط اللي بيخلّي الغلطة تستخبى لحد ما تكتب Script بـ bash).

القاعدة الذهبية

حط أي متغير جوه `"..."` دايماً: `"$var"` و `"$@"` . الاستثناء النادر هو لما تكون عايز التقطيع عن قصد. ودي أهم قاعدة في الـ Scripting (↩ 12.2 ص 144).

7.6 الـ Command Substitution: $()

كل يوم بتعمل Backup لمجلد، وعايز اسم الملف فيه تاريخ النهارده. هتكتب التاريخ بإيدك كل يوم؟ وتنسى يوم وتكتب فوق نسخة إمبارح؟ الأحلى إنك تخلي أمر `date` هو اللي يكتب التاريخ، وتحط ناتجه جوه الاسم. `$(cmd)` معناها: نفّذ الأمر ده، وحط **ناتجه** مكانه في السطر.

تشريح الأمر

```
tar -czf "backup-$(date +%F).tar.gz" ~/docs
 1    2        3          4          5
```

1. `tar -czf` اعمل أرشيف مضغوط (↩ 10.8 ص 123).
2. `"backup-` بداية اسم الملف، والـ Double quotes بتسمح بالـ `$()` جواها.
3. `$(date +%F)` نفّذ `date` بالشكل `%F` = `2026-10-22`، وحط الناتج هنا. الاسم النهائي: `backup-2026-10-22.tar.gz`.
4. `.tar.gz"` باقي الاسم.
5. `~/docs` اللي هيتأرشف.

الـ Backticks

`` `date` `` هي الشكل القديم لـ `$(date)`. بتشتغل، بس صعب تتداخل (`$(dirname $(pwd))` سهلة، وبالـ Backticks لازم `\``) وشكلها بيتلخبط مع الـ Single quote. استخدم `$()` دايماً. وفيه كمان `$((2+3))` بقوسين = حساب أرقام، مش أمر (↩ 12.2 ص 144).

7.7 الـ Exit Codes و && و || و ;

إنت كمبرمج JavaScript عارف الـ `try/catch` وإن الـ Function ممكن ترجع قيمة تقولك نجحت ولا لأ. الأوامر نفس الفكرة: كل أمر بيخلص بيسيب "ورقة صغيرة" فيها رقم بيقول نجح ولا فشل. ومحدش بيشوف الورقة دي غير لو سأل عليها. ودي اللي بتخلّي الأوامر تاخد قرارات: "لو البناء نجح شغّل المشروع، لو فشل ابعتلي رسالة".

كل أمر لما يخلص بيرجع رقم: **0 = نجح**، وأي رقم تاني (1 لـ 255) = فشل. الرقم ده بيتحفظ في المتغير الخاص `$?`.

Terminal

```
user@MacBook ~ % ls /etc/hosts
/etc/hosts
user@MacBook ~ % echo $?
0
user@MacBook ~ % ls /nope
ls: /nope: No such file or directory
user@MacBook ~ % echo $?
1
```

على Linux نفس الأمر رسالته `ls: cannot access '/nope'` والكود بتاعه 2. الرقم بالظبط بيختلف من أداة لأداة، المهم إنه مش صفر.

الكود	المعنى
`0`	نجاح
`1`	خطأ عام (grep: ملقاش حاجة)
`2`	استخدام غلط / Argument غلط
`126`	الملف موجود بس مش قابل للتشغيل (Permission، ↩ 6.1 ص 58)
`127`	command not found
`128+N`	اتقتل بـ Signal رقم N: `130` = Ctrl+C (2)، `137` = kill -9 (↩ 9.3 ص 100)

جدول 7.4 Exit codes متعارف عليها

الرمز	المعنى	مثال
`a ; b`	نفّذ a وبعدها b **مهما حصل**	`cd /tmp ; ls`
`a && b`	b بس لو a **نجح** (AND)	`make && ./app`
`a || b`	b بس لو a **فشل** (OR)	`ping -c1 x || echo down`
`a &`	a في الخلفية (↩ 9.4 ص 101)	`sleep 60 &`
`( a; b )`	في Subshell: الـ cd جوه ميأثرش بره	`(cd /tmp && ls)`
`{ a; b; }`	Group في نفس الـ Shell	`{ date; ls; } > out`

جدول 7.5 ربط الأوامر

تشريح الأمر

```
mkdir -p build && cd build || echo "failed"
```

1. `mkdir -p build` اعمل المجلد (↩ 5.3 ص 50).
2. `&&` لو نجح بس، كمّل.
3. `cd build` ادخله.
4. `||` لو اللي قبلها فشل (أي حتة في السلسلة)، نفّذ اللي بعدها.
5. `echo "failed"` رسالة الفشل. خد بالك: `a && b || c` مش if/else حقيقي؛ لو b نفسها فشلت، c هتتنفذ برضه.

ليه ; خطيرة أحياناً

`cd /old/project ; rm -rf *` : لو الـ cd فشل (مسار غلط)، الـ rm هيتنفذ في المجلد اللي إنت فيه دلوقتي! استخدم `&&` دايماً لما الأمر التاني معتمد على الأول (↩ 15.1 ص 182).

7.8 المتغيرات الخاصة

المتغير	يرجع
`$?`	Exit code آخر أمر
`$$`	الـ PID بتاع الـ Shell الحالي (↩ 9.1 ص 97)
`$!`	الـ PID بتاع آخر أمر اتبعت للخلفية بـ `&`
`$0`	اسم الـ Shell أو الـ Script
`$1` `$2` … و `$@` و `$#`	الـ Arguments بتاعة Script (↩ 12.6 ص 150)
`$_`	آخر Argument في الأمر اللي فات (زي `$!`)

جدول 7.6 متغيرات الـ Shell الخاصة

7.9 مشاكل شائعة

مشكلة 1 **الملف بقى فاضي بعد `sort file > file`**

كتبت `sort names.txt > names.txt` والملف اتمسح! لأن الـ Shell بيفتح `> names.txt` ويفضّيه **قبل** ما sort يبدأ يقرا.

الحل: اكتب في ملف تاني وبعدين انقله: `sort names.txt > tmp && mv tmp names.txt`، أو استخدم `sort -o names.txt names.txt` (↩ 8.6 ص 88)، أو `sponge` من `brew install moreutils`.

مشكلة 2 **الـ Script بيتكسر على أسماء فيها مسافات**

كل حاجة شغالة لحد ما ييجي ملف اسمه `My Photo.jpg`.

الحل: Quotes حوالين كل متغير `"$f"`، واستخدم `find -print0 | xargs -0` (↩ 8.9 ص 90).

ملخص الفصل

- 0 = stdin، 1 = stdout، 2 = stderr، و `/dev/null` ثقب أسود.
- `>` يمسح و `>>` يضيف، و `2>&1 > f` بالترتيب ده.
- `|` يوصل stdout بـ stdin، والأوامر بتشتغل مع بعض.
- الـ Glob بيوسعه الـ Shell قبل الأمر، و `{}` بيولد كلمات.
- `'...'` حرفي، و `"..."` بيفك المتغيرات، و `"$var"` دايماً.
- `$()` ناتج أمر، و `$?` الـ Exit code، و `&&` لو نجح و `||` لو فشل.

صح ولا غلط؟

1. `cmd 2>&1 > f` بيحط الأخطاء في الملف. صح / غلط
2. في `rm *.txt` ، الأمر rm هو اللي بيدور على الملفات. صح / غلط
3. `echo '$HOME'` بتطبع `$HOME` زي ما هي. صح / غلط
4. الـ Exit code 0 معناه إن الأمر فشل. صح / غلط

أسئلة مراجعة

1. اكتب أمر يشغّل `find /` ويحفظ النتائج في ملف ويرمي الأخطاء.
2. إيه الفرق بين `>` و `>>` و `|` ؟
3. اشرح ليه `rm $f` خطر و `rm "$f"` لأ.
4. إيه الفرق بين `a ; b` و `a && b` ؟ امتى كل واحد؟
5. إيه هي `$?` و `$$` و `$!` ؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل 10 ملفات `note1.txt` لـ `note10.txt` بأمر واحد، وبعدين اعرض اللي رقمها من 1 لـ 9 بس بـ Glob.
2. متوسط اكتب سطر واحد يعمل مجلد باسم تاريخ النهارده، ويدخله، ويكتب فيه ملف `info.txt` فيه اسمك والـ PID بتاع الـ Shell، ولو أي خطوة فشلت يطبع "failed".
3. صعب اعمل Here-doc بيكتب ملف فيه `$HOME` حرفياً في سطر، وقيمته الحقيقية في سطر تاني، في نفس الـ Here-doc.

08

المرحلة 3 · قوة الـ Shell

الفصل التامن

معالجة النصوص والبحث

تخيل إنك خلصت مشروع React فيه 300 ملف، وقبل ما ترفعه افتكرت إنك سايب console.log في حتت كتير. هتفتح الملفات واحد واحد؟ أو عندك Log فيه نص مليون سطر، وعايز تعرف أكتر 10 IPs عملوا طلبات فاشلة. في Excel ده ساعة، وفي الـ Terminal سطر واحد. الفصل ده بيشرح عُدّة معالجة النصوص: grep و sed و awk و cut و sort و uniq و wc و tr و tee و xargs، وبعدين find و mdfind (Spotlight) للبحث عن الملفات، وفي الآخر بنركبهم في Pipelines حقيقية. وهنا بالذات هتقابل فروقات الماك عن Linux كتير، فخلي عينك على صناديق "على Linux".

بعد الفصل ده هتكون عارف

- grep للبحث جوه الملفات بكل الـ Flags المهمة
- أساسيات الـ Regex المشتركة بين grep و sed و awk
- sed للاستبدال و awk للأعمدة
- cut و sort و uniq و wc و tr
- tee و xargs وإزاي يكملوا الـ Pipes
- find و mdfind (Spotlight) و locate والفرق بينهم وبين grep
- فروقات BSD المهمة: sed -i '' و grep -P و xargs
- Pipelines لتحليل لوجات حقيقية

طول الفصل هنشتغل على ملف لوج بسيط بالشكل ده (اسمه `access.log`، زي لوجات nginx):

access.log

```
192.168.1.10 - - [22/Oct/2026:09:14:02 +0300] "GET /index.html HTTP/1.1" 200 5120
10.0.0.7 - - [22/Oct/2026:09:14:05 +0300] "GET /admin HTTP/1.1" 403 210
192.168.1.10 - - [22/Oct/2026:09:15:11 +0300] "POST /login HTTP/1.1" 401 98
172.16.0.3 - - [22/Oct/2026:09:16:40 +0300] "GET /img/logo.png HTTP/1.1" 404 0
```

كل سطر أعمدة مفصولة بمسافات: **$1** الـ IP، و **$4** الوقت، و **$6** الـ Method، و **$7** المسار، و **$9** الـ Status code، و **$10** الحجم.

يعني إيه Method و Status code هنا؟

السطر ده بيسجّل كل زيارة للموقع. الـ **Method** هي نوع الطلب: `GET` = "هاتلي الصفحة"، و `POST` = "خد البيانات دي". والـ **Status code** رقم السيرفر رد بيه: `200` تمام، `404` الصفحة مش موجودة، `403` ممنوع، و `5xx` مشكلة في السيرفر. الشرح الكامل في كتاب الإنترنت (فصل 7 و 8)، بس ده كفاية علشان تفلتر الـ Log دلوقتي.

8.1 grep: دوّر على سطور

فاكر Cmd+F اللي بتدور بيه في أي صفحة؟ `grep` هو نفس الفكرة، بس بيدوّر في ملف واحد أو في ألف ملف مرة واحدة، وبيطبعلك كل سطر فيه الكلمة ومكانه بالظبط. خلينا نرجع لمشكلة الـ `console.log` اللي في أول الفصل، بس المرة دي هندوّر على كلمة `todo` اللي بنسيبها لنفسنا في الكود:

تشريح الأمر

```
grep -rniw --include="*.py" "todo" ~/project
```

1. `grep` — Global Regular Expression Print: اطبع كل سطر فيه النمط.
2. `-rniw` — أربعة مدموجين: `-r` (Recursive: دوّر في المجلد وكل اللي جواه، نفس معنى -r في cp و rm)، `-n` (اطبع رقم السطر)، `-i` (Ignore case: todo = TODO)، `-w` (Word: كلمة كاملة، فـ `todos` متتحسبش).
3. `--include="*.py"` — دوّر في ملفات Python بس. الـ Quotes مهمة عشان الـ Shell ميوسعش النجمة قبل grep (↩ 7.4 ص 75).
4. `"todo"` — النمط (Pattern). بيتعامل معاه كـ Regex.
5. `~/project` — فين يدوّر. لو مفيش مسار ومفيش -r، grep بيقرا من stdin (يعني بيشتغل في Pipe).

Flag	المعنى
`-i`	تجاهل حالة الحروف
`-v`	Invert: اعكس، السطور اللي **مفيهاش** النمط (`-v` هنا مش Verbose!)
`-c`	Count: عدد السطور المطابقة بس
`-n`	رقم السطر
`-l`	أسماء الملفات اللي فيها تطابق بس
`-r` / `-R`	Recursive (على Linux `-R` بيتبع الـ Symlinks، وعلى الماك الاتنين واحد تقريباً)
`-w` / `-x`	كلمة كاملة / السطر كله بالظبط
`-E`	Extended Regex: `( ) ? + |` من غير `\` (= `egrep` القديم)
`-F`	Fixed: النمط نص عادي مش Regex (أسرع، و `.` بتبقى نقطة)
`-o`	Only: اطبع الجزء المطابق بس مش السطر كله
`-A 3` / `-B 3` / `-C 3`	Context / Before / After: سطور بعد/قبل/حوالين التطابق
`-q`	Quiet: متطبعش حاجة، استخدم الـ Exit code بس (للـ Scripts، ↩ 7.7 ص 78)
`-P`	Perl Regex (زي `\d` والـ Lookahead). **مش موجود في grep بتاع الماك**: استخدم `-E`، أو واستخدم `brew install grep` `ggrep -P`، أو `rg`

جدول 8.1 أهم Flags بتاعة grep

v- بتختلف من أمر للتاني

في معظم الأوامر `-v` = Verbose (`cp -v` ، `mkdir -v` ، `tar -v`)، لكن في `grep` = Invert، وفي `sort -V` (كبيرة) = Version sort. ودي من أشهر المفاجآت. شوف جدول الـ Flags المشتركة (↩ 2.5 ص 21)، ولما تشك: `--help` .

Regex في دقيقتين

الرمز	المعنى	مثال
`.`	أي حرف واحد	`h.t` = hat ,hot
`*`	اللي قبلها 0 مرة أو أكتر (مش زي الـ Glob!)	`ab*c` = ac ,abbc
`+`	مرة أو أكتر (ERE)	`[0-9]+`
`?`	اختياري (ERE)	`colou?r`
`^` / `$`	بداية / نهاية السطر	`^#` سطور كومنت
`[abc]` و `[^abc]`	حرف من دول / مش من دول	`[0-9]`
`a|b`	دي أو دي (ERE)	`error|fail`
`(...)`	جروب (ERE)	`(ab)+`
`{n,m}`	عدد مرات (ERE)	`[0-9]{1,3}`
`\`	الغي معنى الرمز	`\.` نقطة حقيقية

جدول 8.2 أساسيات الـ Regular Expressions

Regex مشترك بين grep و sed و awk

نفس الرموز دي بتشتغل في الـ 3 أوامر (وفي `find -regex` و `less` بـ `/` و vim وبايثون). BRE (الافتراضي في grep و sed) محتاج `\(\)` ، `\+` `\|` ، و ERE (`grep -E` و `sed -E` و awk دايماً) من غير Backslash. ابدأ بـ `-E` على طول وريّح دماغك.

وخلي بالك: `*` في الـ Regex غير `*` في الـ Glob؛ والنمط حطه في Single quotes عشان الـ Shell ميلمسوش.

تشريح الأمر

```
grep -Eo '^[0-9]{1,3}(\.[0-9]{1,3}){3}' access.log
```

1. `grep` دوّر.
2. `-Eo` `-E` Regex موسّع، و `-o` اطبع الجزء المطابق بس (الـ IP مش السطر).
3. `'` Single quote: متخليش الـ Shell يلمس الرموز (↩ 7.5 ص 76).
4. `^` من أول السطر.
5. `[0-9]{1,3}` رقم من 1 لـ 3 خانات.
6. `(\.[0-9]{1,3}){3}` نقطة حقيقية (`\.`) ورقم، والجروب ده يتكرر 3 مرات.
7. `access.log` الملف.

8.2 sed: عدّل النص وهو ماشي

الموقع بتاعك اتنقل لـ HTTPS، وعندك ملف إعدادات فيه 40 لينك لسه بادئين بـ `http://`. في VS Code هتعمل Find & Replace، تمام. بس لو الملف على سيرفر، أو عايز تعمل ده في 50 ملف مرة واحدة، أو جوه Script بيشتغل لوحده؟ هنا بييجي دور `sed`: "Replace & Find" بالأوامر. وخلي بالك، ده من أكتر الأوامر اللي بتختلف بين الماك و Linux، فركز معايا في الـ `-i`:

تشريح الأمر

```
sed -i.bak -E 's/http:\/\//https:\/\//g' config.txt
```

1. `sed` Stream EDitor: بيعدّل كل سطر وهو بيعدي.
2. `-i.bak` In-place: عدّل الملف نفسه بدل ما تطبع، و `.bak` لازقة فيها = احتفظ بنسخة قديمة `config.txt.bak`. على الماك الـ `-i` **لازم** ياخد امتداد للنسخة، ولو مش عايز نسخة خالص تكتب `-i ''` (علامتين تنصيص فاضيين). والشكل `-i.bak` اللازق ده بيشتغل على الماك و Linux الاتنين، فهو الأضمن في الـ Scripts.
3. `-E` Extended Regex (نفس grep).
4. `s` Substitute: أمر الاستبدال. الشكل `s/القديم/الجديد/flags`.
5. `/http:\/\//` النمط القديم. الـ `/` هي الفاصل، فأي `/` جوه النص لازم تتكتب `\/`.
6. `https:\/\//` النص الجديد.
7. `g` Global: كل التطابقات في السطر، مش أول واحد بس.
8. `config.txt` الملف.

عايز	الماك (BSD)	(GNU) Linux
عدّل من غير نسخة	`sed -i '' 's/a/b/' f`	`sed -i 's/a/b/' f`
عدّل واحتفظ بنسخة	`sed -i .bak 's/a/b/' f` أو `-i.bak`	`sed -i.bak 's/a/b/' f`
يشتغل على الاتنين	`sed -i.bak 's/a/b/' f && rm f.bak`، أو `gsed -i` من `brew install gnu-sed`	

جدول 8.3 sed -i على الماك مقابل Linux

غيّر الفاصل وارتاح

الفاصل ممكن يبقى أي حرف: `sed 's|http://|https://|g'` أوضح بكتير من غابة الـ `\/`. وجرّب دايماً من غير `-i` الأول عشان تشوف النتيجة.

`sed -n '10,20p' f`	`-n` متطبعش تلقائي، و `p` اطبع السطور من 10 لـ 20 بس.
`sed '/^#/d' f`	`d` = Delete: امسح السطور اللي بتبدأ بـ `#`.
`sed '/^$/d' f`	امسح السطور الفاضية (`^$` = بداية ونهاية من غير حاجة بينهم).
`sed 's/a/b/2' f`	استبدل التطابق التاني بس في كل سطر.
`sed -E 's/([[:alnum:]]+) ([[:alnum:]]+)/\2 \1/' f`	`\1 \2` = اللي اتمسك في الجروب الأول والتاني: بيبدّل كلمتين. و `[[:alnum:]]` = حرف أو رقم، وده شغال في كل نسخ sed (أما `\w` فمش مضمون على الماك).

8.3 awk: لغة الأعمدة

لو `grep` هو Cmd+F، فـ `awk` هو Excel مصغّر جوه الـ Terminal. اسمه جاي من أسامي التلات علماء اللي عملوه سنة 1977 (Aho و Weinberger و Kernighan)، وهو في الحقيقة لغة برمجة صغيرة كاملة. بس إحنا هنستخدم أهم حاجة فيه: إنه بيشوف كل سطر كأنه صف في جدول. `awk` بيقسم كل سطر لأعمدة (`$1`، `$2` ... و `$0` = السطر كله) وبيخليك تكتب شروط وحسابات.

خلينا نجرّبه على ملف `/etc/passwd` اللي اتكلمنا عنه في ↩ 6.5 ص 63. على الماك الملف ده فيه مستخدمين النظام بس، وكل سطر شكله `_www:*:70:70:World Wide Web Server:/Library/WebServer:/usr/bin/false`، وفي أوله سطور تعليقات بتبدأ بـ `#`. عايزين نعرف مين من المستخدمين دول ليه Shell حقيقي يقدر يدخل بيه:

تشريح الأمر

```
awk -F: '!/^#/ && $7 ≠ "/usr/bin/false" { print $1, $7 }' /etc/passwd
```

1. `awk` اقرا سطر سطر، ونفّذ البرنامج اللي بين الـ Quotes على كل سطر.
2. `-F:` Field separator: الأعمدة مفصولة بـ `:` بدل المسافات.
3. `'` Single quotes: عشان `$7` هنا تبع awk مش متغير Shell.
4. `!/^#/` شرط أول: `/^#/` = Regex "السطر يبدأ بـ #"، و `!` قبله = "مش". يعني سيب التعليقات.
5. `&&` و (AND): الشرطين لازم يتحققوا.
6. `$7 ≠ "/usr/bin/false"` شرط تاني: العمود السابع (الـ Shell) مش `/usr/bin/false` (ده "Shell" وهمي بيقفل على طول، بيتحط للمستخدمين اللي مش مفروض يدخلوا).
7. `{ print $1, $7 }` الـ Action: اطبع الاسم والـ Shell. الـ `,` بتحط مسافة بينهم.
8. `/etc/passwd` الملف.

على Linux (السيرفرات)

على السيرفر الملف ده فيه كل المستخدمين، والمثال المشهور هو `awk -F: '$3 >= 1000 { print $1, $7 }' /etc/passwd` : العمود التالت (الـ UID) أكبر من أو يساوي 1000 = مستخدمين حقيقيين.

`awk '{print $NF}' f`	`NF` = Number of Fields، فـ `$NF` = آخر عمود.
`awk 'NR>1' data.csv`	`NR` = Number of Record (رقم السطر): كله ماعدا الهيدر.
`awk '{s+=$10} END {print s}' access.log`	اجمع العمود العاشر، و `END` = بعد آخر سطر اطبع المجموع.
`awk '$9==404 {print $7}' access.log`	المسارات اللي رجعت 404.

8.4 cut: قص الأعمدة

أحياناً مش محتاج كل قوة awk. عندك ملف CSV صدّرته من Google Sheets، وعايز العمود الأول والتالت بس. هنا `cut` هو المقص البسيط:

تشريح الأمر

```
cut -d',' -f1,3 users.csv
```

1. `cut` قص أجزاء من كل سطر.
2. `-d','` Delimiter: الفاصل فاصلة. (الافتراضي Tab، و cut مبيفهمش "مسافات كتير" كفاصل واحد، ساعتها استخدم awk).
3. `-f1,3` Fields: العمود الأول والتالت. وممكن Range: `-f2-4` . و `-c1-10` = أول 10 حروف.
4. `users.csv` الملف.

8.5 tr: بدّل الحروف

زميلك بعتلك ملف معمول على Windows، وأول ما فتحته في ال Terminal لقيت في آخر كل سطر `^M` غريبة، وال Script بتاعك بيبوظ بسببها. أو عايز تحوّل نص كله لحروف كبيرة. `tr` هو أداة "بدّل الحرف ده بالحرف ده" أو "امسح الحرف ده خالص":

تشريح الأمر

```
echo "Hello World" | tr 'a-z' 'A-Z'
```

1. `echo "Hello World"` النص.
2. `|` ودّيه لـ tr، لأن tr بيقرا من stdin **بس**، مبياخدش اسم ملف (استخدم `< file` ، ↩ 7.2 ص 73).
3. `tr` TRanslate: بدّل حروف بحروف.
4. `'a-z'` المجموعة الأولى: الحروف الصغيرة.
5. `'A-Z'` المجموعة التانية: كل حرف يقابل اللي في نفس مكانه. النتيجة `HELLO WORLD` .

`tr -d '\r' < win.txt > unix.txt`	`-d` = Delete: شيل ال Carriage return بتاع ملفات Windows.
`tr -s ' '`	`-s` = Squeeze: المسافات المتكررة تبقى واحدة (مفيد قبل cut).
`tr ':' '\n' <<< "$PATH"`	اعرض ال PATH كل مجلد في سطر (↩ 13.5 ص 163).

8.6 sort و uniq: رتّب وشيل المكرر

الاتنين دول زي الأخين اللي مبيمشوش غير مع بعض. `sort` يرتب، و `uniq` بيشيل المكرر. ولما تجمعهم بتقدر تجاوب على أسئلة زي "مين أكتر واحد بعت طلبات؟" و "أنهي صفحة أكتر صفحة بتفشل؟":

تشريح الأمر

```
1    2     3    4      5
sort -t',' -k3,3nr users.csv
```

1. `sort` رتّب السطور.
2. `-t','` الفاصل فاصلة (في cut اسمه `-d`، وفي sort اسمه `-t`، وفي awk `-F`. نفس الفكرة 3 أسماء!).
3. `-k3,3` Key: رتّب بالعمود التالت لحد التالت (من غيرها `-k3` = من التالت لآخر السطر).
4. `nr` لازقين في الـ Key: `n` Numeric (عشان 10 تيجي بعد 9 مش بعد 1) و `r` Reverse (الأكبر الأول).
5. `users.csv` الملف.

الأمر	Flag	المعنى
sort	`-n` / `-h`	أرقام / أحجام Human (K2 قبل M1)
sort	`-r`	اعكس الترتيب
sort	`-u`	Unique: شيل المكرر
sort	`-o file`	اكتب الناتج في ملف (حتى لو نفس ملف الدخل)
uniq	(من غير)	شيل السطور المكررة **المتجاورة** بس
uniq	`-c`	Count: حط عدد التكرار قبل كل سطر
uniq	`-d` / `-u`	المكرر بس / اللي ملوش تكرار بس

جدول 8.4 Flags بتاعة sort و uniq

> ⚠ **uniq محتاج sort قبله**
>
> `uniq` بيقارن كل سطر باللي قبله بس، فلو التكرارات متفرقة مش هيشوفها. عشان كده هتشوف دايماً `sort | uniq -c` مع بعض.

8.7 wc: عِد السطور والكلمات

تشريح الأمر

```
wc -lwc notes.txt
```

1. `wc` — Word Count: عِد.
2. `-lwc` — `-l` سطور، و `-w` كلمات، و `-c` Bytes (و `-m` حروف، مهمة للعربي لأن الحرف العربي 2 Bytes في UTF-8).
3. `notes.txt` — الملف. ولو من stdin (`ls | wc -l`) مش هيطبع اسم ملف.

Terminal

```
user@MacBook ~ % wc -l notes.txt
      42 notes.txt
user@MacBook ~ % wc -l < notes.txt
      42
```

على الماك `wc` بيحط مسافات قبل الرقم. لو هتستخدم الرقم في Script وعايزه نضيف: `wc -l < notes.txt | tr -d ' '`.

8.8 tee: شوف واحفظ في نفس الوقت

تشريح الأمر

```
./build.sh 2>&1 | tee -a build.log
```

1. `./build.sh 2>&1` — شغّل الـ Script وخلي الأخطاء تمشي مع الناتج (↩ 7.2 ص 73).
2. `|` — Pipe.
3. `tee` — زي حرف T في السباكة: بيكتب الداخل على الشاشة **وفي ملف** في نفس الوقت.
4. `-a` — Append: ضيف على الملف بدل ما تمسحه (زي `>>`).
5. `build.log` — الملف. وعشان tee برنامج مستقل، ينفع تشغله بـ sudo: (`| sudo tee /etc/x`) (↩ 6.6 ص 66).

8.9 xargs: من سطور لـ Arguments

هنا مشكلة غريبة شوية: `find` طلعلك لستة بـ 50 ملف `.tmp`، وعايز تمسحهم. فكتبت `find . -name "*.tmp" | rm` ومحصلش حاجة. ليه؟ بعض الأوامر مبتقراش من stdin، بتاخد Arguments بس (`rm` و `mkdir` و `echo`). `xargs` بيحوّل السطور اللي داخلاله لـ Arguments.

تشريح الأمر

```
find . -name "*.tmp" -print0 | xargs -0 rm -v
```

1. `find . -name "*.tmp"` هات كل ملفات tmp (تحت).
2. `-print0` افصل الأسماء بـ Null character (`\0`) بدل سطر جديد، عشان الأسماء اللي فيها مسافات ومعاها سطور جديدة متتقطعش.
3. `|` Pipe.
4. `xargs` خد اللي جاي من stdin وحطه Arguments للأمر اللي بعده.
5. `-0` الأسماء مفصولة بـ Null (لازم يمشي مع `-print0`).
6. `rm -v` الأمر اللي هيتنفذ: `rm -v ./a.tmp ./b.tmp …` .

على Linux (السيرفرات)

على Linux، `xargs` بيشغّل الأمر مرة واحدة حتى لو مفيش ولا اسم داخل، فبتلاقي الناس كاتبة `xargs -0 -r rm -v` : الـ `-r` = No-run-if-empty. الـ xargs بتاع الماك أصلاً مبيشغّلش الأمر لو مفيش مدخلات، فالـ `-r` ملهاش لازمة عندك، ومتعتمدش عليها في Script هيشتغل على الماك لأن نسخ كتير منه بترفضها بـ `illegal option` . لو محتاج سطر يشتغل في الاتنين، استخدم لوب `while read` (↩ 12.4 ص 148) أو `find ... -delete` .

`xargs -n 1`	Argument واحد لكل مرة تشغيل.
`xargs -I{} cp {} /backup/`	`-I{}` : حط كل سطر مكان `{}` (بيشغل مرة لكل سطر).
`xargs -P 4`	Parallel: شغّل 4 في نفس الوقت.

8.10 find و mdfind: دوّر على ملفات

فاكر لما كنت عايز تلاقي كل ملفات الـ Log الكبيرة اللي واكلة المساحة؟ أو كل مجلدات `node_modules` المنسية في مشاريع قديمة؟ الـ Finder بيدوّر بالاسم وخلاص، لكن `find` بيدوّر بأي حاجة: الاسم، والحجم، والتاريخ، والنوع، والصلاحيات، وكمان بيعمل حاجة على كل ملف بيلاقيه:

تشريح الأمر

```
find ~/projects -type f -name "*.log" -size +10M -mtime -7 -exec ls -lh {} \;
```

1. `find` دوّر على ملفات بخصائصها (الاسم، الحجم، التاريخ...) بالمشي في المجلدات فعلاً.
2. `~/projects` نقطة البداية (لازم تيجي الأول، و `.` = هنا). على الماك المسار ده **إجباري**: `find -name x` من غير مسار هتطلعلك Error (على Linux بتشتغل وتعتبره `.`).
3. `-type f` ملفات عادية بس (`d` مجلدات، `l` Links). في find الشروط كلمة كاملة بشَرطة واحدة.
4. `-name "*.log"` الاسم بيطابق الـ Glob ده، والـ Quotes إجبارية عشان find هو اللي يفهم النجمة مش الـ Shell. و `-iname` من غير حالة الحروف.
5. `-size +10M` أكبر من 10 ميجا (`+` أكبر، و `-` أصغر، ومن غيرهم = بالظبط).
6. `-mtime -7` Modified time: اتعدّل في آخر 7 أيام (`-mmin -30` = آخر 30 دقيقة).
7. `-exec ls -lh {}` نفّذ أمر على كل ملف لقيته، و `{}` = مكان اسم الملف.
8. `\;` نهاية الـ `-exec`. الـ `\` عشان الـ Shell ميفهمش `;` إنها فاصل أوامر. ولو كتبت `+` بدلها، find هيجمّع الملفات كلها في أمر واحد (أسرع).

الشرط	المعنى
`-name` / `-iname` / `-path`	بالاسم / بالاسم من غير حالة / بالمسار كله
`-type f|d|l`	النوع
`-size +1G`	الحجم (`k M G`)
`-mtime` / `-mmin` / `-newer f`	وقت التعديل
`-user ahmed` / `-perm 777`	المالك / الصلاحيات (↩ 6.2 ص 59)
`-maxdepth 2`	متنزلش أكتر من مستويين (حطها بدري)
`-empty`	ملفات أو مجلدات فاضية
`-o` / `! -name x`	OR / NOT (و AND هو الافتراضي)
`-delete`	امسح اللي لقيته (خطر: جرّب من غيرها الأول)
`-print0`	للاستخدام مع `xargs -0`

جدول 8.5 أهم شروط find

mdfind: الـ Spotlight من الـ Terminal

الماك عنده ميزة مش موجودة على Linux: الـ **Spotlight** (اللي بتفتحه بـ Cmd+Space) طول الوقت شغال في الخلفية وعامل فهرس بكل ملفاتك، وكمان بمحتواها. والأمر `mdfind` بيخليك تسأل الفهرس ده من الـ Terminal، فالنتيجة بتطلع لحظياً:

تشريح الأمر

```
mdfind -onlyin ~/projects -name package.json
```

1. `mdfind` — Metadata find: دوّر في فهرس ال Spotlight.
2. `-onlyin ~/projects` — دوّر جوه المجلد ده بس (من غيرها بيدوّر في الجهاز كله).
3. `-name package.json` — بالاسم. ومن غير `-name` بيدوّر في **المحتوى** كمان: `mdfind "API_KEY"` هيجيبلك كل ملف فيه الكلمة دي.

وفيه أخوه `mdls file.pdf` اللي بيعرض كل ال Metadata بتاعة ملف (عدد الصفحات، أبعاد الصورة، اتنزل منين)، و `mdutil -s /` بيقولك الفهرسة شغالة ولا لأ.

	`find`	`grep -r`	`mdfind`
بيدوّر على	ملفات بخصائصها	**محتوى** جوه الملفات	بالاسم أو المحتوى أو ال Metadata
بيقرا	شجرة الملفات حالاً	محتوى كل ملف	فهرس Spotlight الجاهز
السرعة	متوسطة	أبطأ (بيقرا كل حاجة)	لحظي
محدّث؟	دايماً	دايماً	تقريباً لحظي، بس مبيفهرسش كل حاجة (زي المجلدات المخفية وبعض مجلدات النظام)
استخدمه لما	"الملفات الكبيرة اللي اتعدلت امبارح"	"فين السطر اللي فيه API_KEY في المشروع"	"فين ملف package.json بتاع المشروع القديم"

جدول 8.6 find مقابل grep مقابل mdfind

على Linux: locate

تشريح الأمر

```
locate -i nginx.conf
```

1. `locate` — دوّر في **قاعدة بيانات** فيها أسماء كل الملفات (سريع جداً لأنه مبيمشيش في القرص). على التوزيعات الحديثة الحزمة اسمها `plocate`.
2. `-i` — من غير حالة الحروف.
3. `nginx.conf` — جزء من الاسم أو المسار. القاعدة بتتحدّث يومياً، فالملفات الجديدة أوي مش هتظهر غير بعد `sudo updatedb`.

الماك عنده `locate` برضه، بس قاعدة البيانات بتاعته مش متفعّلة افتراضياً، و `mdfind` أحسن منه في كل حاجة تقريباً.

بدائل حديثة

`rg` (ripgrep) بديل أسرع لـ `grep -r` وبيتجاهل اللي في `.gitignore` وبيفهم Regex زي `\d`، و `fd` بديل أبسط لـ find. سطّبهم بـ `brew install ripgrep fd` (↩ 11.1 ص 129). وعلى Ubuntu الحزمة اسمها `fd-find` والأمر `fdfind`.

8.11 Pipelines حقيقية لتحليل اللوجات

دلوقتي جه وقت نجمع كل اللي اتعلمناه. تخيل إن موقعك بقى بطيء فجأة، ومديرك بيسألك: "مين اللي بيضرب السيرفر بالطلبات دي؟". الإجابة سطر واحد، بس كل حتة فيه أمر من الأوامر اللي فاتت. اقراه من الشمال لليمين كأنه قصة:

تشريح الأمر

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
```

1	`awk '{print $1}' access.log`	طلّع العمود الأول (الـ IP) من كل سطر.
2	`| sort`	رتّبهم عشان المتشابه يبقى جنب بعض.
3	`| uniq -c`	اجمع المتكرر وحط عدده قدامه: `58 192.168.1.10`.
4	`| sort -rn`	رتّب بالرقم، الأكبر الأول.
5	`| head -10`	أول 10 بس (↩ 5.5 ص 53). النتيجة: أكتر 10 IPs بيبعتوا طلبات.

`awk '{print $9}' access.log | sort | uniq -c | sort -rn`	توزيع الـ Status codes (كام 200 وكام 404...).
`awk '$9 ~ /^5/' access.log | wc -l`	عدد أخطاء السيرفر (أي كود بيبدأ بـ 5). الـ `~` = يطابق Regex.
`awk '$9==404 {print $7}' access.log | sort | uniq -c | sort -rn | head`	أكتر صفحات مش موجودة الناس بتطلبها.
`grep "22/Oct/2026:09" access.log | wc -l`	عدد الطلبات في الساعة 9.
`tail -f access.log | grep --line-buffered " 500 "`	راقب الأخطاء لايف. `--line-buffered` عشان grep يطبع كل سطر فوراً وهو في Pipe.
`log show --last 1h --style compact | grep -i error`	نفس الفكرة على Logs النظام في الماك (على Linux: `journalctl --since today`، ↩ 9.5 ص 103).

8.12 مشاكل شائعة

مشكلة 1 grep بيطلع سطور مش عايزها لما أدور على 1.2.3.4

النقطة في الـ Regex = أي حرف، فـ `1.2.3.4` هتطابق `1a2b3c4` كمان.

الحل: `grep -F "1.2.3.4"` (نص ثابت) أو هرّب النقط `'1\.2\.3\.4'`.

مشكلة 2 sed: 1: "config.txt": invalid command code c

نسخت من شرح Linux `sed -i 's/a/b/' config.txt` وطلعلك الرسالة الغريبة دي. اللي حصل إن `sed` بتاع الماك افتكر إن `'s/a/b/'` هو امتداد النسخة (قيمة الـ `-i`)، وإن `config.txt` هو الأمر اللي هينفذه، فأول حرف فيه `c` اتقرا كأمر sed!

الحل: `sed -i '' 's/a/b/' config.txt` على الماك، أو `sed -i.bak ...` اللي بيشتغل في الاتنين (↩ 2.6 ص 23).

مشكلة 3 grep: P -- invalid option

نسخت `grep -oP '\d+'` من Stack Overflow. الـ `-P` مش موجود في grep بتاع الماك.

الحل: `grep -oE '[0-9]+'`، أو `brew install grep` واستخدم `ggrep -oP`، أو `rg -o '\d+'`.

مشكلة 4 find: paths must precede expression

كتبت `find . -name *.txt` وفي المجلد ملفين txt، فالـ Shell وسّع النجمة وبقى الأمر `find . -name a.txt b.txt`.

الحل: Quotes دايماً حوالين النمط: `-name "*.txt"` (↩ 7.4 ص 75).

ملخص الفصل

- `grep` سطور، و `sed` استبدال، و `awk` أعمدة وحسابات، و `cut` قص سريع.
- الـ Regex مشترك بينهم، واستخدم `-E` والـ Single quotes.
- `sort | uniq -c | sort -rn | head` = "الأكتر تكراراً"، أشهر Pipeline في الدنيا.
- `tee` شاشة + ملف، و `xargs` سطور ← Arguments.
- `find` بالخصائص وحالاً (والمسار إجباري على الماك)، و `mdfind` من فهرس Spotlight لحظياً، و `grep -r` بالمحتوى.
- على الماك: `sed -i ''` (أو `-i.bak` للاتنين)، ومفيش `grep -P`، و `xargs` مش محتاج `-r`.

صح ولا غلط؟

1. `grep -v` معناها Verbose. صح / غلط
2. `uniq` لوحده بيشيل كل التكرارات في الملف حتى لو متفرقة. صح / غلط
3. على الماك، `sed -i 's/a/b/' f` هتشتغل زي Linux بالظبط. صح / غلط
4. في awk، `$NF` هو آخر عمود. صح / غلط

؟ أسئلة مراجعة

1. اكتب أمر يطلع عدد السطور اللي فيها "error" (بأي حالة) في كل ملفات `.log` في مجلد.
2. استبدل كل `localhost` بـ `127.0.0.1` في ملف مع الاحتفاظ بنسخة.
3. إيه الفرق بين `-d` في cut و `-t` في sort و `-F` في awk؟
4. ليه بنستخدم `-print0` مع `xargs -0` ؟
5. إمتى تستخدم find وإمتى mdfind وإمتى grep؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اطبع أسماء كل المستخدمين في `/etc/passwd` مترتبين أبجدياً (من غير سطور التعليقات).
2. متوسط من `access.log` : أكتر 5 مسارات طلبتها IPs رجعلها 403 أو 401.
3. صعب لاقي أكبر 10 ملفات في الـ home بتاعك، واعرض حجمهم بشكل مقروء، من غير ما تشوف أي Permission denied.

09

المرحلة ٤ · النظام والشبكة

الفصل التاسع

العمليات والخدمات

الماك بقى بطيء فجأة، والمروحة شغالة كأنها طيارة، و Chrome مش راضي يقفل. فبتفتح Activity Monitor وتلاقي لستة طويلة بأسامي غريبة ومش عارف مين فيهم المذنب. أو شغّلت سيرفر Node في الـ Terminal، وقفلت التابة، والسيرفر مات معاها. الفصل ده عن العمليات (Processes): إزاي تشوفها من الـ Terminal، وتعرف مين واكل الـ CPU والرامات، وتبعتلها Signals، وتشغلها في الخلفية. وبعدين الخدمات اللي بتشتغل لوحدها على الماك بـ launchd و launchctl و brew services، وإزاي تقرا الـ Logs بـ log show و log stream. ومعاهم المقابل على السيرفرات: systemctl و journalctl.

بعد الفصل ده هتكون عارف

- يعني إيه Process و PID و PPID، وليه launchd هو PID 1 على الماك
- الـ Signals وجدول أشهرها (والأرقام اللي بتختلف بين الماك و Linux)
- kill و killall و pkill وإمتى -9
- & و jobs و fg و bg و Ctrl+Z و nohup و caffeinate
- launchd و launchctl و LaunchAgents و brew services
- log show و log stream، ومقارنتهم بـ systemctl و journalctl

9.1 ps: صورة للعمليات دلوقتي

افتح **Activity Monitor** (من Spotlight: Cmd+Space واكتب Activity). اللستة الطويلة اللي قدامك دي كل البرامج اللي شغالة على الماك دلوقتي: Chrome ومعاه عشرين "Google Chrome Helper"، و Spotify، و VS Code، وحاجات تانية كتير إنت مشغلتهاش بنفسك. كل سطر في اللستة دي اسمه **Process**: برنامج "عايش" وشغال في الذاكرة دلوقتي. والـ Terminal بيوريك نفس اللستة دي، بس بتفاصيل أكتر، وتقدر تفلترها وترتبها وتتحكم فيها بالأوامر.

كل برنامج شغال = **Process** ليه رقم فريد **PID**، وليه "أب" اتولد منه (**PPID**)، زي ما شفنا في fork/exec (↩ 1.2 ص 10). وأول Process في الماك كله، الجد الكبير اللي كل العمليات اتولدت منه، اسمه `launchd` ورقمه PID 1. (على Linux الجد ده اسمه `systemd`).

تشريح الأمر

```
ps aux -m | head -6
1  2  3     4
```

1. `ps` — Process Status: صورة (Snapshot) للعمليات في اللحظة دي.
2. `aux` — من غير شَرطة! ده أسلوب BSD القديم (والماك BSD أصلاً): `a` عمليات كل المستخدمين، و `u` عرض User-oriented (أعمدة المستخدم والـ CPU والـ MEM)، و `x` كمان العمليات اللي ملهاش Terminal (الخدمات).
3. `-m` — رتّب بالـ Memory (الأكبر الأول). و `-r` بدلها = رتّب بالـ CPU.
4. `| head -6` — الهيدر + أكتر 5 (↩ 7.3 ص 74).

Terminal

```
user@MacBook ~ % ps aux -m | head -3
USER       PID  %CPU %MEM      VSZ    RSS   TT  STAT STARTED      TIME COMMAND
ahmed     4120  12.4  9.8 44123456 812340   ??  S     8:02AM   4:11.32 /Applications/Google
Chrome.app/Contents/MacOS/Google Chrome
_mysql     901   1.1  4.2 41901232 350112   ??  Ss    7:58AM   0:40.10
/opt/homebrew/opt/mysql/bin/mysqld
```

العمود	المعنى
`PID`	رقم العملية (اللي هتبعتله kill)
`%CPU` / `%MEM`	نسبة المعالج والرامات
`VSZ` / `RSS`	الذاكرة الافتراضية / الحقيقية في الرامات (الأهم)، بالـ Kilobytes
`TT`	الـ Terminal المربوطة بيه (زي `s000` = `/dev/ttys000`)، و `??` = مفيش، يعني خدمة (↩ 1.1 ص 8). على Linux اسم العمود `TTY` والـ "مفيش" بتبقى `?`
`STAT`	الحالة: `R` شغال، `S` نايم (أقل من 20 ثانية)، `I` Idle نايم من زمان، `U` مستني حاجة ومبيتقتلش (على Linux اسمها `D`)، `Z` Zombie، `T` متوقف، و `s` Session leader، و `+` في الـ Foreground
`COMMAND`	الأمر اللي شغّلها

جدول 9.1 أعمدة ps aux

`ps -ef`	نفس الفكرة بأسلوب Unix/POSIX (بشرطة): `-e` كل العمليات، و `-f` Full format (فيه PPID). شغال على الماك و Linux.
`ps -o pid,ppid,command -p 1234`	`-o` اختار الأعمدة، و `-p` عملية معينة. (على Linux العمود اسمه `cmd` أو `command`).

`pgrep -lf node`	هات الـ PIDs اللي فيها node، و `-l` اعرض الاسم، و `-f` دوّر في الأمر كامل. أنضف من `ps aux | grep node`. خلي بالك: على Linux `pgrep -a` بتعرض الأمر كامل، لكن على الماك `-a` معناها حاجة تانية (ضيف العمليات الأب).
`pstree -p 1`	العمليات كشجرة أب وأبناء. مش جاي مع الماك: `brew install pstree`.

الـ Terminal و Activity Monitor بيتكلموا عن نفس الحاجة

جرّب: شغّل `sleep 500` في تابة، وافتح Activity Monitor، ودوّر على `sleep`: هتلاقيه بنفس الـ PID اللي طلعلك في `pgrep -l sleep`. دبل كليك عليه هيوريك الـ Parent Process، وهتلاقيه zsh بتاعك. ولو دوست **Quit** من Activity Monitor، ده نفس `kill` اللي جاي (و **Force Quit** = `kill -9`).

9.2 top و htop: مراقبة لايف

الـ `ps` صورة فوتوغرافية، لقطة واحدة وخلاص. لكن لو الماك بيسخن دلوقتي وعايز تتفرج على مين بيعمل كده لحظة بلحظة، محتاج "فيديو" مش صورة. وده `top`: Activity Monitor جوه الـ Terminal، والأهم إنه بيشتغل كمان على أي سيرفر هتدخله بـ SSH ومفيهوش شاشة أصلاً:

تشريح الأمر

```
top -o cpu -U ahmed
```

1. `top` — جدول بيتحدث كل ثانية فيه الحمل والرامات والعمليات.
2. `-o cpu` — Order: رتّب بالـ CPU. على الماك اسم المفتاح بحروف صغيرة: `cpu` أو `mem` أو `pid` أو `time`. (على Linux بيتكتب `-o %CPU`).
3. `-U ahmed` — عمليات المستخدم ده بس. على الماك الحرف **كبير**؛ الـ `-u` الصغيرة في top بتاع الماك معناها حاجة تانية (رتّب بالـ CPU). على Linux العكس: `top -u ahmed`.

تشريح الأمر

```
top -l 1 -n 5 -o mem
```

1. `-l 1` — Logging mode: خد عينة واحدة (Sample 1) واطبعها وخلّص، بدل الشاشة التفاعلية. مفيد جداً في الـ Scripts أو تبعته لـ Pipe.
2. `-n 5` — Number: اعرض أول 5 عمليات بس.
3. `-o mem` — رتّب بالرامات.

المفتاح	بيعمل
`o` ثم اكتب `cpu` أو `mem`	غيّر الترتيب (على Linux: `P` / `M` على طول)
`S`	ابعت Signal لـ PID (على Linux: `k`)
`U`	اعرض مستخدم معين
`?`	المساعدة، فيها كل المفتاتيح
`q`	اخرج

جدول 9.2 مفاتيح جوه top على الماك

load average وعدد الـ Cores

السطر الأول فيه `Load Avg: 2.52, 2.80, 3.10` = متوسط عدد العمليات اللي عايزة CPU في آخر 1 و 5 و 15 دقيقة. قارنه بعدد الـ Cores: `sysctl -n hw.ncpu` على الماك (أو `nproc` على Linux). لو 8 Cores والـ Load = 8، يبقى الجهاز مشغول 100%. وتشوفه بسرعة بـ `uptime` . و `htop` (`brew install htop`) نفس الفكرة بألوان وماوس وشجرة (`F5`)، و `F9` للـ Kill، وبيشتغل بنفس الشكل على الماك والسيرفر. وللذاكرة على الماك تحديداً: `memory_pressure` بيقولك الضغط على الرامات طبيعي ولا لأ، زي اللون الأخضر والأصفر في Activity Monitor.

9.3 الـ Signals و kill

لما تدوس `Ctrl+C` على برنامج شغال، إيه اللي بيحصل؟ إنت مش بتقطع عنه الكهربا. إنت بتبعتله "رسالة" بتقوله "لو سمحت وقّف". والبرنامج المحترم بيستلم الرسالة، ويحفظ شغله، ويقفل بأدب. والرسايل دي اسمها **Signals**، وفيه منها أنواع كتير، كل واحد ليه معنى.

وعشان كده `kill` اسمها مضلل: هي مش بتقتل، هي بتبعت **Signal** (رسالة) للعملية، والعملية تقرر تعمل إيه (ماعدا KILL و STOP، دول النظام بينفذهم غصب عنها).

الماك	Linux	الاسم	بيتبعت إمتى	المعنى	تتقفش؟
`1`	`1`	`SIGHUP`	قفلت الـ Terminal	Hang up، وخدمات كتير بتفهمها "اقرا الإعدادات تاني"	أيوه
`2`	`2`	`SIGINT`	`Ctrl+C`	Interrupt: وقّف لو سمحت	أيوه
`3`	`3`	`SIGQUIT`	`Ctrl+\`	اقفل واعمل Core dump	أيوه
`9`	`9`	`SIGKILL`	`kill -9` / Force Quit	موت فوراً، من غير تنظيف	**لأ**
`15`	`15`	`SIGTERM`	`kill` (الافتراضي) / Quit	اقفل بأدب: احفظ ونضّف واخرج	أيوه
`17`	`19`	`SIGSTOP`	`kill -STOP`	اتجمد	**لأ**

الماك	Linux	الاسم	بيتبعت إمتى	المعنى	تتقفش؟
18	20	SIGTSTP	Ctrl+Z	وقّف مؤقتاً (Terminal stop)	أيوه
19	18	SIGCONT	bg / fg	كمّل بعد التوقف	أيوه

جدول 9.3 أشهر الـ Signals (الأرقام على الماك وعلى Linux)

لاحظ إن الأرقام من 17 لـ 19 مختلفة بين الماك و Linux! عشان كده **استخدم الأسماء مش الأرقام**: `kill -STOP` شغالة في الاتنين، أما `kill -19` على الماك هتعمل CONT بدل STOP. اعرضهم كلهم بـ `kill -l`. وافتكر إن الـ Exit code بتاع عملية اتقتلت بـ Signal = 128 + رقمها (↩ 7.7 ص 78).

تشريح الأمر

```
kill -TERM 4120
```

1. `kill` ابعت Signal (أمر Builtin في zsh و bash، وفيه نسخة `/bin/kill` كمان، ↩ 1.3 ص 11).
2. `-TERM` أي Signal. ينفع `-15` أو `-SIGTERM` أو `-s TERM`. ومن غيرها = TERM أصلاً.
3. `4120` الـ PID (من ps أو pgrep). ممكن أكتر من واحد.

`pkill -f "node server.js"`	ابعت TERM لأي عملية `-f` (Full) الأمر بتاعها كله فيه النص ده.
`killall "Google Chrome"`	كل العمليات اللي اسمها **بالظبط** كده (والـ Quotes علشان المسافة). على الماك والـ Linux آمنة. (على Solaris القديم killall بتقتل كل حاجة!).
`kill -HUP $(pgrep -o nginx)`	قول للـ Master بتاع nginx يقرا الإعدادات (`-o` = Oldest). `$()` من ↩ 7.6 ص 77.
`kill -0 1234`	Signal صفر = متبعتش حاجة، بس قولي العملية موجودة ولا لأ (Exit code).

الترتيب الصح

ابدأ بـ `kill PID` (TERM) واستنى ثواني. لو مقفلتش، ساعتها `kill -9`. الـ 9 مبتديش فرصة للبرنامج يحفظ ويقفل ملفاته، وممكن تسيب بيانات بايظة (خصوصاً قواعد البيانات). وده نفس الفرق بين Quit و Force Quit في الماك.

9.4 الخلفية: & و jobs و fg و bg و nohup

شغّلت `npm run dev` والـ Terminal "اتحجز": مش قادر تكتب أي أمر تاني لحد ما توقفه. طب لو عايز السيرفر يفضل شغال وإنت تكمل شغلك في نفس التابة؟ أو شغّلت حاجة هتاخد ساعة وعايز تقفل الـ Terminal وتمشي؟ هنا بييجي دور الخلفية (Background):

تشريح الأمر

```
1      2              3      4  5
nohup python3 server.py > server.log 2>&1 &
```

1. `nohup` — No Hang Up: شغّل الأمر وخليه يتجاهل SIGHUP، فميموتش لما تقفل الـ Terminal أو الـ SSH يقطع.
2. `python3 server.py` — الأمر نفسه.
3. `> server.log` — الناتج لملف (لو مكتبتهاش، nohup بيكتب في `nohup.out`).
4. `2>&1` — والأخطاء لنفس الملف (↩ 7.2 ص 73).
5. `&` — في الآخر: شغّله في الخلفية ورجّعلي الـ Prompt فوراً. الـ Shell بيطبع `[1] 5531` = رقم الـ Job والـ PID، والـ PID بيتحفظ في `$!` (↩ 7.8 ص 80).

Terminal

```
user@MacBook ~ % sleep 300
^Z
zsh: suspended  sleep 300
user@MacBook ~ % bg %1
[1]  + continued  sleep 300
user@MacBook ~ % jobs -l
[1]  + 5602 running    sleep 300
user@MacBook ~ % fg %1
[1]  + continued  sleep 300
```

ده شكل zsh. في bash نفس الأوامر بتطبع `[1]+ Stopped sleep 300` و `[1]+ Running`.

الأمر	بيعمل
`cmd &`	شغّل في الخلفية من الأول
`Ctrl+Z`	وقّف اللي شغال قدامك مؤقتاً (SIGTSTP)
`jobs -l`	اعرض الـ Jobs في الـ Shell ده، و `-l` معاها الـ PID
`bg %1`	كمّل Job رقم 1 في الخلفية
`fg %1`	رجّعه قدامك (Foreground). `%+` = آخر واحد، و `%-` اللي قبله
`kill %1`	الـ `%` معناها رقم Job مش PID
`disown %1`	افصل الـ Job عن الـ Shell (زي nohup بعد ما بدأ)

جدول 9.4 التحكم في الـ Jobs

zsh بيقتل الـ Jobs لما تقفل

لو كتبت `exit` وفيه Jobs شغالة، zsh هيقولك `you have running jobs`، ولو كتبت `exit` تاني هيبعتلهم HUP ويموتوا. علشان كده استخدم `nohup` أو `disown` لأي حاجة عايزها تفضل عايشة.

caffeinate: متنامش يا ماك!

شغلة هتاخد ساعتين (Build طويل، أو بتنزل ملف كبير)، والماك نام بعد 10 دقايق ووقّفها؟ حط `caffeinate` قبل الأمر: `caffeinate -i npm run build`. الـ `-i` = امنع الـ Idle sleep طول ما الأمر ده شغال، وأول ما يخلص الماك يرجع لطبيعته. و `caffeinate -d` كمان بتمنع الشاشة تطفي (↩ 14.3 ص 171).

الأحسن من nohup

للشغل الطويل على سيرفر، `tmux` أحسن بكتير: تقفل وترجع تلاقي كل حاجة زي ما هي (↩ 14.6 ص 177). وللخدمات الدائمة، اعملها خدمة حقيقية (الجزء الجاي).

9.5 الخدمات على الماك: launchd و launchctl و brew services

تخيل كده: إنت بتتعلم Backend، وسطّبت PostgreSQL عشان مشروعك. كل يوم الصبح تفتح الماك، وتفتح الـ Terminal، وتكتب أمر طويل عشان تشغل الداتابيز، وتسيب التابة دي مفتوحة طول اليوم. ولو قفلتها بالغلط، التطبيق بتاعك يقع. ده تعب ملوش لازمة. اللي إنت عايزه حاجة تشغّل Postgres **لوحدها** أول ما الماك يفتح، وتشغّلها تاني لو وقعت، وتكتب أخطاءها في ملف تقدر ترجعله.

الحاجة دي على الماك اسمها **launchd**: نفس الجد الكبير PID 1 اللي شفناه في أول الفصل. هو اللي بيشغل كل حاجة على الماك تقريباً: الـ Dock، و Spotlight، و Wi-Fi، وأي خدمة إنت تضيفها. وكل خدمة بيتوصف شغلها في ملف صغير اسمه **plist** (Property List)، وهو ملف XML فيه "اسم الخدمة إيه، وتشغّل إيه، وإمتى". والأداة اللي بتكلم بيها launchd اسمها `launchctl`.

الطريق السهل: brew services

قبل ما ندخل في الـ plist والتفاصيل، الخبر الحلو: لو الخدمة متسطبة من Homebrew (Postgres و MySQL و Redis و nginx...)، Homebrew بيكتبلك الـ plist ويسجله مع launchd بأمر واحد (↩ 11.3 ص 131):

تشريح الأمر

```
brew services start postgresql@17
```

1. `brew services` — الجزء من Homebrew اللي بيدير الخدمات. هو في الحقيقة "واجهة لطيفة" فوق launchctl.
2. `start` — شغّلها دلوقتي **وكمان** مع كل مرة تعمل Login (زي `systemctl enable --now` على Linux). لو عايزها تشتغل مرة واحدة بس من غير ما تتسجل: `brew services run`.
3. `postgresql@17` — اسم الـ Formula بالظبط زي ما سطبتها (الـ `@17` = نسخة 17).

```
Terminal
user@MacBook ~ % brew services list
Name           Status  User  File
nginx          none
postgresql@17  started ahmed ~/Library/LaunchAgents/homebrew.mxcl.postgresql@17.plist
redis          error 1 ahmed ~/Library/LaunchAgents/homebrew.mxcl.redis.plist
```

الأمر	بيعمل
`brew services list`	كل الخدمات وحالتها: `started` شغالة، `none` مش متسجلة، `error` وقعت ومعاها الـ Exit code
`start` / `stop`	شغّل وسجّل مع الـ Login / وقّف وشيل التسجيل
`restart`	وقّف وشغّل (بعد ما تعدّل الإعدادات)
`run`	شغّل دلوقتي بس، من غير تسجيل
`info redis`	تفاصيل خدمة واحدة: الـ PID والـ plist وفين اللوج
`sudo brew services start nginx`	كـ root: تشتغل مع الـ Boot نفسه قبل أي Login (الـ plist بيروح `/Library/LaunchDaemons`). محتاجها لو عايز بورت أقل من 1024

جدول 9.5 أوامر brew services

طب لو `redis` طلعت `error`؟ أول حاجة: اللوج. خدمات Homebrew غالباً بتكتب في `/opt/homebrew/var/log/`، ف `tail -f /opt/homebrew/var/log/redis.log` (↩ 5.5 ص 53). ولو مفيش هناك، `brew services info redis` بيقولك فين.

تحت الغطا: الـ plist و LaunchAgents و LaunchDaemons

طب لو عايز تشغّل حاجة **إنت** كاتبها؟ Script بيعمل Backup كل ساعة، أو سيرفر Node صغير. هنا محتاج تكتب الـ plist بإيدك. والأول لازم تعرف هتحطه فين، لأن المكان هو اللي بيحدد "مين" بيشغّله و"إمتى":

المكان	النوع	بيشتغل إمتى وبصلاحية مين
`~/Library/LaunchAgents`	Agent بتاعك	لما **إنت** تعمل Login، بصلاحياتك (مكانك الطبيعي، من غير sudo)
`/Library/LaunchAgents`	Agent لكل المستخدمين	لما أي حد يعمل Login، بصلاحياته (برامج زي Zoom بتحط حاجات هنا)
`/Library/LaunchDaemons`	Daemon	مع الـ Boot، قبل أي Login، كـ root (محتاج sudo)
`/System/Library/Launch*`	بتاعة Apple	محمية بـ SIP، متلمسهاش (↩ 6.8 ص 69)

جدول 9.6 أماكن ملفات launchd

الفرق بين **Agent** و **Daemon** ببساطة: الـ Agent "موظف عندك إنت"، بيشتغل جوه جلستك وبيقدر يكلم الشاشة والـ Clipboard. الـ Daemon "موظف عند النظام"، بيشتغل في الخلفية من قبل ما حد يفتح، ومالوش علاقة بأي مستخدم. للشغل اليومي كمطوّر، 99% من الوقت إنت محتاج **LaunchAgent** في `~/Library/LaunchAgents` .

خلينا نعمل واحد: Script بيكتب التاريخ في ملف كل 5 دقايق. الملف اسمه لازم يبقى بالشكل "الدومين بالمقلوب" (Reverse DNS) زي `com.ahmed.hello.plist` :

~/Library/LaunchAgents/com.ahmed.hello.plist

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>             <string>com.ahmed.hello</string>
  <key>ProgramArguments</key>
  <array>
    <string>/bin/sh</string>
    <string>-c</string>
    <string>date >> /Users/ahmed/hello.log</string>
  </array>
  <key>RunAtLoad</key>         <true/>
  <key>StartInterval</key>     <integer>300</integer>
  <key>StandardErrorPath</key> <string>/Users/ahmed/hello.err</string>
</dict>
</plist>
```

المفتاح	معناه
`Label`	اسم الخدمة الفريد، ولازم يطابق اسم الملف. ده اللي هتستخدمه في كل أوامر launchctl
`ProgramArguments`	الأمر وكل Argument في `<string>` لوحده. **مفيش Shell**، فمفيش `~` ولا `>>` ولا `$HOME` ؛ علشان كده لفّينا الأمر في `/bin/sh -c` ، وكتبنا المسار كامل
`RunAtLoad`	شغّلها أول ما تتحمّل (يعني مع الـ Login)
`StartInterval`	كررها كل كام ثانية (300 = 5 دقايق). أو `StartCalendarInterval` لوقت معين زي cron (↩ 12.8 ص 152)
`KeepAlive`	`<true/>` = لو وقعت، شغّلها تاني على طول (للسيرفرات)
`StandardErrorPath` / `StandardOutPath`	فين يروح الناتج والأخطاء. من غيرهم الناتج بيضيع
`EnvironmentVariables` / `WorkingDirectory`	الفولدر اللي تشتغل فيه، والمتغيرات. افتكر إن launchd مش بيقرا `~/.zshrc` ، فال `PATH` بتاعك مش موجود (↩ 13.5 ص 163)

جدول 9.7 أهم مفاتيح الـ plist

اتأكد إن الملف سليم قبل ما تسجله: `plutil -lint ~/Library/LaunchAgents/com.ahmed.hello.plist` لازم يقول `OK`. وبعدين سجّله مع launchd:

تشريح الأمر

```
launchctl bootstrap gui/$(id -u) ~/Library/LaunchAgents/com.ahmed.hello.plist
   1         2          3                    4
```

1. `launchctl` أداة التحكم في launchd.
2. `bootstrap` حمّل الـ plist ده وسجّله (ولو فيه `RunAtLoad` هيشتغل فوراً). وعكسها `bootout`.
3. `gui/$(id -u)` الـ **Domain**: "جلسة الشاشة بتاعة المستخدم رقم كذا". `id -u` بيرجع رقمك (غالباً 501، ↩ 6.5 ص 63). للـ Daemons الـ Domain اسمه `system` ومعاه sudo.
4. `~/Library/...plist` مسار الملف.

الأمر	بيعمل
`launchctl list | grep ahmed`	هل متسجلة؟ العمود الأول PID (أو `-` لو مش شغالة دلوقتي)، والتاني آخر Exit code
`launchctl print gui/$(id -u)/com.ahmed.hello`	كل حاجة عنها: الحالة، آخر Exit code، عدد مرات التشغيل، المسارات (زي `systemctl status`)
`launchctl kickstart -k gui/$(id -u)/com.ahmed.hello`	شغّلها دلوقتي، و `-k` اقتل النسخة الشغالة الأول (= restart)
`launchctl bootout gui/$(id -u)/com.ahmed.hello`	وقّفها وشيل تسجيلها (لحد ما تعمل bootstrap تاني أو Login)
`launchctl disable gui/$(id -u)/com.ahmed.hello`	امنعها تتحمّل خالص حتى مع الـ Login، و `enable` عكسها
`launchctl load -w file.plist` / `unload`	الأوامر القديمة. لسه شغالة وهتلاقيها في شروحات كتير، بس Apple بتعتبرها Legacy

جدول 9.8 أوامر launchctl الحديثة

⚠ عدّلت الـ plist ومفيش حاجة اتغيرت؟

launchd بيقرا الملف مرة واحدة وقت الـ bootstrap. أي تعديل بعد كده محتاج `bootout` وبعدين `bootstrap` تاني (مفيش `daemon-reload` زي Linux). وأشهر سبب إن خدمتك بتقع بـ Exit code `127` = "Command not found": إنك كتبت `node` بدل `/opt/homebrew/bin/node`، لأن launchd معندوش الـ PATH بتاعك.

System Settings بيوريك نفس الحاجة

افتح **System Settings ← General ← Extensions & Login Items**، تحت *Allow in the Background* هتلاقي كل ال Agents وال Daemons اللي البرامج سجلتها، وتقدر تقفلها من هناك. ولما تعمل bootstrap لأول مرة، الماك ممكن يبعتلك Notification "Background Items Added". ده طبيعي، وده الماك بيحميك من البرامج اللي بتسجل نفسها من وراك.

9.6 اللوجات على الماك: log show و log stream

الخدمة وقعت، أو ال Wi-Fi بيفصل كل شوية، أو برنامج بيقفل لوحده. أول سؤال أي حد خبير هيسألهولك: "اللوج بيقول إيه؟". على الماك، معظم البرامج والنظام نفسه بيكتبوا في مكان واحد ضخم اسمه **Unified Logging**، وبتقراه بأمر واحد: `log`. (والواجهة الرسومية بتاعته هي برنامج **Console**).

المشكلة إن اللوج ده فيه **آلاف** الرسايل في الدقيقة. لو كتبت `log stream` لوحده هتشوف شلال مبيقفش. فالسر كله في إنك تفلتر بال `--predicate`:

تشريح الأمر

```
log stream --predicate 'process == "nginx"' --level info
```

1	`log stream`	اعرض الرسايل الجديدة لايف أول ما تتكتب (زي `tail -f`، ↩ ص 53). `Ctrl+C` للخروج.
2	`--predicate '...'`	الفلتر: شرط مكتوب بلغة شبه SQL. لاحظ ال Single quotes بره وال Double جوه (↩ ص 76).
3	`process == "nginx"`	الرسايل اللي جاية من البرنامج ده بس. ممكن كمان `subsystem == "com.apple.wifi"`.
4	`--level info`	الافتراضي بيعرض default وأعلى؛ `info` بتزوّد التفاصيل، و `debug` أكتر وأكتر.

تشريح الأمر

```
log show --last 1h --predicate 'eventMessage CONTAINS[c] "error"' --style compact
```

1	`log show`	اقرا اللي **اتكتب قبل كده** (مش لايف).
2	`--last 1h`	آخر ساعة. ينفع `30m` أو `2d`، أو `--start "2026-10-22 09:00:00"`.
3	`eventMessage CONTAINS[c] "error"`	نص الرسالة فيه كلمة error، و `[c]` = Case-insensitive (زي `grep -i`).
4	`--style compact`	سطر قصير لكل رسالة بدل الشكل الطويل. (وفيه `json` لو هتعالجه ببرنامج).

`log show --last 5m --predicate 'process == "node"' | tail -20`	أي برنامج بيكتب في Unified Log تقدر تفلتره بالاسم. وزي أي أمر، ينفع تكمّله بـ Pipe (↩ ص 94 8.11).
`log show --last boot | head`	من أول آخر Boot (مفيد لو الماك عمل Restart لوحده).
`ls ~/Library/Logs/DiagnosticReports`	تقارير ال Crash: كل برنامج وقع ليه ملف `.ips` هنا فيه آخر لحظاته.

`tail -f /opt/homebrew/var/log/nginx/error.log`	برامج كتير (خصوصاً من Homebrew) لسه بتكتب في ملفات نصية عادية. لو الـ `log` مطلعش حاجة، دوّر عليها.

f- و u- بمعاني متكررة

`-f` = Follow في `tail -f` و `journalctl -f` و `docker logs -f`، بس = Force في `rm -f` و `cp -f` و = File في `tar -f` و = Full في `pgrep -f`. و `-u` = User في ps و sudo و crontab (و `top -u` على Linux)، بس = Unit في journalctl و = Unique في sort، و `top -u` على الماك حاجة تالتة خالص. السياق هو اللي بيحدد (↩ 2.5 ص 21).

على Linux: systemctl و journalctl

على Linux (السيرفرات)

لما تدخل سيرفر Linux بـ SSH، مفيش launchd ولا brew services. الخدمات (nginx، ssh، docker) بيديرها **systemd** في أغلب توزيعات Linux الحديثة (Ubuntu و Debian و Fedora و Arch). كل خدمة اسمها **Unit** وملفها بينتهي بـ `.service` (هو المقابل للـ plist، بس بصيغة INI بسيطة بدل XML). ونفس الأفكار اللي اتعلمتها فوق موجودة بأسماء تانية: الجدول ده خريطتك، والشرح الكامل تحته.

الفكرة	الماك	Linux
PID 1	`launchd`	`systemd`
ملف وصف الخدمة	`.plist` (XML)	`.service` (INI)
خدماتك من غير sudo	`~/Library/LaunchAgents`	`~/.config/systemd/user` + `systemctl --user`
خدمات النظام	`/Library/LaunchDaemons`	`/etc/systemd/system`
شغّل وسجّل	`brew services start` / `launchctl bootstrap`	`systemctl enable --now`
الحالة	`launchctl print` / `brew services info`	`systemctl status`
إعادة تشغيل	`launchctl kickstart -k`	`systemctl restart`
بعد تعديل الملف	`bootout` ثم `bootstrap`	`systemctl daemon-reload`
اللوج لايف	`log stream --predicate`	`journalctl -u NAME -f`
اللوج القديم	`log show --last 1h`	`journalctl --since "1 hour ago"`

جدول 9.9 launchd على الماك مقابل systemd على Linux

تشريح الأمر

```
sudo systemctl enable --now nginx
```

1. `sudo` محتاج root لأنك بتغير النظام (↩ 6.6 ص 66).
2. `systemctl` أداة التحكم في systemd.
3. `enable` شغّلها تلقائياً مع كل Boot (بتعمل Symlink، ↩ 4.5 ص 43).
4. `--now` وكمان شغّلها دلوقتي (= start + enable).
5. `nginx` اسم الـ Unit (الـ `.service` اختيارية).

الأمر	بيعمل
`systemctl status nginx`	شغالة؟ من إمتى؟ PID وآخر سطور لوج (مش محتاج sudo)
`start` / `stop` / `restart`	شغّل / وقّف / وقّف وشغّل
`reload`	اقرا الإعدادات من غير ما تقفل (لو الخدمة بتدعمه)
`enable` / `disable`	مع الـ Boot ولا لأ
`is-active` / `is-enabled`	إجابة كلمة واحدة + Exit code (للـ Scripts)
`list-units --type=service --state=failed`	الخدمات اللي فشلت
`daemon-reload`	بعد ما تعدّل ملف service.
`systemctl --user ...`	خدماتك الشخصية من غير sudo

جدول 9.10 أوامر systemctl

تشريح الأمر

```
journalctl -u nginx -f --since "1 hour ago" -p err
```

1. `journalctl` اقرا اللوجات اللي systemd بيجمعها (الـ Journal).
2. `-u nginx` Unit: لوجات الخدمة دي بس.
3. `-f` Follow: كمّل اعرض الجديد لايف، **نفس** `tail -f` (↩ 5.5 ص 53).
4. `--since "1 hour ago"` من ساعة. ينفع `today` أو `"2026-10-22 09:00"`.
5. `-p err` Priority: الأخطاء وأخطر بس.

`journalctl -b`	لوجات الـ Boot الحالي، و `-b -1` اللي قبله (مفيد لو الجهاز وقع).

`journalctl -xe`	`-x` شرح إضافي، و `-e` روح للآخر. أول حاجة بعد ما خدمة تفشل.
`journalctl --disk-usage`	اللوجات واكلة قد إيه، و `--vacuum-time=2weeks` امسح الأقدم.

مش كل الأنظمة systemd

الماك launchd زي ما شفنا، و Alpine بيستخدم OpenRC (`rc-service`)، وجوه Docker Containers غالباً مفيش systemd خالص. و WSL2 بيدعم systemd بس لازم يتفعّل في `/etc/wsl.conf` .

9.7 مشاكل شائعة

مشكلة 1 **Address already in use: البورت مشغول**

شغّلت السيرفر قالك البورت 3000 مستخدم، لأن نسخة قديمة لسه شغالة في الخلفية (غالباً في تابة نسيتها).

الحل: اعرف مين ماسكه: `lsof -nP -iTCP:3000 -sTCP:LISTEN` (↩ 10.9 ص 125) وخد الـ PID واعمله `kill` . أو في سطر واحد: `kill $(lsof -t -iTCP:3000 -sTCP:LISTEN)` ، والـ `-t` بتطبع الـ PID بس.

مشكلة 2 **البورت 5000 أو 7000 مشغول ومفيش حاجة شغالة!**

شغّلت Flask أو سيرفر على 5000، وقالك مشغول، و `lsof` بيقولك إن اللي ماسكه اسمه `ControlCenter` . ده مش فيروس: ده **AirPlay Receiver** في الماك، بيسمع على 5000 و 7000.

الحل: يا إما غيّر البورت بتاعك (`flask run --port 5001`)، يا إما اقفل AirPlay Receiver من System Settings ← General ← Handoff & AirDrop.

مشكلة 3 **العملية مش بتموت حتى بـ kill -9**

لو حالتها `U` (على Linux) `D` فهي مستنية قرص أو Network share أو USB واقع، ولو `Z` فهي ميتة أصلاً (Zombie) ومستنية أبوها يقرا الـ Exit code بتاعها.

الحل: لـ Zombie: اقتل أو أعد تشغيل الأب (`ps -o ppid= -p PID`). لـ U/D: صلّح الجهاز/الشبكة اللي مستنياها (افصل الـ Disk بـ `diskutil unmount force`) أو Restart.

مشكلة ٤ **قتلت عملية ورجعت تاني لوحدها**

عملت `kill` لبرنامج، وبعد ثانية لقيته شغال تاني بـ PID جديد.

الحل: ده معناه إن launchd بيشغله بـ `KeepAlive`، فهو بيصحيه كل ما يموت. متقتلوش، وقّفه من عند launchd: `brew services stop NAME` أو `launchctl bootout gui/$(id -u)/LABEL`. تعرف الـ Label من `launchctl list | grep NAME`.

ملخص الفصل

- كل عملية ليها PID و PPID، والجد الكبير على الماك `launchd` (PID 1). `ps aux -m` صورة و `top -o cpu` / `htop` لايف، و Activity Monitor نفس الكلام بالماوس.
- `kill` بيبعت Signal: TERM (15) الأول، و KILL (9) آخر حل. واستخدم الأسماء مش الأرقام لأن 17-19 بتختلف بين الماك و Linux.
- `&` للخلفية، و `Ctrl+Z` + `fg` / `bg`، و `nohup` عشان متموتش مع الـ Terminal، و `caffeinate -i` عشان الماك مينامش.
- `brew services start/stop/list` أسهل طريق للخدمات، وتحتها launchd والـ plist في `~/Library/LaunchAgents` و `launchctl bootstrap/bootout/kickstart/print`.
- `log stream` / `log show` مع `--predicate` للوجات. وعلى Linux: `systemctl` و `journalctl -u -f`.

صح ولا غلط؟

1. `kill PID` من غير رقم Signal بتبعت SIGKILL. صح / غلط
2. البرنامج يقدر يتجاهل SIGTERM بس مايقدرش يتجاهل SIGKILL. صح / غلط
3. launchd بيقرا `~/.zshrc` قبل ما يشغّل الـ Agent، فالـ PATH بتاعك متاح جوه الـ plist. صح / غلط
4. `Ctrl+Z` بيقفل البرنامج نهائياً. صح / غلط

أسئلة مراجعة

1. إزاي تعرف أكتر 5 عمليات واكلين رامات على الماك؟ (بطريقتين)
2. إيه الفرق بين `kill` و `pkill` و `killall`؟
3. بدأت أمر طويل ونسيت الـ `&`، إزاي تبعته للخلفية من غير ما توقفه؟
4. Redis متسطب من Homebrew ومش شغال بعد الـ Restart، إيه الأوامر اللي هتستخدمها عشان تعرف وتصلح؟
5. إيه الفرق بين LaunchAgent و LaunchDaemon، وكل واحد بيتحط فين؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل شغّل `sleep 600 &` مرتين، واعرضهم بـ jobs، واقتل التاني برقم الـ Job.
2. متوسط اكتب سطر واحد يطبع "running" لو فيه عملية اسمها Finder، و "stopped" لو لأ (من غير grep).
3. صعب اعمل LaunchAgent (`~/Library/LaunchAgents/com.YOU.hello.plist`) بيكتب التاريخ في ملف كل دقيقة، وسجّله، واتأكد إنه شغال بـ `launchctl print`، وبعدين شيله. (ولو عندك سيرفر Linux: اعمل نفس الفكرة بـ systemd user service).

10

المرحلة 4 · النظام والشبكة

الفصل العاشر

الشبكات والاتصال بالسيرفرات

قاعد في كافيه، والماك واصل بال Wi-Fi، وعلامة النت مليانة، بس الموقع اللي شغال عليه مش بيفتح. تعمل Refresh عشر مرات. تقفل ال Wi-Fi وتفتحه. ولا حاجة. طب المشكلة فين؟ في الكافيه؟ في ال DNS؟ في السيرفر؟ في البورت؟ المحترفين مبيخمنوش، بيمشوا بالترتيب ويسألوا كل طبقة سؤال. والفصل ده هيعلمك الترتيب ده، وبعدين هياخدك للخطوة اللي هتعملها قريب جداً كمطوّر: إنك تأجر أول سيرفر (VPS) وتدخل عليه من الماك بـ ssh بالمفاتيح، وتنقل ملفاتك بـ scp و rsync و tar، وتعرف مين فاتح أنهي بورت بـ lsof.

بعد الفصل ده هتكون عارف

- ping و ifconfig و networksetup لتشخيص الاتصال على الماك
- curl و wget وكل Flags بتاعتهم المهمة
- DNS بـ dig و scutil و dscacheutil، و mDNS والـ local.
- ssh بالمفاتيح (Ed25519) وملف ~/.ssh/config والـ Keychain
- نقل الملفات بـ scp و rsync (و openrsync بتاع الماك) و tar
- البورتات و lsof على الماك (و ss على Linux)

كلمات هتقابلها في الفصل ده (والشرح الكامل في كتاب الإنترنت)

الفصل ده بيستخدم كلمات شبكات ممكن تكون أول مرة تسمعها، فخد معناها في سطر، والحكاية كاملة في **كتاب الإنترنت والويب و DevTools** (الكتاب 03):

- **IP**: رقم عنوان الجهاز على الشبكة، زي `192.168.1.23` (كتاب الإنترنت فصل 3).
- **Port**: رقم "الباب" اللي البرنامج بيسمع عليه جوه الجهاز، زي `3000` أو `443`. الـ IP = العمارة، والـ Port = رقم الشقة.
- **DNS**: اللي بيحوّل `google.com` لـ IP (فصل 5 هناك).
- **Request / Response**: إنت بتبعت طلب فيه **Method** (زي GET = هات، POST = خد) و **Headers** (معلومات على الظرف) و **Body** (المحتوى)، والسيرفر يرد برقم اسمه **Status code** (200 = تمام، 404 = مش موجود، 500 = السيرفر وقع). فصول 7 لـ 9 هناك.
- **HTTPS / TLS**: التشفير اللي بيحمي الكلام (فصل 12 هناك). و **Token**: "كارنيه" نصي بيثبت إنت مين، وبيتبعت في Header اسمه Authorization.
- **JSON**: شكل نصي للبيانات شبه الـ Object في JavaScript: `{"name": "Ali", "age": 30}`.

مش لازم تفهمهم 100% دلوقتي. نفّذ الأوامر، ولما تخلص كتاب الإنترنت ارجع للفصل ده، هتلاقيه بقى سهل جداً.

10.1 التشخيص: ping و ip

خلينا نرجع للكافيه. الدكتور الشاطر مبيقولش "إنت عيان" وخلاص، بيكشف بالترتيب: الحرارة، الضغط، النبض. والشبكة نفس الكلام، فيه "سلم" وبتطلعه درجة درجة، وأول درجة تفشل هي مكان المشكلة:

1. **عندي IP أصلاً؟** (الماك واخد عنوان من راوتر الكافيه؟)
2. **أوصل للراوتر؟**
3. **أوصل للنت؟** (`ping 8.8.8.8`)
4. **ال DNS شغال؟** (الاسم بيتحول لـ IP؟)
5. **السيرفر بيسمع على البورت؟** (`curl`)

أول أداة على السلم هي `ping` : زي ما تنادي على حد في أوضة تانية "إنت هناك؟" وتستنى يرد، وتقيس هو رد في قد إيه:

تشريح الأمر

```
ping -c 4 -W 2000 8.8.8.8
```

1. `ping` ابعت رسايل ICMP صغيرة واستنى الرد، وقيس الوقت.
2. `-c 4` Count: 4 مرات بس. على الماك و Linux من غيرها بيفضل شغال لحد `Ctrl+C` (على Windows بيبعت 4 افتراضياً).
3. `-W 2000` Wait: استنى الرد لكل رسالة 2000 **ملي ثانية** (ثانيتين) بالكتير. خلي بالك: على الماك الرقم بالملي ثانية، وعلى Linux بالثواني (`-W 2`). لو كتبت `-W 2` على الماك هيستنى 2 ملي ثانية بس وكل حاجة هتبان فاشلة! ولو عايز حد أقصى للأمر كله: `-t 5` (ثواني).
4. `8.8.8.8` الهدف (IP أو اسم). لو IP شغال والاسم لأ، المشكلة في ال DNS.

طب الدرجتين الأولانيين؟ "عندي IP؟" و"الراوتر فين؟". على الماك كل كارت شبكة ليه اسم قصير: ال Wi-Fi غالباً `en0` ، وال Ethernet أو ال Adapter `en1` أو أكتر، و `lo0` هو الماك بيكلم نفسه (`127.0.0.1`). وعشان تتأكد مين مين:

تشريح الأمر

```
networksetup -listallhardwareports
```

1. `networksetup` أداة الماك الرسمية لإعدادات الشبكة من ال Terminal (هي نفسها اللي ورا System Settings ← Network). كتير من أوامرها محتاجة sudo لو هتغيّر حاجة.
2. `-listallhardwareports` اعرض كل كروت الشبكة: الاسم اللي إنت تعرفه (Wi-Fi) قصاد اسم الجهاز (`en0`) وال MAC address.

Terminal

```
user@MacBook ~ % networksetup -listallhardwareports
Hardware Port: Wi-Fi
Device: en0
Ethernet Address: a4:83:e7:12:34:56

user@MacBook ~ % ipconfig getifaddr en0
192.168.1.23
```

`ipconfig getifaddr en0`	ال IP بتاعي على ال Wi-Fi، سطر واحد ومن غير زحمة. (مش `ipconfig` بتاع Windows، نفس الاسم بس أداة تانية).
`ifconfig en0`	كل تفاصيل الكارت: `inet` = IPv4، و `inet6` = IPv6، و `status: active` . على الماك ifconfig لسه هي الأداة الأساسية.
`route -n get default`	ال Default gateway (الراوتر) فين: سطر `gateway: 192.168.1.1` . و `netstat -rn` بيعرض جدول ال Routing كله.
`networksetup -getinfo Wi-Fi`	ال IP وال Router وال Subnet بتوع ال Wi-Fi، وهل جايين من DHCP ولا يدوي.
`traceroute google.com`	الطريق خطوة خطوة لحد السيرفر (أو `mtr` لايف بعد `brew install mtr` ، و `tracert` على Windows).
`networkQuality`	Speed test رسمي من Apple جوه ال Terminal: سرعة الرفع والتنزيل وال Responsiveness. (بال Q كبيرة).

على Linux: الأداة اسمها ip

على أغلب السيرفرات الحديثة `ifconfig` مش متسطبة أصلاً، وبدلها `ip` :

`ip -br addr`	عناويني إيه؟ `-br` = Brief: سطر لكل كارت. (بديل `ifconfig` القديم). أسامي الكروت هناك زي `eth0` أو `ens3` .
`ip route`	ال Default gateway (الراوتر) فين: السطر اللي يبدأ بـ `default via` .

ping مش دليل

سيرفرات كتير بتقفل ICMP، فممكن ping يفشل والموقع شغال عادي. الاختبار الأدق للخدمة نفسها هو `curl` للبورت (تحت).

10.2 curl و wget: كلّم السيرفرات

إنت كمطوّر JavaScript أكيد كتبت `fetch("https://api.example.com/users")` قبل كده. طب لو ال API مش راجع البيانات اللي متوقعها، المشكلة في الكود بتاعك ولا في السيرفر نفسه؟ أسرع طريقة تعرف: تسأل السيرفر بنفسك، من غير متصفح ولا كود. وده `curl` : هو حرفياً `fetch` بس من ال Terminal، وجاي جاهز مع الماك. وكل Flag فيه بيقابل حاجة إنت عارفها: `-X` هي ال `method` ، و `-H` هي ال `headers` ، و `-d` هي ال `body` .

وهتشوف curl كمان في كل صفحة "Install" تقريباً، زي سطر تثبيت Homebrew نفسه (↩ 11.3 ص 131). فخلينا نفهم الشكل المشهور ده حرف حرف:

تشريح الأمر

```
curl -fsSL -o get.sh https://example.com/install.sh
```

1. `curl` — Client URL: ابعت طلب لـ URL واطبع الرد على stdout. بيدعم HTTP و HTTPS و FTP وغيرهم.
2. `-fsSL` — أربعة مدموجين ومشهورين جداً: `-f` Fail (لو السيرفر رجع 404 أو 500 افشل بـ Exit code بدل ما تطبع صفحة الخطأ)، و `-s` Silent (من غير شريط التقدم)، و `-S` Show error (بس اطبع الأخطاء رغم -s)، و `-L` Location (اتبع الـ Redirects).
3. `-o get.sh` — Output: احفظ في الملف ده بدل الشاشة. (و `-O` كبيرة = احفظ بنفس اسم الملف اللي في الـ URL).
4. `https://example.com/install.sh` — الـ URL. لو فيه `&` أو `?` حطه في Quotes (↩ 7.5 ص 76).

Flag	المعنى	مثال
`-I`	Head: الـ Headers بس	`curl -I https://site.com`
`-i`	Include: الـ Headers + الـ Body	
`-v`	Verbose: كل تفاصيل الاتصال (TLS و Headers)	للتشخيص
`-X POST`	الـ Method	`-X DELETE`
`-H "..."`	Header	`-H "Authorization: Bearer $TOKEN"`
`-d '...'`	Data: Body (وبيخلي الطلب POST تلقائياً)	`-d 'name=ali'`
`--json '...'`	JSON body + الـ Headers المناسبة (curl 7.82+)	`--json '{"a":1}'`
`-u user:pass`	Basic auth	
`-k`	تجاهل أخطاء الشهادة (للتجربة بس!)	
`-w '%{http_code}'`	اطبع معلومة بعد الطلب	تحت
`-C -`	كمّل تحميل اتقطع	

جدول 10.1 أهم Flags بتاعة curl

تشريح الأمر

```
curl -s -o /dev/null -w '%{http_code}\n' https://example.com
```

1. `curl -s` — من غير شريط تقدم.
2. `-o /dev/null` — ارمي الـ Body في الثقب الأسود (↩ 7.1 ص 72).
3. `-w '%{http_code}\n'` — Write-out: اطبع الـ Status code بس وسطر جديد. أسرع فحص "الموقع شغال؟".
4. `https://example.com` — الـ URL.

تشريح الأمر

```
wget -c -O ubuntu.iso https://releases.ubuntu.com/ubuntu.iso
```

1. `wget` — Web get: متخصص في التحميل (curl متخصص في الطلبات). مش جاي مع الماك: `brew install wget`. ولو مش عايز تسطبه، `curl -C - -L -o ubuntu.iso URL` بيعمل نفس الشغلانة.
2. `-c` — Continue: كمّل التحميل من مكان ما وقف.
3. `-O ubuntu.iso` — اسم الملف. خد بالك: `-O` الكبيرة في wget = اسم ملف تختاره، وفي curl = استخدم اسم الـ URL. نفس الحرف بمعنيين!
4. `https://...` — الـ URL (هنا مسار توضيحي).

o- و O- و v- بين الأوامر

`-o` = Output في curl و sort (`sort -o`) و gcc، و `-v` = Verbose في curl و ssh و rsync و cp، و `-q` = Quiet في wget و grep و apt. ولما تشوف Flag مش فاهمه في سكربت حد، `man curl` ثم `/` واكتب الـ Flag (↩ 2.4 ص 20).

10.3 الـ DNS: dig و nslookup

الكمبيوترات بتكلم بعض بالأرقام (`142.250.185.78`)، وإحنا بنحفظ أسامي (`google.com`). اللي بيترجم بينهم هو الـ **DNS**: دليل التليفونات بتاع النت. ولما الـ DNS يقع، النت يبان "واقع" رغم إنه شغال تمام؛ `ping 8.8.8.8` شغال، بس `ping google.com` بيقولك `cannot resolve`. وده أشهر سبب لمشكلة الكافيه بتاعتنا.

تشريح الأمر

```
dig +short MX gmail.com @1.1.1.1
```

1. `dig` Domain Information Groper: اسأل ال DNS. جاي مع الماك. (على Debian/Ubuntu في حزمة `dnsutils` أو `bind9-dnsutils`).
2. `+short` ال Options بتاعة dig بتبدأ بـ `+` مش `-`! معناها: الإجابة بس من غير التفاصيل.
3. `MX` نوع السجل: `A` (IPv4، الافتراضي)، و `AAAA` (IPv6)، و `MX` (إيميل)، و `CNAME` (اسم مستعار)، و `TXT`، و `NS`.
4. `gmail.com` الدومين.
5. `@1.1.1.1` اسأل ال DNS server ده بالذات بدل الافتراضي. لو الإجابة اختلفت، يبقى ال DNS عندك هو المشكلة.

`nslookup example.com`	أبسط وموجود على Windows و macOS و Linux.
`dscacheutil -q host -a name example.com`	بيحل الاسم زي ما البرامج على الماك بتحله فعلاً (بيقرا `/etc/hosts` والـ Cache كمان). `dig` بيسأل ال DNS server على طول ومبيبصش على `/etc/hosts`، فممكن الاتنين يختلفوا!
`scutil --dns | grep nameserver`	ال DNS servers اللي الماك بيستخدمها فعلاً دلوقتي (اللي جاية من الراوتر أو من ال VPN).
`networksetup -setdnsservers Wi-Fi 1.1.1.1 8.8.8.8`	غيّر ال DNS بتاع ال Wi-Fi (و `Empty` مكان الأرقام = رجّع الافتراضي). ممكن يطلب باسورد الأدمن.
`sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder`	امسح ال DNS cache. محتاجها لما تغيّر سجل DNS لموقعك والماك لسه فاكر القديم (↩ 6.6 ص 66).

الأسامي اللي بتنتهي بـ .local

عندك ماك تاني أو طابعة على نفس الشبكة؟ مش محتاج تعرف ال IP: `ping ahmeds-macbook.local`. ده اسمه **mDNS** (أو Bonjour): الأجهزة على نفس الشبكة بتعلن عن نفسها بأساميها من غير DNS server. اسم الماك بتاعك تشوفه بـ `scutil --get LocalHostName`، وتتفرج على الخدمات المعلنة حواليك بـ `dns-sd -B _ssh._tcp` (`Ctrl+C` للخروج). وده نفس اللي بيخلي AirDrop و AirPlay يلاقوا الأجهزة.

على Linux: resolvectl g getent

`getent hosts example.com`	بيحل الاسم زي ما البرامج بتحله فعلاً (بيقرا `/etc/hosts` كمان). المقابل لـ dscacheutil.
`resolvectl status`	ال DNS servers اللي الجهاز بيستخدمها (systemd-resolved). المقابل لـ `scutil --dns`.

10.4 ssh: ادخل على سيرفر بعيد

جه اليوم الكبير: خلّصت مشروع Node وعايز تنزله على النت. أجّرت VPS صغير بـ 5 دولار في الشهر من DigitalOcean أو Hetzner، وجالك إيميل فيه حاجتين: IP زي `203.0.113.10` واسم مستخدم. مفيش شاشة، مفيش ماوس، مفيش Finder. السيرفر ده جهاز في Data center في بلد تانية، والطريقة الوحيدة إنك تكلمه هي إنك تفتح Terminal على الماك وتعمل `ssh`. ساعتها الـ Terminal بتاعك يبقى "شباك" على جهاز تاني خالص: كل أمر تكتبه بيتنفذ هناك.

والحلو إن `ssh` جاي مع الماك، وكل اللي جاي في الجزء ده بيشتغل بنفس الشكل على الماك و Linux لأنه نفس البرنامج (OpenSSH):

تشريح الأمر

```
1   2          3                    4
ssh -p 2222 -i ~/.ssh/id_ed25519 deploy@203.0.113.10
```

1. `ssh` — Secure SHell: افتح Shell على جهاز تاني، والاتصال كله متشفر.
2. `-p 2222` — Port: البورت (الافتراضي 22). وخد بالك إن scp بيستخدم `-P` كبيرة لنفس الحاجة!
3. `-i ~/.ssh/id_ed25519` — Identity: المفتاح الخاص اللي هتدخل بيه (لو مش الافتراضي).
4. `deploy@203.0.113.10` — `user@host`: المستخدم على السيرفر، والسيرفر (IP أو اسم). من غير `user@` بيستخدم اسمك المحلي.

Terminal

```
user@MacBook ~ % ssh deploy@203.0.113.10 'df -h /'
Filesystem  Size  Used Avail Use% Mounted on
/dev/sda1    40G   18G   20G  48% /
```

لو حطيت أمر بعد العنوان، ssh بينفذه هناك ويرجعلك الناتج ويقفل. والـ Quotes بتحدد إن الـ `df -h /` يتنفذ **هناك** مش عندك. ولاحظ إن الناتج شكل Linux (`/dev/sda1`) مش شكل الماك، لأن الأمر اتنفذ على السيرفر.

طب كل مرة هتكتب باسورد؟ لأ. الباسوردات بتتسرق وبتتخمن، وفيه بوتات على النت بتجرب آلاف الباسوردات على أي سيرفر جديد في أول ساعة. الحل اللي المحترفين بيستخدموه: **المفاتيح**. تخيلها كده: إنت بتعمل "قفل" و"مفتاح" مع بعض. القفل (المفتاح العام `.pub`) بتركّبه على باب كل سيرفر عايز تدخله، وتوزعه براحتك. والمفتاح (الخاص) بيفضل في جيبك، على الماك بتاعك بس، ومحدش يشوفه أبداً.

المفاتيح: ادخل من غير باسورد (وبأمان أكتر)

تشريح الأمر

```
ssh-keygen -t ed25519 -C "ahmed@laptop"
```

1. `ssh-keygen` — اعمل زوج مفاتيح: خاص (يفضل عندك ومحدش يشوفه) وعام (`.pub`، بيتحط على السيرفرات).
2. `-t ed25519` — Type: نوع Ed25519 (حديث وصغير وسريع). من OpenSSH 9.5 هو الافتراضي أصلاً، بس كتابته بتضمن النوع على النسخ الأقدم.
3. `-C "ahmed@laptop"` — Comment: تعليق في آخر المفتاح العام عشان تعرف بتاع مين. هيسألك على مكان الحفظ (Enter = `~/.ssh/id_ed25519`) و Passphrase (حط واحدة!).

تشريح الأمر

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub deploy@203.0.113.10
```

1. `ssh-copy-id` — انسخ المفتاح العام لملف `~/.ssh/authorized_keys` على السيرفر وظبط صلاحياته (↩ 6.2 ص 59). موجود على نسخ الماك الحديثة؛ لو مش لاقيه: `brew install ssh-copy-id`، أو انسخ محتوى الـ `.pub` يدوي (`pbcopy < ~/.ssh/id_ed25519.pub` وتلزقه في لوحة تحكم السيرفر).
2. `-i ~/.ssh/id_ed25519.pub` — المفتاح العام (`.pub`) اللي هيتنسخ.
3. `deploy@203.0.113.10` — السيرفر، وهيطلب الباسورد آخر مرة.

المفتاح الخاص = مفتاح الشقة

عمرك ما تبعت `id_ed25519` (من غير pub.) لحد ولا ترفعه على GitHub. وصلاحياته `600` والمجلد `700` وإلا ssh هيرفضه (↩ 6.2 ص 59).

تشريح الأمر

```
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

1. `ssh-add` — حمّل المفتاح في الـ **ssh-agent** (برنامج صغير شغال في الخلفية بيفتكر المفاتيح المفتوحة)، عشان متكتبش الـ Passphrase مع كل اتصال.
2. `--apple-use-keychain` — ميزة خاصة بالماك: احفظ الـ Passphrase في الـ **Keychain** (نفس المكان اللي Safari بيحفظ فيه الباسوردات)، فتكتبها مرة واحدة في عمرك. (في نسخ قديمة كانت `-K`).
3. `~/.ssh/id_ed25519` — المفتاح الخاص.

10.5 ملف ~/.ssh/config: اختصر كل ده

~/.ssh/config

```
Host prod
    HostName 203.0.113.10
    User deploy
    Port 2222
    IdentityFile ~/.ssh/id_ed25519

Host *
    ServerAliveInterval 60
    AddKeysToAgent yes
    UseKeychain yes
```

ملف `~/.ssh/config` ده "دفتر العناوين" بتاعك: بدل ما تحفظ IP وبورت واسم مستخدم ومكان مفتاح لكل سيرفر، تديله اسم قصير. وبعد كده `ssh prod` = الأمر الطويل كله. والسطرين الأخيرين خاصين بالماك: حمّل المفاتيح في ال Agent تلقائياً، وخد ال Passphrase من ال Keychain. وكمان `scp` و `rsync` و `git` بيفهموا `prod` نفس الاسم.

المفتاح	بيساوي
`Host prod`	الاسم المختصر اللي هتكتبه (و `*` = كل السيرفرات)
`HostName`	العنوان الحقيقي
`User` / `Port` / `IdentityFile`	`user@` و `-p` و `-i`
`ProxyJump bastion`	`-J` : ادخل عن طريق سيرفر وسيط
`LocalForward 5433 localhost:5432`	`-L` : افتح بورت عندك يوصل لبورت هناك
`ServerAliveInterval 60`	ابعت نبضة كل دقيقة عشان الاتصال ميقطعش
`UseKeychain yes`	على الماك بس: خد ال Passphrase من ال Keychain (على Linux السطر ده بيعمل Error، فلو بتنقل الملف لسيرفر حط قبله `IgnoreUnknown UseKeychain`)

جدول 10.2 مفاتيح ~/.ssh/config

تشريح الأمر

```
ssh -N -L 5433:localhost:5432 prod
```

1. `ssh` اتصل.
2. `-N` No command: متفتحش Shell، أنا عايز النفق بس.
3. `-L 5433:localhost:5432` Local forward: `بورتي:العنوان_من_ناحية_السيرفر:بورته`. أي حاجة توصل لـ localhost:5433 عندك تعدي في النفق المتشفر لـ PostgreSQL على السيرفر.
4. `prod` الاسم من ملف الـ config.

10.6 scp: انسخ من وإلى السيرفر

دخلت السيرفر. تمام. بس الكود لسه على الماك بتاعك. محتاج تنقله. الـ Finder مش هيساعدك هنا، ومفيش Drop & Drag لسيرفر في بلد تانية. هنا بييجي دور `scp`: هو `cp` اللي إنت عارفه، بس عرف يسافر:

تشريح الأمر

```
scp -P 2222 -r ./site deploy@203.0.113.10:/var/www/
```

1. `scp` Secure CoPy: زي `cp` بالظبط بس أحد الطرفين (أو الاتنين) على جهاز تاني (↩ 5.4 ص 51).
2. `-P 2222` البورت، بـ **P كبيرة** (عشان `-p` الصغيرة في scp = Preserve التواريخ زي cp).
3. `-r` Recursive: مجلد بكل اللي فيه، نفس `cp -r`.
4. `./site` المصدر المحلي.
5. `deploy@203.0.113.10:/var/www/` الوجهة: `user@host:path`. الـ **:** هي اللي بتقول "ده على جهاز تاني". من غير مسار بعدها = الـ home هناك. وعشان تنزّل من السيرفر اعكس الترتيب.

scp من جوه

من OpenSSH 9.0، الأمر `scp` بيستخدم بروتوكول SFTP من جوه بدل البروتوكول القديم (والـ `-O` بترجّع القديم لو سيرفر قديم). الاستخدام زي ما هو، بس للنسخ الكبير والمتكرر `rsync` أحسن.

10.7 rsync: المزامنة الذكية

تخيل إن عندك 2000 ملف على السيرفر، وغيرت سطر واحد في ملف واحد. scp هينقل الـ 2000 تاني كل مرة. ده زي ما تنقل عفش الشقة كله كل ما تشتري كرسي جديد. `rsync` أذكى: بيقارن الطرفين وينقل **الفرق بس**:

تشريح الأمر

```
1     2      3          4          5          6
rsync -avzP --delete --dry-run ./site/ prod:/var/www/site/
```

1. `rsync` — Remote sync: بينقل **الفرق بس**. أول مرة بينسخ كله، والمرات الجاية الملفات اللي اتغيرت بس.
2. `-avzP` — `-a` (= Archive: `-rlptgoD` : Recursive و Links و Permissions و Times و Group و Owner و Devices؛ يعني نسخة طبق الأصل)، و `-v` Verbose، و `-z` اضغط في الطريق، و `-P` = `--partial --progress` (شريط تقدم ويكمل لو اتقطع).
3. `--delete` — امسح من الوجهة أي حاجة اتمسحت من المصدر (مرآة كاملة). خطيرة لو المسار غلط!
4. `--dry-run` — (= `-n`) جرّب: اعرض اللي هيحصل من غير ما تعمل حاجة. شيلها لما تتأكد.
5. `./site/` — المصدر. **ال / في الآخر مهمة**: `site/` = "محتويات site"، و `site` = "المجلد site نفسه" (هيبقى `/var/www/site/site`).
6. `prod:/var/www/site/` — الوجهة على السيرفر باسم ال config (rsync بيستخدم ssh تلقائياً).

المقارنة بين cp و mv و rsync في ↩ ص 51 5.4. وبورت مختلف مع rsync: `-e "ssh -p 2222"` (أو حطه في ال config وخلاص).

الـ rsync اللي جاي مع الماك مش rsync!

لسنين طويلة Apple كانت شايلة rsync نسخة `2.6.9` من سنة 2006 (لنفس سبب bash 3.2: رخصة GPLv3، ↩ ص 28 3.4). ومن macOS 15.4 الأمر `/usr/bin/rsync` بقى في الحقيقة **openrsync**، نسخة بترخيص BSD بتدعم جزء بس من ال Options. جرّب `rsync --version` : هيقولك `openrsync: protocol version 29` . الأوامر اللي في الكتاب ده شغالة عليه، بس لو سكربت من النت فيه Option غريب وقالك `unrecognized option` ، سطّب الأصلي: `brew install rsync` . والـ Homebrew بيحطه في `/opt/homebrew/bin` قبل `/usr/bin` في ال PATH، فيكسب (↩ ص 163 13.5).

10.8 tar: أرشف واضغط

إنت عارف ال Zip: كليك يمين في ال Finder ← Compress. طب ليه كل حاجة على السيرفرات وفي GitHub Releases بتبقى `.tar.gz`؟ لأن `tar` أقدم من ال Zip، وبيحافظ على الصلاحيات وال Owners وال Links (اللي اتكلمنا عنهم في ↩ ص 59 6.2) وده مهم جداً على السيرفر. الفكرة على مرحلتين: tar بيلم الملفات في "شنطة" واحدة، و gzip بيضغط الشنطة:

تشريح الأمر

```
tar -czvf backup.tar.gz --exclude="node_modules" project/
```

1. `tar` — Tape ARchive: اجمع ملفات كتير في ملف واحد.
2. `-czvf` — `-c` Create: اعمل أرشيف، و `-z` اضغطه بـ gzip، و `-v` Verbose (اعرض الأسماء)، و `-f` File: اسم الأرشيف **جاي بعدها على طول**، فلازم `f` تبقى آخر حرف.
3. `backup.tar.gz` — اسم الأرشيف (Argument بتاع `-f`).
4. `--exclude="node_modules"` — متدخلش ده.
5. `project/` — اللي هيتأرشف.

الحرف	المعنى
`c` / `x` / `t`	Create / eXtract / lisT (اعرض من غير فك)
`z` / `j` / `J`	gzip `.gz` / bzip2 `.bz2` / xz `.xz` (و tar بتاع الماك، bsdtar، بيعرف لوحده وقت الفك، و GNU tar كمان)
`f`	اسم الأرشيف بعدها
`-C dir`	فك في المجلد ده: `tar -xzf a.tar.gz -C /tmp`

جدول 10.3 حروف tar

تشريح الأمر

```
tar -czf - project/ | ssh prod 'tar -xzf - -C /srv'
```

1. `tar -czf -` — أرشف واضغط، والـ `-` مكان اسم الملف = اكتب على stdout (اصطلاح مشترك بين أوامر كتير: `-` = stdin أو stdout).
2. `project/` — المجلد.
3. `|` — Pipe (ssh ↩ 7.3 ص 74).
4. `ssh prod` — ادخل على السيرفر، والـ stdin بتاعك يتنقل لهناك.
5. `'tar -xzf - -C /srv'` — هناك: فك (`-x`) من stdin (`-f -`) في `/srv`. نسخ مجلد كامل من غير ملف وسيط.

> ⚠ **ملفات ._ الغريبة على السيرفر**
>
> عملت `tar` على الماك وفكيته على Linux، ولقيت جنب كل ملف ملف تاني اسمه `._index.js` ؟ ده tar بتاع الماك بيحفظ الـ Extended attributes (↩ 6.7 ص 67) في ملفات "AppleDouble" دي. السيرفر مش محتاجها. امنعها كده: `COPYFILE_DISABLE=1 tar -czf backup.tar.gz project/` (أو `--no-mac-metadata`). ونفس القصة مع `.DS_Store`: ضيف `--exclude=".DS_Store"`.

10.9 البورتات و lsof

فكر في الـ IP على إنه عنوان عمارة، والـ **Port** (رقم من 0 لـ 65535) هو رقم الشقة جوه العمارة. الماك بتاعك عمارة واحدة، بس فيها "سكان" كتير: سيرفر الـ React على شقة 3000، و Postgres على 5432، و AirPlay على 5000. ولما متصفحك يروح لـ `localhost:3000` فهو بيخبط على شقة معينة. البورتات تحت 1024 محتاجة root عشان برنامج يسمع عليها.

البورت	الخدمة	البورت	الخدمة
22	SSH	3306	MySQL
53	DNS	5432	PostgreSQL
80 / 443	HTTPS / HTTP	6379	Redis
25 / 587	SMTP	3000 / 8000 / 8080	سيرفرات تطوير

جدول 10.4 بورتات مشهورة

والسؤال اللي هيقابلك كل أسبوع تقريباً: "مين اللي قاعد في شقة 3000؟". على الماك الأداة اسمها `lsof` (List Open Files)، لأن في Unix كل حاجة ملف، حتى الاتصال بالشبكة:

تشريح الأمر

```
lsof -nP -iTCP -sTCP:LISTEN
 1    2   3      4
```

1. `lsof` — List Open Files: اعرض الملفات المفتوحة ومين فاتحها. والـ Sockets بتاعة الشبكة بتتعد ملفات كمان.
2. `-nP` — `-n` No DNS: متحولش الـ IPs لأسامي، و `-P` Port numbers: اعرض `3000` مش اسمه (`hbci`). الاتنين بيخلوه أسرع بكتير وأوضح.
3. `-iTCP` — Internet: الاتصالات الـ TCP بس. ممكن `-i :3000` لبورت معين، أو `-iUDP`.
4. `-sTCP:LISTEN` — State: اللي في حالة LISTEN بس، يعني اللي مستني حد يكلمه (السيرفرات)، مش الاتصالات الطالعة بتاعة Chrome.

```
Terminal
user@MacBook ~ % lsof -nP -iTCP -sTCP:LISTEN
COMMAND     PID  USER   FD   TYPE  DEVICE SIZE/OFF NODE NAME
ControlCe   612 ahmed   10u  IPv4  0x1a2b      0t0  TCP *:5000 (LISTEN)
node       4433 ahmed   23u  IPv6  0x3c4d      0t0  TCP *:3000 (LISTEN)
postgres    901 ahmed    7u  IPv4  0x5e6f      0t0  TCP 127.0.0.1:5432 (LISTEN)
```

عمود `NAME` هو المهم: `*:3000` = بيسمع على كل الكروت، و `127.0.0.1:5432` = من الماك نفسه بس (الفرق ده مشروح تحت). ومن غير `sudo` هتشوف عملياتك إنت بس؛ ضيف `sudo` قبله لو عايز تشوف خدمات النظام والـ root كمان.

`lsof -nP -i :3000`	مين بيستخدم البورت ده (سامع أو متصل).
`lsof -t -i :3000`	`-t` Terse: الـ PID بس، عشان تديه لـ kill: `kill $(lsof -t -i :3000)` (↩ 9.3 ص 100).
`netstat -an -p tcp | grep LISTEN`	الطريقة القديمة، شغالة على الماك بس من غير أسامي البرامج (netstat بتاع الماك مفيهوش `-p` بمعنى Process، هنا `-p tcp` معناها Protocol).

على Linux: ss

على السيرفر `lsof` ممكن متكونش متسطبة، والأداة الأساسية هي `ss`:

تشريح الأمر

```
sudo ss -tulnp
 1   2   3
```

1. `sudo` عشان تشوف أسماء عمليات المستخدمين التانيين.
2. `ss` Socket Statistics: البديل الحديث لـ `netstat` (اللي في حزمة net-tools القديمة، وبنفس الحروف تقريباً: `netstat -tulnp`).
3. `-tulnp` `-t` TCP، و `-u` UDP، و `-l` Listening (اللي مستني اتصالات بس)، و `-n` Numeric (أرقام البورتات مش أسماءها، أسرع وأوضح؛ نفس `-n` بتاعة ping و ls -n)، و `-p` Process (مين البرنامج والـ PID، ↩ 9.1 ص 97).

```
Terminal
user@server:~$ sudo ss -tlnp
State   Recv-Q Send-Q Local Address:Port  Peer Address:Port Process
LISTEN  0      511          0.0.0.0:80         0.0.0.0:*     users:
(("nginx",pid=812,fd=6))
LISTEN  0      128        127.0.0.1:5432       0.0.0.0:*     users:
(("postgres",pid=901,fd=5))
```

0.0.0.0 مقابل 127.0.0.1

`0.0.0.0:80` (أو `*:80` في lsof) = بيسمع من أي حد على الشبكة. و `127.0.0.1:5432` = من الجهاز نفسه بس. لو قاعدة بياناتك على `0.0.0.0` ومفيش Firewall، الدنيا كلها تقدر تجرب تدخلها، ونفس الكلام على الماك وإنت في الكافيه: أي حد على نفس الـ Wi-Fi يقدر يوصل لسيرفر التطوير بتاعك لو سامع على `*`. (الماك عنده Firewall في System Settings ← Network ← Firewall، وبيبقى مقفول افتراضياً). وعشان تختبر بورت على جهاز تاني: `nc -zv host 443` (`-z` افحص بس، و `-v` قولي النتيجة).

10.10 مشاكل شائعة

مشكلة 1 (publickey) Permission denied

السيرفر مش لاقي مفتاحك في `authorized_keys`، أو بتدخل بمستخدم غلط، أو الصلاحيات واسعة.

الحل: `ssh -v prod` واقرا هو جرّب أنهي مفتاح (سطور `Offering public key`). اتأكد من الـ User، ومن `chmod 700 ~/.ssh` و `600 authorized_keys` على السيرفر.

مشكلة 2 WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!

بصمة السيرفر اختلفت عن اللي محفوظة في `~/.ssh/known_hosts`. ده طبيعي لو السيرفر اتعمله Reinstall، وخطير لو مفيش سبب (ممكن حد في النص).

الحل: لو متأكد إن السيرفر اتغير فعلاً: `ssh-keygen -R 203.0.113.10` يشيل البصمة القديمة، وادخل تاني وراجع البصمة الجديدة.

مشكلة 3 ssh بيسألني على الـ Passphrase كل مرة

عملت Restart للماك، ورجع ssh يسألك على الـ Passphrase مع كل اتصال.

الحل: مرة واحدة: `ssh-add --apple-use-keychain ~/.ssh/id_ed25519`، وحط `AddKeysToAgent yes` و `UseKeychain yes` تحت `Host *` في `~/.ssh/config`. بعدها الماك هيفتكرها من الـ Keychain.

مشكلة 4 الموقع بيفتح من موبايلي ومش بيفتح من الماك

غالباً الماك فاكر IP قديم للموقع (DNS cache)، أو فيه سطر قديم في `/etc/hosts` إنت نسيته من مشروع فات.

الحل: `grep example /etc/hosts` وشيل السطر لو موجود (`sudo nano /etc/hosts`)، وبعدين امسح الـ Cache: `sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder`. وقارن `dig +short example.com` بـ `dscacheutil -q host -a name example.com`.

ملخص الفصل

- شخّص من تحت لفوق: `ipconfig getifaddr en0` و `route -n get default` ثم `ping` ثم `dig` ثم `curl` ثم `lsof -i`. وعلى Linux: `ip` و `ss`.
- `curl -fsSL` للتحميل في Scripts، و `-I` للـ Headers، و `-w '%{http_code}'` للفحص.
- `ssh-keygen -t ed25519` و `ssh-copy-id`، و `ssh-add --apple-use-keychain`، وملف `~/.ssh/config` بيختصر كل حاجة.
- `ssh -p` صغيرة و `scp -P` كبيرة، و rsync بالـ `/` في آخر المصدر وجرّب بـ `--dry-run`.
- rsync الماك = openrsync؛ للنسخة الكاملة `brew install rsync`.
- `tar -czf` للضغط و `-xzf` للفك (و `COPYFILE_DISABLE=1` عشان ملفات `._`)، و `lsof -nP -iTCP -sTCP:LISTEN` مين بيسمع على إيه.
- DNS الماك: `scutil --dns` و `dscacheutil -flushcache`، والأجهزة القريبة بـ `.local`.

صح ولا غلط؟

1. لو ping فشل يبقى الموقع أكيد واقع. صح / غلط
2. `scp -p 2222` بتحدد البورت. صح / غلط
3. `rsync -a src/ dst/` و `rsync -a src dst/` بيدوا نفس النتيجة. صح / غلط
4. المفتاح اللي بينتهي بـ `.pub` هو اللي بيتحط على السيرفر. صح / غلط

أسئلة مراجعة

1. موقع مش بيفتح: رتّب الأوامر اللي هتستخدمها للتشخيص وقول كل واحد بيختبر إيه.
2. اشرح كل حرف في `curl -fsSL`.
3. اكتب Host في ssh config لسيرفر على بورت 2200 بمستخدم admin.
4. إيه وظيفة كل جزء في `lsof -nP -iTCP -sTCP:LISTEN`؟ وإيه المقابل على سيرفر Linux؟
5. ليه `ping -W 2` على الماك ممكن يخلي كل الردود تبان فاشلة؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعرف الـ IP بتاع `github.com` بـ dig من DNS server تاني غير الافتراضي، واعرف الـ DNS servers اللي الماك بيستخدمها دلوقتي.
2. متوسط اكتب سطر يطبع "UP" لو موقع رجع 200 و "DOWN" غير كده، باستخدام curl و `&&` و `||`.
3. صعب اعمل مفتاح Ed25519 جديد، وحطه في ssh config باسم `lab` لسيرفر، أو للماك نفسه: فعّل **Remote Login** من System Settings ← General ← Sharing، واستخدم `HostName localhost`. وانسخ مجلد بـ rsync مع `--dry-run` الأول.

11

المرحلة 4 · النظام والشبكة

الفصل الحادي عشر

Homebrew ومديري الحزم: تسطيب البرامج صح

افتكر آخر مرة سطبت فيها برنامج على الماك: دخلت الموقع، دوّرت على زرار Download وسط الإعلانات، نزل ملف dmg.، فتحته، سحبت الأيقونة لـ Applications، وبعدين نسيت الـ dmg. في Downloads للأبد. وبعد شهر البرنامج عايز Update، وكل برنامج بيعمل Update بطريقته. طب لو عندك 30 أداة؟ ولو جبت ماك جديد وعايز ترجّع كل ده؟ الـ App Store مبيعرفش حاجة عن git و node و wget. هنا بييجي دور Homebrew: مدير حزم (Package Manager) بيجيب البرنامج من مخزن موثوق، ويسطب اللي محتاجه معاه، ويحدّث كل حاجة بأمر واحد. الفصل ده هيخليك تسطب وتحدّث وتشيل وتنقل كل برامجك بـ brew، وبعدين هنشوف إخواته على السيرفرات (apt و dnf و pacman) وعلى Windows، لأن الفكرة واحدة والأسامي بس اللي بتتغير.

بعد الفصل ده هتكون عارف

- ✅ يعني إيه Package و Repository و Dependency، و Formula و Cask و Bottle
- ✅ تسطيب Homebrew نفسه و brew shellenv و /opt/homebrew
- ✅ أوامر brew اليومية: install و upgrade و search و info و services و doctor
- ✅ Brewfile و brew bundle عشان تنقل كل برامجك لماك جديد
- ✅ apt على Debian و Ubuntu، و dnf على Fedora، و pacman على Arch
- ✅ winget و choco على Windows، وجدول مقارنة واحد لكل العمليات

11.1 الفكرة: المخزن والحزمة والاعتماديات

فكّر في الـ App Store على الموبايل. إنت مبتدورش على موقع كل برنامج، ومبتقلقش إن الملف متعدل أو فيه فيروس، ولما ييجي Update بتدوس "Update All". مدير الحزم هو نفس الفكرة بالظبط، بس لأدوات المطوّرين، ومن الـ Terminal. وفيه 3 كلمات هتسمعها كتير:

Package

البرنامج متغلّف ومعاه معلومات: الإصدار وبيحتاج إيه. في Homebrew اسمه **Formula** (أداة Terminal) أو **Cask** (برنامج app.)، وعلى Linux ملف `.deb` أو `.rpm`.

Repository

المخزن: مكان رسمي فيه آلاف الحزم، ومعاها بصمات (Checksums) فإنت متأكد إنها متعدلتش.

Dependencies

المكتبات اللي البرنامج محتاجها. تسطب `ffmpeg` فتلاقي brew جاب معاه 30 مكتبة لوحده. المدير بيحلها ويسطبها.

والقاعدة دايماً: **حدّث القائمة** (إيه الجديد في المخزن) ← **رقّي** (سطّب الجديد) ← **سطّب** اللي محتاجه. على الماك brew بيشتغل **من غير sudo**، لأنه بيسطب في فولدر بتاعك إنت. أما مديري Linux فبيسطبوا للنظام كله ومحتاجين `sudo` (↩ 6.6 ص 66).

ليه الماك مفيهوش واحد جاهز؟

Apple عندها الـ App Store للبرامج العادية، و `softwareupdate` لتحديثات النظام نفسه (↩ 14.3 ص 171)، بس معندهاش مدير حزم رسمي لأدوات المطوّرين زي Linux. فالمجتمع عمل **Homebrew** سنة 2009، وبقى عملياً "الـ apt بتاع الماك". فيه بدائل زي **MacPorts** و **Nix**، بس Homebrew هو اللي هتلاقيه في 90% من الشروحات وصفحات الـ Install.

11.2 تسطيب Homebrew نفسه

أول حاجة brew مش جاي مع الماك، فلازم تسطبه مرة واحدة في عمرك (لكل ماك). والطريقة الرسمية من `brew.sh` سطر واحد طويل شكله مخيف شوية. خلينا نفككه بدل ما ننسخه ونلزقه وإحنا مغمضين:

تشريح الأمر

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

1. `/bin/bash` — شغّل bash بالمسار الكامل (bash 3.2 بتاع الماك، مش zsh)، لأن السكربت مكتوب لـ bash (↩ 3.4 ص 28).
2. `-c "..."` — Command: نفّذ النص ده كأمر.
3. `$( ... )` — Command substitution: شغّل اللي جوه الأول وحط ناتجه مكانه (↩ 7.6 ص 77). يعني النص اللي bash هينفذه = محتوى السكربت نفسه.
4. `curl -fsSL URL` — نزّل السكربت بالـ Flags المشهورة اللي شرحناها (↩ 10.2 ص 115): افشل لو فيه Error، ساكت، بس اعرض الأخطاء، واتبع الـ Redirects.

السكربت بيعمل كام حاجة: بيتأكد إن عندك **Xcode Command Line Tools** (فيهم git والـ Compilers، ولو مش موجودين بيسطبهم، أو تسطبهم إنت بـ `xcode-select --install`)، وبيعمل الفولدر `/opt/homebrew`، وبيطلب باسورد الأدمن **مرة واحدة بس** عشان يعمل الفولدر ده ويخليه ملكك. وبعد كده عمرك ما هتحتاج sudo مع brew. وفي الآخر بيطبعلك حاجة زي دي:

Terminal

```
==> Next steps:
- Run these commands in your terminal to add Homebrew to your PATH:
    echo >> /Users/ahmed/.zprofile
    echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> /Users/ahmed/.zprofile
    eval "$(/opt/homebrew/bin/brew shellenv)"
```

ناس كتير بتعدّي السطور دي، وبعدين يكتبوا `brew` يلاقوا `zsh: command not found: brew`. ليه؟ لأن zsh بيدور على الأوامر في الـ PATH بس (↩ 13.5 ص 163)، و `/opt/homebrew/bin` مش فيه لسه. والسطر ده بيصلح ده:

تشريح الأمر

```
eval "$(/opt/homebrew/bin/brew shellenv)"
```
1

1. `/opt/homebrew/bin/brew shellenv` — brew نفسه بيطبع شوية أوامر `export` فيها `PATH` و `HOMEBREW_PREFIX` و `MANPATH` الصح للماك بتاعك. جرّبه لوحده من غير eval وشوف بيطبع إيه.
2. `$( ... )` — خد الناتج ده كنص.
3. `eval "..."` — نفّذ النص ده كأوامر جوه الـ Shell الحالي، فالـ PATH يتغير فعلاً.
4. `>> ~/.zprofile` — والسطر اللي قبله بيحطه في `~/.zprofile` عشان يتنفذ لوحده في كل Terminal جديدة (ليه zprofile بالذات؟ ↩ 13.1 ص 157).

الماك	الـ Prefix	ملاحظات
Apple Silicon (M1 وما بعده)	`/opt/homebrew`	المستوى الأول من الدعم (Tier 1). كل الأمثلة في الكتاب ده عليه
Intel	`/usr/local`	Homebrew نزّل دعمه لـ Tier 3، وأعلن إنه هيبطل يشتغل عليه حوالي سبتمبر 2027. و macOS 27 نفسه مبقاش بينزل على Intel
Linux	`/home/linuxbrew/.linuxbrew`	أيوه، brew بيشتغل على Linux كمان، بس هناك apt أشهر

جدول 11.1 Homebrew بيتسطب فين

وعشان متحفظش الجدول ده: `brew --prefix` بيقولك المكان على جهازك، وفي السكربتات استخدم `$(brew --prefix)` بدل ما تكتب المسار بإيدك.

بس مش curl | bash ده خطر؟

سؤال ممتاز. تشغيل سكربت من النت على طول معناه إنك واثق في المصدر 100%، وهنفصّل ده في فصل الخطر (↩ 15.6 ص 187). مع Homebrew المصدر هو الـ Repository الرسمي على GitHub، وده مقبول. ولو حابب تبقى أحرص: نزّل السكربت الأول بـ `curl -fsSL -o install.sh URL`، واقراه بـ `less install.sh`، وبعدين شغله. أو استخدم ملف `.pkg` الرسمي من صفحة الـ Releases بتاعة Homebrew على GitHub.

11.3 brew كل يوم: install و upgrade و info

خلاص brew متسطب. خلينا نعيش يوم عادي مع مطوّر JavaScript: محتاج `wget`، وعايز VS Code، وعايز تعرف إيه اللي محتاج تحديث. كل ده من نفس المكان:

تشريح الأمر

```
brew install --cask visual-studio-code
```

1. `brew`: Homebrew: مدير الحزم الأشهر على macOS. بيشتغل **من غير sudo**.
2. `install`: سطّب. وقبلها brew بيعمل `update` لوحده لو القائمة قديمة (تقدر تمنع ده بـ `HOMEBREW_NO_AUTO_UPDATE=1`).
3. `--cask`: برنامج بواجهة رسومية (app.) بيتحط في `/Applications` زي ما كنت بتسحبه بإيدك. من غيرها = أداة Terminal (Formula) زي `brew install wget`. وفي الحقيقة brew ذكي كفاية إنه يعرف لوحده لو الاسم Cask، بس كتابتها أوضح.
4. `visual-studio-code`: الحزمة. الاسم بتلاقيه بـ `brew search code` أو من موقع `formulae.brew.sh`.

	Formula	Cask
إيه هو	أداة Terminal أو مكتبة	برنامج app. بواجهة، أو Font، أو Driver
أمثلة	`git` و `node` و `wget` و `tree` و `postgresql@17`	`google-chrome` و `visual-studio-code` و `iterm2` و `docker-desktop`
بيتحط فين	`/opt/homebrew/Cellar/` ومعاه Symlink في `/opt/homebrew/bin`	`/Applications`
بيجي إزاي	**Bottle** (نسخة متجمعة جاهزة) غالباً، أو بيتبني من الكود لو مفيش	الـ dmg. أو zip. الرسمي من الشركة نفسها

جدول 11.2 Formula قصاد Cask

Terminal

```
user@MacBook ~ % brew info node
==> node: stable 24.x (bottled), HEAD
Open-source, cross-platform JavaScript runtime environment
https://nodejs.org/
Installed
/opt/homebrew/Cellar/node/24.x (2,700 files, 90MB) *
==> Dependencies
Required: brotli, c-ares, icu4c, libnghttp2, libuv, openssl@3, ...
```

الأرقام بالظبط هتختلف على جهازك حسب آخر إصدار؛ المهم شكل المعلومات: الإصدار، و bottled يعني جاهزة متجمعة، والمكان في الـ Cellar، والاعتماديات.

الأمر	بيعمل
`brew update`	حدّث brew نفسه وقائمة الحزم. **مبيرقّيش** برامجك (زي `apt update`)
`brew outdated`	إيه اللي ليه إصدار أحدث (قبل ما ترقّي)

الأمر	بيعمل
`brew upgrade`	رقّي كل حاجة، الـ Formulae والـ Casks. أو `brew upgrade node` لحزمة واحدة
`brew search wget`	دوّر بالاسم (و `brew search /regex/` بـ Regex)
`brew info wget`	الإصدار والوصف والاعتماديات ومتسطب ولا لأ
`brew list`	كل المتسطب، و `--formula` أو `--cask` لنوع واحد
`brew leaves`	اللي إنت سطبته بنفسك بس (مش الاعتماديات). أنضف لستة
`brew uninstall wget`	شيل. و `brew autoremove` بعدها تشيل الاعتماديات اللي محدش بقى محتاجها
`brew cleanup`	امسح النسخ القديمة والـ Downloads المتخزنة (brew بيعملها لوحده كل فترة). `-n` = Dry run
`brew doctor`	اكتشف المشاكل في الإعداد. أول حاجة تعملها لو brew بيتصرف غريب
`brew deps --tree ffmpeg`	شجرة الاعتماديات. و `brew uses --installed openssl@3` العكس: مين محتاجها
`brew pin node`	ثبّت الإصدار ده ومترقيهوش مع `upgrade` (و `unpin` عكسها)
`brew install python@3.12`	إصدار معين، لو الـ Formula بتاعته موجود
`brew services start redis`	شغّل خدمة مع كل Login (↩ 9.5 ص 103)
`brew tap user/repo`	ضيف مخزن تاني غير الرسمي (شركات كتير بتوزّع أدواتها كده)

جدول 11.3 أوامر brew اليومية

روتين أسبوعي

`brew update && brew outdated` عشان تشوف الجديد، وبعدين `brew upgrade`، وكل كام شهر `brew autoremove && brew cleanup`. ولو حابب تعملهم في سطر: `brew update && brew upgrade && brew cleanup` (↩ 7.7 ص 78).

Node: brew ولا nvm؟

كمطوّر JavaScript هتحتاج إصدارات Node مختلفة لمشاريع مختلفة (مشروع قديم على Node 18 ومشروع جديد على 24). `brew install node` بيديك إصدار واحد بس (الأحدث)، وبيترقى لوحده مع `brew upgrade` وممكن يكسر مشروع قديم. الأفضل لـ Node تحديداً: مدير إصدارات زي `fnm` أو `nvm` (`brew install fnm`)، وبعدين `fnm install 22` و `fnm use 22` جوه كل مشروع. يعني brew يسطب الأداة، والأداة تدير الإصدارات.

npm و node_modules و package.json في دقيقة

هتقابل الكلمات دي في كل الكتب الجاية (Git و APIs و Docker)، فخلينا نعرفها من دلوقتي. لما تسطّب Node، بييجي معاه **npm**: مدير الحزم بتاع JavaScript، زي brew بالظبط بس لمكتبات الكود. الفرق المهم: brew بيسطّب برامج على الماك كله، و npm بيسطّب مكتبات **جوه فولدر المشروع** نفسه.

- `npm install express` : نزّل مكتبة express وحطها في فولدر اسمه **node_modules** جوه المشروع. الفولدر ده ممكن يبقى مئات الميجات، وبيتعمل تاني بأمر واحد، فعمره ما بيترفع على Git.
- **package.json**: ملف في أول المشروع فيه اسم المشروع والمكتبات اللي محتاجها والأوامر الجاهزة (Scripts). فأي حد ياخد المشروع يكتب `npm install` بس، وكل المكتبات تنزل.
- `npm run dev` أو `npm run build` : شغّل Script مكتوب في package.json. الـ **Build** = تحويل الكود لنسخة نهائية مضغوطة جاهزة للنشر (غالباً في فولدر `dist`).
- `npx tool` : شغّل أداة من npm مرة واحدة من غير ما تسطّبها.

الشرح الكامل بكل تفاصيله في كتاب **عالم الفرونت إند** (فصول 13 لـ 15).

لو حابب تتعمق

تقدر تعدّيه في أول قراية

إيه اللي بيحصل فعلاً لما تكتب `brew install wget` ؟ brew بيقرا وصف الـ Formula (ملف Ruby صغير، أو من نسخة JSON بيجيبها من `formulae.brew.sh` بدل ما ينزّل Repository كامل بـ git)، ويحل الاعتماديات، وينزّل الـ **Bottle** (أرشيف `.tar.gz` فيه البرنامج متجمع جاهز لنسخة الماك ونوع المعالج بتاعك) ويتأكد من بصمته (SHA-256)، ويفكه في:

جوه /opt/homebrew

```
/opt/homebrew/Cellar/wget/1.25.0/bin/wget    ← البرنامج الحقيقي، فولدر لكل إصدار
/opt/homebrew/opt/wget  → ../Cellar/wget/1.25.0   ← Symlink ثابت لآخر إصدار
/opt/homebrew/bin/wget  → ../Cellar/wget/1.25.0/bin/wget   ← ده اللي في الـ PATH
```

الـ Symlinks دي (↩ 4.5 ص 43) هي سر إن الترقية سهلة: النسخة الجديدة بتتفك في فولدر جديد جنب القديم، والـ Links بتتحول عليها، وبس. وعشان كده `brew unlink wget` بيشيل الأمر من الـ PATH من غير ما يمسح حاجة، و `brew link` يرجّعه. والـ Formulae اللي "keg-only" (زي `openssl@3` أو `node@22`) مبتتعملهاش Link في `bin` عمداً، عشان متتخانقش مع نسخة الماك؛ و `brew info` بيقولك تضيف إيه للـ PATH لو عايزها. والـ Casks حكاية تانية: بتنزّل الـ dmg. الرسمي وتحط الـ app. في `/Applications` ، وتشيل علامة الـ Quarantine عادي زي أي برنامج من النت (↩ 6.7 ص 67).

11.4 Brewfile: نقل كل برامجك لماك جديد

جالك ماك جديد من الشغل. يا فرحتك! وبعدين تفتكر: لازم أسطب git و node و wget و tree و jq و VS Code و Chrome و iTerm2 و Docker و... وكل واحدة لوحدها؟ وأنا أصلاً فاكر سطبت إيه؟ هنا بقى بييجي دور الـ **Brewfile**: ملف نصي صغير فيه لستة كل حاجة، ويتسطب كله بأمر واحد.

تشريح الأمر

```
brew bundle dump --file=~/Brewfile
```
(1) (2) (3)

1. `brew bundle` الجزء اللي بيتعامل مع الـ Brewfiles.
2. `dump` اكتب لستة بكل اللي متسطب دلوقتي (Formulae و Casks و Taps، وحتى برامج الـ App Store لو عندك `mas`).
3. `--file=~/Brewfile` اكتبها هنا. من غيرها بيكتب `Brewfile` في الفولدر الحالي. ضيف `--force` عشان يكتب فوق ملف موجود.

~/Brewfile

```
tap "homebrew/bundle"
brew "fnm"
brew "git"
brew "jq"
brew "tree"
brew "wget"
cask "google-chrome"
cask "iterm2"
cask "visual-studio-code"
```

على الماك الجديد: سطّب Homebrew، وانقل الملف ده (أو حطه في Repo على GitHub مع ملفات الإعدادات بتاعتك، ↩ 13.1 ص 157)، واكتب `brew bundle --file=~/Brewfile`. روح اعمل شاي، وارجع تلاقي كل حاجة متسطبة. و `brew bundle check` بيقولك لو فيه حاجة ناقصة، و `brew bundle cleanup` بيقولك إيه المتسطب ومش في الملف (ضيف `--force` عشان يشيله فعلاً).

أدوات الماك الأخرى

برامج الـ App Store؟ `brew install mas`، وبعدين `mas list` و `mas install ID`. تحديثات macOS نفسه؟ مش من brew خالص، دي `softwareupdate --list` (↩ 14.3 ص 171). وحزم JavaScript؟ دي بتاعة `npm` جوه كل مشروع، مش brew. القاعدة: brew للأدوات اللي بتستخدمها على مستوى الماك، و npm للمكتبات اللي مشروعك بيعتمد عليها.

11.5 على السيرفر: apt في Debian و Ubuntu و Mint

أول ما تدخل VPS بـ ssh (↩ 10.4 ص 119)، أغلب الظن إنه Ubuntu. ومفيش Homebrew هناك؛ فيه `apt`، وهو اللي هتسطب بيه nginx و Node و Postgres. والفرق الكبير عن brew: هنا التسطيب على مستوى النظام كله، فمحتاج `sudo` في كل أمر بيغيّر حاجة:

تشريح الأمر

```
1    2    3      4   5          6
sudo apt update && sudo apt upgrade -y
```

1. `sudo` صلاحيات root.
2. `apt` Advanced Package Tool: الواجهة الحديثة السهلة (فيها ألوان وشريط تقدم).
3. `update` نزّل **قائمة** الحزم الجديدة من المخازن. مبيسطبش حاجة!
4. `&&` لو نجح بس، كمّل (↩ 7.7 ص 78).
5. `sudo apt upgrade` رقّي كل الحزم اللي ليها إصدار أحدث.
6. `-y` Yes: وافق على كل الأسئلة أوتوماتيك. (نفس `-y` في dnf و choco، و winget بيستخدم `--accept-package-agreements`). brew مبيسألش أصلاً فمش محتاجها.

تشريح الأمر

```
1        2                      3
sudo apt install --no-install-recommends curl git
```

1. `sudo apt install` سطّب.
2. `--no-install-recommends` سطّب الضروري بس من غير الحزم "المقترحة" (بيوفر مساحة على السيرفرات و Docker).
3. `curl git` حزمة أو أكتر.

الأمر	بيعمل
`apt search nginx`	دوّر في الأسماء والوصف
`apt show nginx`	تفاصيل: الإصدار والحجم والاعتماديات
`apt list --installed`	كل المتسطب (و `--upgradable` اللي ليه تحديث)
`sudo apt remove nginx`	شيل البرنامج وسيب ملفات الإعدادات
`sudo apt purge nginx`	شيله بإعداداته
`sudo apt autoremove`	شيل الاعتماديات اللي محدش بقى محتاجها
`sudo apt full-upgrade`	رقّي حتى لو محتاج يشيل حزم (للترقيات الكبيرة)
`sudo apt install ./app.deb`	سطّب ملف deb محلي (الـ `./` مهمة عشان يعرف إنه ملف مش اسم حزمة)
`dpkg -L curl`	الحزمة دي حطت ملفات فين (الأداة اللي تحت apt)

جدول 11.4 أوامر apt

apt ولا apt-get؟

`apt` معمول للاستخدام التفاعلي، وشكل ناتجه ممكن يتغير بين الإصدارات (وبيحذّرك: *apt does not have a stable CLI interface*). في الـ Scripts استخدم `apt-get` و `apt-cache`، نفس الأوامر تقريباً بس ثابتة.

11.6 على السيرفر: dnf في Fedora و RHEL و Rocky

لو الشركة اللي هتشتغل فيها شغالة على Red Hat أو Rocky Linux، هتلاقي `dnf` مكان apt. نفس الفكرة بالظبط، أسامي مختلفة شوية:

تشريح الأمر

```
sudo dnf install -y htop
     1   2      3  4
```

1. `dnf` مدير حزم عيلة Red Hat (حزم `.rpm`). من Fedora 41 الأمر `dnf` بقى بيشغّل **dnf5** (إعادة كتابة أسرع بـ C++)، والأوامر الأساسية زي ما هي.
2. `install` سطّب. ومفيش `update` منفصلة لازم قبلها: dnf بيحدّث الـ Metadata لوحده لما تبقى قديمة.
3. `-y` وافق تلقائي.
4. `htop` الحزمة.

وباقي الأوامر: `dnf upgrade` (رقّي الكل، و `dnf update` اسم تاني لنفس الحاجة)، و `dnf remove`، و `dnf search`، و `dnf info`، و `dnf list --installed`، و `dnf autoremove`. وعلى RHEL/CentOS القديم هتلاقي `yum` بنفس الأوامر تقريباً.

11.7 على Linux: pacman في Arch و Manjaro

Arch مش شائع على السيرفرات، بس هتقابل ناس كتير بيحبوه على أجهزتهم، وهتشوف أوامره في شروحات كتير. وأوامره شكلها غريب في الأول لأنها حروف بدل كلمات:

تشريح الأمر

```
sudo pacman -Syu neovim
     1   2345  5
```

1. `pacman` PACkage MANager بتاع Arch: الأوامر حروف كبيرة بدل كلمات.
2. `-S` Sync: سطّب من المخزن (العملية الأساسية).
3. `y` حدّث قاعدة البيانات (زي `apt update`).
4. `u` Upgrade: رقّي كل المتسطب. فـ `-Syu` = "حدّث كل الجهاز"، وكتابتها مع التسطيب بتمنع مشاكل الترقية الجزئية.
5. `neovim` الحزمة.

الأمر	بيعمل
`pacman -Ss word`	Search في المخزن
`pacman -Qi pkg`	Query info: معلومات حزمة متسطبة (`-Q` = المتسطب)
`pacman -Rns pkg`	Remove + الاعتماديات اللي ملهاش لازمة (s) + ملفات الإعدادات (n)
`pacman -Qdt`	الحزم اليتيمة (Dependencies محدش محتاجها)

جدول 11.5 حروف pacman

> **متعملش -Sy لوحدها**
>
> `pacman -Sy pkg` (من غير u) بتحدّث القائمة وتسطب حزمة جديدة على نظام قديم = "Partial upgrade"، ودي ممنوعة رسمياً في Arch وممكن تكسر مكتبات. يا `-S` بس، يا `-Syu` .

11.8 ولزمايلك على Windows: winget و choco

مش كل فريقك على ماك. لو زميلك على Windows سألك "أسطب Git إزاي من ال Terminal؟"، الإجابة دي:

تشريح الأمر

```
winget install --id Git.Git -e --source winget
```

1. `winget` Windows Package Manager الرسمي من Microsoft، جاي مع Windows 10 (الحديث) و 11 جوه "App Installer". بيشتغل في PowerShell أو cmd.
2. `install` سطّب.
3. `--id Git.Git` بال ID الدقيق (اعرفه بـ `winget search git`) بدل الاسم اللي ممكن يطابق كذا حاجة.
4. `-e` Exact: طابق ال ID بالظبط (حروف كبيرة وصغيرة).
5. `--source winget` من المخزن الرسمي (فيه كمان `msstore` = Microsoft Store).

تشريح الأمر

```
choco install nodejs-lts -y
```

1. `choco` Chocolatey: مدير حزم مجتمعي أقدم لـ Windows، لازم تسطبه الأول ومحتاج PowerShell كـ Administrator.
2. `install nodejs-lts` سطّب الحزمة.
3. `-y` وافق تلقائي (نفس apt و dnf).

11.9 جدول المقارنة الكبير

ودلوقتي الحصيلة. لو حفظت أوامر brew، إنت عملياً تعرف التانيين، لأن الأفكار واحدة. الجدول ده هتحتاجه أول مرة تدخل سيرفر:

العملية	brew	apt	dnf	pacman
حدّث القائمة	`update`	`update`	`makecache` (تلقائي)	`-Sy`
رقّي الكل	`upgrade`	`upgrade`	`upgrade`	`-Syu`
سطّب	`install`	`install`	`install`	`-S`
شيل	`uninstall`	`remove`	`remove`	`-R`
دوّر	`search`	`search`	`search`	`-Ss`
معلومات	`info`	`show`	`info`	`-Si` / `-Qi`
المتسطب	`list`	`list --installed`	`list --installed`	`-Q`
نضّف	`autoremove` / `cleanup`	`autoremove`	`autoremove`	`-Rns $(pacman -Qdtq)`
خدمات	`services`	`systemctl` (↩ 9.5 ص 103)		
صيغة الحزمة	Formula / Bottle / Cask	`.deb`	`.rpm`	`.pkg.tar.zst`
محتاج sudo/Admin	لأ	أيوه	أيوه	أيوه

جدول 11.6 نفس العملية في brew ومديري حزم Linux

العملية	brew	winget	choco
حدّث القائمة	`update`	`source update`	—
رقّي الكل	`upgrade`	`upgrade --all`	`upgrade all`
سطّب	`install`	`install`	`install`
شيل	`uninstall`	`uninstall`	`uninstall`
دوّر	`search`	`search`	`search`
معلومات	`info`	`show`	`info`
المتسطب	`list`	`list`	`list`
نضّف	`cleanup`	—	—
صيغة الحزمة	Cask / Formula	Manifest	`.nupkg`
محتاج sudo/Admin	لأ	أحياناً	أيوه

جدول 11.7 brew قصاد مديري Windows

نفس الكلمات في كل المديرين

لاحظ إن `install` و `search` و `info` شبه ثابتين في كل مكان. ده أسلوب **Subcommands** (`tool verb args`) زي `git commit` و `docker run` و `brew services start` (↩ 2.1 ص 17). pacman بس اللي بيستخدم حروف.

على Linux: flatpak و snap

دول مديري حزم "عابرين للتوزيعات": البرنامج بييجي ومعاه كل مكتباته في صندوق معزول. Ubuntu بيستخدم `snap` افتراضياً لحاجات زي Firefox، و Fedora بيفضّل `flatpak` للبرامج الرسومية. مميزاتهم: آخر إصدار وعزل أمني؛ عيوبهم: حجم أكبر وأحياناً بطء في الفتح. والأوامر: `snap install x` و `flatpak install flathub org.x.App` .

11.10 مشاكل شائعة

مشكلة 1 **zsh: command not found: brew**

سطبت Homebrew والتسطيب خلص بنجاح، بس الأمر مش موجود. عدّيت سطور "Next steps" في آخر التسطيب.

الحل: `echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile` وبعدين افتح تابة جديدة (أو `source ~/.zprofile`). اتأكد بـ `which brew` إنه بيقول `/opt/homebrew/bin/brew` .

مشكلة 2 Error: Cannot install in Homebrew on ARM processor in Intel default prefix (/usr/local)!

ال Terminal بتاعك شغال تحت **Rosetta** (المترجم اللي بيشغل برامج Intel على Apple Silicon)، فـ brew فاكر نفسه على Intel.

الحل: `uname -m` لازم يقول `arm64`؛ لو قال `x86_64`، اقفل ال Terminal، واعمل Get Info على Terminal.app وشيل علامة "Open using Rosetta"، وافتحه تاني.

مشكلة 3 brew doctor بيطلّع Warnings كتير

زي "Unbrewed header files" أو "config scripts in PATH". مش كلها مشاكل حقيقية.

الحل: اقرا كل Warning، هو بيقولك السبب والحل. لو brew شغال تمام ومفيش حاجة بايظة، أغلبها ينفع تتجاهلها. اللي بتهم: أي حاجة عن ال PATH أو صلاحيات `/opt/homebrew` (الحل غالباً `sudo chown -R $(whoami) /opt/homebrew`، ↩ 6.3 ص 61).

مشكلة 4 على Ubuntu: Could not get lock /var/lib/dpkg/lock-frontend

فيه apt تاني شغال (غالباً التحديث التلقائي في الخلفية).

الحل: استنى دقيقتين، أو اعرف مين: `ps aux | grep -E 'apt|dpkg'`. **متمسحش** ملف ال lock بإيدك إلا لو متأكد إن مفيش حاجة شغالة.

مشكلة 5 على Ubuntu: Unable to locate package

القائمة قديمة أو الاسم غلط أو الحزمة في مخزن مش مفعّل (universe مثلاً).

الحل: `sudo apt update` الأول، ثم `apt search` بجزء من الاسم.

ملخص الفصل

- Repository موثوق + Dependencies + Package بتتحل لوحدها. في brew: Formula للأدوات و Cask للبرامج و Bottle = نسخة جاهزة.
- Homebrew بيتسطب مرة بسطر `curl`، وبعدها `eval "$(/opt/homebrew/bin/brew shellenv)"` في `~/.zprofile`. `/opt/homebrew` على Apple Silicon و `/usr/local` على Intel.
- brew من غير sudo: `update` للقائمة، ثم `outdated`، و `upgrade` و `info` و `leaves` و `doctor` و `services`.
- `brew bundle dump` و `brew bundle` لنقل كل برامجك لماك جديد.
- على السيرفر apt: `update` للقائمة و `upgrade` للترقية، و `apt-get` في ال Scripts. dnf (dnf5 من Fedora 41) بيحدّث لوحده، و pacman `-Syu` دايماً. و winget الرسمي على Windows.

صح ولا غلط؟

1. `brew update` يبرقّي البرامج المتسطبة. صح / غلط
2. `apt purge` بيشيل ملفات الإعدادات كمان. صح / غلط
3. `brew install` محتاجة sudo. صح / غلط
4. على ماك Apple Silicon، Homebrew بيتسطب في `/opt/homebrew` . صح / غلط

؟ أسئلة مراجعة

1. إيه الفرق بين `brew update` و `brew upgrade` ؟ وإيه المقابل في apt؟
2. إيه الفرق بين Formula و Cask؟ اديني مثالين لكل واحد.
3. اشرح السطر `eval "$(/opt/homebrew/bin/brew shellenv)"` ، وليه بيتحط في `~/.zprofile` ؟
4. اكتب أمر تسطيب `git` على macOS و Ubuntu و Fedora و Arch و Windows.
5. ليه `pacman -Sy pkg` خطر؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعرف إصدار `wget` المتاح في Homebrew قبل ما تسطبه، وهل محتاج اعتماديات.
2. متوسط اعرض عدد الـ Formulae اللي إنت سطبتها بنفسك (من غير الاعتماديات) بـ Pipe واحد.
3. متوسط اعمل Brewfile لماكك في `~/dotfiles/Brewfile` ، وافتحه واتأكد إنه فيه كل برامجك، وجرّب `brew bundle check` عليه.
4. صعب اكتب سطر واحد يعرف مدير الحزم الموجود (brew أو apt أو dnf أو pacman) ويطبع اسمه، باستخدام `command -v` و `&&` و `||` .

12

المرحلة 5 · الأتمتة والتخصيص

الفصل التاني عشر

الـ Bash Scripting: خلّي الأوامر تشتغل لوحدها

إنت كمبرمج JavaScript عارف القاعدة الذهبية: لو كتبت نفس الكود 3 مرات، حطه في Function. الـ Terminal نفس الكلام بالظبط. كل يوم الصبح بتفتح المشروع، وتعمل git pull، وتشغل Postgres، وتشغل npm run dev، وتفتح VS Code؟ 5 أوامر كل يوم، 300 يوم في السنة. الـ Script هو الـ Function بتاعة الـ Terminal: ملف فيه نفس الأوامر اللي اتعلمتها، ومعاها متغيرات وشروط ولوبات ودوال، وتشغله بكلمة واحدة. ومش برمجة معقدة: لو فهمت الفصول اللي فاتت، إنت عملياً عارف 80% منه. الفصل ده بيبني معاك Scripts حقيقية خطوة خطوة، ويعلمك إزاي تخليها آمنة متكسرش في نص الشغل، وإزاي تكتبها بحيث تشتغل على الماك (bash 3.2 والأدوات الـ BSD) وعلى السيرفر Linux من غير مفاجآت.

بعد الفصل ده هتكون عارف

- الـ Shebang وتشغيل الـ Script، وإيه اللي بيحصل مع bash 3.2 على الماك
- المتغيرات والـ Quoting والحساب
- test و [ ] و [[ ]] والمقارنات
- if و case و for و while
- الدوال و local و return
- $1 و $@ و $# و shift
- set -euo pipefail و trap
- Scripts عملية كاملة تشتغل على الماك و Linux (date -v و stat -f و sed -i '')
- جدولة الـ Scripts بـ launchd و cron

12.1 أول Script: الـ Shebang

خلينا نبدأ صغير خالص. افتح VS Code (أو `nano hello.sh`) واكتب 3 سطور. السطر الأول غريب شوية، وده أهم سطر في الملف كله:

hello.sh

```
#!/usr/bin/env bash
# أول Script
echo "Hello, $USER! Today is $(date +%A)."
```

تشريح الأمر

```
#!/usr/bin/env bash
```

1. `#!` Shebang (bang + hash): لازم يكون أول حرفين في الملف. بيقول للـ Kernel "شغّل الملف ده بالبرنامج اللي بعدي".
2. `/usr/bin/env` برنامج بيدوّر على اللي بعده في الـ PATH (↩ 1.4 ص 12). أحسن من `#!/bin/bash` لأن bash ممكن يكون في مكان تاني. على الماك ده مهم جداً: لو سطبت bash 5 من Homebrew، هيلاقيه في `/opt/homebrew/bin/bash` الأول، أما `#!/bin/bash` فهيشغّل دايماً bash 3.2 القديم (↩ 3.4 ص 28).
3. `bash` المفسّر. لو كتبت `sh` هيبقى POSIX بس من غير مميزات bash (↩ 3.2 ص 27).

بتكتب لأنهي bash؟

جوه الـ Script اكتب `echo "$BASH_VERSION"` مرة وشوف: على ماك مفيهوش bash brew هيقولك `3.2.57(1)-release`، وعلى Ubuntu `5.x`. الفصل ده بيعلّم bash العادي اللي شغال في الاتنين، ولما حاجة تحتاج bash 4 أو أحدث هنقولك صراحة. والقاعدة: لو الـ Script هيتشغل على ماكات ناس تانية، اكتبه لـ bash 3.2، أو اطلب منهم `brew install bash`. ولو هو ليك إنت بس، ممكن كمان تكتبه لـ zsh بـ `#!/bin/zsh`، بس الـ Syntax بيختلف في حاجات (زي الـ Arrays تحت).

تشريح الأمر

```
chmod +x hello.sh && ./hello.sh
```

1. `chmod +x hello.sh` اديله صلاحية التشغيل (↩ 6.2 ص 59). من غيرها: `Permission denied` (Exit 126).
2. `&&` لو نجح.
3. `./hello.sh` شغّله. الـ `./` لازمة لأن المجلد الحالي مش في الـ PATH. وبديلها: `bash hello.sh` (ساعتها مش محتاج x ولا Shebang).

ShellCheck

سطّب `shellcheck` بـ `brew install shellcheck` (↩ 11.3 ص 131)، وفيه VS Code Extension بيعلّم على الغلط وإنت بتكتب زي ESLint بالظبط. وشغّله على أي Script: `shellcheck hello.sh`. بيلقط 90% من الغلطات الشائعة (Quotes ناقصة، متغيرات مش متعرفة...) قبل ما تكسر حاجة.

12.2 المتغيرات والحساب

في JavaScript بتكتب `let name = "Ahmed"`. في bash الفكرة نفسها، بس فيه قاعدة واحدة هتوقعك أول أسبوع لحد ما تتعود عليها: **ممنوع المسافات** حوالين الـ `=`. خلينا نشوف ليه:

تشريح الأمر

```
name="Ahmed"; count=3; echo "${name}_backup $((count * 2))"
```

1. `name="Ahmed"` تعريف متغير: **من غير مسافات** حوالين `=`. لو كتبت `name = "Ahmed"`، الـ Shell هيفهم إنك عايز تشغل أمر اسمه name.
2. `;` فاصل أوامر (↩ 7.7 ص 78).
3. `count=3` كل حاجة في bash نص؛ الرقم بيتعامل كرقم بس جوه الحساب.
4. `"${name}_backup` الأقواس `{}` بتحدد اسم المتغير بالظبط؛ من غيرها `$name_backup` هيدوّر على متغير اسمه name_backup.
5. `$((count * 2))` Arithmetic expansion: حساب أرقام صحيحة (`+ - * / %`)، وجواها المتغير من غير `$` عادي. النتيجة `Ahmed_backup 6`.

الشكل	يرجع	مثال (f=photo.jpg)
`${var:-def}`	القيمة، أو def لو فاضي	`${PORT:-8080}`
`${var:?msg}`	اقفل الـ Script بالرسالة لو فاضي	`${1:?need a file}`
`${#var}`	الطول	`9`
`${var%.*}`	شيل أقصر نهاية تطابق	`photo`
`${var##*.}`	شيل أطول بداية تطابق	`jpg`
`${var/old/new}`	استبدال أول تطابق (و `//` للكل)	`${f/jpg/png}`
`${var^^}` / `${var,,}`	كابيتال / سمول (bash 4+، **مش** في bash الماك 3.2)	`PHOTO.JPG`

جدول 12.1 توسيعات المتغيرات المفيدة

طب على الماك بـ bash 3.2، أعمل Uppercase إزاي؟ بأداة خارجية بتشتغل في كل مكان: `upper=$(printf '%s' "$f" | tr '[:lower:]' '[:upper:]')` (↩ 8.5 ص 88). وفي zsh فيه شكل خاص بيه: `${(U)f}` و `${(L)f}`.

تشريح الأمر

```
read -rp "Your name: " user
```

1. `read` — Builtin: اقرا سطر من stdin وحطه في متغير.
2. `-rp` — `-r` Raw: متفهمش `\` كـ Escape (استخدمها دايماً)، و `-p` Prompt: اطبع السؤال ده الأول. (و `-s` Silent للباسوردات).
3. `"Your name: "` — نص السؤال.
4. `user` — اسم المتغير (من غير `$` وإنت بتعرّفه).

Arrays

`files=(a.txt "my file.txt" c.txt)` ثم `"${files[@]}"` = كل العناصر كل واحد Argument، و `${#files[@]}` = العدد، و `${files[0]}` = الأول. (الـ Index بيبدأ من 0 في bash ومن 1 في zsh!، يعني نفس السطر بيطبع `echo ${files[1]}` في bash و `my file.txt` في zsh و `a.txt`). الـ Arrays العادية دي شغالة في bash 3.2، أما الـ Associative arrays (زي Object في JS: `declare -A user`) فمحتاجة bash 4+ (↩ 3.4 ص 28).

12.3 الشروط: test و [ ] و [[ ]]

هنا أول مكان الـ bash هيستغرب فيه مخك الـ JavaScript. في JS بتكتب `if (size > 100)` والـ `>` عملية مقارنة. في bash `>` معناها Redirection (↩ 7.2 ص 73)، وهتعمل ملف اسمه 100! السبب إن `if` في bash بتشتغل بطريقة مختلفة خالص من جذورها: `if` مش بتقيّم "صح وغلط" زي اللغات التانية: هي بتشغّل أمر وتشوف الـ **Exit code** (0 = صح، ↩ 7.7 ص 78). و `[` نفسها **أمر** (اسم تاني لـ `test`)، وعشان كده المسافات حواليها إجبارية.

تشريح الأمر

```
size=$(stat -f%z "$file"); if [[ -f "$file" && $size -gt 100 ]]; then echo big; fi
```

1. `size=$(stat -f%z "$file")` — حجم الملف بالبايت. `stat -f%z` دي صيغة الماك (BSD)؛ على Linux بتبقى `stat -c%s` (↩ 2.6 ص 23). ولو عايز سطر يشتغل في الاتنين: `wc -c < "$file"` .
2. `if` — شغّل اللي بعدي، ولو رجع 0 نفّذ الـ then.
3. `[[` — الـ Test المطوّر في bash/zsh: أأمن من `[` (مش بيعمل Word splitting ولا Glob للمتغيرات، وبيفهم `&&` و `||` و `=~` جواه). مسافة بعده إجباري.
4. `-f "$file"` — الملف موجود وملف عادي.
5. `&&` — AND جوه الـ `[[ ]]` . (في `[ ]` القديمة: `[ ] && [ ]`).
6. `$size -gt 100` — Greater Than: مقارنة **أرقام**.
7. `]]` — النهاية، بمسافة قبلها.
8. `; then` — الـ `;` بدل سطر جديد، و then = نفّذ لو صح.
9. `echo big; fi` — الأمر، و `fi` (if بالمقلوب) = النهاية.

الاختبار	صح لو	الاختبار	صح لو
`-e f`	موجود (أي نوع)	`-z "$s"`	النص فاضي (Zero)
`-f f`	ملف عادي	`-n "$s"`	النص مش فاضي
`-d f`	مجلد	`"$a" == "$b"`	نصين متساويين (`=` في `[`)
`-L f`	Symlink	`"$a" != "$b"`	مختلفين
`-r` / `-w` / `-x`	تقدر تقرا / تكتب / تشغّل (↩ 6.1 ص 58)	`$s =~ ^[0-9]+$`	Regex (في `[[` بس، ↩ 8.1 ص 82)
`-s f`	موجود وحجمه أكبر من صفر	`-eq -ne`	أرقام: يساوي / لا يساوي
`f1 -nt f2`	f1 أحدث (Newer Than)	`-lt -le -gt -ge`	أقل / أقل أو يساوي / أكبر / أكبر أو يساوي

جدول 12.2 اختبارات test و [[ ]]

> ⚠ **== و -eq مش زي بعض**
>
> `[[ 10 == 010 ]]` غلط (نصوص مختلفة)، و `[[ 10 -eq 010 ]]` صح (أرقام). و `<` و `>` جوه `[[ ]]` مقارنة **أبجدية** مش أرقام، وجوه `[ ]` هتبقى Redirection وهتعمل ملف! للأرقام استخدم `-lt` أو `(( a < b ))` .

تشريح الأمر

```
case "$1" in start|up) run ;; stop) halt ;; *) echo "usage" ;; esac
```

1. `case "$1" in` — قارن قيمة أول Argument بالأنماط اللي جاية.
2. `start|up)` — نمط (Glob مش Regex) و `|` = أو، و `)` نهاية النمط.
3. `run ;;` — الأوامر، و `;;` = خلصنا الحالة دي.
4. `stop) halt ;;` — حالة تانية.
5. `*)` — أي حاجة تانية (Default).
6. `esac` — case بالمقلوب = النهاية.

12.4 اللوبات: for و while

نقلت 200 صورة من الـ iPhone للماك، ولقيتها كلها `.heic`، والموقع اللي عايز ترفع عليه مبيقبلش غير JPG. هتفتح كل صورة في Preview وتعمل Export؟ 200 مرة؟ دي بالظبط وظيفة اللوب، والماك فيه أداة صور جاهزة اسمها `sips`:

تشريح الأمر

```
for f in *.heic; do sips -s format jpeg "$f" --out "${f%.heic}.jpg"; done
```

1. `for f in` — لكل عنصر في القائمة، حطه في المتغير `f`. (زي `for (const f of files)` في JS).
2. `*.heic` — القائمة: الـ Glob بيتوسع لأسماء الملفات (↩ 7.4 ص 75). ينفع `{1..10}` أو `"$@"` أو `$(cat list)`. خلي بالك إن الماك مش حساس لحالة الحروف في أسامي الملفات، بس الـ Glob حساس: `*.heic` مش هيلقط `IMG_1.HEIC`؛ اكتب `*.[hH][eE][iI][cC]` أو فعّل `shopt -s nocaseglob`.
3. `; do` — بداية الجسم.
4. `sips -s format jpeg "$f" --out "${f%.heic}.jpg"` — Scriptable Image Processing System: أداة جاية مع الماك. `-s format jpeg` = غيّر الصيغة، و `--out` = احفظ هنا، و `${f%.heic}` = الاسم من غير الامتداد. (على Linux المقابل `convert` أو `magick` من ImageMagick، وموجودة على الماك كمان بـ `brew install imagemagick`).
5. `; done` — النهاية.

تشريح الأمر

```
1     2       3                  4                  5
while IFS= read -r line; do echo "> $line"; done < urls.txt
```

1. `while` كرر طول ما الأمر بعدي يرجع 0.
2. `IFS=` Internal Field Separator فاضي **للأمر ده بس**: متشيلش المسافات من أول وآخر السطر.
3. `read -r line` اقرا سطر في `line` . لما الملف يخلص يرجع 1 فاللوب يقف.
4. `; do echo "> $line"; done` الجسم.
5. `< urls.txt` الـ stdin بتاع اللوب كله من الملف (↩ 7.2 ص 73). ده الطريقة الصح لقراية ملف سطر سطر، مش `for l in $(cat f)` اللي بتقطع على المسافات.

`for ((i=1; i<=5; i++)); do …; done`	لوب بعداد على طريقة C.
`until ping -c1 -W1000 host &>/dev/null; do sleep 2; done`	`until` = كرر لحد ما ينجح: "استنى لحد ما السيرفر يرد". (`-W1000` = ثانية على الماك بالملي، وعلى Linux `-W1` ، ↩ 10.1 ص 113).
`break / continue`	اخرج من اللوب / روح للّفة الجاية.

12.5 الدوال

الدوال في bash هي نفس فكرة الـ Functions في JS: كود بتكتبه مرة وتنادي عليه كتير. بس فيه 3 فروق هتلاحظها: مفيش Parameters بالاسم (بتوصلها بـ `$1` و `$2`)، والمتغيرات Global افتراضياً (عكس `let`)، و `return` مبترجّعش قيمة. خلينا نعمل دالة `log` هتستخدمها في كل Script بعد كده:

Terminal

```
log() {
    local level="$1"; shift
    printf '[%s] %s: %s\n' "$(date +%T)" "$level" "$*" >&2
}
log INFO "Backup started"
```

تشريح الأمر

```
log() { local level="$1"; shift; printf '%s\n' "$*" >&2; }
```

1. `log()` تعريف دالة اسمها log. الأقواس فاضية دايماً (الـ Arguments مش بتتكتب هنا).
2. `{` بداية الجسم (مسافة بعدها، و `;` قبل `}` لو على سطر واحد).
3. `local level="$1"` = المتغير ده جوه الدالة بس، ومن غيرها بيبقى Global ويخبط في باقي الـ Script. و `$1` هنا = أول Argument **للدالة** مش للـ Script.
4. `shift` زقّ الـ Arguments: الـ `$2` يبقى `$1` ... (تحت).
5. `printf '%s\n' "$*"` اطبع كل الباقي كنص واحد. `printf` أدق وأضمن من `echo` في الـ Scripts.
6. `>&2` = `1>&2` : ودّي الناتج لـ stderr، عشان رسايل اللوج متتخلطش بالبيانات (↩ 7.1 ص 72).
7. `}` النهاية.

return مقابل echo

`return N` بترجّع **Exit code** بس (0 لـ 255). عشان ترجّع قيمة، اطبعها واستلمها بـ `$()` : `result=$(get_ip)` (↩ 7.6 ص 77).

12.6 الـ Arguments: $1 و $@ و $#

لما تكتب `ls -la ~/Downloads`، الـ `-la` و `~/Downloads` دول Arguments بيوصلوا لبرنامج ls. والـ Script بتاعك نفس الكلام: أي حاجة تكتبها بعد اسمه بتوصله في متغيرات جاهزة (زي `process.argv` في Node):

المتغير	القيمة	المعنى
`$0`	`./deploy.sh`	اسم الـ Script
`$1` `$2` `$3`	`prod` و `v 2` و `-f`	كل Argument برقمه (و `${10}` بأقواس بعد 9)
`$#`	`3`	العدد
`"$@"`	`"prod" "v 2" "-f"`	كلهم، كل واحد لوحده (**ده اللي هتستخدمه**)
`"$*"`	`"prod v 2 -f"`	كلهم كنص واحد

جدول 12.3 متغيرات الـ Arguments (لـ `./deploy.sh prod "v 2" -f`)

تشريح الأمر

```
[[ $# -lt 1 ]] && { echo "usage: $0 <env>" >&2; exit 2; }
```

1. `[[ $# -lt 1 ]]` لو عدد الـ Arguments أقل من 1...
2. `&&` ...نفّذ اللي بعده.
3. `{ echo "usage: $0 <env>" >&2;` اطبع طريقة الاستخدام لـ stderr. الـ `<env>` جوه Quotes فمش Redirection.
4. `exit 2; }` اقفل الـ Script بـ Exit code 2 (استخدام غلط). والـ `{ ;}` بتجمع الأمرين (↩ 7.7 ص 78).

Options زي الأوامر الحقيقية

عايز الـ Script بتاعك ياخد `-v` و `-o file` زي `ls`؟ استخدم `getopts` Builtin:

```
while getopts "vo:" opt; do case $opt in v) verbose=1;; o) out=$OPTARG;; esac; done
```

. الـ `:` بعد o = "بتاخد قيمة" (↩ 2.2 ص 18).

12.7 الوضع الصارم: set -euo pipefail و trap

أغرب حاجة في bash للي جاي من JavaScript: لو أمر فشل في نص الـ Script، bash **بيكمل عادي** على السطر اللي بعده! مفيش Exception بيوقف كل حاجة. تخيل Script بيعمل `cd build` (وفشل لأن الفولدر مش موجود) وبعدين `rm -rf *`. هيمسح الفولدر اللي إنت فيه دلوقتي. السطر الجاي هو "الـ Strict mode" بتاع bash، وبيتكتب في أول أي Script جاد:

تشريح الأمر

```
set -euo pipefail
```

1. `set` Builtin بيغير سلوك الـ Shell.
2. `-e` Errexit: أول أمر يفشل، اقفل الـ Script فوراً بدل ما تكمّل على أساس غلط.
3. `u` Nounset: لو استخدمت متغير مش متعرّف (غلطة في الاسم)، اعتبره خطأ واقفل. ده بيمنع كارثة `rm -rf "$DIR/"` لما DIR يطلع فاضي (↩ 15.1 ص 182). وفخ خاص بالماك: في bash أقدم من 4.4 (يعني 3.2 بتاع الماك)، `"${arr[@]}"` لـ Array **فاضية** بيعتبرها مش متعرّفة ويقفل بـ `unbound variable`. الحل الآمن هناك: `${arr[@]+"${arr[@]}"}`.
4. `o pipefail` الـ Pipe يعتبر فاشل لو **أي** أمر فيه فشل، مش آخر واحد بس (افتراضياً `false | true` = نجاح!).

⚠ e- مش سحر

`set -e` مبيشتغلش جوه شرط `if` أو قبل `&&` و `||`، ومبيقفلش لو الأمر الفاشل جوه `$()` في بعض الحالات. اعتبره شبكة أمان، مش بديل إنك تفحص الأوامر المهمة بنفسك. والأمر اللي متوقع يفشل عادي: `grep x f || true`.

تشريح الأمر

```
tmp=$(mktemp -d); trap 'rm -rf "$tmp"' EXIT
```

1. `tmp=$(mktemp -d)` اعمل مجلد مؤقت باسم عشوائي آمن واحفظ مساره (`-d` = Directory).
2. `;` بعدها.
3. `trap` Builtin: "لما يحصل الحدث ده، نفّذ الأمر ده".
4. `'rm -rf "$tmp"'` الأمر، في Single quotes عشان يتفك وقت التنفيذ مش دلوقتي.
5. `EXIT` الحدث: خروج الـ Script بأي طريقة (نجاح أو فشل أو Ctrl+C). وينفع `INT` و `TERM` (↩ 9.3 ص 100).

12.8 Scripts عملية كاملة

كفاية قطع غيار، خلينا نركّب العربية. أول Script: كل يوم عايز نسخة من فولدر المشروع متضغطة وعليها التاريخ، ومش عايز الـ Backups تاكل الهارد، فنسيب آخر 7 بس. ده كان شغل 10 دقايق يدوي كل يوم، وهيبقى سطر واحد:

backup.sh: باك أب بتاريخ ويمسح القديم

```
#!/usr/bin/env bash
set -euo pipefail

SRC="${1:?usage: backup.sh <dir> [dest]}"
DEST="${2:-$HOME/backups}"
KEEP=7
stamp=$(date +%F_%H%M)
name="$(basename "$SRC")_$stamp.tar.gz"

mkdir -p "$DEST"
tar -czf "$DEST/$name" -C "$(dirname "$SRC")" "$(basename "$SRC")"
echo "saved: $DEST/$name ($(du -h "$DEST/$name" | cut -f1))"

# keep newest $KEEP only
ls -1t "$DEST"/*.tar.gz | tail -n +$((KEEP + 1)) | while IFS= read -r old; do rm -v -- "$old"; done
```

كل سطر فيه حاجة اتعلمتها: `${1:?}` (فوق)، و `tar -C` (↩ 10.8 ص 123)، و `du -h | cut -f1` (↩ 8.4 ص 87)، و `tail -n +8` = من السطر التامن للآخر، و `while IFS= read -r` (↩ 12.4 ص 148). ليه مش زي ما `xargs -r rm` هتلاقي في شروحات Linux؟ لأن الـ `-r` دي GNU بس، و xargs بتاع الماك ممكن يرفضها، واللوب بـ while شغال في الاتنين، ولو مفيش ولا ملف قديم مش بيعمل حاجة (↩ 8.9 ص 90). والـ `ls -1t` هنا آمن لأن إحنا اللي عاملين الأسماء (من غير مسافات). والـ Script ده كله اتكتب عشان يشتغل على الماك و Linux من غير تعديل: `date +%F` و `tar -czf` و `du -h` نفس الشكل في الاتنين.

تشريح الأمر

```
days_ago() { date -v-"$1"d +%F 2>/dev/null || date -d "$1 days ago" +%F; }
```

1. `days_ago()` دالة بترجع تاريخ من N يوم. محتاجينها لأن حساب التواريخ هو أكتر مكان الماك و Linux بيختلفوا فيه (↩ 2.6 ص 23).
2. `date -v-"$1"d` صيغة الماك (BSD): `-v` = Adjust value، و `-7d` = اطرح 7 أيام. وتقدر تجمع: `date -v-1m -v+2d` (شهر لورا ويومين لقدام). وعلى Linux `-v` مش موجودة فهتفشل.
3. `+%F` الشكل `2026-10-15`، نفسه في الاتنين.
4. `2>/dev/null ||` لو فشلت (يعني إحنا على Linux)، اخفي رسالة الخطأ وجرّب اللي بعدها (↩ 7.7 ص 78).
5. `date -d "$1 days ago"` صيغة GNU على Linux: `-d` = Date string بالإنجليزي.

عايز	الماك (BSD)	Linux (GNU)
تاريخ من أسبوع	`date -v-7d +%F`	`date -d '7 days ago' +%F`
من Timestamp لتاريخ	`date -r 1760000000`	`date -d @1760000000`
تاريخ تعديل ملف	`date -r file.txt` أو `stat -f %Sm file.txt`	`date -r file.txt` أو `stat -c %y file.txt`
حجم ملف بالبايت	`stat -f%z file`	`stat -c%s file`
تعديل ملف في مكانه	`sed -i '' 's/a/b/' f`	`sed -i 's/a/b/' f`
يشتغل في الاتنين	`sed 's/a/b/' f > f.tmp && mv f.tmp f`، و `wc -c < f`، و `find . -mtime +7`	

جدول 12.4 التواريخ والملفات في الـ Scripts: الماك قصاد Linux

تاني Script: عندك لستة مواقع (موقعك، الـ API، صفحة الـ Docs) وعايز تعرف كلهم شغالين ولا لأ في ثانية:

sitecheck.sh: افحص قائمة مواقع

```
#!/usr/bin/env bash
set -uo pipefail
fail=0
while IFS= read -r url; do
    [[ -z "$url" || "$url" == \#* ]] && continue
    code=$(curl -s -o /dev/null -m 10 -w '%{http_code}' "$url")
    if [[ "$code" == 2* || "$code" == 3* ]]; then
        printf '%-40s OK  %s\n' "$url" "$code"
    else
        printf '%-40s BAD %s\n' "$url" "$code"; fail=1
    fi
done < "${1:-sites.txt}"
exit "$fail"
```

هنا مستخدمين `-e` لأ عن قصد، عشان موقع واقع ميوقفش الفحص. و `\#*` = سطر بيبدأ بـ # (كومنت)، و `-m 10` = استنى 10 ثواني بالكتير، و `%-40s` = نص بعرض 40 متساوي على الشمال. وفي الآخر الـ Exit code بيقول لو فيه فشل، فينفع تحطه في cron أو CI (↩ 10.2 ص 115).

شغّله أوتوماتيك: launchd (و cron)

الـ Script جاهز، بس لسه إنت اللي بتشغله. الخطوة الأخيرة إنه يشتغل لوحده كل يوم. على الماك الطريقة اللي Apple بتوصي بيها هي LaunchAgent (↩ 9.5 ص 103) بمفتاح `StartCalendarInterval`:

جزء من ~/Library/LaunchAgents/com.ahmed.backup.plist

```
<key>ProgramArguments</key>
<array>
  <string>/Users/ahmed/bin/backup.sh</string>
  <string>/Users/ahmed/projects/app</string>
</array>
<key>StartCalendarInterval</key>
<dict>
  <key>Hour</key>   <integer>3</integer>
  <key>Minute</key> <integer>0</integer>
</dict>
<key>StandardOutPath</key>  <string>/Users/ahmed/backup.log</string>
<key>StandardErrorPath</key><string>/Users/ahmed/backup.log</string>
```

الميزة الكبيرة على cron: لو الماك كان نايم أو مقفول الساعة 3، launchd بيشغّل الـ Script أول ما يصحى، و cron كان هيفوّتها خالص. (كمّل الملف بـ `Label` زي المثال في فصل العمليات، وسجّله بـ `launchctl bootstrap gui/$(id -u) ...`).

cron: على السيرفر (وشغال على الماك كمان)

`crontab -e` وضيف سطر: `0 3 * * * /home/ahmed/bin/backup.sh /home/ahmed/docs >> /home/ahmed/backup.log 2>&1`. الخمس خانات: دقيقة، ساعة، يوم الشهر، الشهر، يوم الأسبوع (0 = الأحد). يعني كل يوم الساعة 3 الفجر. استخدم مسارات مطلقة لأن cron بيشتغل بـ PATH صغير جداً (↩ 13.5 ص 163). وعلى الأنظمة اللي فيها systemd فيه بديل اسمه systemd timers. و cron لسه شغال على الماك بنفس الشكل (بمسارات `/Users/...`)، بس عشان يوصل لفولدرات زي Documents و Desktop لازم تدي `/usr/sbin/cron` صلاحية **Full Disk Access** (↩ 6.8 ص 69)، وده من أسباب إن launchd أريح.

12.9 مشاكل شائعة

مشكلة 1 — bad interpreter: /bin/bash^M

الملف اتكتب على Windows (أو زميلك بعتهولك منه) ونهاية كل سطر فيها `\r\n` بدل `\n`.

الحل: على الماك: `sed -i '' $'s/\r$//' script.sh`، والـ `$'...'` لازمة لأن sed بتاع الماك مبيفهمش `\r` لوحده، فالـ Shell هو اللي بيحولها. على Linux: `sed -i 's/\r$//' script.sh`. أو `brew install dos2unix`. وظبط VS Code على LF من الشريط اللي تحت (↩ 8.5 ص 88).

مشكلة 2 — declare: -A: invalid option أو bad substitution

الـ Script شغال على السيرفر، ولما شغلته على الماك وقع عند `declare -A` أو `${var,,}` أو `mapfile`.

الحل: دي مميزات bash 4+ والماك بيشغّل 3.2. يا إما `brew install bash` واستخدم `#!/usr/bin/env bash` (مش `#!/bin/bash`)، يا إما اكتبها بطريقة 3.2 (↩ 3.4 ص 28).

مشكلة 3 — date: illegal option -- d

نسخت سطر `date -d yesterday` من شرح على النت والماك رفضه.

الحل: على الماك `date -v-1d`، أو استخدم دالة `days_ago` اللي فوق عشان تشتغل في الاتنين، أو `brew install coreutils` واستخدم `gdate -d yesterday`.

مشكلة 4 — [: too many arguments

كتبت `[ $name == "Ahmed Ali" ]` والمتغير فيه مسافة فاتقسم لكلمتين.

الحل: `"$name"` بالـ Quotes، أو استخدم `[[ ]]` (↩ 7.5 ص 76).

ملخص الفصل

- `#!/usr/bin/env bash` + `chmod +x` + `./script`.
- `var=value` من غير مسافات، و `"$var"` و `${var:-default}` و `$(( ))`.
- `if` بتشوف الـ Exit code، و `[[ ]]` أأمن، و `-eq` للأرقام و `==` للنصوص.
- `for f in *` و `while IFS= read -r line; done < file`.
- الدوال بـ `local`، و `"$@"` لكل الـ Arguments، و `$#` عددهم.
- `set -euo pipefail` و `trap ... EXIT` و shellcheck.
- على الماك: `#!/usr/bin/env bash`، وخلي بالك من bash 3.2 (مفيش `declare -A` ولا `${v,,}`)، و `date -v` و `stat -f` و `sed -i ''`. واجدول بـ LaunchAgent أو cron.

صح ولا غلط؟

1. `name = "Ali"` طريقة صحيحة لتعريف متغير. صح / غلط
2. `[` أمر حقيقي وعشان كده محتاج مسافات حواليه. صح / غلط
3. `return` في bash ممكن ترجّع نص. صح / غلط
4. من غير `pipefail`، فشل أول أمر في Pipe مش بيأثر على الـ Exit code. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `"$@"` و `"$*"`؟
2. اكتب شرط يتأكد إن أول Argument مجلد موجود، ولو لأ يقفل بـ Exit code 1.
3. ليه `for line in $(cat file)` غلط؟ وإيه البديل؟
4. اشرح كل حرف في `set -euo pipefail`.
5. إمتى تستخدم `trap`؟
6. اكتب سطر بيطبع تاريخ امبارح ويشتغل على الماك و Linux.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل Script بياخد اسم ويطبع "أهلاً يا NAME"، ولو مفيش اسم يسأل بـ read.
2. متوسط Script بيرتب ملفات مجلد `~/Downloads` في مجلدات حسب الامتداد (pdf و jpg و zip و dmg...). جرّبه الأول على نسخة في `/tmp`.
3. صعب Script فيه Options بـ getopts: `-n` Dry run و `-d dir`، بيمسح الملفات الأقدم من 30 يوم في المجلد، ويطبع الإجمالي.

13

المرحلة 5 · الأتمتة والتخصيص

الفصل التالت عشر

تخصيص الـ Shell: ملفات الإعدادات والـ Aliases والـ Prompt

عملت alias حلو، وقعدت تجربه وانبسطت، وقفلت الـ Terminal. تاني يوم فتحته، لقيته اختفى كأنه محصلش.
أو سطبت Homebrew، وفي التابة دي شغال، وفي التابة الجديدة zsh بيقولك command not found:
brew. الحاجتين دول سببهم واحد: الـ Shell بيصحى كل مرة

بعد الفصل ده هتكون عارف

- أنهي ملف بيتقري إمتى: login مقابل interactive، ولي Terminal الماك بيفتح login دايماً
- zshenv. و zprofile. و zshrc. على الماك، و bashrc. و bash_profile. على السيرفر
- path_helper و /etc/paths وليه brew shellenv في zprofile.
- الـ Aliases وإمتى تستخدم دالة بدلها
- PATH وإزاي تضيف مجلد صح (و path array في zsh)
- متغيرات البيئة و export، والأسرار في Keychain
- تخصيص الـ Prompt في zsh (و bash)

13.1 مين بيقرا إيه وإمتى؟

تخيل موظف استقبال في فندق، كل يوم الصبح بيجي الشغل وهو فاقد الذاكرة تماماً. مش فاكر اسمه، ولا الغرف، ولا أي حاجة. فالمدير ساب له على المكتب دفتر تعليمات، وأول حاجة بيعملها كل يوم إنه يقراه من الأول للآخر. الـ Shell هو الموظف ده بالظبط: كل تابة جديدة بتفتحها هي Shell جديد "ناسي" كل حاجة، وأول ما يصحى بيقرا ملفات الإعدادات بتاعته. اللي مكتوب فيها يفتكره، واللي إنت كتبته في الـ Terminal بس (زي الـ alias بتاع امبارح) راح مع التابة اللي اتقفلت.

طب أنهي دفتر؟ هنا الحكاية بتتعقد شوية، لأن الـ Shell بيتفتح بطريقتين، وكل طريقة بتقرا ملفات مختلفة:

Login shell

"أول دخول": SSH على سيرفر، أو `su -`، أو... **كل تابة في Terminal.app و iTerm2 على الماك**. الماك بيفتح login دايماً، على عكس Linux. اعرف في zsh بـ `[[ -o login ]] && echo login`.

Interactive non-login

لما تكتب `zsh` أو `bash` جوه Shell، أو كل تاب Terminal جديد على Linux Desktop. وفي الاتنين الـ Shell "تفاعلي": مستنيك تكتب.

والـ zsh (الـ Shell الافتراضي على الماك، ↩ 3.4 ص 28) عنده 4 ملفات في الـ Home، بيتقروا بالترتيب ده في Login shell (اللي هو كل تابة على الماك):

الترتيب	الملف	بيتقري إمتى	تحط فيه إيه
1	`~/.zshenv`	**دايماً**، حتى في الـ Scripts و `zsh -c`	تقريباً ولا حاجة. متغيرات بسيطة جداً لو لازم
2	`/etc/zprofile` ثم `~/.zprofile`	Login بس	`eval "$(/opt/homebrew/bin/brew shellenv)"` وإضافات الـ PATH
3	`/etc/zshrc` ثم `~/.zshrc`	أي Shell تفاعلي	**كل حاجة تانية**: aliases ودوال و Prompt و setopt و Plugins
4	`~/.zlogin`	Login، بعد .zshrc	نادراً ما هتحتاجه

جدول 13.1 ترتيب ملفات zsh في تابة Terminal جديدة على الماك

القاعدة العملية على الماك

اتنين بس: `~/.zprofile` فيه سطر Homebrew والـ PATH، و `~/.zshrc` فيه كل الباقي. وانسى `.zshenv` و `.zlogin` لحد ما تحتاجهم فعلاً. ولو الملف مش موجود اعمله: `touch ~/.zshrc`، وافتحه في VS Code بـ `code ~/.zshrc` أو `open -e ~/.zshrc` في TextEdit.

لو حابب تتعمق

تقدر تعدّيه في أول قراية

ليه الـ PATH مكانه `.zprofile` مش `.zshenv`، رغم إن الاتنين بيتقروا؟ بسبب برنامج صغير اسمه **path_helper**. ملف `/etc/zprofile` بتاع Apple فيه سطر بيشغّله، وهو بيبني الـ PATH من الأول من ملف `/etc/paths` (فيه `/usr/local/bin` و `/usr/bin` و `/bin` و `/usr/sbin` و `/sbin`) ومن الملفات اللي في `/etc/paths.d/`، وبعدين يحط القديم بتاعك **بعدهم**. فلو حطيت `/opt/homebrew/bin` في `.zshenv`، path_helper هيزقه لآخر الـ PATH، وساعتها `git` بتاع Apple القديم هيكسب على git بتاع brew (↩ 1.4 ص 12). أما `.zprofile` بتاعك فبيتقري **بعد** `/etc/zprofile`، فاللي تحطه فيه هو اللي يكسب. وبالمناسبة، `/etc/zshrc` بتاع Apple كمان بيعمل حاجات حلوة لوحده: بيظبط الـ History، وبيقرا `/etc/zshrc_Apple_Terminal` اللي بيخلي Terminal.app يفتكر المجلد بتاع كل تابة ويرجعها زي ما كانت بعد الـ Restart. متعدّلش في ملفات `/etc` دي؛ أي تحديث للماك ممكن يرجّعها.

وعلى bash (السيرفر، أو لو غيرت الـ Shell)

bash عنده نظام شبه كده بأسامي تانية، وده اللي هتقابله على كل سيرفر Linux. والجدول ده فيه كل الـ Shells مع بعض للمقارنة:

الـ Shell	Login	Interactive	ملاحظة
bash	`/etc/profile` ثم أول واحد موجود من: `~/.bash_profile` ← `~/.bash_login` ← `~/.profile`	`~/.bashrc`	Ubuntu و Debian: `~/.profile` بيعمل source لـ `~/.bashrc`
zsh	`~/.zshenv` ثم `~/.zprofile` ثم `~/.zshrc` ثم `~/.zlogin`	`~/.zshenv` ثم `~/.zshrc`	`.zshenv` بيتقري دايماً حتى في Scripts

ال Shell	Login	Interactive	ملاحظة
fish	`~/.config/fish/config.fish` (والملفات في `conf.d/`)		جوه الملف `status is-login`
PowerShell	الملف اللي في `$PROFILE`		اعرف مكانه: `echo $PROFILE`
sh/dash	`~/.profile`	الملف اللي في `$ENV`	POSIX (↩ 3.2 ص 27)

جدول 13.2 ملفات البدء لكل Shell

> **القاعدة العملية لـ bash**
>
> حط كل حاجة (aliases ودوال و Prompt) في `~/.bashrc` ، وخلي `~/.bash_profile` سطر واحد بيقرا الـ bashrc: `[[ -f ~/.bashrc ]] && source ~/.bashrc` . كده الإعدادات واحدة في كل الحالات. وده مهم أكتر على الماك لو غيرت الـ Shell لـ bash: Terminal.app بيفتح Login، فبيقرا `~/.bash_profile` بس ومبيبصش على `~/.bashrc` خالص.

تشريح الأمر

```
source ~/.zshrc
```

1. `source` — Builtin: نفّذ الملف ده **جوه الـ Shell الحالي** (مش في Shell جديد زي `./script`)، عشان التغييرات (aliases ومتغيرات) تفضل. اختصارها النقطة: `. ~/.zshrc` (↩ 1.3 ص 11).
2. `~/.zshrc` — الملف. بعد أي تعديل اعمل source أو افتح تاب جديد (Cmd+T)، أو `exec zsh` يبدأ Shell نضيف مكان الحالي. (في bash: `source ~/.bashrc`).

13.2 الـ Aliases: اختصارات

هتلاحظ بعد أسبوعين إنك بتكتب نفس الأوامر الطويلة كل شوية: `ls -lah` و `git status` و `cd ~/projects/my-app` .
الـ Alias هو "اسم دلع" للأمر: كلمة قصيرة إنت تختارها، والـ Shell يبدلها بالأمر الطويل قبل ما ينفذ:

تشريح الأمر

```
alias ll='ls -lahG'
```

1. `alias` — Builtin: لما أكتب كلمة، استبدلها بنص تاني قبل التنفيذ.
2. `ll` — الاسم الجديد. **مفيش مسافات** حوالين `=` (زي المتغيرات بالظبط، ↩ 12.2 ص 144).
3. `='ls -lahG'` — الأمر الحقيقي، في Single quotes عشان يتحفظ زي ما هو. و `-G` = ألوان في ls بتاع الماك (↩ 5.1 ص 48).

```
~/.zshrc: aliases مفيدة على الماك
alias ll='ls -lahG'
alias ..='cd ..'
alias ...='cd ../..'
alias rm='rm -I'              # اسأل مرة لو أكتر من 3 ملفات
alias cp='cp -i' mv='mv -i'
alias grep='grep --color=auto'
alias ports='lsof -nP -iTCP -sTCP:LISTEN'
alias myip='ipconfig getifaddr en0'
alias update='brew update && brew upgrade && brew cleanup'
alias flushdns='sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder'
alias o='open .'              # افتح المجلد الحالي في Finder
alias zshrc='code ~/.zshrc'
```

لاحظ إن كل واحد فيهم حاجة اتعلمناها: `lsof` من فصل الشبكات (↩ 10.9 ص 125)، و `brew` من فصل الحزم (↩ 11.3 ص 131)، و `open` من أدوات الماك (↩ 14.3 ص 171). ده الهدف: ال Aliases بتاعتك هي "ذاكرة" كل اللي اتعلمته.

نفس الفكرة على سيرفر Ubuntu

```
~/.bashrc على السيرفر
alias ll='ls -lah --color=auto'
alias ports='sudo ss -tulnp'
alias update='sudo apt update && sudo apt upgrade'
```

ال `--color=auto` بتاعة GNU ls، و `ss` بدل lsof، و apt بدل brew (↩ 2.6 ص 23).

`alias`	من غير Arguments: اعرض كل ال Aliases.
`type ll`	ll ده إيه؟ (alias ولا دالة ولا برنامج، ↩ 1.3 ص 11).
`unalias ll`	امسحه من ال Session دي.
`\rm file`	ال `\` قبل الأمر = تجاهل ال Alias مرة واحدة (أو `command rm`).
`alias -s json=code`	zsh بس: Suffix alias. اكتب اسم ملف `data.json` لوحده ويتفتح في VS Code.

⚠ الـ Alias مبيقبلش Arguments في النص

ال Alias بيستبدل الكلمة الأولى بس، والباقي بيتحط في الآخر. فلو عايز Argument في النص (`mkdir X && cd X`)، استخدم **دالة**. وال Aliases مبتشتغلش جوه ال Scripts افتراضياً، فمتعتمدش عليها هناك.

13.3 دوال في ملف الإعدادات

لما ال Alias ميكفيش، يعني محتاج Argument في النص أو شرط أو أكتر من خطوة، هنا بقى بييجي دور الدالة. ودي نفس الدوال اللي كتبناها في ال Scripts، بس بتعيش في ال Shell بتاعك على طول:

~/.zshrc: دوال

```
mkcd() { mkdir -p -- "$1" && cd -- "$1"; }
extract() {
    case "$1" in
        *.tar.gz|*.tgz) tar -xzf "$1" ;;
        *.tar.xz)       tar -xJf "$1" ;;
        *.zip)          unzip "$1" ;;
        *) echo "unknown: $1" >&2; return 1 ;;
    esac
}
bak() { cp -a -- "$1" "$1.$(date +%F).bak"; }
```

نفس الدوال اللي في ال Scripts (↩ 12.5 ص 149)، بس هنا بتعيش في ال Shell بتاعك. وال `--` معناها "خلصت ال Options"، عشان لو اسم المجلد بيبدأ بـ `-` (↩ 2.3 ص 19). و `mkcd` لازم تبقى دالة مش Script، لأن Script بيشتغل في Shell تاني وال `cd` بتاعه مبيأثرش عليك.

تشريح الأمر

```
cdf() { cd "$(osascript -e 'tell app "Finder" to POSIX path of (insertion location as alias)')"; }
```

1. `cdf()` — "cd to Finder": دالة بتنقلك للفولدر المفتوح قدامك في ال Finder دلوقتي. عكس `open .` بالظبط.
2. `osascript -e '...'` — شغّل AppleScript من ال Terminal. ده "الكوبري" بين ال Terminal وبرامج الماك (Finder و Music و Safari...).
3. `tell app "Finder" to POSIX path of (insertion location as alias)` — اسأل ال Finder عن الفولدر اللي في الشباك الأمامي، ورجّعه كمسار Unix عادي (`Users/ahmed/Downloads/`). أول مرة الماك هيسألك تسمح لل Terminal يتحكم في Finder (↩ 6.8 ص 69).
4. `cd "$( ... )"` — روح للمسار ده، وال Quotes عشان المسافات في الأسامي (↩ 7.5 ص 76).

13.4 متغيرات البيئة و export

افتكر `process.env.API_URL` في Node. منين بييجي ال `API_URL` ده؟ من هنا بالظبط: من ال Shell اللي شغّل Node. فيه نوعين متغيرات: متغير **Shell** عادي (ال Shell بس اللي يشوفه، زي متغير `let` جوه Function)، ومتغير **بيئة** (Environment)

بيتورّث لأي برنامج بيتشغل من ال Shell، و Node من ضمنهم.

تشريح الأمر

```
export EDITOR="code --wait" API_URL="https://api.example.com"
```

1. `export` — Builtin: خلي المتغيرات دي جزء من البيئة، فـ git و crontab و Node وأي Script يشوفوها.
2. `EDITOR="code --wait"` — متغير مشهور: المحرر اللي البرامج بتفتحه (git commit و crontab -e و visudo).
   `code --wait` = افتح VS Code واستنى لحد ما تقفل الملف (من غير `--wait` هيفتكر git إنك خلصت فوراً). لو بتحب ال Terminal: `nano` أو `vim`.
3. `API_URL="..."` — ممكن أكتر من واحد في نفس السطر. والأسماء بالكابيتال اصطلاح لمتغيرات البيئة. وده اللي هيوصل لـ `process.env.API_URL`.

Terminal

```
user@MacBook ~ % city=Cairo; zsh -c 'echo "child sees: $city"'
child sees:
user@MacBook ~ % export city; zsh -c 'echo "child sees: $city"'
child sees: Cairo
user@MacBook ~ % TZ=Asia/Tokyo date      # variable for this command only
Fri Oct 23 03:00:00 JST 2026
user@MacBook ~ % PORT=4000 npm run dev   # same idea: process.env.PORT = "4000"
```

المتغير	فيه إيه
`HOME` / `USER` / `SHELL`	ال home / اسمك / ال Login shell
`PATH`	مجلدات البرامج (تحت)
`PWD` / `OLDPWD`	المجلد الحالي / اللي قبله (`cd -`)
`EDITOR` / `VISUAL` / `PAGER`	المحرر / ال Pager (less)
`LANG` / `LC_ALL`	اللغة والترميز (`en_US.UTF-8`)، بتأثر على sort والتاريخ
`TERM` / `TERM_PROGRAM`	نوع ال Terminal (`xterm-256color`) / اسم البرنامج على الماك: `Apple_Terminal` أو `iTerm.app` أو `vscode` (↩ 1.1 ص 8)
`HISTFILE` / `SAVEHIST` / `HISTSIZE`	حجم ال History في الذاكرة / في الملف (zsh) / ومكانه: `~/.zsh_history` (↩ 14.4 ص 174)

جدول 13.3 متغيرات بيئة مهمة

`env`	اعرض كل متغيرات البيئة (أو `printenv HOME` لواحد).
`unset API_URL`	امسح المتغير.

`env -i zsh -f`	Shell نضيف من غير أي متغيرات ولا إعدادات (`-f` = متقراش ولا ملف rc). أحسن طريقة تعرف المشكلة في الـ `.zshrc` بتاعك ولا لأ. (في bash: `env -i bash --norc`).

> **الأسرار**
>
> متحطش Tokens وباسوردات في `~/.zshrc` لو ملف الإعدادات ده بيترفع على GitHub (dotfiles). حطها في ملف منفصل `~/.secrets` بصلاحية `600` واعمله source (↩ 6.2 ص 59)، وفي المشاريع استخدم ملف `.env` ومعاه سطر في `.gitignore`. وعلى الماك فيه حل أشيك: الـ **Keychain**.

تشريح الأمر

```
export GITHUB_TOKEN="$(security find-generic-password -s github-token -w)"
```

1 `security`	أداة الماك للتعامل مع الـ Keychain (نفس المكان اللي Safari بيحفظ فيه باسوردااتك، ومتشفر بباسورد الماك).
2 `find-generic-password -s github-token`	دوّر على باسورد اسم الخدمة (Service) بتاعته `github-token`.
3 `-w`	اطبع الباسورد نفسه بس. ومرة واحدة قبل كده تكون حفظته: `security add-generic-password -a "$USER" -s github-token -w` (من غير قيمة بعد `-w` هيسألك عليها، فمتتكتبش في الـ History).
4 `export ... "$( )"`	حطه في متغير بيئة. كده ملف `.zshrc` ينفع يترفع على GitHub، لأن مفيهوش أي سر.

13.5 الـ PATH: ضيف مجلد صح

افتكر رسالة `zsh: command not found: brew` من فصل الحزم. أو إنك نزّلت أداة، وحطيتها في فولدر، وكتبت اسمها والـ Shell مش لاقيها. الـ Shell مش بيدوّر في الماك كله على البرنامج (ده هياخد دقايق مع كل أمر!). هو بيدوّر في لستة فولدرات محددة بس، بالترتيب، ودي اسمها **PATH** (↩ 1.4 ص 12). خلينا نشوفها على ماك عليه Homebrew:

Terminal

```
user@MacBook ~ % echo "$PATH" | tr ':' '\n'
/opt/homebrew/bin
/opt/homebrew/sbin
/usr/local/bin
/System/Cryptexes/App/usr/bin
/usr/bin
/bin
/usr/sbin
/sbin
```

أول اتنين جايين من `brew shellenv`، والباقي من `/etc/paths` اللي path_helper بيقراه (فوق). و `/System/Cryptexes/...` ده فولدر Apple بتحدّث منه أدوات النظام بسرعة، متقلقش منه. طب عايز تضيف فولدر لأدواتك إنت؟

تشريح الأمر

```
export PATH="$HOME/.local/bin:$PATH"
```

1. `export` متغير بيئة (فوق).
2. `PATH=` بنعيد تعريف الـ PATH...
3. `"$HOME/.local/bin` المجلد الجديد. استخدم `$HOME` مش `~` لأن الـ `~` جوه الـ Quotes مبتتوسعش.
4. `:` الفاصل بين المجلدات.
5. `$PATH"` القيمة القديمة كلها. **متنساهاش**، وإلا `ls` و `cat` هيختفوا. والترتيب مهم: اللي في الأول بيكسب (↩ 1.4 ص 12).

السطر ده مكانه `~/.zprofile` تحت سطر Homebrew. وفي zsh فيه طريقة أنضف: الـ PATH موجود كمان كـ **Array** اسمها `path` (بحروف صغيرة) ومربوطة بيه أوتوماتيك:

تشريح الأمر

```
typeset -U path; path=("$HOME/.local/bin" $path)
```

1. `typeset -U path` Unique: امنع التكرار في الـ Array دي. من غيرها كل `source ~/.zprofile` هيضيف نفس الفولدر تاني، والـ PATH يطول ويطول.
2. `path=( ... $path)` الـ Array الجديدة = الفولدر بتاعي + الـ Array القديمة. ومن غير `:` ولا Quotes على `$path`، zsh بيفهم.

بعد ما تضيف برنامج

الـ Shell بيفتكر أماكن البرامج (Hash table)، فلو نقلت أو سطبت برنامج والـ Shell لسه مش شايفه، اعمل `rehash` في zsh (أو `hash -r` في bash). و `which -a python3` بيعرض كل النسخ بالترتيب: غالباً هتلاقي `/opt/homebrew/bin/python3` وبعده `/usr/bin/python3` بتاع Apple. ولو خرّبت الـ PATH بالغلط في الـ Session دي: `export PATH=/opt/homebrew/bin:/usr/bin:/bin:/usr/sbin:/sbin` وصلّح الملف.

13.6 الـ Prompt على مزاجك

الـ Prompt ده الحتة اللي بتبص عليها آلاف المرات في اليوم: `ahmed@MacBook ~ %`. ليه متخليهاش تقولك حاجة مفيدة؟ إنت في أنهي فولدر، وأنهي Git branch، وهل آخر أمر نجح ولا فشل. في zsh الـ Prompt هو قيمة المتغير `PROMPT` (وفيه كمان `RPROMPT` بيظهر على يمين السطر)، وشفنا أجزاءه في ↩ 1.6 ص 14:

تشريح الأمر

```
PROMPT='%F{green}%n@%m%f %F{blue}%1~%f %(?.%F{green}.%F{red})%#%f '
```

1. `PROMPT=` — المتغير، و Single quotes عشان الرموز توصل زي ما هي.
2. `%F{green}…%f` — لون: `%F{اللون}` ابدأ، و `%f` رجّع العادي. الألوان بالاسم (`red green blue yellow magenta cyan`) أو برقم من 0 لـ 255 (`%F{208}` برتقالي). وأسهل بكتير من أكواد bash.
3. `%n@%m` — `%n` = اسم المستخدم، و `@` حرفياً، و `%m` = اسم الجهاز.
4. `%1~` — آخر جزء من المسار بس (`~/projects/my-app` بدل `my-app`). و `%~` = المسار كله.
5. `%(?.X.Y)` — شرط: لو آخر أمر نجح (Exit 0) اكتب X، غير كده Y (↩ 7.7 ص 78). هنا X = أخضر و Y = أحمر.
6. `%#` — لو مستخدم عادي و `%` لو root، والمسافة في الآخر عشان الكلام ميلزقش فيه.

ضيف الـ Git branch

في zsh فيه Module جاهز اسمه `vcs_info`. ضيف دول في `~/.zshrc`:

```
s_info; precmd() { vcs_info; }; setopt prompt_subst; RPROMPT='%F{yellow}${vcs_info_msg_0_}%f'
```

و `precmd` دالة zsh بيشغلها قبل ما يرسم كل Prompt، و `prompt_subst` بتخليه يفك `${...}` جوه الـ Prompt كل مرة. هتلاقي `(git)-[main]-` على يمين السطر جوه أي Repo.

وفي bash: PS1

على السيرفر (أو لو بتستخدم bash) الـ Prompt اسمه `PS1` (و `PS2` هو الـ `>` بتاع السطر الكمّلة)، والرموز مختلفة والألوان أصعب شوية:

على Linux (السيرفرات)

تشريح الأمر

```
1      2          3       4     5       6          7
PS1='\[\e[1;32m\]\u@\h\[\e[0m\]:\[\e[1;34m\]\w\[\e[0m\]\$ '
```

1. `PS1=` المتغير، و Single quotes عشان الـ `\` توصل زي ما هي.
2. `\[\e[1;32m\]` لون: `\e[` = بداية كود لون (ANSI)، و `1` = Bold، و `32` = أخضر، و `m` النهاية. و `\[ \]` حواليه = "دي حروف مش بتاخد مكان"، من غيرها السطر يتلخبط لما يطول.
3. `\u@\h` `\u` = اسم المستخدم، و `@` حرفياً، و `\h` = اسم الجهاز.
4. `\[\e[0m\]` Reset: رجّع اللون الطبيعي.
5. `:` حرفياً.
6. `\[\e[1;34m\]\w` أزرق، و `\w` = المسار الحالي (و `\W` = آخر جزء بس).
7. `\[\e[0m\]\$` Reset، و `\$` = `$` لو مستخدم عادي و `#` لو root، ومسافة.

المعنى	bash (`PS1`)	zsh (`PROMPT`)
المستخدم	`\u`	`%n`
الجهاز	`\h`	`%m`
المسار كامل / آخر جزء	`\w` / `\W`	`%~` / `%1~`
$ أو #	`\$`	`%#` (بيطلع %)
الوقت	`\t`	`%*`
Exit code آخر أمر	`$?` (جوه دالة)	`%?`
لون	`\[\e[32m\]`	`%F{green}…%f`
سطر جديد	`\n`	`$'\n'`

جدول 13.4 رموز الـ Prompt: bash مقابل zsh

الطريق السهل: Starship

`starship` Prompt جاهز وسريع بيشتغل على zsh و bash و fish و PowerShell بنفس الإعدادات: بيعرض فرع git ونسخة Node/Python ووقت آخر أمر. سطّبه بـ `brew install starship`، وضيف `eval "$(starship init zsh)"` في آخر `~/.zshrc` (وفي bash: `starship init bash` في `~/.bashrc`). أغلب الـ Themes بتحتاج **Nerd Font** عشان الأيقونات: `brew install --cask font-jetbrains-mono-nerd-font` واختاره من إعدادات Terminal.app أو iTerm2. وفي zsh كمان فيه Themes مع oh-my-zsh (↩ 14.8 ص 179).

13.7 مشاكل شائعة

مشكلة 1 الـ alias اشتغل وبعد ما قفلت اختفى

كتبته في الـ Terminal بس، ومحطيتوش في ملف.

الحل: `echo "alias ll='ls -lahG'" >> ~/.zshrc && source ~/.zshrc`. خد بالك من `>>` مش `>`، وإلا هتمسح الملف كله (↩ 15.5 ص 186).

مشكلة 2 brew شغال في تابة ومش شغال في التانية

عملت `eval "$(/opt/homebrew/bin/brew shellenv)"` في التابة بس، أو حطيته في `~/.zshrc` وإنت فاتح bash، أو كتبته في `~/.bash_profile` وإنت على zsh.

الحل: اعرف الـ Shell بـ `echo $SHELL` (↩ 3.8 ص 32)، وحط السطر في `~/.zprofile` لـ zsh (أو `~/.bash_profile` لـ bash)، وافتح تابة جديدة.

مشكلة 3 zsh compinit: insecure directories, run compaudit for list

رسالة بتطلع مع كل تابة بعد ما سطبت Homebrew أو oh-my-zsh: zsh لاقي فولدرات الـ Completion ليها صلاحية كتابة للـ Group، واعتبرها خطر.

الحل: `compaudit` بيعرض الفولدرات، وبعدين `compaudit | xargs chmod g-w` يشيل صلاحية الكتابة للـ Group منها (↩ 6.2 ص 59).

مشكلة 4 على macOS الـ bashrc. مش بيتقري

لو غيرت الـ Shell لـ bash: الـ Terminal الـ macOS بيفتح Login shell، فبيقرا `~/.bash_profile` بس. ولو الـ Shell zsh (الافتراضي من 2019) يبقى الملف `~/.zshrc` أصلاً.

الحل: اعرف الـ Shell بـ `echo $SHELL`، ولو bash ضيف سطر الـ source في `~/.bash_profile`.

ملخص الفصل

- كل تابة على الماك Login shell. zsh بيقرا `.zshenv` ← `.zprofile` ← `.zshrc` ← `.zlogin`. اتنين بس بيهموك: `.zprofile` لـ Homebrew والـ PATH، و `.zshrc` لكل الباقي.
- `path_helper` في `/etc/zprofile` بيعيد ترتيب الـ PATH، وعشان كده brew shellenv في `.zprofile`.
- على السيرفر bash: Login بيقرا `.bash_profile` / `.profile`، و Interactive بيقرا `.bashrc`.
- `source` بينفّذ الملف في الـ Shell الحالي. `alias x='...'` للاختصار البسيط، والدالة لما تحتاج Arguments.
- `export` بيورّث المتغير للبرامج (`process.env`)، و `PATH="$HOME/bin:$PATH"` أو `path=(... $path)`، والأسرار في الـ Keychain بـ `security`.
- `PROMPT` في zsh (`%n %m %~ %#` و `%F{color}`)، و `PS1` في bash، أو Starship.

صح ولا غلط؟

1. كل تابة جديدة في Terminal.app بتفتح Login shell، فبتقرا `~/.zprofile`. صح / غلط
2. `./myscript.sh` فيه `cd /tmp` هيغيّر مجلدك الحالي. صح / غلط
3. متغير من غير export مبيوصلش للبرامج اللي بتشغلها. صح / غلط
4. أحسن مكان لسطر `brew shellenv` هو `~/.zshenv` عشان بيتقري دايماً. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين login و interactive shell؟ وإيه اللي بيفتحه Terminal.app؟ وأنهي ملف لكل واحد في zsh وفي bash؟
2. اكتب alias اسمه `gs` لـ `git status` وخليه دايم.
3. إيه الغلط في `export PATH="~/bin"`؟
4. ليه `mkcd` لازم تبقى دالة مش Script؟
5. إيه هو path_helper، وإزاي بيأثر على مكان سطر Homebrew؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل ضيف 3 aliases بتستخدمها كل يوم لـ `~/.zshrc` وجرّبهم في تاب جديد.
2. متوسط اعمل مجلد `~/.local/bin`، وحط فيه Script، وضيفه للـ PATH في `~/.zprofile`، وشغّله من أي مكان باسمه بس.
3. متوسط احفظ Token وهمي في الـ Keychain بـ `security add-generic-password`، واقراه في متغير بيئة من `~/.zshrc`، واتأكد إن `node -e 'console.log(process.env.MY_TOKEN)'` بيطبعه.
4. صعب اعمل Prompt يبقى أحمر لو آخر أمر فشل وأخضر لو نجح، وفيه الـ Git branch على اليمين (`%(?.%F{green}.%F{red})` و `vcs_info`). ولو عندك سيرفر: نفس الفكرة في bash بـ `PROMPT_COMMAND`.

14

المرحلة 5 · الأتمتة والتخصيص

الفصل الرابع عشر

السرعة والإنتاجية: الاختصارات والـ History و tmux

أكيد اتفرجت على فيديو لمبرمج شاطر، وهو بيكتب في الـ Terminal والأوامر بتطير قدامه، وإنت لسه بتدوس السهم الشمال 40 مرة عشان تصلح حرف في أول السطر. السر مش إنه حافظ أوامر أكتر منك؛ السر إنه مبيكتبش نفس الحاجة مرتين، وإن صوابعه عارفة شوية اختصارات. الفصل ده عن الحاجات اللي هتوفرلك ساعات: التنقل في السطر (وإزاي تخلي زرار Option يشتغل على الماك)، واختصارات Terminal.app و iTerm2، والبحث في الـ History، والـ Tab completion، وبعدين صندوق

بعد الفصل ده هتكون عارف

- اختصارات الكيبورد في السطر (Readline/ZLE)، و Option كـ Meta على الماك
- Terminal.app مقابل iTerm2 واختصارات Cmd
- أدوات الماك: open و pbcopy/pbpaste و say و caffeinate و defaults و softwareupdate و screencapture
- البحث في الـ History بـ Ctrl+R و !! و !$
- الـ Tab completion
- tmux: Sessions و Windows و Panes و Detach
- fzf للبحث التفاعلي
- oh-my-zsh: إيه هو وإمتى يستاهل

14.1 اختصارات السطر

كتبت أمر طويل: `rsync -avzP --delete ./site/ prod:/var/www/site/`، واكتشفت إن فيه غلطة في أول كلمة. هتمسك السهم الشمال وتستنى؟ لأ. `Ctrl+A` وإنت في أول السطر في لحظة. الاختصارات دي جاية من مكتبة اسمها **Readline** (اللي bash بيستخدمها)، و zsh (بمحرر السطر بتاعه اسمه ZLE) و fish بيدعموا نفسها تقريباً. ونفسها شغالة في `python` و `mysql` وأي برنامج بيستخدم Readline، وكمان معظمها في macOS في أي مربع نص. حاجة مشتركة تتعلمها مرة.

الاختصار	بيعمل
`Ctrl+A` / `Ctrl+E`	أول السطر / آخره
`Alt+B` / `Alt+F`	كلمة لورا / لقدام (على الماك `Option` = Alt، وشوف التنبيه تحت)
`Ctrl+W`	امسح الكلمة اللي قبل المؤشر
`Ctrl+U`	امسح من المؤشر لأول السطر (في zsh: السطر كله)

الاختصار	بيعمل
`Ctrl+K`	امسح من المؤشر لآخر السطر
`Ctrl+Y`	الصق آخر حاجة اتمسحت بـ W أو U أو K
`Ctrl+R`	دوّر في الـ History (تحت)
`Ctrl+L`	امسح الشاشة (زي `clear`)
`Ctrl+C`	الغي الأمر الشغال أو السطر (SIGINT)
`Ctrl+D`	End-of-file: لو السطر فاضي = اخرج من الـ Shell
`Ctrl+Z`	وقّف الأمر مؤقتاً (SIGTSTP، ↩ 9.4 ص 101)
`Alt+.`	حط آخر Argument من الأمر اللي فات (اضغط تاني للأقدم)
`Ctrl+X Ctrl+E`	افتح السطر في المحرر (`$EDITOR`) للأوامر الطويلة. في bash جاهز، وفي zsh محتاج 3 سطور في `~/.zshrc`: `t-command-line; zle -N edit-command-line; bindkey '^X^E' edit-command-line`

جدول 14.1 أهم الاختصارات

زرار Option مش شغال كـ Alt؟

جرّبت `Option+B` في Terminal.app وطلعلك حرف غريب زي `∫` بدل ما ترجع كلمة؟ ده طبيعي: الماك بيستخدم Option عشان يكتب رموز خاصة. عشان يشتغل كـ **Meta/Alt**: في **Terminal.app**: Settings ← Profiles ← Keyboard ← علّم على *Use Option as Meta key*. وفي **iTerm2**: Settings ← Profiles ← Keys ← خليها *Left Option key* `Esc+`. بعدها كل اختصارات Alt في الكتاب ده (و fzf تحت) هتشتغل. وحاجة كمان في Terminal.app: `Option+Click` في أي حتة في السطر بينقل المؤشر هناك على طول.

Ctrl+S جمّد الشاشة؟

لو دوست `Ctrl+S` بالغلط والـ Terminal "وقف"، ده XOFF قديم بيوقف الناتج. `Ctrl+Q` بترجّعه. وتقدر تقفل الميزة دي بـ `stty -ixon` في `~/.zshrc` (أو `~/.bashrc`)، وساعتها Ctrl+S تبحث لقدام في الـ History.

Ctrl+C في الـ Terminal مش نسخ

في الـ Terminal، `Ctrl+C` = SIGINT (↩ 9.3 ص 100). وده من أحلى حاجات الماك: النسخ واللصق Cmd+C/Cmd+V زي أي برنامج، فمفيش لخبطة. على Linux بيبقوا `Ctrl+Shift+C` و `Ctrl+Shift+V`، وفي Windows Terminal `Ctrl+C` بتنسخ لو فيه نص متحدد.

14.2 Terminal.app و iTerm2: اختصارات Cmd

الماك جاي معاه **Terminal.app**، وهو كويس جداً وكفاية لأغلب الناس. بس هتسمع كتير عن **iTerm2** (`brew install --cask iterm2`)، وهو الأشهر بين المطوّرين على الماك: بيقسم الشاشة، وفيه بحث أقوى، و Hotkey window تنزل من فوق بزرار واحد، وألوان وإعدادات أكتر. وفيه كمان بدائل أحدث زي **Ghostty** و **Warp**، والـ Terminal اللي جوه VS Code (`Ctrl+`). كلهم بيشغّلوا نفس zsh ونفس الأوامر (↩ 1.1 ص 8)؛ الفرق في "الشباك" بس. والاختصارات دي شغالة في Terminal.app و iTerm2:

الاختصار	بيعمل
Cmd+T / Cmd+N	تابة جديدة / شباك جديد (بنفس الفولدر الحالي)
Cmd+W	اقفل التابة (بيسألك لو فيه حاجة شغالة)
Cmd+Shift+[/ Cmd+Shift+]	التابة اللي قبل / اللي بعد، و Cmd+1...Cmd+9 برقمها
Cmd+K	امسح الشاشة **والـ Scrollback** كله (أقوى من `Ctrl+L` اللي بيزق بس)
Cmd+F	دوّر في الناتج اللي على الشاشة
Cmd++ / Cmd+- / Cmd+0	كبّر / صغّر الخط / رجّعه
Cmd+D / Cmd+Shift+D	iTerm2: قسّم الشاشة رأسي / أفقي (و Cmd+Option+الأسهم للتنقل). في Terminal.app Cmd+D بتقسم نفس التابة لجزئين تتفرج بيهم على الـ Scrollback
Cmd+Click على رابط	افتح الـ URL في المتصفح (في Terminal.app: Cmd+Double-click)
اسحب ملف من Finder للـ Terminal	بيكتب مساره كامل في السطر، ومعمول له Escape للمسافات (↩ 4.3 ص 41)
Cmd+,	الإعدادات: الخط والألوان والـ Profiles

جدول 14.2 اختصارات شباك الـ Terminal على الماك

14.3 أدوات الماك اللي مش في أي حتة تانية

لحد دلوقتي أغلب اللي اتعلمناه بيشتغل على الماك و Linux مع شوية فروقات. بس الماك عنده صندوق أدوات خاص بيه، بتربط الـ Terminal بباقي الماك: الـ Finder والـ Clipboard والصوت والإعدادات. ودي الحاجات اللي هتخلي الناس تبصلك وتقول "إنت عملت ده إزاي؟!". خلينا نفتح الصندوق حاجة حاجة:

open: افتح أي حاجة

بدل ما تروح للـ Finder وتدوّر على الفولدر اللي إنت فيه في الـ Terminal، اكتب `open .` وهيتفتح قدامك. و `open` بتعمل بالظبط اللي الدبل كليك بيعمله: الملف يتفتح بالبرنامج الافتراضي بتاعه.

تشريح الأمر

```
open -a "Visual Studio Code" .
```

1. `open` افتح الحاجة دي زي ما الـ Finder بيفتحها (ملف أو فولدر أو URL أو برنامج).
2. `-a "Visual Studio Code"` Application: بالبرنامج ده بالذات بدل الافتراضي. الاسم زي ما هو في `/Applications` من غير `.app`، وفي Quotes عشان المسافات.
3. `.` الفولدر الحالي (↩ 4.3 ص 41). هيفتح المشروع كله في VS Code.

`open .`	افتح الفولدر الحالي في Finder.
`open report.pdf`	افتحه بالبرنامج الافتراضي (Preview).
`open https://localhost:3000`	افتح الرابط في المتصفح الافتراضي.
`open -R package.json`	Reveal: افتح Finder وعلّم على الملف ده.
`open -e notes.txt`	افتحه في TextEdit.
`open -a Safari index.html`	افتح صفحة HTML في Safari بالذات (عشان تجرب الموقع عليه).

على Linux: xdg-open

على Linux Desktop المقابل اسمه `xdg-open file`، وعلى Windows `start file`. وعلى السيرفر مفيش أصلاً، مفيش شاشة!

pbcopy و pbpaste: الـ Clipboard من الـ Terminal

عايز تنسخ المفتاح العام بتاع ssh عشان تلزقه في GitHub. هتعمل `cat` وتحدد بالماوس وتتمنى متكونش نسيت حرف؟ لأ:

تشريح الأمر

```
pbcopy < ~/.ssh/id_ed25519.pub
```

1. `pbcopy` PasteBoard copy: أي حاجة توصله على stdin بتروح الـ Clipboard، كأنك عملت Cmd+C.
2. `< ~/.ssh/id_ed25519.pub` خلي الـ stdin بتاعه هو الملف ده (↩ 7.2 ص 73). دلوقتي روح GitHub ودوس Cmd+V.

`pwd | pbcopy`	انسخ المسار الحالي.
`pbpaste | jq . | pbcopy`	خد JSON من الـ Clipboard، ونسّقه بـ `jq`، ورجّعه للـ Clipboard (Pipe ببدأ وبيخلص في الـ Clipboard!) (↩ 8.11 ص 94).

`pbpaste > data.csv`	الصق اللي نسخته من Excel أو Numbers في ملف.
`git log --oneline -5 | pbcopy`	آخر 5 Commits جاهزين تلزقهم في Slack.

say و osascript: الماك بيكلمك

شغّلت Build هياخد 5 دقايق، وقعدت تبص على الشاشة مستنيه. ليه؟ خلي الماك يقولك لما يخلص:

تشريح الأمر

```
npm run build && say "build done" || say "build failed"
```

1. `npm run build` الأمر الطويل.
2. `&& say "build done"` لو نجح (Exit 0)، الماك يقول الجملة دي بصوت عالي (↩ 7.7 ص 78). و `say -v '?'` بيعرض كل الأصوات المتاحة.
3. `|| say "build failed"` لو فشل، يقول دي.

ولو مش عايز صوت، اعمل Notification عادي زي أي برنامج: `osascript -e 'display notification "Build finished" with title "npm"'`. و `osascript` ده نفسه اللي استخدمناه في دالة `cdf` (↩ 13.3 ص 161): بيشغّل AppleScript، يعني يقدر يتحكم في أي برنامج على الماك تقريباً.

defaults: الإعدادات المخفية

كل إعداد في الماك، حتى اللي مش ظاهر في System Settings، متخزن في ملفات plist (↩ 9.5 ص 103). و `defaults` هي الأداة اللي بتقرا وتكتب فيهم:

تشريح الأمر

```
defaults write com.apple.screencapture location ~/Pictures/Screenshots
```

1. `defaults write` اكتب إعداد. (و `read` اقرا، و `delete` رجّعه للافتراضي).
2. `com.apple.screencapture` الـ Domain: البرنامج أو الجزء من النظام صاحب الإعداد. ده بتاع الـ Screenshots.
3. `location ~/Pictures/Screenshots` المفتاح وقيمته: احفظ الـ Screenshots هنا بدل الـ Desktop المزحوم. (اعمل الفولدر الأول بـ `mkdir -p`).

`defaults write com.apple.finder AppleShowAllFiles -bool true; killall Finder`	اعرض الملفات المخفية في Finder على طول (زي `Cmd+Shift+.` بس دايم). و `killall Finder` عشان يعيد التشغيل ويقرا الإعداد (↩ 9.3 ص 100).
`defaults read com.apple.dock autohide`	اقرا قيمة إعداد واحد. ومن غير المفتاح: كل إعدادات الـ Dock.

`defaults delete com.apple.dock autohide; killall Dock`	رجّع الإعداد للافتراضي.

defaults قوية بس بحذر

إنت بتكتب في إعدادات البرامج مباشرة. اقرا القيمة الأول (`defaults read`) واحفظها عشان ترجعلها. وفيه مواقع كاملة بتجمع أوامر defaults مفيدة؛ متشغلش أي أمر منها قبل ما تفهمه، نفس قاعدة ↩ 15.6 ص 187.

وكمان...

الأمر	بيعمل
`caffeinate -i cmd`	امنع الماك ينام طول ما الأمر شغال، و `-d` الشاشة كمان (↩ 9.4 ص 101)
`mdfind -name invoice`	دوّر بـ Spotlight في ثانية (↩ 8.10 ص 91)
`softwareupdate --list`	تحديثات macOS المتاحة. و `sudo softwareupdate --install --all` سطّبها، و `--history` اللي اتسطب قبل كده
`screencapture -i -c`	Screenshot: `-i` حدد بالماوس (زي Cmd+Shift+4)، و `-c` للـ Clipboard بدل ملف. و `screencapture -T 5 shot.png` بعد 5 ثواني
`sw_vers`	نسخة macOS بالظبط (`ProductVersion: 27.0.1` مثلاً)
`system_profiler SPHardwareDataType`	الموديل والمعالج والرامات والـ Serial
`pmset -g batt`	نسبة البطارية وهل الشاحن موصّل
`qlmanage -p file.png`	Quick Look (زي المسافة في Finder) من الـ Terminal
`shortcuts run "My Shortcut"`	شغّل Shortcut عملته في برنامج Shortcuts
`networkQuality`	Speed test (↩ 10.1 ص 113)
`diskutil list`	كل الـ Disks والـ Partitions (↩ 15.2 ص 183)

جدول 14.3 صندوق أدوات الماك

14.4 الـ History: متكتبش حاجة مرتين

امبارح كتبت أمر ssh طويل فيه بورت و IP ومفتاح، واشتغل بعد 20 محاولة. النهارده محتاجه تاني. فاكره؟ طبعاً لأ. بس الـ Shell فاكر. كل أمر بتكتبه بيتحفظ (في zsh: `~/.zsh_history` أو اللي في `HISTFILE`، وفي bash: `~/.bash_history`). والأسهم فوق وتحت بتمشي فيه، بس فيه طريقة أسرع بكتير:

```
Terminal
bck-i-search: ssh_
ssh -p 2222 deploy@203.0.113.10
```

ده شكل zsh. في bash نفس الفكرة بتظهر كده: `(reverse-i-search)`ssh': ssh -p 2222 ...` .

`Ctrl+R` واكتب أي جزء من الأمر: بيلاقي آخر أمر فيه الكلمة دي. `Ctrl+R` تاني = اللي أقدم منه، و `Enter` = نفّذ، و `→` أو `Ctrl+E` = حطه في السطر عشان تعدّل، و `Ctrl+G` = الغي.

الشكل	بيتحول لـ	مثال
`!!`	الأمر اللي فات كله	`sudo !!` نسيت sudo (↩ 6.6 ص 66)
`!$`	آخر Argument في الأمر اللي فات	`mkdir new && cd !$`
`!^` / `!*`	أول Argument / كل الـ Arguments	`ls a b` ثم `rm !*`
`!123`	أمر رقم 123 في `history`	
`!-2`	الأمر قبل اللي فات	
`!ssh`	آخر أمر بيبدأ بـ ssh	خطر: بينفّذ على طول
`^old^new`	الأمر اللي فات مع استبدال	`^dev^prod`
`!ssh:p`	اطبعه من غير ما تنفذه (Print)	جرّب قبل ما تنفذ

جدول 14.4 توسيعات الـ History (علامة التعجب)

تشريح الأمر

```
history 1 | grep -i docker | tail -5
```

1. `history 1` — Builtin: اعرض الأوامر القديمة بأرقامها، من أول أمر رقم 1. في zsh `history` لوحدها بتعرض آخر 16 أمر بس، فال `1` مهمة. (في bash `history` لوحدها بتعرض كله).
2. `| grep -i docker` — اللي فيها docker بأي حالة حروف (↩ 8.1 ص 82).
3. `| tail -5` — آخر 5 (↩ 5.5 ص 53).

```
~/.zshrc: History أحسن
HISTSIZE=50000                  # في الذاكرة
SAVEHIST=50000                  # في الملف
setopt SHARE_HISTORY            # كل التابات تشوف History بعض على طول
setopt HIST_IGNORE_ALL_DUPS     # متكررش نفس الأمر
setopt HIST_IGNORE_SPACE        # الأمر اللي بيبدأ بمسافة ميتحفظش
setopt EXTENDED_HISTORY         # احفظ وقت كل أمر (history -i يعرضه)
```

لو حابب تتعمق

تقدر تعدّيه في أول قراية

ليه الأمر اللي كتبته في تابة مش لاقيه بـ Ctrl+R في تابة تانية على Terminal.app؟ لأن `/etc/zshrc_Apple_Terminal` بتاع Apple بيعمل ميزة اسمها **Shell sessions**: كل تابة ليها History منفصل متخزن في `~/.zsh_sessions/`، عشان لما الماك يعمل Restart ويرجّع التابات، كل تابة ترجع بالـ History بتاعها. ده لطيف، بس بيلخبط الناس. لو عايز History واحد مشترك لكل التابات: `SHARE_HISTORY` اللي فوق، ولو عايز تقفل ميزة Apple خالص: ضيف `SHELL_SESSIONS_DISABLE=1` في `~/.zshenv` (ده من الحالات القليلة اللي `.zshenv` مكانها الصح، لأن الإعداد لازم يتقري قبل `/etc/zshrc`، ↩ 13.1 ص 157).

نفس الكلام في bash على السيرفر

~/.bashrc: History أحسن

```
HISTSIZE=50000                  # في الذاكرة
HISTFILESIZE=100000             # في الملف
HISTCONTROL=ignoreboth          # ignorespace + ignoredups
HISTTIMEFORMAT='%F %T  '        # وقت كل أمر
shopt -s histappend             # ضيف متمسحش الملف لما تقفل كذا تاب
```

مسافة قبل الأمر = سري

مع `ignoreboth` (أو `ignorespace`)، أي أمر بتبدأه بمسافة مبيتحفظش. مفيد لما تكتب Token في الأمر: `export TOKEN=...`. وفي zsh الإعداد اسمه `setopt HIST_IGNORE_SPACE` (اللي حطيناه فوق).

14.5 الـ Tab Completion

عايز تدخل `~/Library/Application Support/Code/User`. هتكتبها حرف حرف وتتلخبط في المسافات والحروف الكبيرة؟ ولا تكتب `cd ~/Li` و Tab و `App` و Tab و `Co` و Tab؟ اكتب أول كام حرف واضغط `Tab`: الـ Shell يكمّل اسم الأمر أو الملف أو المجلد. لو فيه أكتر من احتمال، `Tab Tab` يعرضهم كلهم. ده بيوفر كتابة **وكمان** بيمنع الغلط في الأسماء، ولو الـ Tab مكمّلش يبقى المسار غلط أصلاً.

`cd ~/Doc<Tab>`	يكمّل لـ `~/Documents/`.
`git che<Tab>`	يكمّل Subcommands (لو حزمة `bash-completion` متسطبة).
`ssh pr<Tab>`	يكمّل أسماء السيرفرات من `~/.ssh/config` (↩ 10.5 ص 121).
`kill -<Tab>`	في zsh و fish: يعرض الـ Options بشرحها.

الـ Shell	الوضع الافتراضي
bash	ملفات وأوامر، والباقي بحزمة `bash-completion` (موجودة غالباً على Ubuntu، وعلى الماك مع brew bash `brew install bash-completion@2`)
zsh (الماك)	قوي جداً بعد `autoload -Uz compinit && compinit` في `~/.zshrc` (oh-my-zsh بيعملها)، وقائمة تمشي فيها بالأسهم بـ `zstyle ':completion:*' menu select`. وعشان يكمّل أوامر الأدوات اللي من Homebrew (`brew` و `git` و `gh`)، حط قبل `compinit`: `FPATH="$(brew --prefix)/share/zsh/site-functions:$FPATH"`
fish	جاهز من غير إعداد، وكمان Autosuggestions رمادي من الـ History (`→` للقبول)
PowerShell	`Tab` بيلف على الاحتمالات، و `Ctrl+Space` قائمة

جدول 14.5 الـ Completion في كل Shell

14.6 tmux: شغلك ميضيعش

الساعة 11 بالليل، وإنت داخل على السيرفر بـ ssh من الماك، ومشغّل ترحيل قاعدة بيانات هياخد ساعة. وفجأة الـ Wi-Fi فصل، أو قفلت غطا الماك بالغلط. الـ SSH قطع، والـ Shell اللي على السيرفر مات، والترحيل مات معاه في النص. كارثة. SSH قطع؟ مع tmux مفيش مشكلة: البرامج بتشتغل جوه **Session** على السيرفر، وإنت بتتصل بيها وتفصل براحتك. وكمان بيقسم الشاشة لـ Panes.

شكل 14.1 Session فيها Windows (زي التابات)، وكل Window ممكن تتقسم Panes.

تشريح الأمر

```
tmux new -s work
1    2   3
```

1. `tmux` — Terminal MUltipleXer. على الماك: `brew install tmux`، وعلى السيرفر غالباً موجود أو `sudo apt install tmux`. ومفيد على الماك نفسه كمان: Session فيها السيرفر والـ Tests واللوج، تقفل الـ Terminal وترجعلها زي ما هي.
2. `new` — اعمل Session جديدة (= `new-session`).
3. `-s work` — Session name: سمّيها عشان ترجعلها بالاسم.

تشريح الأمر

```
tmux attach -t work
```
(1) (2)

1. `tmux attach` ارجع لـ Session موجودة (= `a`).
2. `-t work` Target: أنهي واحدة. اعرضهم بـ `tmux ls`.

جوه tmux كل الاختصارات بتبدأ بـ **Prefix**: `Ctrl+B` سيبها، وبعدين اضغط الحرف.

المفتاح	بيعمل	المفتاح	بيعمل
`d`	**Detach**: اخرج وسيبها شغالة	`%`	قسّم رأسي (يمين وشمال)
`c`	Window جديدة	`"`	قسّم أفقي (فوق وتحت)
`n` / `p`	الـ Window الجاية / اللي قبلها	الأسهم	اتنقل بين الـ Panes
`0` ... `9`	روح لـ Window برقمها	`z`	كبّر الـ Pane (Zoom) وارجع
`,`	غيّر اسم الـ Window	`x`	اقفل الـ Pane
`[`	Copy mode: اطلع لفوق في الناتج (`q` للخروج)	`?`	كل الاختصارات

جدول 14.6 اختصارات tmux (بعد Ctrl+B)

الماوس وبدائل

عايز تختار الـ Pane بالماوس وتعمل Scroll عادي؟ ضيف `set -g mouse on` في `~/.tmux.conf`. و `screen` أقدم وموجود على سيرفرات كتير (`screen -S name` و `Ctrl+A d` و `screen -r name`). و `zellij` بديل حديث بقوايم اختصارات ظاهرة على الشاشة.

14.7 fzf: البحث التفاعلي

افتكر Cmd+P في VS Code: تكتب "usrctrl" فيلاقيلك `userController.js` من وسط 500 ملف. ده اسمه Fuzzy search. و `fzf` بيجيب الميزة دي لأي حاجة في الـ Terminal (`brew install fzf`). (Fuzzy Finder) بياخد أي قائمة على stdin ويديك مربع بحث تفاعلي: تكتب حروف متفرقة من اللي عايزه فيلاقيه، والسطر اللي تختاره يطلع على stdout. يعني بيشتغل في أي Pipe (↩ 7.3 ص 74).

تشريح الأمر

```
vim "$(fzf --preview 'head -40 {}')"
```

1. `vim` افتح الملف اللي هيتختار.
2. `"$(` Command substitution في Quotes (↩ 7.6 ص 77).
3. `fzf` من غير stdin بيعرض كل ملفات المجلد والمجلدات اللي تحته.
4. `--preview 'head -40 {}'` جنب القائمة اعرض أول 40 سطر من الملف اللي عليه المؤشر، و `{}` = السطر الحالي (نفس فكرة `{}` في find و xargs).

~/.zshrc أو ~/.bashrc (أو أحدث fzf 0.48)

```
source <(fzf --zsh)        # zsh
eval "$(fzf --bash)"       # bash
fzf --fish | source        # fish (في config.fish)
```

بعدها بتاخد 3 اختصارات: `Ctrl+R` بحث History أقوى بكتير، و `Ctrl+T` اختار ملف وحطه في السطر، و `Alt+C` اختار مجلد وادخله (على الماك محتاج Option كـ Meta، فوق). (النسخ الأقدم من 0.48 بتستخدم ملفات `key-bindings` بدل السطور دي).

`git branch | fzf | xargs git checkout`	اختار Branch تفاعلياً.
`ps aux | fzf | awk '{print $2}' | xargs kill`	اختار عملية واقتلها (↩ 9.1 ص 97).
`code "$(fzf)"`	دوّر على ملف وافتحه في VS Code.

14.8 oh-my-zsh باختصار

بعد ما عرفت إن zsh بيعمل كل ده، هتلاقي ناس كتير بتقولك "سطّب oh-my-zsh وخلاص". خلينا نفهم ده إيه الأول. **oh-my-zsh** إطار إعدادات لـ zsh: فيه مئات Themes و Plugins جاهزة (git aliases، Completion، إلخ). بيتسطب بـ Script من موقعه الرسمي ohmyz.sh، وده نوع `curl | sh` اللي لازم تفهمه قبل ما تشغله (↩ 15.6 ص 187). بعد التسطيب، كل الإعدادات في `~/.zshrc`:

~/.zshrc

```
ZSH_THEME="robbyrussell"
plugins=(git z sudo)        # sudo: Esc مرتين يحط sudo قبل الأمر
```

متكترش

كل Plugin بيبطّأ فتح الـ Terminal شوية. ابدأ بـ 3 أو 4. ولو عايز أخف: zsh عادي + Starship (↩ 13.6 ص 164) + `zsh-autosuggestions` (اقتراح رمادي من الـ History وإنت بتكتب، `→` للقبول) و `zsh-syntax-highlighting` (بيلوّن الأمر أحمر لو مش موجود قبل ما تدوس Enter). الاتنين بـ `brew install`، وبعدين في آخر `~/.zshrc`: `source $(brew --prefix)/share/zsh-autosuggestions/zsh-autosuggestions.zsh` وبنفس الشكل للتاني. أو جرّب fish اللي فيه الحاجات دي جاهزة (↩ 3.5 ص 29).

ملخص الفصل

- `Ctrl+A/E` للأول والآخر، و `Ctrl+W/U/K` للمسح، و `Ctrl+L` للشاشة.
- على الماك فعّل Option as Meta عشان اختصارات Alt، و Cmd+T و Cmd+K و Cmd+F للشباك.
- `Ctrl+R` للبحث، و `!!` و `$!` و `Alt+.` لإعادة الاستخدام، و `history 1` في zsh و `SHARE_HISTORY`.
- أدوات الماك: `open` و `pbcopy` / `pbpaste` و `say` و `osascript` و `defaults` و `caffeinate` و `softwareupdate` و `screencapture`.
- `Tab` دايماً: أسرع ومبيغلطش في الأسماء.
- tmux: `new -s` و `Ctrl+B d` و `attach -t`.
- fzf بيحوّل أي قائمة لبحث تفاعلي، و oh-my-zsh إطار لـ zsh.

صح ولا غلط؟

1. `Ctrl+D` على سطر فاضي بيقفل الـ Shell. صح / غلط
2. `$!` هو الأمر اللي فات كله. صح / غلط
3. لو قفلت الـ Terminal بعد Detach من tmux، البرامج جواه بتموت. صح / غلط
4. زرار Option في Terminal.app بيشتغل كـ Alt (Meta) من غير أي إعداد. صح / غلط

أسئلة مراجعة

1. كتبت أمر طويل ونسيت sudo في الأول: إزاي تصلحه بأقل كتابة؟
2. إيه الفرق بين `Ctrl+C` و `Ctrl+Z` و `Ctrl+D`؟
3. اشرح خطوات تشغيل أمر طويل على سيرفر بـ tmux والرجوع له تاني يوم.
4. اكتب سطر بينسخ محتوى `package.json` للـ Clipboard، وسطر بيفتح الفولدر الحالي في VS Code.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اكتب أمر طويل، وامشي فيه بـ `Ctrl+A` و `Alt+F`، وامسح كلمة بـ `Ctrl+W` ورجّعها بـ `Ctrl+Y`.
2. متوسط طلّع أكتر 10 أوامر بتستخدمها من الـ History (تلميح: `history 1` و awk و sort و uniq).
3. متوسط اعمل دالة `notify` في `~/.zshrc` بتشغّل أي أمر وبعدين تعمل Notification فيها "نجح" أو "فشل" (`notify npm test`).
4. صعب اعمل Session tmux فيها Window مقسومة 3 Panes: `htop` و `tail -f` على لوج والتالت فاضي، واعمل Detach و Attach.

15

المرحلة 6 · الاحتراف بأمان

الفصل الخامس عشر

الأوامر الخطيرة وإزاي تحمي نفسك

لما تمسح ملف من الـ Finder، بيروح الـ Trash، وتقدر ترجعه. ولو فضيت الـ Trash، الماك بيسألك

بعد الفصل ده هتكون عارف

- rm -rf وأشهر الطرق اللي بيمسح بيها حاجات مش قصدك، وإيه اللي SIP بيحميه وإيه لأ
- dd و diskutil وليه dd اسمه Disk Destroyer
- chmod -R 777 و chown -R على مسارات النظام
- الـ Fork bomb وتشريحها رمز رمز
- > اللي بيمسح الملف من غير ما يسأل
- تشغيل Scripts من النت بـ curl و sh بأمان
- قائمة حماية عملية

متجربش الأوامر دي على الماك بتاعك

كل الأمثلة الخطيرة هنا للقراية والفهم. لو عايز تجرب، جرّب جوه Container (`docker run --rm -it ubuntu`) مع Docker Desktop أو OrbStack) أو Virtual Machine (زي UTM) مش مهم تبوظ. ولو مفيش، اعمل فولدر تجارب في `/tmp` واشتغل جواه بس، واستخدم `echo` قبل أي أمر عشان تشوف هيعمل إيه.

15.1 rm -rf: المسح من غير رجعة

حكاية حقيقية بتتكرر كل يوم: واحد عايز يمسح فولدر `node_modules` القديم من مشروع عشان يوفر مساحة (أي حد اشتغل JavaScript عارف إن الفولدر ده ممكن يبقى 800 ميجا). كتب الأمر بسرعة، وإيده سبقت مخه، وحط مسافة واحدة زيادة:

تشريح الأمر

```
1  2     3              4
rm -rf ~/projects/ my-app/node_modules
```

1. `rm` امسح (↩ 5.4 ص 51). ومن غير Trash: الملف مش بيروح أي حتة، المكان بتاعه بيتعلّم "فاضي" وخلاص.
2. `-rf` `-r` كل اللي جوه، و `-f` Force: متسألش ومتشتكيش لو حاجة مش موجودة. الاتنين مع بعض = ولا سؤال ولا تحذير.
3. `~/projects/` المصيبة: **مسافة** بالغلط قبل my-app، فبقى عندنا Argumentين (↩ 2.1 ص 17). الأول هو `~/projects/` كله! كل مشاريعك.
4. `my-app/node_modules` والتاني ده (نسبي للفولدر الحالي، وغالباً مش موجود، و `-f` هتسكت عنه). القصد كان `~/projects/my-app/node_modules` بس.

ومفيش `sudo` هنا خالص! ملفاتك إنت ملكك، فـ rm مش محتاج أي صلاحية زيادة عشان يمسحها. ونفس الغلطة على سيرفر بـ sudo: `sudo rm -rf /var/www/ site` = الموقع كله.

الأمر	ليه خطير
`rm -rf "$DIR/"`	لو `DIR` فاضي أو اسمه غلط، بقى `rm -rf /` (حصلت فعلاً في Script تسطيب Steam على Linux سنة 2015 ومسحت ملفات ناس). الحل: `set -u` و `${DIR:?}` (↩ 12.7 ص 151)
`rm -rf ~ /tmp/x`	مسافة بعد `~` = ال home كله: Desktop و Documents و Photos و ~/Library
`cd /x ; rm -rf *`	لو ال cd فشل، المسح حصل في المجلد الحالي. استخدم `&&` (↩ 7.7 ص 78)
`rm -rf .*`	قديماً كان بيطابق `..` ويطلع للأب؛ الإصدارات الحديثة من bash و rm بتمنعها، بس متعتمدش على كده
`rm -rf /*`	rm بيرفض `rm -rf /` على الماك و Linux، لكن `/*` بيتوسع لـ `/Applications /Users /bin ...` فالحماية **مبتشتغلش** (↩ 7.4 ص 75)

جدول 15.1 طرق rm -rf بيمسح حاجات مش قصدك

طب و SIP مش بيحميني؟

SIP (↩ 6.8 ص 69) ومعاه ال System volume اللي للقراية بس (↩ 4.6 ص 44) بيحموا **macOS نفسه**: `/System` و `/bin` و `/usr` (ماعدا `/usr/local`). فلو حد عمل `sudo rm -rf /*` على الماك، الماك نفسه غالباً هيفضل يقوم. بس كل حاجة **بتاعتك**: `/Users` و `/Applications` و `/opt/homebrew` و `/Library`، مش محمية خالص وهتتمسح. يعني SIP بيحمي البيت، مش العفش. والعفش حمايته الوحيدة **Backup**: فعّل Time Machine (تحت).

--preserve-root على Linux

GNU coreutils من سنين بيرفض `rm -rf /` إلا لو كتبت `--no-preserve-root` صراحة. وعلى Ubuntu 25.10 وما بعده اللي بقى بيستخدم rust-coreutils (uutils) لأوامر كتير، الحماية دي موجودة برضه، بس الفكرة: **الحماية بتغطي حالة واحدة بس**، وعادة الأمان عندك أهم.

15.2 dd: الـ Disk Destroyer

يوم ما هتحتاج تعمل فلاشة Ubuntu عشان تسطبه على لابتوب قديم أو Raspberry Pi، هتلاقي الشرح بيقولك استخدم `dd`. وهنا لازم تبقى صاحي جداً، لأن dd أخطر أمر في الكتاب ده: بيكتب على Disk كامل، Byte بـ Byte، من غير ما يسأل ومن غير ما يفهم إنت بتكتب على إيه. أول خطوة على الماك: اعرف الفلاشة اسمها إيه بالظبط.

```
Terminal
user@MacBook ~ % diskutil list external
/dev/disk4 (external, physical):
   #:                       TYPE NAME                    SIZE       IDENTIFIER
   0:     FDisk_partition_scheme                        *32.0 GB    disk4
   1:             Windows_FAT_32 USBSTICK                 32.0 GB    disk4s1
```

شوف كلمة `external, physical` والحجم (32 جيجا = الفلاشة). ال SSD الداخلي بتاع الماك غالباً `disk0`، وحواليه حاجات زي `disk3` (`synthesized`) دي ال APFS Containers بتاعة النظام. أي رقم غير الفلاشة = متلمسوش. والأحسن: اعمل

`diskutil list` من غير الفلاشة، وبعدين ركّبها واعمله تاني، وشوف إيه الجديد.

تشريح الأمر

```
diskutil unmountDisk /dev/disk4 && sudo dd if=ubuntu.iso of=/dev/rdisk4 bs=4m
```

1. `diskutil unmountDisk /dev/disk4` فك ربط كل الـ Partitions بتاعة الفلاشة (من غير ما تطلّعها)، لأن الماك مش هيسيب dd يكتب على Disk مفتوح.
2. `dd` انسخ Bytes خام من مكان لمكان. أسلوب الـ Arguments قديم `key=value` من غير شَرطة خالص.
3. `if=ubuntu.iso` Input File: المصدر.
4. `of=/dev/rdisk4` Output File: الوجهة، وهنا **Disk كامل**. الـ `r` قبل disk = Raw: طريق مباشر للجهاز من غير Cache الماك، وبيخلي الكتابة أسرع بكتير. لو كتبت رقم غلط، الـ Disk ده اتمسح في ثواني ومفيش سؤال. عشان كده اسمه الشعبي Disk Destroyer.
5. `bs=4m` Block size: اقرا واكتب 4 ميجا كل مرة. على الماك بحرف صغير `m` (و GNU dd بيكتبها `4M`).

dd ساكت تماماً وهو شغال. عايز تعرف وصل لفين؟ اضغط `Ctrl+T`: ده بيبعت Signal اسمها `SIGINFO` (موجودة في BSD والماك ومش في Linux)، و dd بيرد بيطبع وصل كام Byte. ولما يخلص: `diskutil eject /dev/disk4`.

dd على Linux

تشريح الأمر

```
sudo dd if=ubuntu.iso of=/dev/sdb bs=4M status=progress conv=fsync
```

1. `dd` انسخ Bytes خام من مكان لمكان. أسلوب الـ Arguments قديم `key=value` من غير شَرطة خالص.
2. `if=ubuntu.iso` Input File: المصدر.
3. `of=/dev/sdb` Output File: الوجهة، وهنا **جهاز تخزين كامل** (فلاشة). لو كتبت `/dev/sda` بالغلط (غالباً القرص الأساسي)، النظام كله اتمسح في ثواني، ومفيش سؤال. عشان كده اسمه الشعبي Disk Destroyer.
4. `bs=4M` Block size: اقرا واكتب 4 ميجا كل مرة (أسرع من الافتراضي 512 Byte).
5. `status=progress` اعرض التقدم (GNU dd).
6. `conv=fsync` اتأكد إن البيانات اتكتبت فعلاً على الجهاز قبل ما تخلص.

وقبلها: `lsblk -o NAME,SIZE,MODEL,MOUNTPOINTS` عشان تعرف أي جهاز هو الفلاشة بالحجم والموديل (المقابل لـ `diskutil list`).

قبل أي dd

اعرف الفلاشة بالحجم و `external`، ويُفضّل تشيلها وترجعها وتشوف أنهي اسم اختفى وظهر. وفكّر في أداة برسومات (balenaEtcher، `brew install --cask balenaetcher`) لو مش مضطر، بتمنعك تختار الـ Disk الداخلي أصلاً.

15.3 chmod -R 777 و chown -R على النظام

تشريح الأمر

```
sudo chmod -R 777 /
 1    2  3   4
```

1. `sudo chmod` غيّر الصلاحيات كـ root (↩ 6.2 ص 59).
2. `-R` على كل حاجة جوه.
3. `777` أي حد يقرا ويكتب ويشغّل أي حاجة. وكمان بيشيل الـ setuid من أوامر زي `sudo` و `passwd`.
4. `/` النظام كله. النتيجة على Linux: sudo يرفض يشتغل (*/etc/sudoers is world writable*)، و SSH يرفض المفاتيح، وفي حالات كتير الحل الوحيد إعادة تسطيب. وعلى الماك SIP هيمنعه من ملفات النظام المحمية، بس كل اللي مش محمي (`/Users` و `/Library` و `/private/etc` و `/opt/homebrew`) هيتبهدل، وحاجات زي sudo و ssh هتبدأ ترفض تشتغل.

والنسخة "الماكية" الأشهر من الغلطة دي: نصايح قديمة على النت بتقول `sudo chown -R $(whoami) /usr/local` أو `sudo chmod -R 777 /usr/local` عشان Homebrew يشتغل. على ماك Apple Silicon ده ملوش أي لازمة أصلاً (Homebrew في `/opt/homebrew`، ↩ 11.2 ص 130)، والـ 777 خطر في أي حتة.

و `777` حتى على مجلد واحد غلط: أي برنامج أو مستخدم على الجهاز (أو أي ثغرة في موقعك) يقدر يعدّل ملفاتك. لو حاجة "مش شغالة غير بـ 777"، المشكلة في **المالك** غالباً، صلّحها بـ `chown` وصلاحية مناسبة (↩ 6.3 ص 61). ونفس الخطر في `sudo chown -R ahmed /usr`: هيكسر برامج النظام.

15.4 الـ Fork Bomb

ده "أمر" مشهور بيتبعت كنكتة في المنتديات. شكله رموز بريئة، بس بيخلي الجهاز يقف في ثواني.

تشريح الأمر

```
:(){ :|:& };:
```

1. `:` اسم دالة! الـ `:` حرف مسموح في أسماء الدوال في bash. يعني ده زي `bomb` بالظبط.
2. `()` تعريف دالة (↩ 12.5 ص 149).
3. `{` بداية جسم الدالة.
4. `:|:` الدالة بتنادي نفسها، وتعمل Pipe لنسخة تانية من نفسها (↩ 7.3 ص 74). يعني كل نسخة بتعمل نسختين.
5. `&` في الخلفية، عشان الـ Shell ميستناش (↩ 9.4 ص 101).
6. `};` نهاية الدالة.
7. `:` شغّلها. 1 ← 2 ← 4 ← 8 ... لحد ما جدول العمليات يتملي والجهاز ميقدرش يشغّل ولا حتى `kill` .

بالشكل المقروء: `bomb() { bomb | bomb & }; bomb` . الحماية: حد أقصى لعدد العمليات لكل مستخدم. على الماك الحد ده موجود أصلاً من الـ Kernel: `sysctl kern.maxprocperuid` بيقولك الرقم (غالباً بضعة آلاف)، و `ulimit -u` بيعرض الحد في الـ Shell بتاعك. فالماك مش هيموت خالص، بس هيبقى بطيء جداً ومش هتقدر تفتح ولا برنامج لحد ما تعمل Restart. وعلى Linux: `ulimit -u 4096` ، أو `nproc` في `/etc/security/limits.conf` ، و systemd بيحط حد افتراضي للـ Tasks لكل مستخدم في توزيعات كتير.

15.5 > اللي بيمسح من غير ما يسأل

قعدت ساعة تظبط `~/.zshrc` بتاعك: Aliases ودوال و Prompt بالألوان. وبعدين عايز تضيف alias واحد بسرعة، فكتبت `echo "alias gs='git status'" > ~/.zshrc` . علامة `>` واحدة بدل اتنين. وكل شغل الساعة راح، والملف فيه سطر واحد بس. ومفيش Undo.

الأمر	اللي بيحصل
`echo "x" > ~/.zshrc`	كنت عايز تضيف سطر، الملف كله اتمسح وبقى فيه x. الصح `>>`
`sort f > f`	الملف بيتفضى قبل ما يتقري (↩ 7.2 ص 73)
`> important.log`	سطر لوحده = فضّي الملف فوراً
`cmd > /dev/disk4`	كتابة على القرص الخام زي dd (على Linux `/dev/sda`)
`crontab -r`	مش Redirection بس جنب `-e` على الكيبورد: بيمسح كل مهام cron من غير سؤال

جدول 15.2 Redirection ممكن تمسح ملف

تشريح الأمر

```
setopt noclobber
```

1 `setopt` Builtin في zsh: فعّل Option (و `unsetopt` تقفلها). حطه في `~/.zshrc` (↩ 13.1 ص 157).

2 `noclobber` خلي `>` ترفض تكتب فوق ملف موجود: zsh هيقولك `zsh: file exists: notes.txt`. ولو قاصد تكتب فوقه: `>|` أو `>!` (↩ 7.2 ص 73).

في bash

تشريح الأمر

```
set -o noclobber
```

1 `set -o` فعّل Option في الـ Shell (و `set +o` تقفلها؛ الـ `+` هنا عكس المتوقع: يعني "اقفل").

2 `noclobber` خلي `>` ترفض تكتب فوق ملف موجود: `cannot overwrite existing file`. ولو قاصد: `>|`.

15.6 curl | sh: تشغيل Script من النت

افتكر سطر تسطيب Homebrew في فصل الحزم (↩ 11.2 ص 130): `/bin/bash -c "$(curl -fsSL ...)"`. ونفس الشكل هتلاقيه في تسطيب oh-my-zsh و nvm و Bun و Deno و Rust. سطر واحد، تلزقه، ويتسطب كل حاجة. سهل جداً. وده بالظبط اللي بيخليه خطير لو المصدر مش موثوق:

تشريح الأمر

```
curl -fsSL https://example.com/install.sh | sudo bash
```

1 `curl -fsSL https://...` نزّل الـ Script (↩ 10.2 ص 115).

2 `|` وابعته على طول...

3 `sudo bash` ...لـ bash بصلاحيات root ينفذه **من غير ما تشوفه**. أي حد يتحكم في الموقع (أو في الطريق لو مش HTTPS) يقدر يعمل أي حاجة في جهازك. ولو التحميل اتقطع في النص، ممكن يتنفذ نص سطر (`rm -rf /usr/local/x` يتقطع لـ `rm -rf /usr`).

الطريقة الآمنة

```
curl -fsSLo install.sh https://example.com/install.sh
less install.sh                          # اقراه
shasum -a 256 install.sh                 # قارنه بالـ Checksum المنشور (Linux: sha256sum على)
bash install.sh                          # من غير sudo لو ينفع
```

curl بيعدّي من Gatekeeper

افتكر علامة `com.apple.quarantine` (↩ 6.7 ص 67): المتصفح بيحطها على أي حاجة بتنزل، فالماك يفحصها ويسألك قبل أول تشغيل. أما `curl` فمبيحطهاش. يعني أي حاجة بتنزل بـ curl وتشغلها، الماك **مش هيفحصها ولا هيسألك**. إنت لوحدك المسؤول. وده سبب زيادة إنك تقرا قبل ما تشغّل.

مش كل sh | curl شر

مشاريع معروفة كتير بتستخدمه (Homebrew و oh-my-zsh و rustup)، والـ Scripts بتاعتهم عادة بتتحط جوه دالة بتتنادى في آخر سطر عشان مشكلة التحميل الناقص. الفكرة: اعرف المصدر، و HTTPS، واقرا لو تقدر، و **متلزقش أوامر من مواقع مش معروفة في الـ Terminal**. حتى اللصق نفسه ممكن يخدعك (نص مخفي في الصفحة)، فالـ Terminals الحديثة بتدعم Bracketed paste اللي بيمنع التنفيذ لحد ما تدوس Enter.

15.7 قائمة الحماية

الحماية	إزاي
شوف قبل ما تنفذ	`echo rm *.log`، و `ls` بنفس الـ Glob الأول (↩ 7.4 ص 75)
Dry run	`rsync -n` و `find` من غير `-delete` و `sed` من غير `-i` و `brew cleanup -n` (وعلى السيرفر `apt -s`)
اسأل قبل المسح	`alias rm='rm -I' cp='cp -i' mv='mv -i'` (↩ 13.2 ص 159)
سلة مهملات	على الماك `brew install trash` وبعدين `trash file` بدل rm، والملف يروح الـ Trash بتاع Finder وترجعه من هناك (↩ 5.4 ص 51). على Linux `trash-put` (trash-cli) أو `gio trash`
Scripts آمنة	`set -euo pipefail` و `"$var"` و `${var:?}` و shellcheck (↩ 12.7 ص 151)
Prompt واضح	اسم الجهاز ولون أحمر لما تكون root أو على Production (↩ 13.6 ص 164)
أقل صلاحيات	sudo للأمر اللي محتاجه بس، ومتشتغلش كـ root
Backups	على الماك **Time Machine** على Disk خارجي (System Settings ← General ← Time Machine)، ومن الـ Terminal: `tmutil startbackup` و `tmutil listbackups`، و `tmutil localsnapshot` بياخد APFS Snapshot في ثانية قبل أي حاجة خطيرة. والقاعدة العامة: نسخ **مجرّبة** (3-2-1: 3 نسخ، على 2 نوع وسائط، 1 بره المكان)، وعلى السيرفرات Snapshots (Btrfs و ZFS و LVM)
مسارات مطلقة	في الأوامر الخطيرة اكتب المسار كامل بدل الاعتماد على المجلد الحالي

الحماية	إزاي
استنى ثانية	قبل Enter على أي أمر فيه `rm` أو `dd` أو `>` أو `-R` أو `sudo` : اقرا السطر تاني

جدول 15.3 عادات وأدوات بتحميك

ملخص الفصل

- `rm -rf` مفيهوش رجعة ولا Trash: خاف من المسافات والمتغيرات الفاضية و `;`. و SIP بيحمي النظام مش ملفاتك.
- `dd of=` غلط = قرص كامل اتمسح؛ `diskutil list` الأول (و `lsblk` على Linux)، و `/dev/rdiskN` و `Ctrl+T` للتقدم.
- `777` مش حل أبداً؛ صلّح المالك.
- الـ Fork bomb دالة بتنادي نفسها مرتين في الخلفية؛ الحماية `ulimit -u`.
- `>` بيمسح، و `setopt noclobber` بيحميك.
- نزّل ← اقرا ← شغّل، بدل `curl | sudo bash`، وافتكر إن curl بيعدّي من Gatekeeper. و Time Machine هو شبكة الأمان الأخيرة.

صح ولا غلط؟

1. GNU rm بيرفض `rm -rf /*` زي ما بيرفض `rm -rf /`. صح / غلط
2. `set -u` ممكن يمنع مسح بسبب متغير فاضي. صح / غلط
3. الـ `:` في الـ Fork bomb هي اسم دالة. صح / غلط
4. SIP بيحمي ملفاتك في `~/Documents` من `rm -rf`. صح / غلط

أسئلة مراجعة

1. اشرح بالظبط ليه `rm -rf "$DIR/"*` خطير، واكتب نسخة آمنة منه.
2. إزاي تتأكد من اسم الفلاشة قبل `dd` على الماك؟ وإيه الفرق بين `/dev/disk4` و `/dev/rdisk4`؟
3. واحد قالك "اعمل chmod 777 والموقع هيشتغل"، إيه ردك وإيه البديل؟
4. اكتب خطوات تشغيل Script تسطيب من الإنترنت بأمان.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل فعّل `setopt noclobber` وجرّب `>` على ملف موجود، وبعدين `>|`.
2. متوسط شوف بعينك التوسيع من غير خطر: اكتب `DIR=""; echo rm -rf "$DIR/"*` (بـ `echo` بس!)، وبعدين جرّب نفس السطر بعد `set -u` و `unset DIR`، وبعدين بـ `"${DIR:?}"`.
3. صعب اكتب دالة `saferm` بتنقل الملفات لـ `~/.trash/التاريخ/` بدل ما تمسحها، وترفض تشتغل على `/` أو `$HOME`. (ولو حابب: خليها تنقل لـ `~/.Trash` عشان تظهر في Trash بتاع Finder).

16

المرحلة 6 · الاحتراف بأمان

الفصل السادس عشر

المشروع النهائي: أداة opskit

افتكر أول يوم فتحت فيه الـ Terminal: شاشة سودا، ومؤشر بيرمش، وإنت مش عارف تكتب إيه. دلوقتي إنت عارف تتنقل في الماك كله، وتدوّر جوه آلاف الملفات، وتسطّب بـ brew، وتدخل سيرفر بـ ssh، وتكتب Scripts. وقت تجمع كل ده في حاجة واحدة بتاعتك. هنبني أداة Terminal حقيقية اسمها opskit: بتطلع تقرير عن صحة الماك (أو السيرفر)، وتحلل لوج سيرفر ويب، وتعمل Backup بتاريخ وتمسح القديم، وتنشر موقع على سيرفر بـ rsync، وتشتغل أوتوماتيك كل يوم بـ launchd. ومكتوبة بحيث تشتغل على الماك بـ bash 3.2 اللي جاي معاه، وعلى Linux من غير تعديل. كل سطر فيها هيرجعك لفصل اتعلمت فيه الحاجة دي.

بعد الفصل ده هتكون عارف

- تخطيط أداة CLI بـ Subcommands
- كتابة Script يشتغل على الماك و Linux (uname و bash 3.2 والأدوات الـ BSD)
- تجميع Pipes و awk و sort في تقارير
- Backup آمن بـ tar و Rotation
- Deploy بـ rsync و ssh config
- التثبيت في PATH و Alias و Completion في zsh
- الجدولة بـ LaunchAgent (و cron على السيرفر)

16.1 الخطة

قبل ما نكتب سطر كود، خلينا نعمل زي أي مطوّر محترم: نخطط. إحنا عايزين أداة واحدة، اسمها `opskit`، وتحتها أوامر فرعية زي `git commit` و `brew install` بالظبط (↩ 2.1 ص 17). كل أمر فرعي بيحل مشكلة حقيقية هتقابلك:

الأمر	بيعمل	على الماك	على Linux
`opskit health`	الحمل والرامات والقرص وأكتر 5 عمليات والبورتات المفتوحة	sysctl و vm_stat و df و lsof و ps	/proc و free و df و ss و ps
`opskit logs FILE`	أكتر IPs، وتوزيع الـ Status codes، وأكتر صفحات 404	awk و sort و uniq و head (نفس الكود)	
`opskit backup DIR`	أرشيف بتاريخ ويحتفظ بآخر 7	tar و ls -t و while read (نفس الكود)	
`opskit deploy`	ينشر مجلد على السيرفر (مع Dry run)	rsync و ssh (نفس الكود)	

جدول 16.1 الأوامر الفرعية في opskit

1. **الهيكل:** Script واحد فيه دالة لكل أمر فرعي، و `case` بيوزّع (↩ 12.3 ص 146).
2. **نظامين في ملف واحد:** `uname -s` بيرجع `Darwin` على الماك و `Linux` على Linux، وبنختار الأوامر على أساسه. والأجزاء المشتركة مكتوبة بالأدوات اللي شكلها واحد في الاتنين (↩ 2.6 ص 23).

3. **bash 3.2:** مفيش `declare -A` ولا `mapfile` ولا `${var,,}`، والـ Array الفاضية محمية من `set -u` (↩ 3.4 ص 28).
4. **الأمان:** `set -euo pipefail` و Quotes و `${var:?}` (↩ 12.7 ص 151).
5. **الرسايل:** دالة `log` بتكتب على stderr، والنتايج على stdout عشان ينفع تعمل لها Pipe (↩ 7.1 ص 72).
6. **الإعدادات:** من متغيرات بيئة بقيم افتراضية `${OPSKIT_DEST:-...}` (↩ 13.4 ص 161).
7. **التثبيت:** في `~/.local/bin` اللي ضفناه للـ PATH في `~/.zprofile` (↩ 13.5 ص 163).

16.2 الكود كامل

ده الكود كله، حوالي 100 سطر. متخافش من طوله: اقراه دالة دالة، وهتلاقي كل سطر فيه حاجة إنت عارفها. وبعده هنشرّح أصعب السطور.

~/.local/bin/opskit (الجزء 1)

```
#!/usr/bin/env bash
# opskit: health / logs / backup / deploy  (macOS first, Linux too, bash 3.2+)
set -euo pipefail

VERSION="1.0"
DEST="${OPSKIT_DEST:-$HOME/backups}"
KEEP="${OPSKIT_KEEP:-7}"
REMOTE="${OPSKIT_REMOTE:-prod:/var/www/site/}"
OS="$(uname -s)"                    # Darwin = macOS, Linux = Linux

log()  { printf '\033[1;34m[opskit]\033[0m %s\n' "$*" >&2; }
die()  { printf '\033[1;31m[error]\033[0m %s\n' "$*" >&2; exit 1; }
need() { command -v "$1" >/dev/null || die "missing: $1"; }

usage() {
    cat <<USAGE
opskit $VERSION
usage: opskit <command> [args]
  health            system report
  logs FILE         analyze an access log
  backup DIR        archive DIR into $DEST (keep $KEEP)
  deploy [-n] DIR   rsync DIR to $REMOTE (-n = dry run)
USAGE
}

mem_mac() {
    local total; total=$(sysctl -n hw.memsize)
    vm_stat | awk -v total="$total" '
        /page size of/          { ps = $8 }
        /Pages active/          { a = $3 }
        /Pages wired down/      { w = $4 }
        /occupied by compressor/{ c = $5 }
        END { used = (a + w + c) * ps
```

```
            printf "used %.1fG of %.1fG\n", used/2^30, total/2^30 }'
}

cmd_health() {
    echo "== $(hostname) · $(date '+%F %T') · $OS"
    if [[ "$OS" == Darwin ]]; then
        local load; load=$(sysctl -n vm.loadavg | awk '{print $2, $3, $4}')
        echo "-- load:   $load (cores: $(sysctl -n hw.ncpu))"
        echo "-- memory: $(mem_mac)"
        echo "-- disk:";  df -h -l \
            | { IFS= read -r hdr; echo "$hdr"; sort -k5 -rn; } | head -6
        echo "-- top cpu:"; ps -Ao pid,user,%cpu,%mem,comm -r | head -6
        echo "-- listening:"
        lsof -nP -iTCP -sTCP:LISTEN 2>/dev/null | awk 'NR > 1 {print $1, $9}' | sort -u
    else
        echo "-- load:   $(cut -d' ' -f1-3 /proc/loadavg) (cores: $(nproc))"
        echo "-- memory:"; free -h | awk 'NR<=2'
        echo "-- disk:";   df -h -x tmpfs -x devtmpfs \
            | { IFS= read -r hdr; echo "$hdr"; sort -k5 -rn; } | head -6
        echo "-- top cpu:"; ps -eo pid,user,%cpu,%mem,comm --sort=-%cpu | head -6
        echo "-- listening:"
        ss -tlnH | awk '{print $4}' | sort -u
    fi
}
```

~/.local/bin/opskit (الجزء 2)

```
cmd_logs() {
    local f="${1:?usage: opskit logs FILE}"
    [[ -r "$f" ]] || die "cannot read: $f"
    echo "== requests: $(wc -l < "$f" | tr -d ' ')"
    echo "-- top IPs:"
    awk '{print $1}' "$f" | sort | uniq -c | sort -rn | head -5
    echo "-- status codes:"
    awk '{print $9}' "$f" | sort | uniq -c | sort -rn
    echo "-- top 404:"
    awk '$9 == 404 {print $7}' "$f" | sort | uniq -c | sort -rn | head -5
}

cmd_backup() {
    local src="${1:?usage: opskit backup DIR}"
    [[ -d "$src" ]] || die "not a directory: $src"
    need tar
    mkdir -p -- "$DEST"
    local base; base="$(basename "$src")"
    local name; name="${base}_$(date +%F_%H%M%S).tar.gz"
    COPYFILE_DISABLE=1 tar -czf "$DEST/$name" --exclude='.DS_Store' \
        -C "$(dirname "$src")" "$base"
    log "saved $DEST/$name ($(du -h "$DEST/$name" | cut -f1))"
    # names are generated by us (no newlines), so ls -t is safe here
    # shellcheck disable=SC2012
```

```
    ls -1t "$DEST"/"$base"_*.tar.gz | tail -n +$((KEEP + 1)) \
        | while IFS= read -r old; do rm -v -- "$old"; done
}

cmd_deploy() {
    local dry=()
    [[ "${1:-}" == "-n" ]] && { dry=(--dry-run); shift; }
    local src="${1:?usage: opskit deploy [-n] DIR}"
    need rsync
    log "deploying ${src%/}/ -> $REMOTE ${dry[*]:-}"
    rsync -az --delete --exclude='.git' --exclude='.DS_Store' \
        ${dry[@]+"${dry[@]}"} "${src%/}/" "$REMOTE"
}

case "${1:-}" in
    health) shift; cmd_health "$@" ;;
    logs)   shift; cmd_logs "$@" ;;
    backup) shift; cmd_backup "$@" ;;
    deploy) shift; cmd_deploy "$@" ;;
    -v|--version) echo "$VERSION" ;;
    ""|-h|--help) usage ;;
    *) usage >&2; exit 2 ;;
esac
```

اتجرّب فين؟

الكود ده اتجرّب على Linux بـ bash 5.2 وكمان بـ bash 3.2.57 (نفس نسخة الماك) عشان نتأكد إنه مفيهوش أي ميزة أحدث، وعدّى من `shellcheck` من غير ولا تحذير. والجزء الخاص بالماك في `health` اتجرّب بنسخ "تمثيلية" من `sysctl` و `vm_stat` و `lsof` بنفس شكل ناتجهم على الماك، مش على ماك حقيقي. فلو شغلته على الماك بتاعك ولقيت رقم شكله غريب، ده تمرين ممتاز: قارن بـ Activity Monitor وصلّحه.

16.3 تشريح أصعب السطور

أغلب الكود واضح لو مشيت معاه. بس فيه 4 سطور كل واحد فيهم بيلخّص فصل كامل، فخلينا نفتحهم:

تشريح الأمر

```
case "$(uname -s)" in Darwin) ... ;; *) ... ;; esac
```

1. `uname -s` اسم الـ Kernel: `Darwin` (الماك، ↩ 2.6 ص 23) أو `Linux`. في الكود حفظناه في `OS` مرة واحدة فوق، وبنسأل عليه بـ `[[ "$OS" == Darwin ]]`.
2. `Darwin)` فرع الماك: `sysctl -n hw.ncpu` بدل `nproc`، و `vm.loadavg` بدل `/proc/loadavg` (الماك معندوش `/proc`، ↩ 4.2 ص 38)، و `lsof` بدل `ss` (↩ 10.9 ص 125)، و `ps -A ... -r` بدل `--sort` (↩ 9.1 ص 97).
3. `*)` أي حاجة تانية = Linux.

تشريح الأمر

```
vm_stat | awk -v total="$total" '/page size of/ { ps = $8 } ... END { printf ... }'
```

1 `vm_stat` أداة الماك اللي بتعرض الذاكرة بعدد "الصفحات" (Pages)، مش بالجيجا. وأول سطر فيها: `(page size of 16384 bytes)` على Apple Silicon.

2 `-v total="$total"` ابعت متغير من bash لجوه awk (الرامات الكلية بالبايت من `sysctl -n hw.memsize`).

3 `/page size of/ { ps = $8 }` لما تلاقي السطر ده، خد الكلمة التامنة (الرقم) (↩ ص 86 8.3). وبنفس الطريقة بناخد Pages active و wired و occupied by compressor. والـ awk بيتجاهل النقطة اللي في آخر كل رقم لوحده لما يحوّله لرقم.

4 `END { ... }` بعد ما السطور تخلص: المستخدم = (active + wired + compressed) × حجم الصفحة، وقسّمه على `2^30` عشان يبقى جيجا. ده تقريب قريب من رقم "Memory Used" في Activity Monitor.

تشريح الأمر

```
ls -1t "$DEST"/"$base"_*.tar.gz | tail -n +$((KEEP + 1)) | while IFS= read -r old; do rm -v -- "$old"; done
```

1 `ls -1t "$DEST"/"$base"_*.tar.gz` أرشيفات المجلد ده بس، واحد في كل سطر (`-1`)، الأحدث الأول (`-t`). الـ Glob بره الـ Quotes عشان يتوسع، والمتغيرات جوه الـ Quotes عشان المسافات (↩ ص 75 7.4).

2 `| tail -n +$((KEEP + 1))` من السطر رقم KEEP+1 للآخر = كل حاجة بعد أحدث 7 (↩ ص 53 5.5).

3 `| while IFS= read -r old` اقرا كل اسم كامل، حتى لو فيه مسافات (↩ ص 148 12.4).

4 `do rm -v -- "$old"; done` امسحه واطبع اسمه، و `--` نهاية الـ Options. ولو مفيش ولا ملف قديم، اللوب مبيلفّش ولا مرة. ليه مش `xargs -r rm`؟ لأن `-r` مش مضمونة على الماك (↩ ص 90 8.9).

على Linux بس: find -printf

لو الأداة هتشتغل على Linux بس، فيه طريقة أقوى مع الأسامي الغريبة:

Terminal

```
find "$DEST" -maxdepth 1 -name "x_*.tar.gz" -printf '%T@ %p\n' \
  | sort -rn | tail -n +8 | cut -d' ' -f2- | xargs -r rm -v --
```

الـ `-printf '%T@ %p\n'` بتطبع وقت التعديل كرقم ثواني ثم المسار، و `cut -d' ' -f2-` بتشيل الرقم. بس `-printf` ميزة GNU find ومش موجودة في find بتاع الماك (المقابل هناك `stat -f '%m %N'`)، وعشان كده opskit استخدم `ls -t`.

تشريح الأمر

```
rsync -az --delete --exclude='.git' ${dry[@]+"${dry[@]}"} "${src%/}/" "$REMOTE"
```

1. `rsync -az` — Archive + ضغط (↩ 10.7 ص 122). شغال مع openrsync بتاع الماك، ومع rsync الأصلي من brew أو على Linux.
2. `--delete` — مرآة: امسح من السيرفر اللي اتمسح عندك.
3. `--exclude='.git'` — متنشرش مجلد git (ولا `.DS_Store`، ↩ 4.3 ص 41).
4. `${dry[@]+"${dry[@]}"}` — Array: يا فاضية (ولا Argument خالص) يا فيها `--dry-run`. الشكل الغريب ده عشان bash 3.2: `"${dry[@]}"` لوحدها مع `set -u` وهي فاضية بتوقع الـ Script بـ `unbound variable` و `${x+y}` معناها "لو x متعرّفة حط y" (↩ 12.2 ص 144). ولو استخدمنا متغير نصي فاضي `"$dry"` كان هيبقى Argument فاضي `''` و rsync يشتكي.
5. `"${src%/}/"` — شيل أي `/` في الآخر وحط واحدة: نضمن "المحتويات" مهما المستخدم كتب.
6. `"$REMOTE"` — الوجهة من الإعدادات (`prod` من ssh config، ↩ 10.5 ص 121).

16.4 التثبيت والتشغيل اليومي

الأداة جاهزة. خلينا نحطها في مكانها ونخليها جزء من يومك:

Terminal

```
user@MacBook ~ % mkdir -p ~/.local/bin
user@MacBook ~ % code ~/.local/bin/opskit          # paste the code, save
user@MacBook ~ % chmod +x ~/.local/bin/opskit
user@MacBook ~ % brew install shellcheck && shellcheck ~/.local/bin/opskit
user@MacBook ~ % opskit health
user@MacBook ~ % opskit logs /opt/homebrew/var/log/nginx/access.log | tee report.txt
user@MacBook ~ % opskit deploy -n ./site           # dry run first
user@MacBook ~ % opskit deploy ./site
```

~/.zshrc

```
export OPSKIT_DEST="$HOME/backups" OPSKIT_KEEP=10
alias ok='opskit'
_opskit() { compadd health logs backup deploy --help --version; }
compdef _opskit opskit
```

`compadd` و `compdef` من نظام الـ Completion بتاع zsh: "لما تضغط Tab بعد opskit، اقترح الكلمات دي" (↩ 14.5 ص 176). لازم ييجوا بعد سطر `compinit` في الملف. والـ Alias من ↩ 13.2 ص 159. (وفي bash: `complete -W "health logs backup deploy --help --version" opskit`).

وعشان الـ Backup يشتغل لوحده كل يوم الساعة 2:30 الفجر، LaunchAgent (↩ 9.5 ص 103):

~/Library/LaunchAgents/com.ahmed.opskit.backup.plist

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>  <string>com.ahmed.opskit.backup</string>
  <key>ProgramArguments</key>
  <array>
    <string>/Users/ahmed/.local/bin/opskit</string>
    <string>backup</string>
    <string>/Users/ahmed/projects</string>
  </array>
  <key>EnvironmentVariables</key>
  <dict>
    <key>PATH</key>  <string>/opt/homebrew/bin:/usr/bin:/bin:/usr/sbin:/sbin</string>
    <key>OPSKIT_KEEP</key> <string>10</string>
  </dict>
  <key>StartCalendarInterval</key>
  <dict><key>Hour</key><integer>2</integer><key>Minute</key><integer>30</integer></dict>
  <key>StandardErrorPath</key> <string>/Users/ahmed/opskit.log</string>
</dict>
</plist>
```

Terminal

```
user@MacBook ~ % plutil -lint ~/Library/LaunchAgents/com.ahmed.opskit.backup.plist
/Users/ahmed/Library/LaunchAgents/com.ahmed.opskit.backup.plist: OK
user@MacBook ~ % launchctl bootstrap gui/$(id -u)
~/Library/LaunchAgents/com.ahmed.opskit.backup.plist
user@MacBook ~ % launchctl kickstart gui/$(id -u)/com.ahmed.opskit.backup    # test it now
user@MacBook ~ % tail ~/opskit.log
```

لاحظ الـ `PATH` جوه الـ plist: launchd مش بيقرا `~/.zprofile`، فمن غيره opskit مش هيلاقي أدوات brew. واللوج في `StandardErrorPath` لأن `log` بتاعتنا بتكتب على stderr. ولو الماك كان نايم الساعة 2:30، launchd هيشغّلها أول ما يصحى. وعايز تقرير صحة كل يوم اتنين؟ نفس الملف باسم تاني، و `health` بدل `backup`، و `<key>Weekday</key><integer>1</integer>` في الـ `StartCalendarInterval`.

على السيرفر: tmux و cron

```
crontab -e
```

```
30 2 * * * $HOME/.local/bin/opskit backup $HOME/docs >> $HOME/opskit.log 2>&1
0  8 * * 1 $HOME/.local/bin/opskit health > $HOME/health-weekly.txt 2>&1
```

Backup كل يوم 2:30 الفجر، وتقرير صحة كل يوم اتنين الساعة 8. و cron بيفهم `$HOME` لأنه بيشغّل السطر بـ sh (↩ 12.8 ص 152). ولو شغّال على سيرفر بعيد، افتح `tmux new -s ops` وشغّل `watch -n 60 opskit health` في Pane، و `tail -f` على اللوج في Pane تاني (↩ 14.6 ص 177). (و `watch` مش جاي مع الماك: `brew install watch`).

كل الكتاب في أداة واحدة

اتفرج: `|` و `>>` و `2>&1` (الفصل 7)، و awk و sort و uniq و ls و while (الفصلين 8 و 12)، و ps و lsof و launchd (الفصل 9)، و tar و rsync و ssh (الفصل 10)، و brew و shellcheck (الفصل 11)، و set و case والدوال والـ Arrays و bash 3.2 (الفصل 12)، و PATH و alias و export و zsh completion (الفصلين 13 و 14)، و `--` والـ Dry run والـ Quotes (الفصل 15)، والفروق بين BSD و GNU (الفصل 2). مفيش سطر مش فاهمه.

16.5 طوّرها بنفسك

1. ضيف `opskit clean` يمسح ملفات `/tmp` بتاعتك الأقدم من 7 أيام (مع `-n` للتجربة).
2. خلي `health` يطلع Exit code 1 لو أي قرص أكتر من 90%، عشان تستخدمه في المراقبة.
3. ضيف خيار `--json` لـ `logs` يطلع النتيجة JSON (تلميح: `awk` أو `jq` من brew).
4. ابعت لنفسك تنبيه لو `health` فشل: على الماك Notification بـ `osascript` أو `say` (↩ 14.3 ص 171)، وعلى السيرفر curl لـ Webhook (↩ 10.2 ص 115).
5. ضيف `opskit battery` للماك بس، بيستخدم `pmset -g batt`، ويطبع رسالة واضحة لو اتشغّل على Linux.

ملخص الفصل

- Script واحد بـ Subcommands عن طريق `case` ودوال، ونظامين بـ `uname -s`.
- stdout للنتايج و stderr للرسايل، والإعدادات من متغيرات بيئة بقيم افتراضية.
- اكتب لـ bash 3.2: `${arr[@]+"${arr[@]}"}` مع `set -u`، ومفيش `xargs -r` ولا `find -printf` على الماك.
- Arguments لـ Arrays اختيارية، و `--` قبل الأسماء، و Dry run قبل أي مسح أو نشر.
- التثبيت في `~/.local/bin` + alias + compdef + LaunchAgent (أو cron على السيرفر).

صح ولا غلط؟

1. رسايل `log` في opskit بتظهر في `report.txt` لما تعمل `| tee report.txt` . صح / غلط
2. `${dry[@]+"${dry[@]}"}` وهي فاضية مبتضيفش أي Argument. صح / غلط
3. الـ LaunchAgent بيقرا `~/.zprofile` فبيلاقي `/opt/homebrew/bin` في الـ PATH لوحده. صح / غلط

؟ أسئلة مراجعة

1. ليه `log` و `die` بيكتبوا على stderr؟
2. اشرح الـ Pipeline بتاع مسح الـ Backups القديمة خطوة خطوة، وليه مستخدمناش `find -printf` .
3. ليه بنستخدم Array لـ `dry` بدل متغير عادي؟ وليه الشكل `${dry[@]+...}` ؟
4. إزاي opskit بيعرف هو شغال على الماك ولا Linux، وإيه الأوامر اللي بتختلف؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل سطّب opskit وشغّل `opskit health` و `opskit --version` ، وقارن رقم الرامات بـ Activity Monitor.
2. متوسط ضيف `opskit clean` بالمواصفات اللي في "طوّرها بنفسك" (تلميح: `find /tmp -user "$USER" -mtime +7`).
3. صعب خلّي `health` يرجع Exit code 1 لو أي قرص فوق 90%، واعمل LaunchAgent بيشغّله كل ساعة ويعمل Notification بس لما يفشل (تلميح: `||` و `osascript`).

ملحق أ

Cheat Sheet وجدول الـ Flags وجدول الرموز

ورقة مراجعة لكل الأوامر المهمة في الكتاب مقسّمة حسب الموضوع، وبعدها جدول الـ Flags المشتركة بين الأوامر ومعانيها المختلفة، وجدول لكل رمز ممكن تشوفه في سطر أوامر.

Navigation	التنقل
`pwd`	أنا فين؟
`ls -lahG`	كل الملفات بالتفاصيل والمخفي والألوان
`ls -lhtr`	بالوقت، الأحدث في الآخر
`cd -`	المجلد اللي قبله
`cd ~ / cd`	الـ home (/Users/you)
`open .`	افتح المجلد في Finder
`cdf`	روح لمجلد Finder (دالة الفصل 13)
`tree -L 2`	شجرة لمستويين (brew install tree)

Files	الملفات
`mkdir -p a/b/c`	مجلدات متداخلة
`touch f`	ملف فاضي / حدّث الوقت
`cp -a src dst`	نسخة طبق الأصل
`cp -c big.mov copy.mov`	Clone على APFS في لحظة
`mv old new`	نقل أو تغيير اسم
`rm -I files`	امسح واسأل مرة (مفيش Trash!)
`trash f`	للـ Trash (brew install trash)
`ln -s target link`	Soft link

Viewing — القراءة

```
cat f
```
اطبع الملف

```
less f
```
اقرا صفحة صفحة (/ للبحث، q للخروج)

```
head -n 20 f
```
أول 20 سطر

```
tail -n 50 -f f
```
آخر 50 وتابع لايف

```
file f
```
نوع الملف الحقيقي

```
stat -x f
```
التفاصيل كاملة (stat -f للصيغ)

```
qlmanage -p f
```
Quick Look

```
wc -l f
```
عدد السطور

Text — النصوص

```
grep -rni 'x' dir
```
دوّر جوه الملفات

```
grep -v x
```
السطور اللي مفيهاش x

```
sed -i '' 's/a/b/g' f
```
استبدال جوه الملف (الماك)

```
awk '{print $1}' f
```
العمود الأول

```
cut -d: -f1 f
```
قص بفاصل

```
sort | uniq -c | sort -rn
```
الأكتر تكراراً

```
tr 'a-z' 'A-Z'
```
بدّل حروف

```
pbpaste | cmd | pbcopy
```
من وإلى الـ Clipboard

Permissions — الصلاحيات

```
chmod 755 f
```
rwxr-xr-x

```
chmod u+x f
```
ضيف تشغيل للمالك

```
sudo chown -R $(whoami):staff dir
```
رجّع الملكية ليك

```
id / groups
```
أنا مين (staff و admin)

```
ls -le@
```
ACL و Extended attributes

```
xattr -d com.apple.quarantine app
```
شيل علامة النت

```
csrutil status
```
SIP شغال؟

Find — البحث عن ملفات

```
find . -name '*.log'
```
بالاسم (المسار إجباري على الماك)

```
find . -type f -size +100M
```
الكبيرة

```
find . -mtime -1
```
اتعدلت النهارده

```
find . -name '*.tmp' -delete
```
دوّر وامسح

```
find . -print0 | xargs -0 cmd
```
بأمان مع المسافات

```
mdfind -name invoice
```
Spotlight: في ثانية

```
mdfind -onlyin ~/Docs 'kMDItemKind == PDF'
```
Spotlight بشرط ومكان

Redirection — التوجيه والربط

`cmd > f`	stdout لملف (يمسح)
`cmd >> f`	ضيف في الآخر
`cmd 2> /dev/null`	اخفي الأخطاء
`cmd > f 2>&1`	الاتنين لملف
`a | b`	ناتج a يدخل b
`a && b`	b لو a نجح
`a || b`	b لو a فشل
`echo $?`	Exit code آخر أمر

Processes — العمليات

`ps aux -m | head`	العمليات بالرامات (r- بالـ CPU)
`top -o cpu / htop`	مراقبة لايف
`pgrep -lf name`	PIDs بالاسم
`kill PID`	اقفل بأدب (TERM)
`kill -9 PID`	اقتل فوراً (Force Quit)
`pkill -f pattern`	بنمط في الأمر
`cmd & / Ctrl+Z ثم bg / fg`	خلفية / وقّف / قدام
`nohup cmd &`	يعيش بعد قفل الـ Terminal
`caffeinate -i cmd`	متخليش الماك ينام

Services — الخدمات (launchd)

`brew services list`	خدمات Homebrew
`brew services start x`	شغّل ومع كل Login
`launchctl list | grep x`	متسجلة؟ PID و Exit code
`launchctl bootstrap gui/$(id -u) f.plist`	سجّل LaunchAgent
`launchctl kickstart -k gui/$(id -u)/label`	إعادة تشغيل
`launchctl bootout gui/$(id -u)/label`	وقّف وشيل
`log stream --predicate 'process == "x"'`	لوج لايف
`log show --last 1h`	لوج آخر ساعة

Network — الشبكة

`ipconfig getifaddr en0`	الـ IP بتاعي
`networksetup -listallhardwareports`	الكروت (Wi-Fi = en0)
`ping -c 4 host`	الاتصال (W- بالملي ثانية)
`dig +short host`	الـ DNS
`scutil --dns`	الـ DNS servers الماك
`sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder`	امسح الـ DNS cache
`curl -I url / curl -fsSLO url`	الـ Headers / حمّل
`lsof -nP -iTCP -sTCP:LISTEN`	مين بيسمع على أنهي بورت

SSH & Transfer — السيرفرات والنقل

`ssh user@host`	ادخل
`ssh -p 2222 host`	بورت تاني
`ssh-keygen -t ed25519`	مفتاح جديد
`ssh-copy-id host`	ركّب المفتاح على السيرفر
`ssh-add --apple-use-keychain key`	الـ Passphrase في الـ Keychain
`scp -P 2222 f host:/p`	انسخ (P كبيرة)
`rsync -avzP --dry-run a/ host:b/`	مزامنة (جرّب الأول)
`ssh -N -L 8080:localhost:80 host`	نفق

Homebrew — الحزم

`brew update && brew outdated`	إيه الجديد
`brew upgrade`	رقّي الكل
`brew install x / --cask x`	أداة / برنامج .app
`brew search x / brew info x`	دوّر / تفاصيل
`brew leaves`	اللي سطبته بنفسك
`brew uninstall x && brew autoremove`	شيل ونضّف
`brew doctor`	اكتشف المشاكل
`brew bundle dump / brew bundle`	Brewfile: احفظ / رجّع

Archives — الأرشيف

`tar -czvf a.tar.gz dir`	اعمل أرشيف مضغوط
`COPYFILE_DISABLE=1 tar -czf a.tgz dir`	من غير ملفات ._ بتاعة الماك
`tar -xzvf a.tar.gz -C /dst`	فك في مكان
`tar -tzf a.tar.gz`	اعرض المحتوى
`zip -r a.zip dir / unzip a.zip`	zip

Servers — على سيرفر Linux

`sudo apt update && sudo apt upgrade`	Debian/Ubuntu
`sudo dnf upgrade`	Fedora
`systemctl status x`	حالة الخدمة
`sudo systemctl enable --now x`	مع الـ Boot ودلوقتي
`journalctl -u x -f`	لوج الخدمة لايف
`ip -br addr / sudo ss -tulnp`	العناوين / البورتات

BSD vs GNU — الماك قصاد Linux

`sed -i '' ` / `sed -i`	تعديل ملف
`date -v-1d` / `date -d yesterday`	امبارح
`stat -f%z` / `stat -c%s`	حجم الملف
`ls -G` / `ls --color`	ألوان
`du -d 1` / `du --max-depth=1`	عمق واحد
`grep -E` / `grep -P`	Regex (مفيش -P على الماك)
`brew install coreutils`	gls و gsed و gdate

Shell config — الإعدادات (zsh)

`~/.zprofile`	Homebrew والـ PATH
`~/.zshrc`	aliases ودوال و Prompt
`source ~/.zshrc` / `exec zsh`	طبّق التعديلات
`alias ll='ls -lahG'`	اختصار
`export PATH="$HOME/.local/bin:$PATH"`	ضيف للـ PATH
`which -a cmd` / `type cmd`	ده إيه وفين
`security find-generic-password -s x -w`	سر من الـ Keychain

Scripting — الـ Scripts

`#!/usr/bin/env bash`	أول سطر (يلاقي bash brew)
`set -euo pipefail`	الوضع الصارم
`"$var"` / `${var:-def}`	استخدم المتغير بأمان
`[[ -f f ]] && …`	شرط
`for f in *.txt; do …; done`	لوب على ملفات
`while IFS= read -r l; do …; done < f`	سطر سطر
`trap 'cleanup' EXIT`	نضّف عند الخروج
`${arr[@]+"${arr[@]}"}`	Array فاضية في bash 3.2

Mac goodies — أدوات الماك

`open -a 'Visual Studio Code' .`	افتح بالبرنامج ده
`pbcopy < f` / `pbpaste > f`	الـ Clipboard
`say "done"`	الماك يتكلم
`osascript -e 'display notification "x"'`	Notification
`defaults write dom key val`	إعداد مخفي
`softwareupdate --list`	تحديثات macOS
`screencapture -i -c`	Screenshot للـ Clipboard
`sw_vers` / `pmset -g batt`	النسخة / البطارية

Keys & tmux	الاختصارات
دوّر في الـ History	`Ctrl+R`
أول / آخر السطر	`Ctrl+A / Ctrl+E`
امسح كلمة / السطر	`Ctrl+W / Ctrl+U`
آخر أمر / آخر Argument	`!! / !$`
امسح الـ Scrollback / الشاشة	`Cmd+K / Ctrl+L`
تابة / قسّم (iTerm2)	`Cmd+T / Cmd+D`
عشان Alt+B و Alt+F	`Option as Meta`
Session جديدة / ارجع	`tmux new -s n / attach -t n`

1.أ جدول الـ Flags المشتركة

نفس الحرف ممكن يبقى ليه معنى مختلف من أمر للتاني. الجدول ده بيجمع أشهر الحروف ومعانيها حسب الأمر. ولما تشك: `cmd --help` أو `man cmd`.

الـ Flag	في الأوامر	المعنى
`-h`	ls و df و du و free و sort	Human-readable: أحجام K و M و G
`-h / --help`	معظم الأوامر	المساعدة (لو h- مش محجوزة لحاجة تانية)
`-r / -R`	cp و rm و grep و chmod و chown	Recursive: المجلد وكل اللي جواه (chmod و chown بالكبيرة بس)
`-r`	ls و sort	Reverse: اعكس الترتيب
`-f`	rm و cp و mv و ln	Force: من غير أسئلة
`-f`	tail و journalctl	Follow: تابع لايف
`-f`	pgrep و pkill	Full: دوّر في الأمر كامل
`-f`	tar	File: اسم الأرشيف بعدها
`-F`	awk / grep	Field separator / Fixed string
`-v`	cp و mv و rm و mkdir و tar و rsync و curl و ssh	Verbose: اشرح اللي بتعمله
`-v`	grep	Invert: اعكس التطابق

الـ Flag	في الأوامر	المعنى
`-i`	cp و mv و rm	Interactive: اسأل قبل
`-i`	grep و sort و locate	Ignore case
`-i`	sed	In-place: عدّل الملف نفسه (على الماك i- '')
`-i`	ssh	Identity: ملف المفتاح
`-n`	head و tail	عدد السطور
`-n`	grep و cat -n و less -N	أرقام السطور
`-n`	sort	Numeric
`-n / -nP`	ss و netstat و lsof	Numeric: أرقام مش أسامي (lsof: n- للـ IP و P- للبورت)
`-n`	sed و rsync	Dry run / متطبعش تلقائي
`-a`	ls	All: حتى المخفي
`-a`	cp و rsync	Archive: نسخة طبق الأصل
`-a`	tee و usermod -aG	Append: ضيف
`-l`	ls	Long: سطر تفاصيل
`-l`	ss / grep / wc	سطور / أسماء ملفات / Listening
`-p`	mkdir	Parents: اعمل الأبهات
`-p`	netstat / read / ss / ssh	Protocol / Prompt / Process / Port (على الماك)
`-P`	scp	Port (كبيرة)
`-c`	grep و uniq	Count: عِد
`-c`	cp / wc / ping / tar	Clone / Bytes / Count / Create على APFS (الماك)
`-o`	sort و curl و gcc	Output: ملف الناتج
`-O`	wget / curl	اسم من الـ URL / اسم تختاره
`-q`	grep و wget و apt	Quiet: من غير ناتج
`-y`	apt و dnf و choco	Yes: وافق تلقائي (brew مبيسألش أصلاً)
`-u`	ps و sudo و top على Linux	User
`-U`	top على الماك	User (كبيرة؛ u- الصغيرة معناها تاني)
`-u`	journalctl / sort	Unit / Unique

الـ Flag	في الأوامر	المعنى
`-s`	read / tr / curl	Silent / Squeeze / Silent
`-t`	lsof / ssh / sort / ls	بس PIDs / TTY / Separator / Time
`-e`	grep / echo / ps	Expression / Escapes / Every
`-x`	stat -x / grep -x / bash -x	Trace / السطر كله / شكل مفصّل (الماك)
`-G`	ls على الماك	Color: ألوان (على Linux --color=auto)
`-W`	ping	Wait: بالملي ثانية على الماك، بالثواني على Linux
`-v`	date على الماك	Adjust: date -v-7d = من أسبوع
`--`	كل الأوامر تقريباً	نهاية الـ Options: اللي بعدها أسماء حتى لو بتبدأ بـ -
`-`	tar و cat و diff	اسم ملف = stdin أو stdout
`--dry-run`	rsync و kubectl و git (وفي apt اسمها -s)	جرّب من غير تنفيذ
`--color=auto`	grep و diff (و ls على Linux)	لوّن على الشاشة بس

جدول أ.1 الـ Flags ومعانيها حسب الأمر

أ.2 جدول الرموز

الرمز	المعنى	مثال
`|`	Pipe: stdout الأول = stdin التاني	`ps aux | grep ssh`
`|&`	Pipe ل stdout و stderr مع بعض (zsh و bash 4+)	`make |& less`
`>`	stdout لملف ويمسح القديم	`ls > list.txt`
`>>`	stdout لملف ويضيف	`date >> log.txt`
`<`	stdin من ملف	`wc -l < f`
`2>`	stderr لملف	`cmd 2> err.txt`
`2>&1`	stderr لنفس مكان stdout	`cmd > all 2>&1`
`&>`	الاتنين لملف (bash/zsh)	`cmd &> all`
`<<EOF`	Here-doc: سطور كـ stdin	`cat <<EOF`
`<<<`	Here-string: نص كـ stdin	`wc -w <<< "a b"`
`>| / !>`	اكتب فوق رغم noclobber (>! في zsh)	`ls >| f`
`&`	شغّل في الخلفية	`sleep 60 &`

الرمز	المعنى	مثال
`&&`	نفّذ لو اللي قبله نجح	`make && ./app`
`||`	نفّذ لو اللي قبله فشل	`cd x || exit`
`;`	فاصل أوامر	`cd /tmp; ls`
`( )`	Subshell	`(cd /tmp && ls)`
`{ ; }`	Group في نفس الـ Shell	`{ date; ls; } > f`
`$VAR / ${VAR}`	قيمة متغير	`echo "${HOME}"`
`$( )`	ناتج أمر (Command substitution)	`echo $(date)`
`` ` ` ``	نفس $() بشكل قديم	`` echo `date` ``
`$(( ))`	حساب أرقام	`echo $((2+3))`
`' '`	نص حرفي 100%	`echo '$HOME'`
`" "`	نص بيفك المتغيرات	`echo "$HOME"`
`\`	الغي معنى الحرف اللي بعده / كمّل السطر	`my\ file`
`*`	أي حروف (Glob)	`ls *.jpg`
`?`	حرف واحد (Glob)	`ls img?.png`
`[ ]`	حرف من مجموعة (Glob)	`ls [ab]*`
`{a,b} / {1..5}`	توليد كلمات (Brace)	`mkdir {css,js}`
`**`	أي عمق (zsh، أو globstar في bash 4+)	`ls **/*.py`
`~`	الـ home	`cd ~/docs`
`.`	المجلد الحالي / source	`./run.sh`
`..`	المجلد الأب	`cd ..`
`/`	الجذر أو فاصل المسار	`/etc/hosts`
`#`	تعليق (أو الـ Prompt root)	`# note`
`#!`	Shebang: أول سطر في Script	`#!/usr/bin/env bash`
`% / $ / #`	Prompt: zsh (الماك) / bash / root	`% ls`
`$0 $! $$ $?`	الاسم / PID الخلفية / PID / Exit code	`echo $?`
`$# $@ $1`	Arguments: عددها / كلها	`"$@"`

الرمز	المعنى	مثال
`$'...'`	نص فيه Escapes زي \n و \r (ANSI-C)	`sed $'s/\r$//'`
`$! / !!`	آخر أمر / آخر Argument	`sudo !!`
`%1`	رقم Job	`fg %1`
`{}`	مكان الاسم في find -exec و xargs -I	`-exec rm {} +`
`; / +`	نهاية find -exec / جمّع	`-exec ls {} \;`
`<name> / [opt]`	في الشرح: قيمتك / اختياري (مش بتتكتب)	`ssh <host>`
`user@host:path`	مسار على جهاز تاني (scp و rsync)	`deploy@srv:/var/www`
`=`	تعريف متغير (من غير مسافات)	`name=Ali`

جدول أ.2 كل رمز في سطر الأوامر ومعناه

ملحق ب

قاموس المصطلحات

كل مصطلح مهم في الكتاب في سطر واحد، مترتب أبجدياً. ارجعله كل ما تقابل كلمة ومش فاكر معناها.

A

Alias

اسم مختصر بيستبدله الـ Shell بأمر أطول قبل التنفيذ.

APFS

Apple File System: نظام ملفات الماك، فيه Clones و Snapshots، وافتراضياً مش حساس لحالة الحروف.

Apple Silicon

معالجات Apple (M1 وما بعده، arm64). Homebrew عليها في /opt/homebrew.

Argument

أي كلمة بعد اسم الأمر: ملف أو قيمة أو Option.

Arithmetic expansion

`$(( ))` : حساب أرقام صحيحة جوه الـ Shell.

Array

متغير فيه أكتر من قيمة: `a=(x y)` و `"${a[@]}"` .

B

Background job

أمر شغال في الخلفية بـ `&` والـ Prompt راجعلك.

bash

Bourne Again SHell: الافتراضي على أغلب توزيعات Linux. الماك شايل نسخة قديمة 3.2 بسبب رخصة GPLv3، والحديثة بـ brew install bash.

Bottle

نسخة جاهزة متجمعة من Formula في Homebrew، بتتنزل بدل ما تتبني من الكود.

Brace expansion

`{a,b}` و `{1..5}` : توليد كلمات قبل التنفيذ.

BSD

عيلة Unix جه منها جزء كبير من الماك؛ أوامر الماك (sed و date و stat) نسخ BSD مش GNU.

Builtin

أمر جوه الـ Shell نفسه (cd و export) مش برنامج على القرص.

C

Cask

حزمة Homebrew لبرنامج بواجهة (.app) زي VS Code: `brew install --cask` .

Command substitution

`$(cmd)` : حط ناتج أمر مكانه في السطر.

Console

الشاشة والكيبورد المتوصلين بالجهاز مباشرة، أو Terminal النظام النصي.

cron

خدمة بتشغّل أوامر على مواعيد متكررة (`crontab -e`).

D

Daemon

برنامج شغال في الخلفية على طول (sshd و nginx)، وغالباً اسمه بينتهي بـ d.

Darwin

اسم الـ Kernel والنظام الأساسي تحت macOS (`uname -s`).

Dependency

مكتبة أو حزمة لازمة عشان برنامج يشتغل.

dotfiles

ملفات الإعدادات المخفية في الـ home (.bashrc و .zshrc و .ssh/config).

E

Environment variable

متغير بيتورّث للبرامج اللي بتتشغل (`export`).

Exit code

رقم بيرجّعه الأمر: 0 نجاح وأي حاجة تانية فشل (`$?`).

F

FHS
Filesystem Hierarchy Standard: مكان كل نوع ملفات في Linux. الماك قريب منه بس عنده /Users و /Applications و /Library و /System.

File descriptor
رقم للقناة المفتوحة: 0 stdin و 1 stdout و 2 stderr.

fish
Shell حديث سهل باقتراحات تلقائية، مش POSIX.

Flag / Option
حرف أو كلمة بشَرطة بتغير سلوك الأمر (`-l` و `--all`).

fork / exec
الـ Shell بينسخ نفسه (fork)، والنسخة بتستبدل نفسها بالبرنامج (exec).

Fork bomb
دالة بتنادي نفسها بشكل متضاعف لحد ما الجهاز يقف.

Formula
حزمة Homebrew لأداة Terminal أو مكتبة: `brew install wget`.

Full Disk Access
صلاحية في Security & Privacy بتسمح لبرنامج (زي Terminal) يوصل للفولدرات المحمية.

G

Gatekeeper
حماية الماك اللي بتفحص البرامج اللي نزلت من النت قبل أول تشغيل.

Glob
نمط أسماء ملفات (`*` و `?` و `[]`) بيوسعه الـ Shell.

GNU coreutils
الأوامر الأساسية (ls و cp و rm...) في Linux. على الماك brew install coreutils بيجيبها بأسامي gls و gcp و gdate.

Group
مجموعة مستخدمين ليهم نفس الصلاحيات على ملفات.

H

Hard link
اسم تاني لنفس الـ inode على نفس الـ Filesystem.

Here-doc
`<<EOF`: كتابة سطور كتير لـ stdin.

History expansion
`!!` و `!$`: إعادة استخدام أوامر قديمة.

home
مجلدك الشخصي (`~` و `$HOME`).

Homebrew
مدير الحزم الأشهر على الماك (`brew`)، بيشتغل من غير sudo.

I

inode
السجل اللي فيه بيانات الملف (الحجم والصلاحيات ومكانه) من غير الاسم.

Interactive shell
Shell بتكتب فيه بنفسك، بيقرا `~/.zshrc` (أو `~/.bashrc` في bash).

J

Job control
التحكم في الأوامر: `&` و Ctrl+Z و fg و bg و jobs.

journalctl
أداة قراية لوجات systemd على Linux (المقابل على الماك: `log show`).

K

Kernel
قلب نظام التشغيل اللي بيدير العمليات والذاكرة والأجهزة.

Keychain
خزنة الباسوردات المتشفرة في الماك، وتوصلها من الـ Terminal بـ `security`.

L

LaunchAgent
خدمة بتشتغل في جلستك لما تعمل Login، ملفها plist في `~/Library/LaunchAgents`.

launchctl
الأداة اللي بتكلم بيها launchd: bootstrap و bootout و kickstart و print.

launchd
الـ PID 1 على الماك، وبيدير كل الخدمات والـ Agents (المقابل لـ systemd).

LaunchDaemon

خدمة نظام بتشتغل مع الـ Boot كـ root، في `/Library/LaunchDaemons`.

Login shell

أول Shell بعد الدخول (SSH أو su -، وكل تابة Terminal على الماك)، بيقرا zprofile. أو bash_profile..

M

man page

صفحة الدليل الرسمي لأي أمر (`man ls`).

mDNS / Bonjour

أسامي الأجهزة على الشبكة المحلية بـ `.local` من غير DNS server.

N

noclobber

Option بتمنع `>` من الكتابة فوق ملف موجود.

O

Octal permissions

الصلاحيات بالأرقام (755 و 644): r=4 و w=2 و x=1.

openrsync

نسخة rsync بترخيص BSD اللي بقت `/usr/bin/rsync` من macOS 15.4.

P

Package manager

أداة بتسطب وتحدّث البرامج من مخازن (brew على الماك، و apt و dnf على Linux).

PATH

قائمة المجلدات اللي الـ Shell بيدوّر فيها على البرامج.

path_helper

أداة في `/etc/zprofile` بتبني الـ PATH من `/etc/paths` و `/etc/paths.d`.

PID / PPID

رقم العملية / رقم العملية الأب.

Pipe

`|`: توصيل stdout أمر بـ stdin أمر تاني.

plist

Property List: ملف إعدادات بصيغة XML (أو Binary) بيستخدمه الماك لكل حاجة، ومنه ملفات launchd.

Port

رقم بيحدد البرنامج اللي بيستقبل الاتصال جوه الجهاز (22 و 443).

POSIX

معيار مشترك لسلوك الـ Shells والأوامر على أنظمة Unix.

PowerShell

Shell من Microsoft، الـ Pipe فيه بيعدّي Objects مش نصوص.

Process

برنامج شغال في الذاكرة ليه PID.

Prompt

النص اللي قبل المؤشر (`user@MacBook ~ %`)، في zsh متغير PROMPT وفي bash PS1.

PTY

Pseudo-terminal: Terminal افتراضية بتعملها برامج زي Terminal.app و iTerm2 و ssh (`/dev/ttys000` على الماك).

Q

Quarantine

علامة `com.apple.quarantine` اللي المتصفح بيحطها على أي حاجة نزلت من النت.

Quoting

استخدام `'` و `"` و `\` عشان تتحكم في فهم الـ Shell للرموز.

R

Readline

المكتبة اللي بتدي اختصارات السطر (Ctrl+A و Ctrl+R) في bash وغيره؛ zsh عنده محرر خاص بيه اسمه ZLE بنفس الاختصارات.

Redirection

توجيه stdin أو stdout أو stderr لملف (`>` و `<` و `2>`).

Regex

Regular Expression: لغة أنماط للبحث في النصوص (grep و sed و awk).

Repository

سيرفر فيه حزم موقّعة بيسحب منه مدير الحزم.

root

المستخدم الأعلى صلاحيات (UID 0)، وكمان اسم المجلد `/`.

Rosetta

مترجم بيشغّل برامج Intel على Apple Silicon.

rsync

أداة مزامنة بتنقل الفرق بس، محلياً أو عبر SSH.

S

Script
ملف فيه أوامر بتتنفذ ورا بعض، أوله Shebang.

setuid
صلاحية بتخلي البرنامج يشتغل بصلاحيات صاحبه (زي passwd).

Shebang
`#!` في أول سطر: بيحدد البرنامج اللي هيشغّل الـ Script.

Shell
البرنامج اللي بيفهم الأوامر وينفذها (bash و zsh و fish).

Signal
رسالة بتتبعت لعملية (TERM و KILL و INT و HUP).

SIP
System Integrity Protection: بيمنع حتى root إنه يعدّل ملفات النظام المحمية.

Soft link (Symlink)
ملف بيشاور على مسار ملف تاني، وبيتكسر لو الأصل اتمسح.

Spotlight
فهرس البحث في الماك؛ من الـ Terminal بـ `mdfind`.

SSH
اتصال متشفر لتشغيل Shell على جهاز تاني.

SSH key
زوج مفاتيح (خاص وعام pub.) للدخول من غير باسورد.

stdin / stdout / stderr
الدخل والخرج والأخطاء: القنوات 0 و 1 و 2.

Sticky bit
على مجلد: محدش يمسح غير ملفاته (`/tmp`).

Subcommand
فعل بعد اسم الأداة (`git commit` و `apt install`).

Subshell
نسخة ابن من الـ Shell (`( )` أو Pipe)، تغييراتها مبترجعش.

sudo
تشغيل أمر واحد بصلاحيات root بباسوردك.

systemd
نظام الـ Init وإدارة الخدمات في أغلب توزيعات Linux (PID 1). المقابل على الماك launchd.

T

Tab completion
Tab بيكمّل أسماء الأوامر والملفات.

tar
أداة أرشفة بتجمع ملفات في ملف واحد، ومعاها ضغط.

TCC
Transparency, Consent, and Control: نظام صلاحيات الخصوصية (Documents و Desktop والكاميرا).

Terminal
البرنامج (أو الجهاز زمان) اللي بيعرض النص ويستقبل الكتابة.

Terminal.app
برنامج الـ Terminal اللي جاي مع الماك، بديله الأشهر iTerm2.

tmux
Terminal multiplexer: Sessions بتفضل شغالة بعد ما تفصل، وتقسيم للشاشة.

TTY
TeleTYpewriter: اسم الـ Terminal (`/dev/ttys000` على الماك، و `/dev/pts/0` على Linux).

U

UID / GID
رقم المستخدم / رقم الجروب.

umask
قناع بيحدد الصلاحيات الافتراضية للملفات الجديدة.

W

Word splitting
تقسيم الـ Shell للقيم على المسافات لو مش جوه Quotes.

X

xargs
بيحوّل سطور stdin لـ Arguments لأمر.

xattr
Extended attributes: بيانات إضافية على الملف زي Quarantine، بتشوفها بـ `ls -l@` و `xattr -l`.

Z

zsh
Shell متوافق تقريباً مع bash بمميزات أكتر، الافتراضي على macOS من Catalina (2019).

ملحق ج

الإجابات والحلول

جاوب بنفسك الأول، وبعدين قارن. لكل فصل: إجابات أسئلة المراجعة، وبعدها صح ولا غلط، وبعدها حلول التحديات. حلول التحديات طريقة واحدة من طرق كتير، فلو حلّك مختلف وشغال فهو صح.

الفصل 1: جوه الـ Terminal

1. الـ Terminal (Terminal.app أو iTerm2) هو الشباك اللي بيعرض ويستقبل الكتابة، والـ Shell (zsh على الماك) هو البرنامج اللي جواه بيفهم الأوامر وينفذها.
2. لأن البرنامج الخارجي بيشتغل في Process ابن، وتغيير المجلد في الابن مش بيرجع للأب؛ فلازم الـ Shell نفسه يغير مجلده.
3. اللي مجلده جاي الأول في الـ PATH (غالباً `/opt/homebrew/bin` قبل `/usr/bin`). اتأكد بـ `which -a node` أو `type -a node`.
4. لأن المجلد الحالي مش في الـ PATH (لأسباب أمنية)، فلازم تقول المسار صراحة: `./` = هنا.

صح ولا غلط: (1) غلط: الاتنين مستقلين، و iTerm2 بيشغّل نفس zsh. · (2) صح. · (3) غلط: `type` (أو `whence -w` في zsh) هو الأدق.

التحديات:

1. المسارين مختلفين، زي `/dev/ttys000` و `/dev/ttys001`: كل تابة ليها Terminal خاصة.
2. `cd` و `pwd` و `type` Builtins، و `ls` برنامج خارجي `/bin/ls` (وممكن يطلع alias).
3. `echo "$PATH" | tr ':' '\n'`: اطبع المتغير، وحوّل كل `:` لسطر جديد.
4. الـ PPID بتاع bash = الـ PID بتاع zsh.

`ps -o pid,ppid,command` بيبيّن السلسلة: Terminal ← login ← `-zsh` ← bash ← ps.

الفصل 2: تشريح أي أمر

1. GNU هي نسخ الأوامر في Linux، و BSD هي النسخ اللي الماك واخدها من FreeBSD، ونفس الأمر بيختلف في Options.

`sed -i` على الماك محتاجة امتداد للنسخة الاحتياطية، فبتتكتب `sed -i '' 's/a/b/' f`.

2. `grep` الأمر، و `--max-count=20` Option طويلة بقيمة (وقف بعد 20 تطابق)، و `-i` تجاهل حالة الحروف، و `error` النمط (Argument)، و `a.log b.log` الملفات.
3. `rm -- --help` أو `rm ./--help`.
4. `head -n` عدد السطور، و `ls -n` أرقام UID/GID، و `sort -n` ترتيب رقمي (و `grep -n` أرقام السطور، و `lsof -n` من غير DNS).
5. `man passwd` بيفتح القسم 1 (الأمر passwd)، و `man 5 passwd` القسم 5 (صيغة ملف `/etc/passwd`).

صح ولا غلط: (1) غلط: `f` بتاخد اللي بعدها، فـ `z` هتبقى اسم الأرشيف. · (2) غلط: `-R` Recursive و `-r` Reverse. · (3) صح.

التحديات:

1. الـ Flag هو `-h` (Human-readable)، مع `-l` أو `-s`.
2. `rm -- -n` و `rm ./-n`.
3. أرشف `logs/` مضغوط على stdout (`-f -`)، و Pipe لـ ssh، وعلى السيرفر فك من stdin في `/backup`. الشرح كامل في فصل الشبكات (tar).

الفصل 3: الـ Shells

1. POSIX معيار مشترك لسلوك الـ Shells والأوامر. Script بـ `sh` هيشتغل على أي نظام Unix (حتى اللي مفيهوش bash زي Alpine، أو dash على Ubuntu).
2. الـ Arrays بتبدأ من 1 في zsh ومن 0 في bash، و zsh مبيقسمش المتغيرات اللي من غير Quotes على المسافات (bash بيقسمها).
3. bash: `export NAME=Ali`، و fish: `set -gx NAME Ali`، و PowerShell: `$env:NAME = "Ali"`.

4. `brew install bash`، وبعدين `echo /opt/homebrew/bin/bash | sudo tee -a /etc/shells`، وبعدين `chsh -s /opt/homebrew/bin/bash`. لازم `/etc/shells` لأن `chsh` بيرفض أي Shell مش مكتوب فيه.
5. لأن bash 4 وما بعده رخصتهم GPLv3 و Apple مش عايزة تلتزم بيها، فوقفت عند 3.2.57. ومش هيشتغل فيه: `declare -A` و `mapfile` و `${var,,}` و `globstar` و `&|`.

صح ولا غلط: (1) غلط: fish مش POSIX والـ Syntax مختلف. · (2) غلط: `$SHELL` = الـ Login shell، مش اللي شغال (اعرفه بـ `ps -p $$`). · (3) صح.

التحديات:

1. `echo $SHELL` و `ps -p $$` و `echo $0`. ممكن يختلفوا لو فتحت Shell جوه Shell.
2. مفيش إجابة واحدة: المهم تلاحظ الاقتراح الرمادي من الـ History والألوان.
3. bash: `for i in 1 2 3; do echo $i; done`، و fish: `for i in 1 2 3; echo $i; end`، و PowerShell: `foreach ($i in 1..3) { $i }`.
4. `/bin/bash -c 'declare -A m; m[a]=1; echo ${m[a]}'` هيقولك `declare: -A: invalid option`، و `/opt/homebrew/bin/bash -c '...'` هيطبع 1.

الفصل 4: الـ Filesystem على الماك

1. SSH في `~/.ssh/config`، و VS Code في `~/Library/Application Support/Code/User/settings.json`، وبرامج Homebrew على M2 في `/opt/homebrew/bin` (والملفات الحقيقية في `/opt/homebrew/Cellar`).
2. `../../Downloads`.
3. الـ Hard link اسم تاني لنفس الـ inode، والـ inode جوه Filesystem واحد بس، فمينفعش لقرص تاني. وللمجلدات ممنوع عشان ميحصلش لف في الشجرة.
4. لأن `/etc` على الماك Symlink لـ `/private/etc`، والملفات الحقيقية هناك. `ls -ld /etc` بيوريك السهم.
5. `du -h -d 1 ~ 2>/dev/null | sort -h | tail -4` (آخر سطر هو الـ home نفسه).

صح ولا غلط: (1) صح: الـ inode بيفضل طول ما فيه اسم بيشاور عليه. · (2) غلط: مفيش `/proc`؛ المعلومات بتيجي من `ps` و `sysctl`. · (3) صح: APFS افتراضياً Case-insensitive. · (4) غلط: نسبي، بيبدأ من المجلد الحالي.

التحديات:

1. مفيش إجابة واحدة؛ المهم إن `cd -` بيرجعك، و `open .` بيفتح نفس المكان في Finder.
2. `sysctl -n hw.memsize` (بالبايت) و `sysctl -n machdep.cpu.brand_string`، و `diskutil info / | grep -i 'file system'` هيقول APFS.
3. الـ Hard link لسه بيطبع المحتوى (نفس الـ inode)، والـ Soft link بيقول `No such file or directory` لأنه بيشاور على اسم اتمسح.
4. الماك هيلاقي الملف عادي لأنه مش حساس للحالة. على سيرفر Linux `import './Test'` و `'./test'` ملفين مختلفين، فالكود اللي شغال عندك ممكن يقع هناك.
5. الملف التنفيذي في `Calculator.app/Contents/MacOS/Calculator`، وتشغله بالمسار ده كامل (والـ app. نفسه مجلد عادي).

الفصل 5: التنقل والملفات

1. `ls -lhtrG ~/Downloads`.
2. `cp -r` بينسخ المحتوى بس، و `-a` بيحافظ على الصلاحيات والتواريخ والـ Links. وعلى الماك `cp -R src dst` بينسخ المجلد نفسه، و `cp -R src/ dst` بينسخ محتوياته بس جوه dst.
3. `mv draft.md final.md`.
4. `tail -n 100 -f app.log`.

صح ولا غلط: (1) غلط: mv بينقل المجلد على طول. · (2) غلط: من السطر التاني للآخر. · (3) غلط: بيقرا أول Bytes في الملف (Magic numbers).

التحديات:

1. `mkdir -p lab/{in,out,tmp} && touch lab/in/{a,b,c}.txt`.

2. `cp -a lab lab-backup` وبعدين `stat -f '%Sm' lab/in/a.txt lab-backup/in/a.txt` : نفس الوقت.
3. `less /var/log/install.log` ثم `G` للآخر، ثم `?error` و `n` للي قبله.

الفصل 6: الصلاحيات

1. `750` و `664` .
2. المفتاح الخاص `600` ، وال Script `755` (أو `700`) لو ليك بس).
3. هيشيلك من كل الجروبات التانية ويحطك في docker بس (وممكن تخسر sudo لو كنت في جروب sudo). الصح `usermod -aG` .
4. علامة `com.apple.quarantine` اللي المتصفح حطها، و Gatekeeper رافض البرنامج لأنه مش موقّع أو مش Notarized. الحل: كليك يمين ← Open (أو System Settings ← Privacy & Security ← Open Anyway)، أو `xattr -d com.apple.quarantine tool` لو واثق في المصدر.
5. `rwx` للكل عشان أي حد يقدر يعمل ملفات، وال `t` (Sticky bit) بتمنع أي حد يمسح ملف مش بتاعه. (على الماك `/tmp` Symlink لـ `/private/tmp`).

صح ولا غلط: (1) غلط: 4 = قراية بس. · (2) صح. · (3) غلط: ال `>` بيعملها ال Shell بصلاحياتك قبل sudo. الحل: `echo hi | sudo tee /etc/test.txt` . · (4) غلط: SIP بيمنع حتى root.

التحديات:

1. قبل: `permission denied` (وتقدر تشغله بـ `bash hi.sh`)، وبعد `chmod +x` : `./hi.sh` بيطبع hi.
2. `mkdir secret && chmod 700 secret && touch secret/f && chmod 600 secret/f` ، و `sudo -u nobody cat secret/f` هيقولك `Permission denied` .
3. الملف اللي من المتصفح عليه `com.apple.quarantine` (وممكن `com.apple.macl` و `kMDItemWhereFroms`)، واللي من `curl -O` مفيهوش، عشان كده curl بيعدّي من Gatekeeper.
4. `(umask 077; touch f; mkdir d; ls -ld f d)` : الملف `-rw-------` والمجلد `drwx------` . وال `( )` عشان التغيير ميأثرش على ال Shell بتاعك.

الفصل 7: قوة الـ Shell

1. `find / -name '*.conf' > found.txt 2>/dev/null` .
2. `>` بيكتب في ملف ويمسح القديم، و `>>` بيضيف في الآخر، و `|` بيدّي الناتج لأمر تاني.
3. من غير Quotes لو اسم الملف `my file.txt` هيتقسم لكلمتين، و rm هيحاول يمسح `my` و `file.txt` (اللي ممكن يكونوا ملفات تانية!). ال Quotes بتخليه اسم واحد.
4. `;` بينفّذ التاني مهما حصل، و `&&` بس لو الأول نجح. استخدم `&&` لما التاني معتمد على الأول (`cd dir && rm *`).
5. `$?` Exit code آخر أمر، و `$$` ال PID ال Shell، و `$!` PID آخر أمر في الخلفية.

صح ولا غلط: (1) غلط: الترتيب مهم؛ الأخطاء بتروح للشاشة. الصح `> f 2>&1` . · (2) غلط: ال Shell هو اللي بيوسّع ال Glob. · (3) صح. · (4) غلط: 0 = نجاح.

التحديات:

1. `touch note{1..10}.txt` ثم `ls note?.txt` .
2. `d=$(date +%F) && mkdir -p "$d" && cd "$d" && printf '%s %s\n' "$USER" "$$" > info.txt || echo failed` .
3. استخدم Here-doc عادي (من غير Quotes حوالين EOF) واكتب `$HOME` في السطر الأول و `\$HOME` في التاني.

الفصل 8: النصوص والبحث

1. `grep -ic error *.log` (لكل ملف)، أو `cat *.log | grep -ic error` للإجمالي.
2. على الماك: `sed -i.bak 's/localhost/127.0.0.1/g' config` (نفس السطر شغال على Linux كمان).
3. التلاتة بيحددوا الفاصل بين الأعمدة، كل أداة بحرف مختلف.
4. عشان أسماء الملفات اللي فيها مسافات أو سطور جديدة: `-print0` بيفصل بـ Null و `-0` بيفهمه.
5. `find` للبحث الدقيق بالاسم والحجم والتاريخ في أي مكان (حتى اللي Spotlight مش بيفهرسه)، و `mdfind` للبحث السريع جداً بالاسم أو المحتوى في الملفات اللي Spotlight فهرسها، و `grep` للبحث جوه نص الملفات في مشروع.

صح ولا غلط: (1) غلط: Invert. · (2) غلط: بيشيل التكرار المتجاور بس، فلازم `sort` قبله. · (3) غلط: على الماك محتاجة `-i ''`، وإلا هيفهم `'s/a/b/'` على إنه امتداد النسخة ويقولك `invalid command code`. · (4) صح.

التحديات:

1. `grep -v '^#' /etc/passwd | cut -d: -f1 | sort`
.
2. `awk '$9 == 401 || $9 == 403 {print $7}' access.log | sort | uniq -c | sort -rn | head -5`
.
3. `find ~ -type f -exec du -h {} + 2>/dev/null | sort -rh | head -10`
.

الفصل 9: العمليات والخدمات

1. `ps aux -m | head -6`، أو `top -o mem` (أو Activity Monitor ← Memory).
2. `kill` بالـ PID، و `pkill` بنمط في الاسم أو الأمر (`-f`)، و `killall` بالاسم بالظبط.
3. `Ctrl+Z` وبعدين `bg`.
4. `brew services list` (الحالة)، و `brew services info redis`، واللوج `tail /opt/homebrew/var/log/redis.log`، وبعدين `brew services restart redis` (أو `start` لو مش متسجلة).
5. الـ LaunchAgent بيشتغل في جلستك لما تعمل Login وبصلاحياتك، في `~/Library/LaunchAgents` (أو `/Library/LaunchAgents` لكل المستخدمين). الـ LaunchDaemon بيشتغل مع الـ Boot كـ root، في `/Library/LaunchDaemons`.

صح ولا غلط: (1) غلط: TERM (15). · (2) صح. · (3) غلط: launchd مبيقراش ملفات zsh، فلازم `EnvironmentVariables` أو مسارات كاملة. · (4) غلط: بيوقفه مؤقتاً.

التحديات:

1. `sleep 600 & sleep 600 & jobs; kill %2`.
2. `pgrep -x Finder >/dev/null && echo running || echo stopped`
.
3. plist فيه `Label` و `ProgramArguments` (`/bin/sh -c 'date >> /Users/YOU/hello.log'`) و `StartInterval` = 60، وبعدين `launchctl bootstrap gui/$(id -u) FILE`، و `launchctl print gui/$(id -u)/com.YOU.hello`، و `launchctl bootout gui/$(id -u)/com.YOU.hello`. وعلى Linux: Unit في `~/.config/systemd/user/` + Timer، و `systemctl --user enable --now`.

الفصل 10: الشبكات

1. `ipconfig getifaddr en0` (عندي IP؟) ← `route -n get default` و `ping` للراوتر ← `ping 8.8.8.8` (النت) ← `dig site.com` (الـ DNS) ← `curl -I https://site.com` (الخدمة) ← على السيرفر `ss -tlnp` أو `lsof -i` (بيسمع؟).
2. `-f` افشل لو HTTP error، و `-s` ساكت، و `-S` بس اعرض الأخطاء، و `-L` اتبع الـ Redirects.
3. `Host myserver` ثم `HostName IP` ثم `User admin` ثم `Port 2200`.
4. `-n` من غير DNS، و `-P` أرقام البورتات، و `-iTCP` اتصالات TCP، و `-sTCP:LISTEN` اللي بيسمع بس. المقابل على Linux: `sudo ss -tlnp`.
5. لأن `-W` على الماك بالملي ثانية، فـ `-W 2` = استنى 2 ملي ثانية بس، وأغلب الردود بتاخد أكتر. الصح `-W 2000`.

صح ولا غلط: (1) غلط: سيرفرات كتير بتقفل ICMP. · (2) غلط: البورت في scp بـ `-P` كبيرة. · (3) غلط: الأولى بتنسخ المحتويات، والتانية بتعمل `dst/src`. · (4) صح.

التحديات:

1. `dig +short github.com @1.1.1.1`، و `scutil --dns | grep nameserver`.
2. `[ "$(curl -s -o /dev/null -w '%{http_code}' https://site.com)" = 200 ] && echo UP || echo DOWN`
.
3. `ssh-keygen -t ed25519 -f ~/.ssh/lab`، و Host lab في الـ config بـ `IdentityFile ~/.ssh/lab`، و `ssh-copy-id -i ~/.ssh/lab.pub lab`، ثم `rsync -avz --dry-run dir/ lab:dir/` وبعدين من غير `--dry-run`.

الفصل 11: Homebrew ومديري الحزم

1. `update` بيحدّث brew وقائمة الحزم، و `upgrade` بيرقّي البرامج نفسها. نفس الفرق بالظبط في `apt update` و `apt upgrade`.
2. Formula = أداة Terminal أو مكتبة (`git` و `wget`)، و Cask = برنامج app. بواجهة (`google-chrome` و `visual-studio-code`).
3. `brew shellenv` بيطبع أوامر `export` للـ PATH والمتغيرات، و `eval $( )` بتاخد الناتج، و `eval` بينفذه في الـ Shell الحالي. ومكانه `~/.zprofile` لأنه بيتقري في كل Login shell (كل تابة على الماك) وبعد path_helper، فـ `/opt/homebrew/bin` يفضل في أول الـ PATH.
4. `brew install git`، و `sudo apt install git`، و `sudo dnf install git`، و `sudo pacman -S git`، و `winget install --id Git.Git -e`.
5. بيحدّث القائمة ويسطب حزمة جديدة على مكتبات قديمة (Partial upgrade)، وده ممكن يكسر النظام.

صح ولا غلط: (1) غلط: بيحدّث القائمة بس. · (2) صح. · (3) غلط: brew بيشتغل من غير sudo. · (4) صح.

التحديات:

1. `brew info wget`: هيعرض الإصدار و Dependencies.
2. `brew leaves | wc -l`.
3.
```
mkdir -p ~/dotfiles && brew bundle dump --file=~/dotfiles/Brewfile
```
، ثم
```
brew bundle check --file=~/dotfiles/Brewfile
```
.
4.
```
for p in brew apt dnf pacman; do command -v $p >/dev/null && { echo $p; break; }; done
```
.

الفصل 12: الـ Scripting

1. `"$@"` كل Argument لوحده، و `"$*"` كلهم نص واحد.
2.
```
[[ -d "$1" ]] || { echo "not a dir" >&2; exit 1; }
```
.
3. لأن `$(cat file)` بيتقسم على المسافات مش السطور، وبيعمل Glob. البديل:
```
while IFS= read -r line; do …; done < file
```
.
4. `-e` اقفل عند أول فشل، و `-u` المتغير المش متعرّف خطأ، و `-o pipefail` الـ Pipe يفشل لو أي جزء فشل.
5. عشان تنضّف (ملفات مؤقتة) أو تعمل حاجة لما الـ Script يخلص أو يتقطع.
6.
```
date -v-1d +%F 2>/dev/null || date -d yesterday +%F
```
.

صح ولا غلط: (1) غلط: من غير مسافات. · (2) صح. · (3) غلط: Exit code بس (0-255). · (4) صح.

التحديات:

1.
```
name="${1:-}"; [[ -z "$name" ]] && read -rp "Name: " name; echo "أهلاً يا $name"
```
.
2.
```
for f in *.*; do [[ -f "$f" ]] || continue; ext="${f##*.}"; mkdir -p "$ext" && mv -- "$f" "$ext/"; done
```
(جوه نسخة في `/tmp` الأول).
3.
```
while getopts "nd:" o; do case $o in n) dry=1;; d) dir=$OPTARG;; esac; done
```
، ثم `find "$dir" -type f -mtime +30` مع `-delete` لو مش dry، و `wc -l` للإجمالي.

الفصل 13: التخصيص

1. Login أول دخول (SSH، وكل تابة في Terminal.app لأنه بيفتح Login دايماً)، و Interactive أي Shell بتكتب فيه. في zsh: Login بيقرا `.zprofile` ثم `.zshrc`، و Interactive بيقرا `.zshrc` (و `.zshenv` دايماً). في bash: Login بيقرا `~/.bash_profile` (أو `.profile`)، و Interactive `~/.bashrc`.
2.
```
echo "alias gs='git status'" >> ~/.zshrc && source ~/.zshrc
```
.
3. الـ `~` جوه Quotes مبتتوسعش، والأهم إنه مسح الـ PATH القديم كله. الصح: `export PATH="$HOME/bin:$PATH"`.
4. لأن الـ Script بيشتغل في Shell ابن، والـ `cd` بتاعه مبيرجعش للـ Shell بتاعك.

5. أداة بيشغّلها `/etc/zprofile`، بتبني الـ PATH من `/etc/paths` و `/etc/paths.d` وتحط اللي كان موجود بعدهم. عشان كده سطر Homebrew مكانه `~/.zprofile` (بيتقري بعدها) مش `~/.zshenv` (قبلها، فيتزق لآخر الـ PATH).

صح ولا غلط: (1) صح. · (2) غلط: الـ Script بيشتغل في Process ابن. · (3) صح. · (4) غلط: path_helper هيعيد الترتيب بعده؛ مكانه `~/.zprofile`.

التحديات:

1. مفيش إجابة واحدة؛ المهم يكونوا في `~/.zshrc` ويشتغلوا في تابة جديدة.
2. `mkdir -p ~/.local/bin`، وحط الـ Script و `chmod +x`، و
```
echo 'export PATH="$HOME/.local/bin:$PAT
H"' >> ~/.zprofile
```
، وافتح تابة جديدة.
3.
```
security add-generic-password -a "$USER" -
s my-token -w
```
، وفي `~/.zshrc`:
```
export MY_TOKEN="$(security find-generic-p
assword -s my-token -w)"
```
.
4. `PROMPT='%(?.%F{green}.%F{red})%1~ %#%f '` مع
```
autoload -Uz vcs_info; precmd() { vcs_inf
o; }; setopt prompt_subst; RPROMPT='${vcs_
info_msg_0_}'
```
. وفي bash:
```
PROMPT_COMMAND='[[ $? == 0 ]] && c=32 || c
=31; PS1="\[\e[${c}m\]\w\$\[\e[0m\] "'
```
.

الفصل 14: الإنتاجية

1. `sudo !!`.
2. `Ctrl+C` يلغي (SIGINT)، و `Ctrl+Z` يوقف مؤقتاً (SIGTSTP)، و `Ctrl+D` نهاية الدخل (ويقفل الـ Shell لو السطر فاضي).
3. `tmux new -s job`، وشغّل الأمر، و `Ctrl+B d`، وتاني يوم `ssh` ثم `tmux attach -t job`.
4. `pbcopy < package.json`، و `code .` (أو `open -a "Visual Studio Code" .`).

صح ولا غلط: (1) صح. · (2) غلط: آخر Argument. · (3) غلط: بتفضل شغالة. · (4) غلط: لازم تفعّل Use Option as Meta key.

التحديات:

1. مفيش إجابة واحدة: جرّب (ولو Alt+F كتب حرف غريب، فعّل Option as Meta).
2.
```
history 1 | awk '{print $2}' | sort | uniq
-c | sort -rn | head -10
```
.
3.
```
notify() { "$@" && m="نجح" || m="فشل"; osa
script -e "display notification \"$m: $*\"
with title \"Terminal\""; }
```
.
4. `tmux new -s mon`، و `Ctrl+B %` و `Ctrl+B "`، وشغّل `htop` و `tail -f`، ثم `Ctrl+B d` و `tmux a -t mon`.

الفصل 15: الأوامر الخطيرة

1. لو `DIR` فاضي أو مش متعرّف، السطر بيبقى `rm -rf /*`. الآمن: `set -u` و `rm -rf -- "${DIR:?}"/*`.
2. `diskutil list external` قبل وبعد تركيب الفلاشة، وشوف الحجم و `external, physical`. `/dev/disk4` بيعدّي على Cache النظام، و `/dev/rdisk4` (Raw) مباشر وأسرع بكتير مع dd.
3. 777 بتخلي أي حد يعدّل ويشغّل. البديل: صلّح المالك (`chown`) وصلاحيات مناسبة (755 للمجلدات و 644 للملفات).
4. نزّل الملف (`curl -fsSLo`)، اقراه (`less`)، قارن الـ Checksum (`shasum -a 256`)، وشغّله من غير sudo لو ينفع.

صح ولا غلط: (1) غلط: `/*` بيتوسع لمسارات والحماية مبتشوفهاش. · (2) صح. · (3) صح. · (4) غلط: SIP بيحمي النظام بس، وملفاتك حمايتها Time Machine.

التحديات:

1. `setopt noclobber; echo a > f; echo b > f` (تاني واحد هيقول `file exists`)، و `echo b >| f` بتكتب عادي.

2. من غير حماية الـ echo بيطبع `rm -rf /Applications /Library /System …` مع `set -u` و `unset DIR` : `DIR: unbound variable` (أو `parameter not set` في zsh). ومع `${DIR:?}` : `parameter null or not set`.

3.
```
saferm(){ local d="$HOME/.trash/$(date +%F)"; mkdir -p -- "$d"; for f in "$@"; do case "$(realpath -- "$f")" in /|"$HOME") echo "refusing: $f" >&2; return 1;; esac; mv -- "$f" "$d/"; done; }
```

.

الفصل 16: المشروع النهائي

1. عشان النتايج بس تروح لـ stdout، فلو عملت Pipe أو `>` الرسايل متتخلطش بالبيانات وتفضل ظاهرة على الشاشة.

2. `ls -1t` الأرشيفات الأحدث الأول، و `tail -n +8` كل حاجة بعد أحدث 7، و `while IFS= read -r` اسم اسم، و `rm -v --` يمسح. ومستخدمناش `find -printf` لأنه GNU بس ومش موجود في find بتاع الماك.

3. لأن الـ Array الفاضية مبتضيفش Argument خالص، أما المتغير الفاضي في Quotes بيبقى `''`. والشكل `${dry[@]+...}` عشان bash 3.2 مع `set -u` بيعتبر الـ Array الفاضية مش متعرّفة.

4. بـ `uname -s` : `Darwin` = الماك. اللي بيختلف: `sysctl` و `vm_stat` بدل `/proc` و `free` ، و `lsof` بدل `ss` ، و `ps -A … -r` بدل `--sort` ، و `df -h -l` بدل `-x tmpfs`.

صح ولا غلط: (1) غلط: log على stderr. · (2) صح. · (3) غلط: launchd مبيقراش ملفات zsh؛ عشان كده حطينا PATH في `EnvironmentVariables`.

التحديات:

1. `opskit --version` بيطبع 1.0، و `health` بيطبع الأقسام الخمسة.

2.
```
cmd_clean(){ local dry=0; [[ "${1:-}" == -n ]] && dry=1; if (( dry )); then find /tmp -maxdepth 1 -user "$USER" -mtime +7 -print; else find /tmp -maxdepth 1 -user "$USER" -mtime +7 -exec rm -rf -- {} +; fi; }
```
وضيفها للـ case.

3. في آخر `cmd_health` :
```
df -h -l | awk 'NR>1 {sub("%","",$5); if ($5+0 > 90) bad=1} END {exit bad}'
```
، والـ plist فيه `StartInterval` = 3600 و `ProgramArguments` :
```
/bin/sh -c 'opskit health >/dev/null || osascript -e "display notification \"Disk > 90%\""'
```

.

الـ Terminal بقى بيتكلم لغتك

أحسن طريقة تثبّت بيها اللي اتعلمته: افتح الـ Terminal كل يوم، وكل ما تعمل حاجة بالماوس اسأل نفسك "أعملها إزاي بأمر؟". ولما تقابل Flag أو رمز مش فاهمه، ارجع لجدول الـ Flags وجدول الرموز في الملحق أ.

```
ahmed@MacBook ~ % exit
```